

THÈSE POUR OBTENIR LE GRADE DE DOCTEUR DE L'UNIVERSITÉ DE MONTPELLIER

En Biostatistiques

École doctorale : Information, Structures, Systèmes

Unité de recherche : Institut Montpellierain Alexander Grothendieck

Apprentissage par renforcement basé sur un modèle pour le contrôle de processus de décision semi-markoviens déterministes par morceaux partiellement observables

Présentée par Orlane Rossini

le 28/11/2025

Sous la direction d'Alice Cleynen
et Benoîte de Saporta et Régis Sabbadin

Devant le jury composé de

Elodie Brunel-Piccinini, Professeure, Université de Montpellier
Alice Cleynen, Directrice de Recherche, Université de Montpellier
Benoîte de Saporta, Professeure, Université de Montpellier
Audrey Durand, Professeure adjointe, Université Laval
Alexandre Génadot, Maître de conférence, Université de Bordeaux
Erwan Le Pennec, Professeur, Ecole Polytechnique
Odalric-Ambrym Maillard, Chargé de Recherche, Inria Lille
Régis Sabbadin, Directeur de Recherche, INRAe Toulouse
Meritxell Vinyals, Chargée de Recherche, INRAe Toulouse

Examinatrice
Co-encadrante
Co-directrice
Rapporteuse
Rapporteur
Rapporteur
Examineur
Co-directeur
Invitée



UNIVERSITÉ
DE MONTPELLIER

Ce n'est pas l'industrie qui mesure le progrès humain. Ce progrès est mesuré par la valeur qu'on accorde à une vie, une vie sans importance, une vie sans le moindre privilège. [...] C'est cela qui définit une époque. C'est cela qui définit une espèce.

LE DOCTEUR
Doctor Who

Remerciements

Je souhaite exprimer toute ma gratitude à Audrey Durand, Alexandre Génadot et Erwan Le Pennec pour avoir accepté de rapporter mon manuscrit et pour leurs commentaires constructifs. J'adresse également mes sincères remerciements à Élodie Brunel et Odalric Maillard d'avoir accepté de faire partie de mon jury. Merci à chacun d'entre vous d'avoir consacré de votre temps à mes travaux de recherche.

Ce manuscrit n'existerait pas sans ma direction de thèse. Benoîte, Alice et Régis, je vous suis profondément reconnaissante pour nos échanges, votre bienveillance et vos conseils précieux tout au long de ces trois années. Chacun de vous a contribué à ma progression d'une manière unique. J'ai énormément appris à vos côtés, bien au-delà du cadre scientifique. Benoîte, merci pour toutes nos discussions sur l'organisation du laboratoire, du département de mathématiques et de l'Université. Alice, grâce à toi j'ai découvert tout un continent en plus d'un super champignon unicellulaire ! Régis, merci pour ton soutien constant, qui m'a souvent redonné le moral. Merci à vous trois, sincèrement.

Enfin, cette thèse n'aurait pas eu la même tête sans Meritxell. D'abord membre de mon CSI, tu as sauté à pied joint dans l'aventure. Tu es devenue ma quatrième encadrante de thèse avec une expertise plus que précieuse sur le deep ! Au-delà de l'aspect scientifique, j'ai trouvé auprès de toi une écoute attentive et de précieux conseils. Merci.

During a period of doubt, Alice showed me that it was possible to link several fields of research and that, even if my subject was temporarily blocked, exploring other avenues could be enriching. With this in mind, Alice and Nikolay agreed to include me in the yeast project. Thank you both for this valuable experience, which not only kept me on track, but also confirmed my desire to continue research. Thank you for our enriching exchanges.

Je tiens à remercier l'équipe BIATSS, sans qui tous mes allers-retours en conférence n'auraient pas été possibles. Merci à Nathalie, Sophie, Céline, Carméla, Brigitte et Karine d'avoir supporté mes râleries sur Étamine. Un grand merci à Laurence et Heinui pour votre travail au sein du département de mathématiques. Plus généralement merci à toute l'équipe administrative pour nos discussions de couloirs, toujours agréables !

Pendant ces trois dernières années, je vous ai souvent appelés les *adultes* du laboratoire. J'ai beaucoup apprécié nos échanges de couloir et vos conseils avisés. François-David, nos discussions sur les LLMs appliqués à la biologie ont été particulièrement stimulantes. Hélène et Bérénice, merci pour ces moments autour du café-e. J'ai pris beaucoup de plaisir à partager

ces instants avec les femmes du labo. Ali, Glawdys et Élodie, cette conférence au Canada a été d'autant plus agréable grâce à votre bonne humeur. Paul, ton aide sur les questions bayésiennes a été précieuse. Enfin, Xavier, merci d'avoir écouté mes idées sur les autoencoders et supporté mes anglicismes.

Ces trois années de thèse ont été d'autant plus agréables grâce à l'ambiance exceptionnelle du laboratoire, portée par notre groupe de doctorant·e·s soudé. Sacha, nos discussions sur le Canada ont été de belles pauses pleines de douceur. Sofian, merci d'avoir voté pour moi quand j'étais bloquée à Québec. Chloé, co-bureau inestimable, tu as répondu à mes questions les plus bêtes sans émettre aucun jugement. Salomé, les sorties patinoires resteront mémorables, et Juliette, mon agrégée préférée, ton écoute et ta joie de vivre ont été salutaires. Et merci à tous les doctorant·e·s du laboratoire pour ces Semdocs, week-end ski, soirées et nos discussions de couloirs. Merci Pierrick, Marwa, Amélie, Christina, Christelle, Julien, Joris, Dark'Toine, Aurelio, Arax, Morgane, Luca, Ulysse, Pierre, et tous ceux que j'oublie ici. Aux futur·e·s doctorant·e·s qui rejoindront ce laboratoire (Abel, Clément, Delfin, Anatole, Malo, Joseph, Andrea, et tous les autres) conservez ces liens soudés : ils transforment l'expérience de la thèse.

Au-delà de ce manuscrit, ces trois années de thèse m'ont offert certains des plus beaux fous rires de ma vie et la chance de les partager avec des personnes formidables. Tanguy, j'espère un jour être à la hauteur de ta relève. *Ma Tchilde*, nos discussions à la fenêtre, à refaire le monde en observant ceux qui faisaient des créneaux, ne sont que le début d'une profonde et belle amitié. Corentin, bravo d'avoir pu supporté mon côté irritant et merci de nous avoir montré le chemin de la déconstruction. Pierre, que notre CDI d'amitié perdure, et qu'il nous mène encore vers plein de villages à découvrir ensemble. Bart', l'enfant-adulte de ce groupe, toujours partant pour tout ! tes imitations de truies me hanteront jusqu'à la fin des temps. Jeanne, merci d'être la force tranquille du groupe, celle qui nous rappelle qu'un vrai rire se mérite. Anne la vilaine... un surnom bien mal choisi pour quelqu'un d'aussi gentil ! Alexandre et Pilou, ce duo capable de nous ramener en enfance (ou à *Beaufland*...). Axel... mille fois merci.

Papa, Maman, je vous dédie ce dernier paragraphe de remerciements. Aucun mot ne saurait exprimer toute ma gratitude pour votre soutien inconditionnel tout au long de ces années. Grâce à vous, j'ai pu croire en moi et avancer avec confiance, et c'est votre amour qui m'a porté jusqu'ici.

Contents

Introduction	2
Résumé en Français	6
1 Markov Decision Processes and Reinforcement Learning	12
1.1 Markov Decision Process Formalism	14
1.1.1 Markov Decision Process Definition	14
1.1.2 Policies	16
1.1.3 Evaluation Criteria of a Policy	17
1.2 Markov Decision Process Extensions	21
1.2.1 Partially Observable MDP	22
1.2.2 Bayes Adaptive MDP	27
1.2.3 Bayes Adaptive Partially Observable MDP	31
1.3 Resolution Methods for MDPs, POMDPs and Bayes Adaptives Extensions . . .	32
1.3.1 Dynamic Programming Algorithms	33
1.3.2 Monte-Carlo Simulations-Based Algorithms	36
1.3.3 Reinforcement-Learning Algorithms	39
1.4 Conclusion	48
2 Impulse Control of Piecewise Deterministic Markov Processes	50
2.1 PDMP, Definition and Examples	51
2.1.1 Generic Definition	51
2.1.2 Semi-Markov Dynamics	55
2.1.3 Simulation	57
2.1.4 Embedded Chains	58
2.2 Impulse Control for PDMPs	60
2.2.1 Continuous versus Impulse Control	61
2.2.2 Definition of an Impulse Control Problem	61
2.2.3 Resolution of Impulse Control Problems for PDMP	65
2.3 Conclusion	66

3	Case Study: Myeloma Cancer Patient Follow-Up	68
3.1	Myeloma Cancer Patient	69
3.1.1	Disease Dynamics	70
3.1.2	Decisions	70
3.2	Modeling Patient Dynamics	71
3.2.1	Construction of a Controlled Process	71
3.2.2	Local Characteristics	73
3.2.3	Transition Kernel	74
3.3	Conclusion	76
4	Construction of ϵ-Optimal Strategies for Fully-Observed Controlled PDMPs	78
4.1	Theoretical Foundations	79
4.1.1	Assumptions	79
4.1.2	Principles of Quantization	81
4.1.3	Numerical Method to Approximate the Value Function	84
4.2	Numerical Construction of a Controlled Trajectory	87
4.2.1	Definition of the Candidate Policy Components	87
4.2.2	Numerical Construction of a Candidate Strategy	88
4.2.3	Cost of the Candidate Strategy	89
4.2.4	Candidate Strategy Optimality	96
4.3	Numerical Application	101
4.3.1	Application to Cancer Treatment Optimization	101
4.3.2	Practical Implementation	103
4.3.3	Numerical Results	103
4.4	Conclusion	104
5	Planning for Partially Observed Impulse Control of PDMPs	106
5.1	Controlled PDMP as a MDP	107
5.1.1	Formulation as a MDP	107
5.1.2	Application to Cancer Treatment Optimization	110
5.2	Partially Observed Controlled PDMP as a POMDP	112
5.2.1	Formulation as a POMDP	112
5.2.2	Application to Cancer Treatment Optimization	113
5.3	Resolution of a POMDP	114
5.3.1	Experimental Design	114
5.3.2	Numerical Results	116
5.4	Conclusion	119
6	Planning under Model Uncertainty for Impulse Control of PDMPs	120
6.1	Controlled PDMP as a BAMDP	122
6.1.1	Controlled PDMP with ill-known parameters	122

6.1.2	Formulation as a BAMDP	122
6.1.3	Application to Cancer Treatment Optimization	125
6.2	Partially-Observed Controlled PDMP as a BAPOMDP	126
6.2.1	Formulation as a BAPOMDP	127
6.2.2	Application to Cancer Treatment Optimization	129
6.3	Solving a BAPOMDP with Deep RL	129
6.3.1	Experimental Design	129
6.3.2	Numerical Results	131
6.4	Comparative Study of Modeling Frameworks	134
6.4.1	Experimental Design	134
6.4.2	Numerical Results	135
6.5	Conclusion	137
7	Stress-responsive Epitranscriptome of Translated RNA in Yeast	140
7.1	Algorithm Specification	141
7.2	Experimental Results	143
7.3	Conclusion	145
	Conclusion and Perspectives	148
	Conclusion	148
	Perspectives	149
	Short-Term Perspectives	149
	Long-Term Perspectives	150
	References	153
A	Proof of the a posteriori update formulas	I
B	Numerical experiment details	I
B.1	Model Parameters	I
B.2	Neural network set hyperparameters	II
B.3	Neural network final hyperparameters	II

Introduction

Piecewise-Deterministic Markov Processes (PDMPs), first introduced by Davis [Dav84], constitute a family of Markov processes characterized by deterministic motion interspersed with random jumps. In contrast to classical Markov chain models, PDMPs not only model the random jumps of a system but also describe a deterministic continuous motion between those jumps. Many real-life phenomena require both discrete random jumps and continuous dynamics for their description. Consequently, PDMPs have been widely applied in various disciplines since their introduction, including biology, economics, medicine, and other fields [AK11; Goa+23; WC23].

The problem of optimally controlling a fully observed PDMP through discrete-time decisions, also known as impulse control of PDMPs, has been formulated by Davis [Dav93] and further extended by subsequent works [BR10; SDG10; BR11; CD22]. While theoretical aspects have been widely studied, the literature on numerical aspects is much scarcer, especially when the jump dates are not observed. In the case of partial observations, this problem can be cast as a Partially Observed Markov Decision Process (POMDP), which itself can be viewed as a continuous MDP (over belief states). There exist few approximation methods for solving this continuous MDP [CS21; Sap+24]. However, these different approaches all assume that the underlying PDMP model is known or, at least, can be simulated.

In many real-world applications, the underlying dynamics of the PDMP are not fully known and must be estimated from limited data. This situation is especially common in medical contexts, where available observations provide only partial information about the patient’s behavior. As a result, estimates of the process dynamics can be uncertain or inaccurate, complicating the design of effective control strategies. To date, no established methods exist for controlling PDMPs when the model is both partially observable and unknown. Recent work has begun to explore this frontier: for instance, [DFD20] investigated the use of Bayesian Reinforcement Learning (BRL) with ordinary differential equations for controlling continuous-time semi-Markov decision processes. Similarly, BRL techniques have been applied to solve POMDPs in general [MT15], but these approaches have yet to be adapted to the specific challenges posed by PDMPs with unknown dynamics.

This thesis addresses the control of PDMPs in difficult settings characterized by partial observability, limited data, uncertain system dynamics, and the inability to conduct direct experiments on real data. The core question we seek to answer is: *How can we get effective control strategies when observations are incomplete and the underlying dynamics are poorly known?* To

Table 0.1: **Overview of Publications and Code Repositories Associated with Each Thesis Chapter**

Chapters	Papers	Code
Chapter 1	[Cle+25] and	Jupyter Notebooks available at
Chapter 2	ongoing book chapter	https://github.com/orlross/phd_notebook
Chapter 4	Ongoing paper	https://github.com/orlross/numerical_esp_opt_strategy
Chapter 5	[Cle+24b; Cle+24a]	https://forge.inrae.fr/orlane.rossini/
Chapter 6	[Ros+25]	controlled_pdmp_po
Chapter 7	Ongoing paper	https://github.com/Arnaroo/Yomics

tackle this challenge, we leverage an innovative framework that transforms the unknown process into a well-characterized, simulable model. This approach explicitly incorporates uncertainty about the underlying dynamics, enabling us to harness powerful existing algorithms for effective control. This work was carried out as part of the ANR HSMM (Hidden Semi-Markov Models: Inference, Control and Applications) project, which provided the scientific basis and funding necessary for this research. It is supported throughout the manuscript by publications and code repositories that are listed in Table 0.1.

This work is interdisciplinary, lying at the crossroads of statistics and process modeling, guided by Alice Cleyen and Benoîte de Saporta. It also involves artificial intelligence, especially Reinforcement Learning (RL) theory, under the supervision of Régis Sabbadin and Meritxell Vinyals¹. To build a common foundation, the manuscript begins with two introductory chapters covering these fields. Chapter 1 provides a comprehensive review of MDPs and their extensions, along with the principal methods for solving sequential decision-making problems. Special emphasis is placed on RL techniques, especially deep RL algorithms, as they form the backbone of the solutions proposed in later chapters. Chapter 2 presents a detailed state-of-the-art review of PDMPs, with a particular focus on semi-Markov processes, which represent the specific class of PDMPs studied in this work. We explain their mathematical formulation, their simulation properties, which are key to our approach, and how they can be controlled, especially in the context of impulse control, which is the framework adopted throughout the thesis. This chapter sets the stage for the application and methods developed later. The purpose of these two chapters is to select, organize, and synthesize the relevant body of knowledge in a way that supports the rest of the thesis and makes the underlying concepts accessible to readers from diverse scientific backgrounds.

Chapter 3 introduces the real-world application that motivates this thesis: the follow-up and monitoring of patients diagnosed with multiple myeloma. We begin by describing the clinical context, the decision-making challenges, and the data limitations inherent to the problem. We then define in detail the PDMP model used to represent the cancer’s evolution over time, which will serve as the basis for all the experiments proposed in subsequent chapters.

With the case study established, the thesis then progresses through increasing levels of com-

¹Research Fellow at INRAE Toulouse

plexity in PDMP control: starting from fully observable systems with known dynamics, and moving toward realistic environments characterized by partial observation and model uncertainty. Each chapter builds upon the previous one, forming a coherent path from theoretical modeling to the application of learning algorithms. The overall contributions include transforming complex models into solvable formulations, the design of simulation-based numerical methods and their integration into a practical healthcare context.

Chapter 4 introduces the first level of complexity, assuming that the PDMP model is fully observable and its dynamics are perfectly known. We propose a numerical method to compute a control strategy close to optimality. The approach is supported by convergence proofs that guarantee the strategy falls within the ϵ -optimal strategies family. The design and analysis of this numerical method constitute an original contribution of the thesis.

Moving to a more complex setting, Chapter 5 introduces partial observability. Using the theoretical foundations established in Chapter 1, we reformulate our problem as a POMDP. Given the high dimensionality and continuous nature of the resulting belief space, classical solution methods are impractical. My contribution in this chapter lies in applying deep RL algorithms to solve this POMDP effectively. This chapter marks the transition from classical to learning-based approaches and demonstrates how deep RL can be used to control PDMPs under realistic conditions.

The last layer of complexity is introduced in Chapter 6 with model uncertainty. Here, we assume that a parameter of the PDMP dynamics is unknown, which further complicates the decision-making process. To address this, we turn the PDMP into a Bayesian Adaptive POMDP (BAPOMDP), which accounts for both partial observability and parametric uncertainty. This framework enables the construction of a simulator that incorporates model belief updates, and the optimization problem is solved using deep RL algorithms, as in the previous chapter.

Although unrelated to the core topic of this thesis, the final Chapter 7 presents a side project conducted within a collaborative effort led by Alice Cleynen and Nikolay Shirokikh². This project focuses on studying the impact of glucose stress on mRNA regulation in yeast. In this project, my contributions included developing numerical methods for processing yeast transcriptome data, as well as performing statistical analyses on the effects of glucose stress on mRNA modifications and their influence on protein translation efficiency. In the scope of this thesis, however, I will only present the algorithm development, while the biological analyses are left outside the main discussion.

²Researcher in molecular biology from the John Curtin School of Medical Research, Australian National University

Résumé en Français

La théorie du contrôle joue un rôle essentiel dans la compréhension et l'optimisation du comportement des systèmes dynamiques complexes dans diverses disciplines scientifiques et techniques. Les Processus de Markov Déterministes par Morceaux (PDMPs) et les Processus de Décision Markovien (MDPs) sont deux cadres clés qui ont émergé pour modéliser et résoudre les problèmes de contrôle dans les systèmes stochastiques. Chacun de ces cadre a ses propres forces, et leur intersection offre des opportunités prometteuses pour aborder une large classe de problèmes, en particulier dans le contexte du contrôle impulsif et de la prise de décision dans les systèmes complexes.

Les PDMPs, introduits dans les années 80 par [Dav84], sont une classe polyvalente de processus stochastiques à temps continu et à espace d'état hybride (discret et continu) qui combinent une dynamique déterministe avec des événements stochastiques discrets, ce qui permet une représentation réaliste des systèmes ayant un comportement hybride. Le contrôle impulsif pour les PDMPs présente un intérêt particulier dans des applications allant de l'allocation des ressources et de la planification de la maintenance aux systèmes biologiques et à la gestion de l'énergie. Ces problèmes consistent à déterminer le moment optimal et l'ampleur des interventions afin de minimiser un coût ou de maximiser une récompense.

D'autre part, les MDPs, introduits dans les années 50 par [Bel58], fournissent un cadre bien établi pour la prise de décision dans l'incertitude, en tirant parti des espaces d'état et d'action discrets, établi à l'origine pour faciliter les approches analytiques et informatiques. Ils ont été étendus à des espaces d'état et d'action continus. Les extensions du cadre classique des MDPs, telles que les processus de décision semi-markoviens, les MDPs partiellement observés ou les MDPs bayés-adaptatif, offrent une plus grande flexibilité dans la modélisation de divers problèmes du monde réel. Les MDPs ont contribué à faire progresser l'étude du contrôle stochastique, notamment grâce à leur compatibilité avec les techniques de programmation dynamique approchée.

La relation entre les PDMPs et les MDPs constitue un axe d'exploration naturel, notamment dans le contexte du contrôle impulsif. En particulier, la reformulation de problèmes de contrôle impulsif pour les PDMPs dans le cadre des MDPs ouvre la voie à l'utilisation d'outils théoriques et numériques bien établis pour les MDPs, tels que la programmation dynamique, les méthodes d'approximation ou d'apprentissage. Cette approche permet de surmonter les difficultés analytiques posées par la nature hybride des PDMPs, tout en profitant de la richesse méthodologique des MDPs.

En réunissant ces deux cadres, et en s'appuyant sur des travaux antérieurs, ce travail contribue au domaine croissant de la recherche à l'interface du contrôle stochastique, de la théorie de la décision, des statistiques, des sciences informatiques et de l'apprentissage par renforcement. Dans cette perspective, j'explore de manière systématique comment les problèmes de contrôle impulsif pour les PDMPs peuvent être formulés, analysés et résolus efficacement dans le cadre des MDPs, afin d'étendre cette intégration au-delà des approches existantes.

J'illustre l'ensemble des concepts étudiés à l'aide d'une étude de cas médicale portant sur les patients atteints de myélome multiple, un cancer qui affecte les plasmocytes, cellules productrices d'anticorps dans la moelle osseuse. La gestion de cette pathologie complexe requiert un équilibre délicat entre l'intensité du traitement et la fréquence des examens médicaux. En effet, un traitement trop agressif peut entraîner une résistance aux médicaments et, en fin de compte, la mort. De plus, la progression de la maladie varie fortement d'un patient à l'autre, ce qui rend indispensables les stratégies de traitement personnalisé [HHB07; Soz+01; Ger+16]. Mettre en œuvre de telles stratégies nécessite toutefois une compréhension fine de la dynamique de la maladie ainsi que des outils prédictifs capables d'anticiper son évolution pour adapter les décisions cliniques en temps réel. Or, cette dynamique est intrinsèquement stochastique, marquée par des évolutions imprévisibles dans le développement des cellules cancéreuses, les taux de mutation et les réponses aux traitements. Les PDMPs permettent de représenter au mieux cette complexité et de soutenir une personnalisation des soins fondée sur des simulations fiables en intégrant à la fois les aspects déterministes et aléatoires de la progression de la maladie. Toutefois, la complexité intrinsèque du problème médical se traduit directement dans le modèle, de sorte qu'il n'existe, à notre connaissance, aucune méthode permettant d'obtenir une résolution explicite de ce type de PDMP.

Les contributions de cette thèse s'organisent autour de plusieurs axes complémentaires, appuyés par des publications, et peuvent être résumées comme suit :

- revue des différents cadres de modélisation et de résolution (*article d'état de l'art [Cle+25] et chapitre de livre soumis en juillet 2025 co-écrit avec Alice Cleynen, Benoîte de Saporta, Régis Sabbadin et Amélie Vernay*³);
- méthode numérique pour expliciter une stratégie ϵ -optimale dans le cas d'un PDMP complètement observé (travail en cours);
- résolution d'un PDMP contrôlé partiellement observé par apprentissage par renforcement profond (*en collaboration avec Meritxell Vinyals*⁴, *acte de conférence [Cle+24a] et communication [Cle+24b]*);
- formulation et résolution par apprentissage par renforcement profond d'un PDMP contrôlé à dynamique partiellement connue et observation partielle (*en collaboration avec Meritxell Vinyals, pré-publication [Ros+25]*).

³Doctorante à l'Université de Montpellier

⁴Chargée de Recherche à l'INRAE Toulouse

L'ensemble de ces contributions est résumé dans la Figure 0.1, qui illustre la structure de la thèse et les relations entre les différents chapitres.

Revue des différents cadres de modélisation et de résolution. Nous proposons une introduction structurée et approfondie aux différents cadres de modélisation et de résolution existants. Dans un premier temps, nous présentons les MDP et leurs extensions, en insistant particulièrement sur les approches de résolution par apprentissage par renforcement. Nous abordons ensuite les PDMPs, avec un focus sur les processus semi-markoviens. Nous en détaillons la formulation mathématique, les propriétés de simulation ainsi que les différentes méthodes de contrôle, en particulier le contrôle impulsif, qui sera adopté tout au long du manuscrit. Au-delà de la synthèse, cette revue fournit le cadre conceptuel dans lequel s'inscrivent nos contributions méthodologiques.

Méthode numérique pour calculer une stratégie ϵ -optimale. [SD12] propose une méthode numérique pour approcher la fonction valeur à partir de la quantification d'une chaîne induite (à temps discret) permettant une approximation des opérateurs de la programmation dynamique. Dans [SDG17], une famille de stratégies ϵ -optimales est construite à partir de l'itération de la fonction valeur obtenue à partir des opérateurs non quantifiés. Par conséquent, pour construire une famille de stratégies ϵ -optimales explicites, nous proposons de remplacer les opérateurs réels dans la construction de [SDG17] par leur approximations de [SD12]. Le principe consiste à extraire, à partir de l'itération de la fonction valeur des opérateurs discrétisés, les dates d'impulsions et les nouveaux points de départ sélectionnés par la programmation dynamique et les appliquer à la trajectoire réelle à contrôler.

En suivant l'approche introduite dans [SDG17], nous définissons un nouveau processus $\{\tilde{X}_t\}$ sur un espace d'état élargi. Ce processus représente une version contrôlée du PDMP original $\{X_t\}$, avec le même flot déterministe mais un mécanisme de saut modifié pour conserver la mémoire des sauts et impulsions passées.

On obtient ainsi une stratégie candidate $\tilde{\mathcal{S}}$ qui est une séquence de dates d'impulsions et de nouveaux points de départ. Nous montrons alors que le coût du PDMP contrôlé par la stratégie $\tilde{\mathcal{S}}$ a une forme récursive.

Nous montrons que cette stratégie candidate est ϵ -optimale. La preuve de convergence s'appuie sur le théorème 5.11 de [SD12], qui garantit que la fonction valeur approchée converge vers la fonction valeur réelle. En combinant ce résultat avec les lemmes intermédiaires de [SD12], nous établissons que le coût de la trajectoire contrôlée par la stratégie candidate est proche de vers la fonction valeur approchée et par extension, de la fonction valeur réelle.

Néanmoins, du fait de sa complexité de calcul, cette méthode n'est applicable qu'aux PDMP de faible dimension et de faible complexité.

Résolution d'un PDMP contrôlé partiellement observé. Dans le cas partiellement observé, où les temps de saut sont cachés, la résolution du PDMP devient plus complexe. Une approche consiste à reformuler le problème comme un MDP Partiellement Observé (POMDP) [CS18; CS23]. Nous proposons une méthode de résolution originale fondée sur l'apprentissage par renforcement profond, qui exploite uniquement un simulateur de trajectoires sans nécessiter

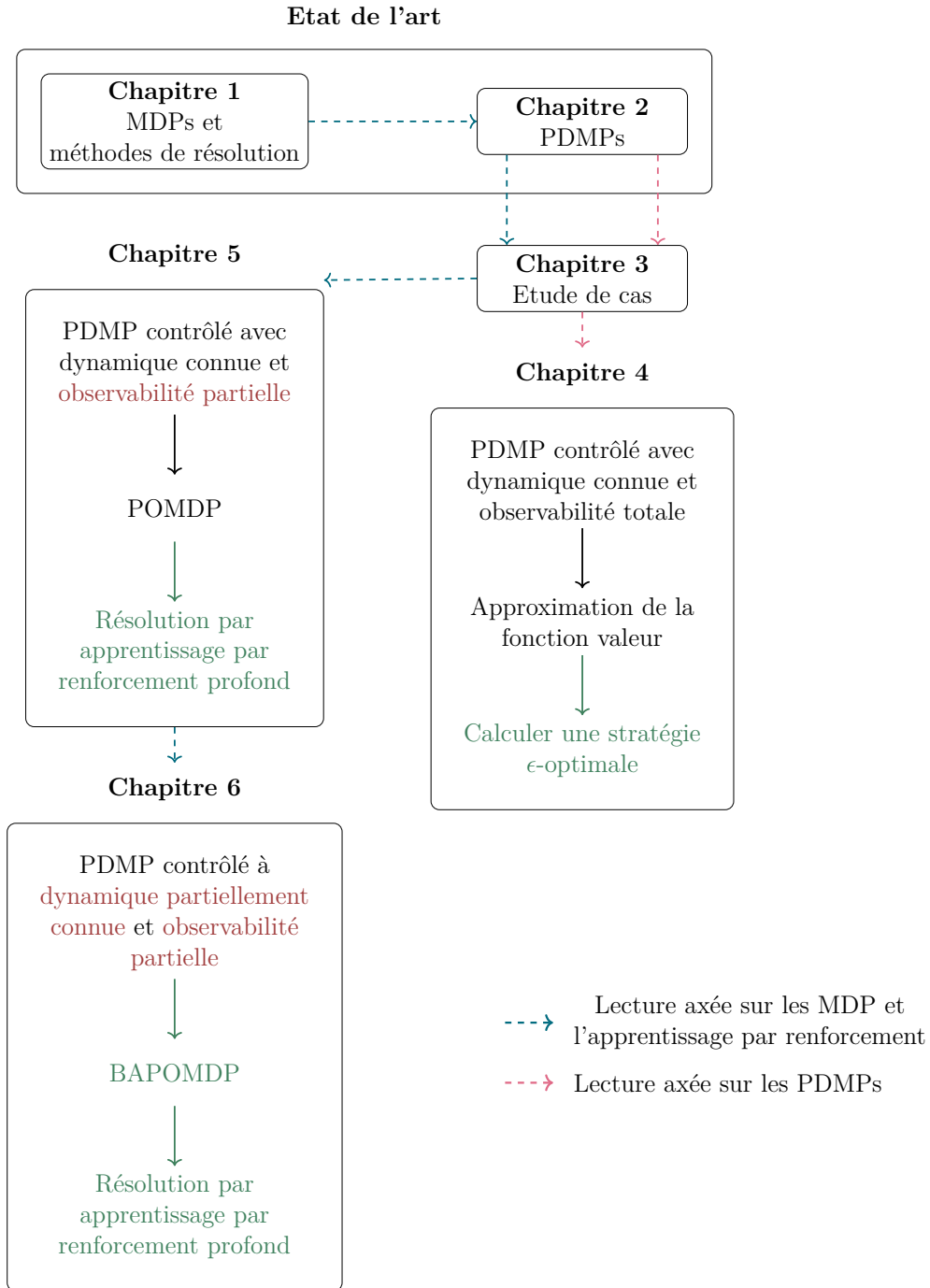


Figure 0.1: **Synthèse des contributions et structure de la thèse.** Les flèches en pointillés indiquent les fils directeurs: en bleu, une lecture orientée MDPs et apprentissage par renforcement, en rose, une approche orientée contrôle impulsif. Les flèches pleines décrivent l'organisation interne des chapitres. Les éléments rouges signalent les difficultés propres aux PDMPs et les éléments verts, les contributions de la thèse.

de réduction de dimension. Un benchmark est mené entre deux algorithmes largement utilisés, *Proximal Policy Optimization* (PPO) et *Deep Q-Networks* (DQN), en testant également l'apport d'un réseau récurrent (LSTM) pour PPO et l'effet de l'action-masking, qui permet d'interdire certaines actions non admissibles. On mesure les performances de chaque algorithme en terme

de coût moyen de la politique sélectionnée. Nos résultats sur le modèle myélome montrent que:

- l'apport du LSTM n'améliore pas les performances de PPO;
- l'*action-masking* améliore nettement les performances de PPO, mais dégrade celles de DQN;
- DQN obtient les meilleures performances en terme de coût moyen, suivi de près par PPO avec *action-masking*;
- les politiques apprises par ces deux algorithmes diffèrent fortement: DQN conduit à une absence totale de traitement, tandis que PPO recommande des traitements long, assurant un meilleur taux de survie.

Ainsi, l'apprentissage profond apparaît comme une piste prometteuse pour contrôler des PDMPs partiellement observés, mais les politiques apprises soulèvent des questions d'interprétabilité et de pertinence clinique.

Formulation et résolution d'un PDMP contrôlé à dynamique partiellement connue et observation partielle. Dans le cadre partiellement observé, lorsque la dynamique du modèle est mal connue, la simulation du PDMP n'est plus possible et l'apprentissage par renforcement direct ne peut être appliqué. Nous proposons alors de reformuler le problème comme un Bayes-Adaptatif POMDP (BAPOMDP). Alors que la plupart des travaux supposent que le prior suit une loi de Dirichlet sur les fonctions de transition et d'observation, notre situation est plus complexe : la fonction de transition résulte d'une combinaison du flot, du noyau et de l'intensité de saut du PDMP. Nous devons donc recourir à une loi continue pour notre prior. Ainsi, bien que le modèle PDMP soit initialement mal connu, le BAPOMDP associé est entièrement défini et simulable.

Pour résoudre ce BAPOMDP, nous appliquons le même benchmark que pour le POMDP à modèle connu. Cette fois, les performances des algorithmes mesurent la robustesse des politiques: nous entraînons les politiques sur le BAPOMDP, puis nous les testons sur un POMDP dont la paramétrisation du flot diffère. Nos résultats sur le modèle myélome montrent que:

- l'utilisation d'un LSTM améliore les performances de PPO;
- l'*action-masking* améliore les performances de PPO et celles de DQN;
- PPO avec *action-masking* obtient les meilleures performances en terme de coût moyen, suivi de près par DQN avec *action-masking*;
- les politiques apprises sont similaires: beaucoup de traitement administrés sur de faibles période de temps avec un délai maximum entre deux visites pour DQN avec *action-masking* et un délai réduit pour PPO avec *action-masking*.

Nous comparons également notre approche bayes-adaptative avec des stratégies d'inférence plus classiques. Dans un POMDP à modèle partiellement connu, l'inconnue peut-être remplacée soit par un estimateur ponctuel, soit par un estimateur bayésien. Nous entraînons alors l'algorithme DQN avec *action-masking* sur trois cadres : le BAPOMDP, le POMDP avec estimateur ponctuel et le POMDP avec estimateur bayésien, puis nous évaluons les politiques obtenues sur un POMDP dont la paramétrisation du flot diffère. Nos résultats sur le modèle myélome montrent que:

- l'entraînement avec le POMDP utilisant un estimateur ponctuel fournit les performances les plus stables;
- l'entraînement avec le POMDP muni d'un prior bayésien donne des performances légèrement supérieures en terme de coûts moyens.

Ces résultats montrent que le cadre BAPOMDP est bien adapté au contrôle des PDMP avec des dynamiques partiellement connues et des observations partielles, fournissant des garanties théoriques supplémentaires par rapport à un POMDP avec un estimateur bayésien.

Le projet levure. En parallèle de ces travaux, je participe à un projet collaboratif dirigé par Alice Cleynen et Nikolay Shirokikh⁵, consacré à l'étude de l'ARN chez la levure *Saccharomyces cerevisiae*. Mes contributions dans ce cadre concernent :

- le développement d'un algorithme de segmentation pour améliorer l'annotation des gènes dans la levure;
- l'analyse de l'impact des modifications de l'ARN messenger induites par le stress glycémique sur l'efficacité de la traduction des protéines.

Ce travail est toujours en cours, et quelques résultats préliminaires sont présentés dans ce manuscrit (Chapitre 7).

⁵The John Curtin School of Medical Research, Australian National University

Chapter 1

Markov Decision Processes and Reinforcement Learning

1.1	Markov Decision Process Formalism	14
1.1.1	Markov Decision Process Definition	14
1.1.2	Policies	16
1.1.3	Evaluation Criteria of a Policy	17
1.2	Markov Decision Process Extensions	21
1.2.1	Partially Observable MDP	22
1.2.2	Bayes Adaptive MDP	27
1.2.3	Bayes Adaptive Partially Observable MDP	31
1.3	Resolution Methods for MDPs, POMDPs and Bayes Adaptive Extensions	32
1.3.1	Dynamic Programming Algorithms	33
1.3.2	Monte-Carlo Simulations-Based Algorithms	36
1.3.3	Reinforcement-Learning Algorithms	39
1.4	Conclusion	48

Markov Decision Processes (MDPs), introduced in the 1950s [Bel57; How62], model scenarios where decisions are made sequentially in a stochastic environment. The goal is to identify an optimal sequence of actions that optimizes a desired outcome, such as minimizing long-term costs or maximizing long-term rewards, while accounting for the system’s inherent uncertainties. In essence, an MDP describes a setting in which an agent observes the current state, selects an action, and receives feedback, with the system evolving probabilistically. MDPs are a foundational class of stochastic control problems, extensively detailed in works such as [Put94a; BDH99].

The versatility of MDPs in capturing stochastic dynamics and optimizing sequential decisions has led to their adoption across diverse fields, such as engineering, computer science, economics and other social sciences, making them a valuable tool for researchers and practitioners alike. In robotics, MDPs have been employed for tasks such as navigation and control [PTL13]. Conversational agents and dialogue systems have leveraged MDPs to model and optimize human-machine interactions [LPE98; You+13]. The medical field has witnessed numerous applications of MDPs, including modeling disease progression and treatment planning for conditions like Alzheimer’s disease [Hoe+10], HIV [KRL16], and diabetes [RCA18]. MDPs have also been used in clinical decision-support systems for critical care [Sch+04].

While MDPs assume full observability of the system’s state, many real-world problems involve uncertainty not only in the dynamics but also in the agent’s perception of the environment. Partially Observable Markov Decision Processes (POMDPs) extend the MDP framework by introducing observation models that capture this limited visibility, requiring the agent to maintain and update a belief (a probability distribution over possible states) based on noisy or incomplete observations [KLC98; Åst65]. Separately, Bayes-Adaptive MDPs (BAMDPs) address model uncertainty by allowing the agent to learn unknown parameters of the environment dynamically, treating them as part of an augmented state space [Duf02; Pou+06]. Combining both challenges, Bayes-Adaptive POMDPs (BAPOMDPs) incorporate partial observability and model uncertainty within a unified framework, enabling the agent to simultaneously infer hidden states and learn model dynamics [RCP07; Ros+11]. These extensions significantly broaden the applicability of MDP-based models to complex, real-world decision-making tasks under uncertainty.

Identifying an optimal policy is crucial in complex decision-making problems. However, the exact computation of optimal policies can be challenging due to the high dimensionality and the continuous nature of the state and action spaces in many real-world problems. Various methods have been developed to obtain an approximate solution to the optimization problem and obtain approximately-optimal policies.

Exact resolution techniques involve solving the decision-making problem with exact calculations, ensuring that the derived policy is indeed optimal given the model, with the caveat that they typically require computing high-dimensional integrals for continuous state or action spaces highly combinatorial problems in the discrete ones. While exact methods are theoretically attractive, they become computationally infeasible for large-scale problems.

Approximation techniques include a range of simulation-based approaches that balance precision and computational efficiency. Monte Carlo Planning is one such category of approaches, leveraging random sampling to evaluate and improve policies. These techniques, such as Monte Carlo Tree Search (MCTS), are particularly effective in sequential decision-making problems by simulating future state-action trajectories and updating estimates based on averaged outcomes.

Reinforcement Learning (RL) offers a robust framework for approximating the value function and deriving near-optimal policies, especially in scenarios where traditional exact methods are impractical. RL focuses on learning optimal policies through interaction with the environment.

In this chapter, we begin by presenting the MDP framework in detail in section 1.1. We then explore key extensions: the POMDP framework to address partial observability in section 1.2.1, the BAMDP framework for model uncertainty in section 1.2.2, and the BAPOMDP framework in section 1.2.3, which combines both complexities. The final section focuses on solution methods, covering exact and approximate algorithms through sections 1.3.1 and 1.3.1, before introducing reinforcement learning approaches in section 1.3.3. Throughout the chapter, a simple example of a squirrel moving between hiding places is developed to illustrate the key concepts. Additionally, the chapter is accompanied by a Jupyter Notebook¹ that provides full code details and an extra example involving a simplified cancer treatment problem. This chapter is an adapted version of an exposition paper [Cle+25] and book chapter co-authored with Alice Cleynen, Benoîte de Saporta, Régis Sabbadin, and Amélie Vernay².

1.1 Markov Decision Process Formalism

In this section, we delve into the formal components of MDPs, including states, actions, constraints, transition function, cost functions and horizon, while also introducing the concept of policies, which determine the action selection process. Additionally, we outline the goals of learning by discussing various optimality criteria. Although MDPs can be extended to cover infinite action spaces, this section will focus on finite-action problems for clarity and simplicity.

1.1.1 Markov Decision Process Definition

Definition 1.1.1: Markov Decision Process

A Markov decision process is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, P, c, C, H \rangle$:

- $\mathcal{S}$ is the set of all possible states of the process.
- $\mathcal{A}$ is the set of actions that the agent can take in each state.
- $P_t : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$, denoted $P(s'|s, a)$, is the time-dependent probabilities of transitioning from one state $s \in \mathcal{S}$ to another $s' \in \mathcal{S}$ after taking a specific action $a \in \mathcal{A}$.
- $c_t : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the time-dependent cost function that assigns a numerical cost to each state-action-state pair.
- $C : \mathcal{S} \rightarrow \mathbb{R}$ is the terminal cost.
- H is the horizon of the process.

Remark 1.1 *The previous definition applies only to finite-horizon MDPs. In infinite-horizon*

¹https://github.com/orlross/phd_notebook

²PhD Student from Montpellier University, supervised by Alice Cleynen and Benoîte de Saporta

MDPs, there is no terminal cost; instead, future rewards are discounted using a discount factor. Additionally, both the transition function and the cost function are stationary, meaning they do not change over time.

Definition 1.1.2: Markov Decision Process with infinite-horizon

A Markov decision process is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, P, c, \alpha \rangle$:

- $\mathcal{S}$ and $\mathcal{A}$ remains the same as in Definition 1.1.1.
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$, denoted $P(s'|s, a)$, is the probabilities of transitioning from one state $s \in \mathcal{S}$ to another $s' \in \mathcal{S}$ after taking a specific action $a \in \mathcal{A}$.
- $c : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the cost function that assigns a numerical cost to each state-action-state pair.
- $\alpha \in (0, 1)$ is the discounted factor.

Remark 1.2 State and action spaces can be continuous, but in this chapter, we focus on finite cases. Continuous state space will be covered in Chapter 2. Continuous action space are beyond the scope of this manuscript [SB18].

Remark 1.3 Depending on the problem, using a reward function may be preferred and is more commonly used in literature. In this case, the minimization problem becomes a maximization problem.

Remark 1.4 The process can also contains actions-related constraints. In this case we define $\mathbb{K} \subset \mathcal{S} \times \mathcal{A}$ the space of admissible actions in state $s \in \mathcal{S}$, with $\mathbb{K}(s) \neq \emptyset, \forall s \in \mathcal{S}$.

Constraints on the admissible actions may also be handled by assigning a prohibitive cost ($+\infty$ for instance) to invalid state-action pairs, preventing them from being chosen.

In finite-horizon MDP, the environment is in a certain initial state $s_0 \in \mathcal{S}$. Then at time t , the agent chooses an action $a \in \mathcal{A}$, applies it to the current state s , and the process evolves to the next state s' with probability $P(s'|s, a)$. A cost $c(s, a, s')$ is computed. This process is iterated until the end: the horizon H of the task can be finite and specified or determined by a discount factor α . This process is illustrated in Figure 1.1.

Example 1.1.1: Squirrel toy model as a MDP

Over the course of a week, you observe a squirrel moving from hiding place to hiding place. Let $m \in \{1, \dots, 4\}$ be the set of hiding places. You only have one camera and each day you can decide to place it in front of a specific hiding place. The aim is to take as many photos of the squirrel as possible over the week.

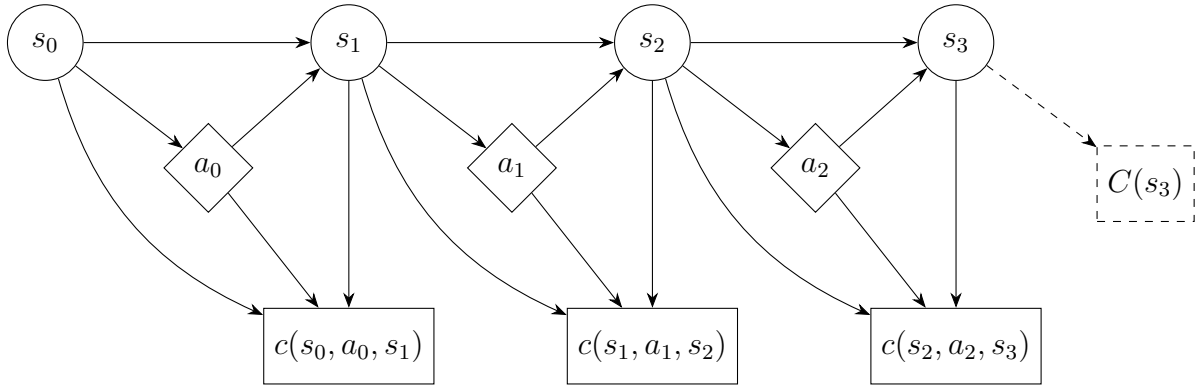


Figure 1.1: Dynamic representation of a Markov Decision Process (MDP) with finite horizon $H = 3$. Circles denote states, diamonds represent actions and rectangles indicate costs associated with state-action-state transitions. The dashed rectangle depicts the terminal cost. The arrows indicate the sequence of steps.

The MDP associated to this example is described by

$$\begin{aligned} \mathbb{S} &= \{1, \dots, 4\}, \\ \mathbb{A} &= \{1, \dots, 4\}, \\ P(s'|s = m, a) &= \sum_{m'=1}^4 p_m^{m'} \mathbb{1}_{\{s'=m'\}}, \\ c(s, a) &= \mathbb{1}_{a \neq m}, \\ H &= 7. \end{aligned}$$

Note that in this example, the transition matrix does not depend on the action, the terminal cost is $C = 0$, and there are no forbidden actions.

To control a MDP, we need to define a policy that specifies how actions are chosen at each decision date.

1.1.2 Policies

Definition 1.1.3: MDP history

Let a MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, C, H \rangle$ be given. For any $0 < t < H$, any sequence $h_t = \langle s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t \rangle$ is called t -history of the MDP. The set of length t histories is denoted $\mathcal{H}_t$. Set also $\mathcal{H}_0 = \mathbb{S}$.

Then, let us define non markovian decision rules and policies:

Definition 1.1.4: Stochastic decision rules

An admissible stochastic decision rule at time t , $\pi_t : \mathcal{H}_t \rightarrow \Delta(\mathbb{A}_t)$ assigns a probability distribution over actions to every t -history.

Interesting subclasses of decision rules are the deterministic and/or markovian decision rules:

Definition 1.1.5: Deterministic decision rule

An admissible decision rule π_t is deterministic if it assigns a unique action to every history (*i.e.* is a Dirac distribution over $\mathbb{A}$).

Definition 1.1.6: Markovian decision rule

An admissible decision rule at time t is markovian if $\pi_t(h_t) = \pi_t(s_t)$, thus $\pi_t : \mathcal{S}_t \rightarrow \Delta(\mathcal{A})$.

Definition 1.1.7: Policy

An admissible policy is a sequence of admissible decision rule: $\pi = (\pi_t)_{t \in \mathbb{N}}$. We denote by Π the set of all admissible policies.

Definition 1.1.8: Stationnary Policy

An admissible stationnary policy is a sequence of admissible decision rule: $\pi = (\pi_t)_{0 \leq t < H}$.

Definition 1.1.9: Stationary policy

An admissible policy $\pi = (\pi_t)_{0 \leq t < H}$ is stationary if all decision rules are identical for all $t \in \mathbb{N}$. In this case, the decision at time t depends only on the current state and not explicitly on time.

Example 1.1.2: Reference policies

Let's introduce three policies to control the process defined in example 1.1.1. Those three policies are Markovian.

Fixed Camera (Deterministic Policy) π_{FC} . The camera is placed in the most likely location and remains there for the duration of the process, without moving.

Random Movement (Stochastic Policy) π_{RM} . The camera moves randomly and looks at one of the four possible hiding spots.

Tracking the Squirrel (Deterministic Policy) π_{TS} . The camera follows the squirrel, always positioning itself wherever the squirrel moved in the previous time step.

1.1.3 Evaluation Criteria of a Policy

The objective in MDPs is to find a policy that minimizes a cost criterion (or equivalently maximizes a reward criterion) over the entire decision horizon. The transition cost, $c(s, a, s')$, represents the immediate benefit of taking action a in state s and transitioning to state s' , while the terminal cost, $C(s_H)$, captures the cost upon reaching the final state of the process. Beyond immediate costs, we must define a criterion that captures the entire dynamic of the process, considering both the short-term effects of an action and its long-term consequences. This is formalized as the total expected cost (summed over the planning horizon H).

The optimization problem associated with a cost criterion involves minimizing the total expected cost over the set of all admissible policies Π .

Finite-horizon optimization criteria.

Definition 1.1.10: Finite-horizon expected cost

Starting from the initial state $s \in \mathbb{S}$ and following the policy $\pi \in \Pi$ the total expected cost is the expected sum of the costs accumulated under the process distribution, up to the horizon H , plus the terminal cost associated with the final state. This can be expressed as:

$$V(\pi, s) = \mathbb{E}_s^\pi \left[\sum_{t=0}^{H-1} c_t(S_t, A_t, S_{t+1}) + C(S_H) \right].$$

Infinite-horizon optimization criteria. Let's first define the two main optimization criteria used in infinite-horizon MDPs.

Definition 1.1.11: Discounted expected cost

In the infinite-horizon case, the agent makes decisions indefinitely. Each transition cost is discounted by a factor $\gamma \in (0, 1)$, which balances the immediate costs against future outcomes. Starting from the initial state $s \in \mathbb{S}$ and following the policy $\pi \in \Pi$ the discounted expected cost is

$$V_\gamma(\pi, s) = \mathbb{E}_s^\pi \left[\sum_{t=0}^{\infty} \gamma^t c(S_t, A_t, S_{t+1}) \right].$$

For $0 < \gamma < 1$, the γ factor ensures the convergence of the series.

The average reward criterion can also be defined (see for example [Put94b]), but will not be considered in the remainder of this manuscript.

Definition 1.1.12: Average cost criterion

Starting from the initial state $s \in \mathbb{S}$ and following the policy $\pi \in \Pi$ the average cost criterion is

$$V_{av}(\pi, s) = \lim_{H \rightarrow \infty} \mathbb{E}_s^\pi \left[\frac{1}{H} \sum_{t=0}^{H-1} c(S_t, A_t, S_{t+1}) \right].$$

Optimization problem definition. The goal is to find an optimal policy π^* that minimizes (or maximizes in case of reward function) the value function V (or V_α in infinite-horizon) for all states $s \in \mathbb{S}$.

Definition 1.1.13: Value function

The optimal value function is then defined,

- in finite horizon:

$$V^*(s) = \min_{\pi \in \Pi} V(\pi, s), \forall s \in \mathbb{S},$$

- in infinite horizon:

$$V_\gamma^*(s) = \min_{\pi \in \Pi} V_\gamma(\pi, s), \forall s \in \mathbb{S}.$$

Theorem 1.1.1 (Bellman equation)

- in finite horizon: the optimal value function V^* is the unique solution to the following linear system of Bellman equations in variables $\{V_t(s)\}_{0 \leq t \leq H, s \in \mathbb{S}}$

$$V_t(s) = \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + V_{t+1}(s')), \quad \forall t \in \{0, \dots, H-1\}, \forall s \in \mathbb{S}$$

where $V_H(s) = C(s)$ defines the terminal values and $V^* = V_0$. Any policy $\pi^* = (\pi_t^*)_{t=0:H-1}$ such that

$$\pi_t^*(s) \in \arg \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + V_{t+1}^*(s')), \quad \forall s \in \mathbb{S},$$

is optimal.

- in infinite horizon: the optimal value function V_γ^* is the unique solution of a system of non-linear equations in variables $\{V_\gamma(s)\}_{s \in \mathbb{S}}$, called Bellman equations

$$V_\gamma(s) = \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + \gamma V_\gamma(s')), \quad \forall s \in \mathbb{S}.$$

Any policy such that

$$\pi^*(s) \in \arg \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + \gamma V_\gamma^*(s')), \quad \forall s \in \mathbb{S},$$

is optimal.

In the finite horizon model, the optimal policy is typically non-stationary. In the infinite horizon discounted model, it has been shown that there exists a stationary policy π^* that is optimal for every starting state [BH61].

Example 1.1.3: Evaluation of reference policies

Let's evaluate the policies defined in Example 1.1.2 on the MDP defined in Example 1.1.1. For this example we consider a transition matrix that does not rely on decision

$$P(s'|s = m) = \begin{bmatrix} 0.4 & 0.1 & 0.3 & 0.2 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.1 & 0.2 & 0.3 & 0.4 \\ 0.3 & 0.2 & 0.1 & 0.4 \end{bmatrix}$$

The cost of each policy, defined in Example 1.1.2, is computed exactly and summarized in the following table.

Policy	Expected Cost
π_{FC}	4.97
π_{RM}	6.37
π_{TS}	0.00

The fixed camera policy π_{FC} maintains a static position, which results in a relatively high cost since it cannot adapt to the squirrel's movements. As a result, it often fails to capture the target effectively.

The random movement policy π_{RM} performs worse than the fixed camera, with an even higher cost. Its random movements do not effectively track the squirrel, leading to inefficient positioning and frequent failures to capture the target.

The tracking policy π_{TS} is the best among the three, as it achieves the lowest possible cost. The camera is always positioned exactly where the squirrel is, ensuring perfect tracking as shown in Figure 1.2.

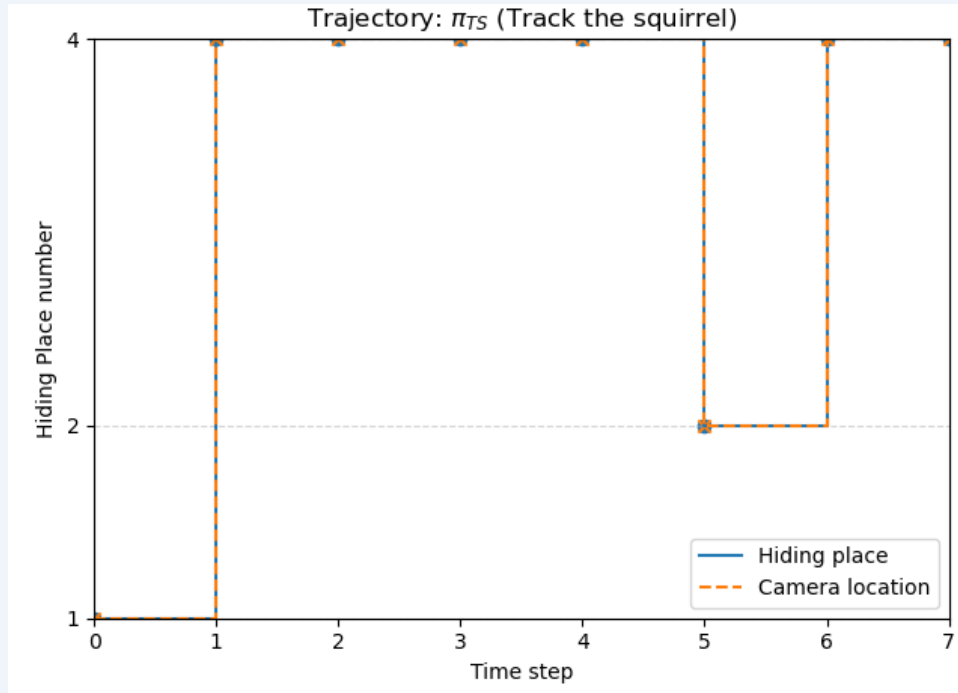


Figure 1.2: An example of trajectory under policy π_{TS} .

Given its zero cost, this is an optimal policy. This optimality will be further demonstrated in Example 1.3.1.

We describe in Section 1.3 a few classical algorithms for computing optimal policies in finite and infinite horizon. Among these algorithms, classical reinforcement-learning algorithms (*e.g.* Q -learning introduced in [WD92a]) make use of a notion related to the optimal-value function of an MDP: the Q -function.

In the finite-horizon discounted case, the Q function is defined as follows.

Definition 1.1.14: Q -function for the total cost criterion

For any $0 \leq t < H$, the Q -function at time t represents the expected cost of selecting action $a \in \mathbb{A}$ in the current state $s \in \mathbb{S}$ and then following policy π up to horizon H .

$$\begin{aligned} Q_t^\pi(s, a) &= \mathbb{E}_s^\pi \left[c(s, a, S_1) + \sum_{n=t+1}^{H-1} c_n(S_n, A_n, S_{n+1}) + C(S_H) \right] \\ &= \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + V_{t+1}(\pi, s')). \end{aligned}$$

Then, if $\{V_t^*\}_{t=0..H}$ form an optimal value function, the optimal Q -function is:

$$Q_t^*(s, a) = \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + V_{t+1}^*(s')), \quad \forall (s, a) \in \mathbb{S} \times \mathbb{A}.$$

Definition 1.1.15: Q -function for the discounted criterion

The Q -function, for the infinite-horizon discounted criterion, represents the expected cost of selecting action $a \in \mathbb{A}$ in the current state $s \in \mathbb{S}$ and then following the policy π forever.

$$\begin{aligned} Q_\gamma^\pi(s, a) &= \mathbb{E}_s^\pi \left[c(s, a, S_1) + \sum_{t=1}^{\infty} \gamma^t c(S_t, A_t, S_{t+1}) \right] \\ &= \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + \gamma V_\gamma(\pi, s')). \end{aligned}$$

Then, if V_γ^* is the optimal value function of the MDP for the discounted criterion, Q_γ^* , the optimal Q -function of the MDP is defined as:

$$Q_\gamma^*(s, a) = \sum_{s' \in \mathbb{S}} P(s'|s, a) (c(s, a, s') + \gamma V_\gamma^*(s')), \quad \forall (s, a) \in \mathbb{S} \times \mathbb{A}.$$

1.2 Markov Decision Process Extensions

MDPs can be extended to model more complex real-world situations. In this section, we focus on two major directions of extension: handling partial observability and managing model uncertainty. To address situations where the agent cannot fully observe the environment's state, we introduce POMDPs. On the other hand, when there is uncertainty about the environment's dynamics, BAMDPs provide a principled way to model and solve decision-making problems under unknown parameters. Combining both challenges leads to the more general BAPOMDP

framework.

1.2.1 Partially Observable MDP

POMDPs. This section is based on the POMDPs definition proposed by [KLC98], which focuses on practical algorithms for computing optimal strategies in settings with finite state, observation and action spaces.

Definition 1.2.1: Partially Observable Markov Decision Process

A finite-horizon POMDP model is a tuple $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$. In addition to the components of a standard MDP introduced in Definition 1.1.1, POMDPs introduce

- a finite set Ω representing the possible observations of the system;
- an observation function

$$O : \mathbb{S} \times \mathbb{A} \times \Omega \rightarrow [0, 1],$$

where $O(s', a, \omega)$ gives the probability of observing $\omega \in \Omega$, after taking action a , resulting in successor state s' ;

- and an initial observation ω_0 , describing the initial imperfect knowledge we have about the initial state.

Remark 1.5 *The process can also contains actions-related constrains. In this case we define $\mathbb{K} \subset \Omega \times \mathbb{A}$ the space of admissible actions in state $\omega \in \Omega$, with $\mathbb{K}(\omega) \neq \emptyset, \forall \omega \in \Omega$.*

In a POMDP, the objective is also to compute a strategy which minimizes the expected sum of future costs. However, a key difference is that, at each time step t , the true state of the system s_t is not directly observed. Thus POMDP histories only include actions and observations $h_t = \langle \omega_0, a_0, \omega_1, \dots, a_{t-1}, \omega_t \rangle$. This process is illustrated in Figure 1.3.

Similarly to solving an MDP, solving a POMDP consists of computing a policy assigning an action to every possible state of current knowledge of the POMDP.

Definition 1.2.2: Stochastic history-dependent POMDP decision rule

Consider a (finite-horizon) POMDP given by $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$. An admissible POMDP decision rule $\pi_t : \mathcal{H}_t \rightarrow \Delta(\mathbb{A})$ assigns a probability distribution over actions in $\mathbb{A}$ to every POMDP history $h_t \in \mathcal{H}_t$.

Definition 1.2.3: POMDP policies

Consider a (finite-horizon) POMDP given by $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$. An admissible POMDP history-dependent policy is a list of admissible history-dependent decision rules $\pi = (\pi_t)_{0 \leq t < H}$. Denote by Π^o the set of all history-dependent policies.

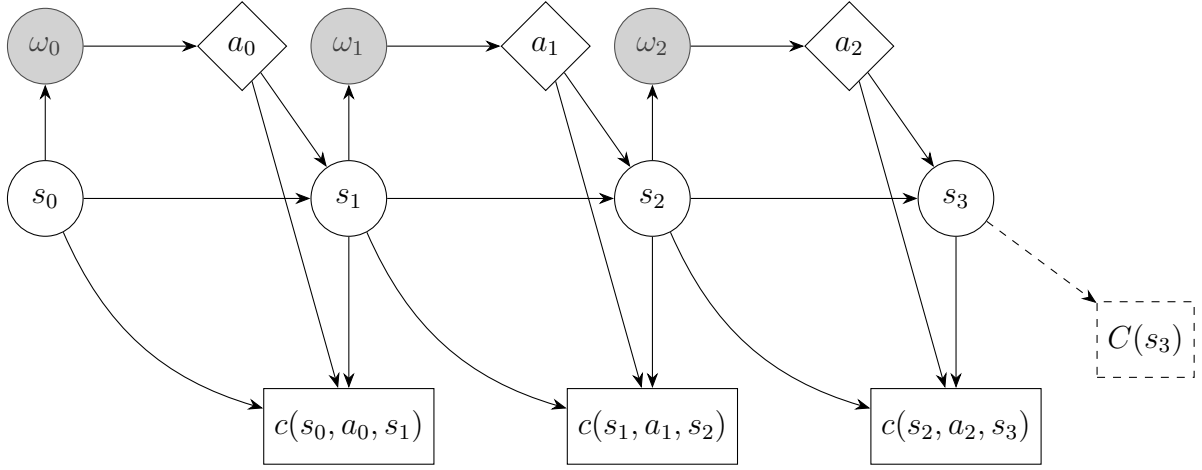


Figure 1.3: **Graphical representation of a Partially Observable Markov Decision Process (POMDP) with finite horizon $H = 3$.** Circles denote states, diamonds represent actions and rectangles indicate costs associated with state-action-state transitions. The dashed rectangle depicts the terminal cost. The arrows indicate the sequence of steps.

A POMDP $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$ can also be seen as an MDP $\langle \bar{\mathbb{S}}, \mathbb{A}, \bar{P}, c, C, H \rangle$ by setting

$$\begin{aligned} \bar{\mathbb{S}} &= \mathbb{S} \times \Omega, \\ \bar{P}(\bar{s}' = (s', \omega') | \bar{s} = (s, \omega), a) &= P(s' | s, a) O(s', a, \omega'), \end{aligned}$$

for all $\bar{s}' = (s', \omega')$ and $\bar{s} = (s, \omega)$ in $\bar{\mathbb{S}}$. In this setting the term partially observed is more obvious. The main difference between this MPD and the POMDP setting is that in the POMDP, only policies in Π^o are considered, and the initial value S_0 is randomly drawn from an initial belief b_0 which is a probability distribution over $\mathbb{S}$: $b_0(s) = \mathbb{P}(S_0 = s | b_0)$ for all $s \in \mathbb{S}$. This setting allows to see Π^o as a subset of Π for the MDP $\langle \bar{\mathbb{S}}, \mathbb{A}, \mathbb{K}, \bar{P}, c, C, H \rangle$, and to directly use definitions 1.1.3 to 1.1.3 to define the cost for a POMDP policy. For criterion V (or V_α in infinite horizon), one has

$$V(\pi, b_0) = \sum_{s \in \mathbb{S}} b_0(s) V(\pi, s).$$

Definition 1.2.4: Optimal policy

Consider the POMDP $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$.

- in finite-horizon: An optimal policy π^* of the POMDP, with respect to criterion $V(\pi, h)$ given in definition 1.1.3 is an admissible policy $\pi^* \in \Pi^o$ such that:

$$V(\pi^*, h_t) \leq V(\pi, h_t), \forall \pi \in \Pi^o, \forall h_t \in \mathcal{H}_t, 0 \leq t \leq H.$$

- in infinite-horizon: An optimal policy π^* of the POMDP, with respect to criterion

$V_\alpha(\pi, h)$ given in definition 1.1.3 is an admissible policy $\pi^* \in \Pi^o$ such that:

$$V(\pi^*, h) \leq V(\pi, h), \forall \pi \in \Pi^o, \forall h \in \mathcal{H}.$$

Belief MDP equivalence. In order to evaluate a POMDP policy and compute an optimal policy, it is useful to notice that a POMDP can be modeled as another particular form of MDP over belief states. The knowledge incorporated in the history h_t can be encapsulated in a belief state, b_t which is a probability distribution over $\mathbb{S}$ and represents an estimation of the unobserved current state. This belief state evolves from the initial belief b_0 , based on the POMDP history $h_t \in \mathcal{H}_t$. When h_t is completed with a new pair of action-observation (a_t, ω_{t+1}) , the belief state b_t is updated as follows. Considering that in a given belief state b_t , an admissible action a_t is applied and leads to a new observation ω_{t+1} then

$$b_{t+1} = \tau(b_t, a_t, \omega_{t+1}).$$

where τ is a deterministic belief transition function. This function is computed through Bayesian updating:

$$\begin{aligned} b_{t+1}(s') &= \mathbb{P}(S_{t+1} = s' | b_t, a_t, \omega_{t+1}) \\ &= \frac{O(s', a_t, \omega_{t+1}) \times \mathbb{P}(S_{t+1} = s' | b_t, a_t)}{\sum_{s'' \in \mathbb{S}} O(s'', a_t, \omega_{t+1}) \times \mathbb{P}(S_{t+1} = s'' | b_t, a_t)} \\ &= \frac{O(s', a_t, \omega_{t+1}) \times \sum_{s \in \mathbb{S}} b_t(s) P(s' | s, a_t)}{\sum_{s'' \in \mathbb{S}} O(s'', a_t, \omega_{t+1}) \times \sum_{s \in \mathbb{S}} b_t(s) P(s'' | s, a_t)} \\ &\propto O(s', a_t, \omega_{t+1}) \times \sum_{s \in \mathbb{S}} b_t(s) P(s' | s, a_t). \end{aligned}$$

A way to deal with partial observability in POMDP is to consider a corresponding belief MDP, which (continuous, multi-dimensional) state space is made of the set of possible belief states of the MDP. The correspondence between POMDP and belief MDP is made explicit in the following definition.

Definition 1.2.5: Belief MDP

A finite-horizon POMDP $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$ can be viewed as a belief MDP, that is an MDP $\langle \mathcal{B}, \mathbb{A}, \mathcal{P}, H, \rho, \rho_H \rangle$ on belief states, where:

- $\mathcal{B} \subset \Delta(\mathbb{S})$ is the continuous set of belief states,
- $\mathbb{A}$ is the action space, unchanged,
- $\mathcal{P}$ is the transition function

$$\mathcal{P}(b_{t+1} | b_t, a_t) = \sum_{\omega' \in \Omega} \sum_{s' \in \mathbb{S}} \mathbb{1}_{b_{t+1} = \tau(b_t, a_t, \omega')} O(s', a_t, \omega') \sum_{s \in \mathbb{S}} b_t(s) P(s' | s, a_t).$$

- $\rho(b, a, b')$ is the cost associated to action a applied in belief state b resulting to b'

$$\rho(b, a, b') = \sum_{s \in \mathbb{S}} b(s) \sum_{s' \in \mathbb{S}} b'(s') c(s, a, s').$$

In essence, ρ corresponds to the expectation of the running cost over the states following distributions given by the beliefs.

- $\rho_H(b)$ is the terminal cost in belief state b

$$\rho_H(b) = \sum_{s \in \mathbb{S}} b(s) C(s).$$

The value of a history-dependent (stochastic or deterministic) policy can now be defined as follows.

Definition 1.2.6: Belief MDP policy value

Consider a finite-horizon POMDP $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, H, \omega_0 \rangle$ and the corresponding belief MDP $\langle \mathcal{B}, \mathbb{A}, \mathcal{P}, H, \rho, \rho_H \rangle$, as well as a Markovian policy π for the belief MDP. The value of π is defined as

$$V(\pi, b_0) = E_{b_0}^{\pi} \left[\sum_{t=0}^{H-1} \rho(b_t, \pi_t(b_t), b_{t+1}) + \rho_H(b_H) \right],$$

where

$$\rho(b_t, \pi_t(b_t), b_{t+1}) = \sum_{a \in \mathbb{A}} \rho(b_t, a, b_{t+1}) \pi_t(b_t)(a).$$

Proposition 1.2.1 *For any POMDP given by $\langle \mathbb{S}, \mathbb{A}, P, \Omega, O, c, C, \omega_0 \rangle$, there always exists an optimal policy that is history-dependant in the space of observations. Moreover, in the finite-horizon case, the corresponding optimal value function is piecewise linear and convex over the continuous belief space.*

Proof. The proof details are available in [Son73]. □

Proposition 1.2.2 *The belief MDP policy value and POMDP policy value are equal. In the case of finite horizon problems, optimal policies are history-dependant and deterministic in general for the POMDP version, whereas markovian and deterministic for the belief MDP.*

Proof. This equality was established in [Åst65]. In finite horizon problems, the corresponding belief MDP optimal policy is Markovian with respect to the belief state and deterministic as well, as detailed in [KLC98]. □

Example 1.2.1: Squirrel toy model as a POMDP

Consider the same setting as in Section 1.1.1 with the squirrel traveling to different hiding places, but now consider you do not fully observe the trajectory of the squirrel. You only observe the presence of the squirrel when the location of the camera corresponds to the squirrel's location. Even then, the time spent by the squirrel in its hiding place remains hidden to you. Denote by $\omega_t = \mathbb{1}_{a_t=m_t}$ the observation at time t , *i.e.* 1 if the squirrel is in the same place as your camera, 0 otherwise. The objective remains the same, *i.e.* maximizing the number of pictures of the squirrel taken over the span of the week.

$$\Omega = \{0, 1\},$$

$$O(s' = m', a, \omega) = \mathbb{1}_{\{a=m'\}}\mathbb{1}_{\{\omega=1\}} + \mathbb{1}_{\{a \neq m'\}}\mathbb{1}_{\{\omega=0\}}.$$

We assume that the camera has just observed the arrival of the squirrel in site $m_0 \in \{1, \dots, 4\}$ and $\omega_0 = 0$.

Let's introduce three policies to control this POMDP.

Random Movement (Stochastic Markov policy) π_{RM} . The camera moves randomly and looks at one of the four possible hiding spots.

Last Seen (Stochastic history-dependant policy) π_{LS} . Positions the camera at the squirrel's last known location; otherwise, it places the camera randomly among possible hiding spots. This approach relies on memory of past observations rather than current state alone.

Max Belief (Deterministic policy) π_{MB} . Move the camera to the site where the belief of squirrel presence is the highest.

The exact cost is summarized in the following table for $\omega_0 = 2$.

Policy	Expected Cost
π_{RM}	6.18
π_{LS}	5.81
π_{MB}	5.48

The Random Movement policy π_{RM} performs the worst with the highest expected cost. Introducing history-dependence with the Last Seen policy π_{LS} reduces the expected cost with 5.81, showing the benefit of even simple memory-based strategies. Finally, the Max Belief policy π_{MB} , which fully exploits the belief state, achieves the best expected cost of 5.48. In Figure 1.4, the belief state evolution under Max Belief policy is presented.

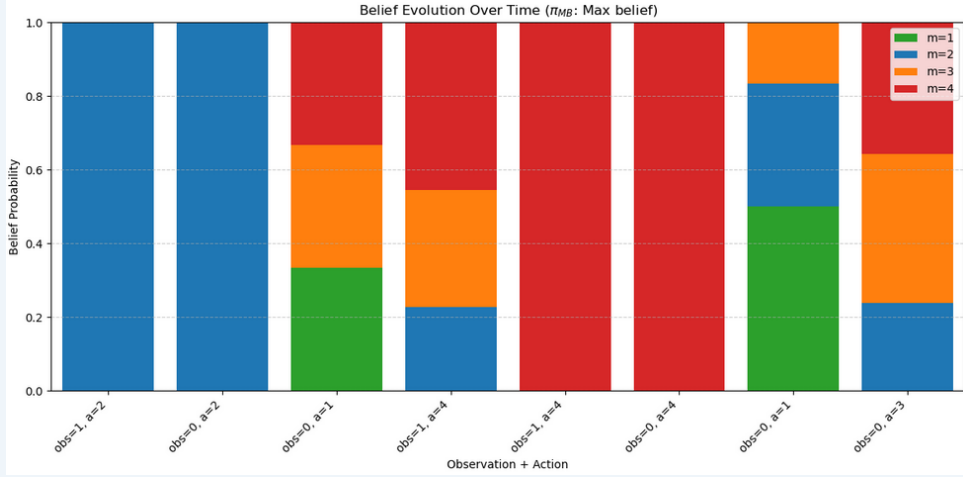


Figure 1.4: An example of belief evolution along a trajectory under policy π_{MB} .

An approximate optimal policy will be computed in Example 1.3.2.

1.2.2 Bayes Adaptive MDP

In this Section we follow the didactic exposition of BAMDP and most of the notations of [Ros+11]. An archetypal BAMDP problem is when we face a MDP $\langle \mathcal{S}, \mathcal{A}, P, c, C, H \rangle$, where $\mathcal{S}$ and $\mathcal{A}$ are finite, the model is initially known, except for the transition probabilities $P(\cdot|s, a)$, $\forall (s, a) \in \mathcal{S} \times \mathcal{A}$. This become challenging for computing optimal policies as the Bellman equations 1.1.1 require knowledge of $P(\cdot|s, a)$, $\forall (s, a) \in \mathcal{S} \times \mathcal{A}$ to assess the impact of a decision on the future states of the process.

BAMDPs address this challenge by embedding the MDP in a Bayesian framework, where we assume that we have a prior distribution over the distribution $P(\cdot|s, a)$, which are assumed independent for all (s, a) . As part of the acquisition of the new state s' , the agent updates the posterior distribution with the information provided by the transition (s, a, s') when it is encountered. Let $\mathcal{S} = \{s^1, \dots, s^n\}$ be the finite state-space and let us define $p^{s,a} = (p_1^{s,a}, \dots, p_n^{s,a})$, where $p_i^{s,a} = P(s^i|s, a)$.

In the Bayesian context, if $p^{s,a}$ is the parameter vector of a multinomial distribution, the conjugate of a multinomial distribution is a Dirichlet distribution, $Dirichlet(\theta)$, parameterized by a count vector $\theta = (\theta_1, \dots, \theta_n) \in \mathbb{N}^n$. The multinomial/Dirichlet framework makes it easy to perform Bayesian updates of the knowledge over p when transitions are experienced. Indeed, assume that from a state s and action a , we observe K transitions from the true distribution $p^{s,a}$, and let $k = (k_1, \dots, k_n)$ represent the number of transitions to $s^1, \dots, s^n$ respectively (with $K = \sum_i k_i$). Then, a Bayesian update of a $Dirichlet(\theta_1, \dots, \theta_n)$ prior on p leads to the posterior distribution over p' , equal to:

$$p' = (p'_1, \dots, p'_n) \sim Dirichlet(\theta_1 + k_1, \dots, \theta_n + k_n). \quad (1.1)$$

The fundamental principle of Bayes-Adaptive MDP is to incorporate model exploration into planning, by considering an augmented MDP, which states $\tilde{\mathbb{S}} = (s, \theta)$ include the hyperparameters θ which can be considered as a memory of past experiences (a sufficient statistics). As in classical MDP, solving a BAMDP requires identifying the optimal decision for all possible states and hyperparameter values. Formally, a BAMDP is defined as:

Definition 1.2.7: BAMDP

A Bayes-adaptive MDP $\langle \tilde{\mathbb{S}}, \tilde{\mathbb{A}}, \tilde{P}, \tilde{c}, \tilde{C}, H \rangle$ is an MDP that can be defined from a MDP $\langle \mathbb{S}, \mathbb{A}, P, c, C, H \rangle$ with ill-known transition matrix P as:

- $\tilde{\mathbb{S}} = \mathbb{S} \times \Theta$, where $\Theta = \{(\theta_{s'}^{s,a})_{s,s' \in \mathbb{S}, a \in \mathbb{A}}\} \subseteq \mathbb{N}^{|\mathbb{S}|^2 \times |\mathbb{A}|}$. States $\tilde{\mathbb{S}} = (s, \theta)$ are usually called hyperstates. Usually, also, an initial belief over P is modeled by specifying an initial set of counts $\{\hat{\theta}_{s'}^{s,a}\}$. This initial set of counts will be assumed to verify $\sum_{s' \in \mathbb{S}} \hat{\theta}_{s'}^{s,a} > 0, \forall s, a$, in order for the initial Dirichlet distributions to be well-defined.
- $\tilde{\mathbb{A}} = \mathbb{A}$. The action space is unchanged.
- $\tilde{P}$ is defined as follows. $\tilde{P}(s', \theta' | s, a, \theta)$ is the probability of transitioning from hyperstate (s, θ) to hyperstate (s', θ') under action a . This writes, by the chain rule:

$$\tilde{P}(s', \theta' | s, a, \theta) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a, \theta_t = \theta) \quad (1.2)$$

$$\times \mathbb{P}(\theta_{t+1} = \theta' | S_t = s, A_t = a, S_{t+1} = s', \theta_t = \theta) \quad (1.3)$$

- $\tilde{c}(s, a, s', \theta, \theta') = c(s, a, s')$, since we assume that the cost function is known.
- $\tilde{C}(s) = C(s)$, since we assume that the terminal cost function is known.

The BAMDP $\langle \tilde{\mathbb{S}}, \tilde{\mathbb{A}}, \tilde{P}, \tilde{c}, \tilde{C}, H \rangle$ is a classical MDP. When H is finite, the size of Θ is bounded by $H^{|\mathbb{S}|^2 |\mathbb{A}|}$ and the size of $\tilde{\mathbb{S}}$ by $|\mathbb{S}| \times H^{|\mathbb{S}|^2 |\mathbb{A}|}$. Note that, in a BAMDP, policies $\pi : \tilde{\mathbb{S}} \rightarrow \mathbb{A}$ choose actions as functions of the current state, s and of the current knowledge about the unknown MDP transition function, summarized by the parameter vector θ .

Let us make clear that the Bayes-Adaptive framework handles parameters' uncertainty in two phases:

- The first phase is completely off-line and does not require to observe any transition of the MDP. In this phase, we use the prior knowledge about transitions and the Multinomial/Dirichlet conjugacy in order to build a BAMDP with augmented state space $\tilde{\mathbb{S}} = \mathbb{S} \times \Theta$, as shown in Figure 1.5 and in Equation 1.2. This MDP can be solved using any MDP solution algorithm (such as those that will be mentioned in the following Sections).
- In the second phase, the decision-maker applies the optimal BAMDP policy $\tilde{\pi} : \mathbb{S} \times \Theta \rightarrow \mathbb{A}$ provided by the first phase to the real-life problem, in which only transitions (s, a, s') drawn from the original unknown matrices P are observed as shown in Figure 1.6. Re-

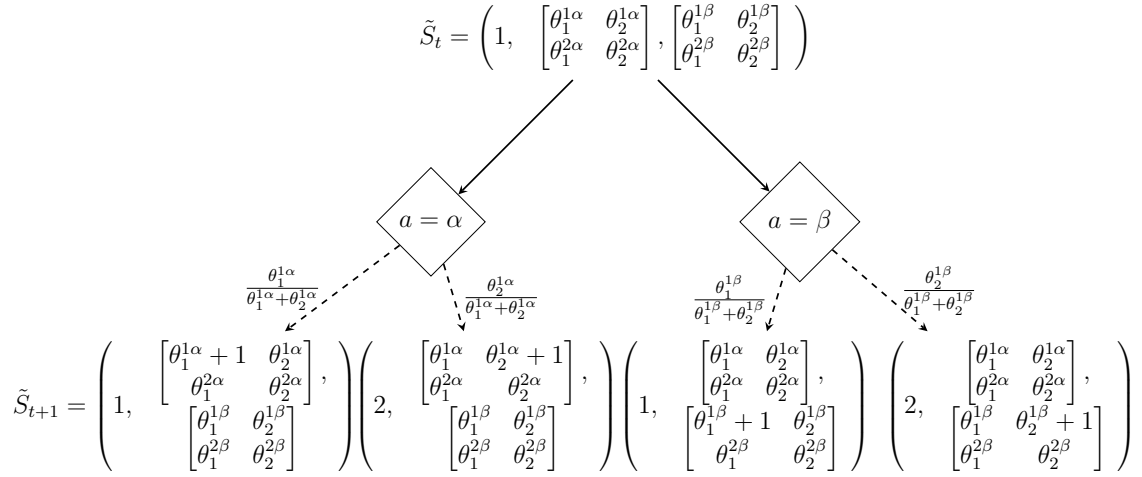


Figure 1.5: **Evolution of the state space of a BAMDP over a single time step.** This figure illustrates a specific example with two possible states $\mathbb{S} = \{1, 2\}$ and two possible actions $\mathbb{A} = \{\alpha, \beta\}$. Diamonds represent actions. The parameters $\theta_{s',a}^{s,a}$ correspond to the transition counts for each (s, a, s') triplet. The dashed arrows indicate transition probability to hyper-states, defined according to the count matrix and the Multinomial/Dirichlet conjugacy. Here, only the case of $s_t = 1$ is shown of the Figure, but the case of $s_t = 2$ is similar.

trieving the hyperparameter values needed to select optimal actions is straightforward since θ and θ' are transition counts and the transition $(s, a, s', \theta) \rightarrow \theta'$ is actually deterministic, as seen in Equation 1.3.

Remark 1.6 *BAMDP can be extended to the case where the cost function is also unknown, but we will leave this case aside. Readers may refer to [Duf02] for an introduction.*

Example 1.2.2: Squirrel toy model as a BAMDP

Going back to the MDP squirrel example presented in Section 1.1.1, we now suppose that the $p_m^{m'}$ are unknown for all $s, s' \in \mathbb{S}$.

To model the uncertainty on these transitions, we use the standard Multinomial/Dirichlet approach

$$(p_m^1, p_m^2, p_m^3, p_m^4) \sim \text{Dirichlet}(\theta_m^1, \theta_m^2, \theta_m^3, \theta_m^4).$$

With each new observation, we update the *a posteriori* distribution of the parameters of the transition matrix $\theta' = \theta + \Delta$, where $\Delta = e_m^{m'}$ is the matrix in $\mathbb{N}^4 \times \mathbb{N}^4$ that has a value of 1 at the position corresponding to the observed mode m and new mode m' , while other entries are 0. Let us illustrate the update process with an example. Suppose that at time t , the squirrel is in hiding place $s = 1$. At this stage, the hyperparameter vector θ is, for instance,

$$\theta = \begin{bmatrix} 2 & 7 & 6 & 3 \\ 3 & 4 & 1 & 2 \\ 1 & 5 & 8 & 1 \\ 3 & 2 & 6 & 3 \end{bmatrix}$$

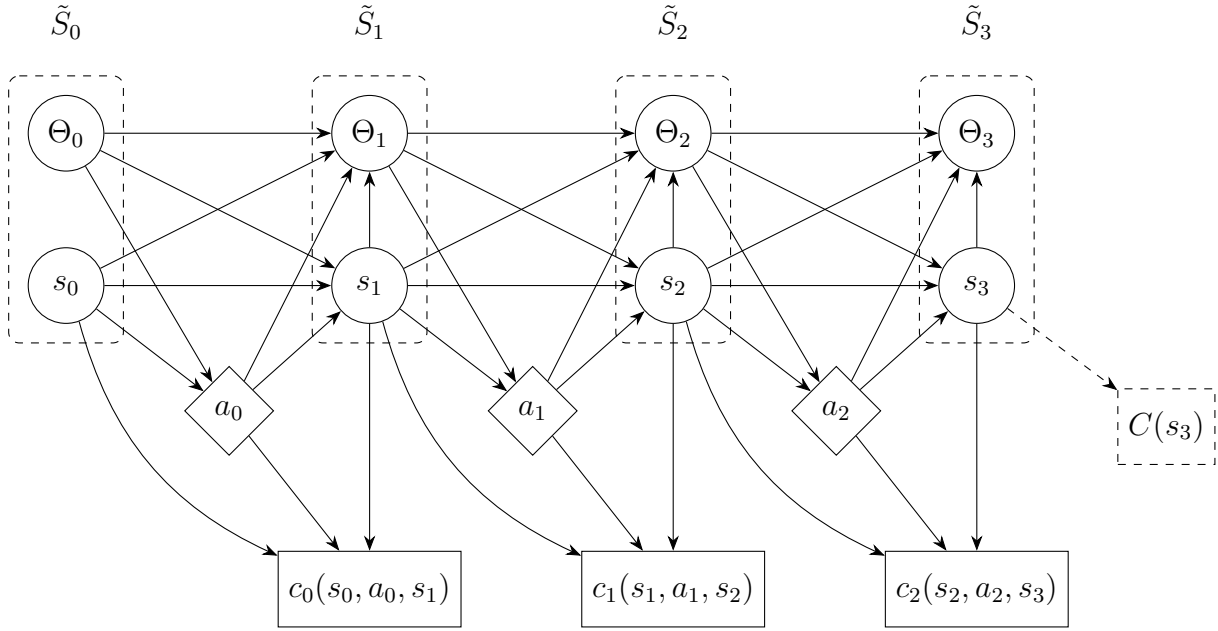


Figure 1.6: **Graphical representation of BAMDP with finite horizon $H = 3$.** Circles denote states and hyperparameters, dashed rounded rectangles denote hyperstates, diamonds represent actions and rectangles indicate costs associated with state-action-state transitions. The dashed rectangle depicts the terminal cost.

At the next step, the squirrel transitions into the hiding place $s' = 3$ while the camera is located at $a = 2$. This observation corresponds to the state-action-state sequence $s = 1, a = 2, s' = 3$. In this case, the parameter θ_1^3 of the Dirichlet distribution is updated. The hyperparameter matrix θ is updated as follows

$$\theta' = \begin{bmatrix} 2 & 7 & 6 & 3 \\ 3 & 4 & 1 & 2 \\ 1 & 5 & 8 & 1 \\ 3 & 2 & 6 & 3 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 7 & 7 & 3 \\ 3 & 4 & 1 & 2 \\ 1 & 5 & 8 & 1 \\ 3 & 2 & 6 & 3 \end{bmatrix}$$

The BAMDP associated with this example builds on the MDP from Section 1.1.1. It is defined by adding

$$\begin{aligned} \tilde{\mathcal{S}} &= \mathcal{S} \times \{0, \dots, H\}^{16}, \\ \tilde{\mathcal{A}} &= \mathcal{A}, \\ \tilde{P} &= \frac{\theta_{m'}^0}{\sum_{m=1}^4 \theta_m^1 + \theta_m^2 + \theta_m^3 + \theta_m^4} \mathbb{1}_{\theta' = \theta + e_m^{m'}}, \\ \tilde{c} &= c. \end{aligned}$$

Resolution of the BAMDP is provided in the accompanying notebook with algorithms presented in Section 1.3.3.

1.2.3 Bayes Adaptive Partially Observable MDP

In [Ros+11], the author proposed an extension of BAMDP to the partially-observed case. In this Bayes-Adaptive POMDP model (BAPOMDP), the state of the process is hidden and the only available information is given through observations $\omega \in \Omega$. In addition, the observation model is parameterized, similarly to the transition model, and the values of the parameters are updated through interactions with the system.

Definition 1.2.8: BAPOMDP

A BAPOMDP is an extension of a BAMDP given by $\langle \bar{\mathbb{S}}, \bar{\mathbb{A}}, \bar{P}, \Omega, O, c, C, H, \bar{b}_0 \rangle$, where

- $\bar{\mathbb{S}} = \tilde{\mathbb{S}} \times \Psi = S \times \Theta \times \Psi$, where

$$\Psi = \left\{ \left(\psi_{s',\omega}^a \right)_{a \in \bar{\mathbb{A}}, s' \in \bar{\mathbb{S}}, \omega \in \Omega} \right\} \subset \mathbb{N}^{|\bar{\mathbb{S}}| \times |\bar{\mathbb{A}}| \times |\Omega|}$$

is an observation count function. The coordinate $\psi_{s',\omega}^a$ counts the number of times ω has been obtained when action a has led to s' . Note that it is not directly observed since s' is not observed.

- $\bar{P}$ is defined using Bayes rule and independence properties as $\bar{P}(s', \theta', \psi', \omega | s, \theta, \psi, a)$ is defined as:

$$\begin{aligned} & \bar{P}(s', \theta', \psi', \omega | s, \theta, \psi, a) \\ &= \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a, \theta_t = \theta) \\ & \quad \times \mathbb{P}(\theta_{t+1} = \theta' | S_t = s, A_t = a, S_{t+1} = s', \theta_t = \theta) \\ & \quad \times \mathbb{P}(\psi_{t+1} = \psi' | \psi_t = \psi, A_t = a, S_{t+1} = s', \omega_{t+1} = \omega) \\ & \quad \times \mathbb{P}(\omega_{t+1} = \omega | A_t = a, S_{t+1} = s', \psi_t = \psi) \\ &= \tilde{P}(s', \theta' | s, a, \theta) \times \mathbb{P}(\psi_{t+1} = \psi' | \psi_t = \psi, A_t = a, S_{t+1} = s', \omega_{t+1} = \omega) \\ & \quad \times \mathbb{P}(\omega_{t+1} = \omega | A_t = a, S_{t+1} = s', \psi_t = \psi). \end{aligned}$$

As for the Dirichlet state transition probability, the observation probability is defined as

$$\mathbb{P}(\omega_{t+1} = \omega | A_t = a, S_{t+1} = s', \psi_t = \psi) = \frac{\psi_{s',\omega}^a}{\sum_{\omega' \in \Omega} \psi_{s',\omega'}^a},$$

and the transition to ψ' is also deterministic

$$\mathbb{P}(\psi_{t+1} = \psi' | \psi_t = \psi, A_t = a, S_{t+1} = s', \omega_{t+1} = \omega) = \begin{cases} 1 & \text{if } \psi_{s',\omega}^a = \psi_{s',\omega}^a + 1 \text{ and} \\ & \psi_{\hat{s},\hat{\omega}}^{\hat{a}} = \psi_{\hat{s},\hat{\omega}}^{\hat{a}} \\ & \forall (\hat{a}, \hat{s}, \hat{\omega}) \neq (a, s', \omega), \\ 0 & \text{otherwise.} \end{cases}$$

- Finally, the initial belief state is defined over $\bar{S}$ as : $\bar{b}_0(s, \theta_0, \psi_0) = b_0(s)$ and $\bar{b}_0(s, \theta, \psi) = 0$ if $(\theta, \psi) \neq (\theta_0, \psi_0)$. The initial counts (θ_0, ψ_0) are assumed to be known.

As for BAMDPs, the BAPOMDP model $\langle \bar{S}, A, \bar{P}, \Omega, O, c, C, H, \bar{b}_0 \rangle$ is a new POMDP over an extended finite state space (provided that H is finite). In [Ros+11] the authors provide exact (for very small problems) and approximate solution algorithms, for solving BAPOMDP, see Section 1.3.

Example 1.2.3: Squirrel toy model as a BAPOMDP

Going back to the BAMDP example presented in example 1.2.2, and POMDP example 1.2.1. we now assume that there is some uncertainty in observing the squirrel. That is, there is only a probability of correctly observing whether the squirrel is at a given location. The new observation function associated with this example is:

$$O(s' = (m'), a, \omega) = \sum_{i=0}^1 \sum_{\omega=0}^1 p_i^\omega \mathbb{1}_{\{a=m'\}},$$

where i is an indicator the observation correctness. We assume $p_i^\omega = (p_0^0, p_0^1, p_1^0, p_1^1)$ is unknown. The parameters of the observation model are given by

$$p_i^\omega \sim \text{Dirichlet}(\psi_0^0, \psi_0^1, \psi_1^0, \psi_1^1),$$

with $(\psi_i^\omega)_{0:1}^{0:1} \in \Psi \subset \mathbb{N}^4$.

The update of the parameters for ψ is similar to that of the parameters for θ presented in example 1.2.2. With each new observation (s', a, ω) , we update the posterior distribution of the parameters of the transition matrix, so that $\psi' = \psi + \Delta$ is a counting matrix, where $\Delta = e_i^j$ is the matrix in $\mathbb{N}^2 \times \mathbb{N}^2$ that has a value of 1 at the position corresponding to $i = \mathbb{1}a \neq m'$, $j = \omega$ and 0 elsewhere, for $s' = (m')$ and ω .

This BAPOMDP is therefore equivalent to a POMDP on a larger state-space. As in the classic POMDP formalism, this uncertainty can be represented using regular POMDP formalism, by maintaining (and updating) a belief over these count vectors.

1.3 Resolution Methods for MDPs, POMDPs and Bayes Adaptive Extensions

The exact computation of optimal policies is often challenging due to the high dimensionality and continuous nature of many real-world problems, that go beyond usual finite MDPs. To address these challenges, various methods have been developed to approximate optimal policies. This section explores these methods, beginning with exact solution techniques and transitioning

to advanced approximation methods, such as reinforcement learning (RL).

Exact solution techniques involve targeting the optimal policy with convergence guaranties, often through smart exploration of the whole set of admissible policies. While these methods are theoretically appealing, they quickly become computationally infeasible for large-scale problems due to the curse of dimensionality [Bel61]. To overcome these limitations, approximation techniques provide a practical alternative. Many of these approaches rely on simulations to achieve a balance between precision and computational efficiency. Monte Carlo Planning (MCP) is a notable example, leveraging random sampling to evaluate and refine policies. Techniques like Monte Carlo Tree Search (MCTS) [Cha+08] excel in sequential decision-making problems by simulating state-action trajectories and updating estimates based on averaged outcomes. Beyond Monte Carlo methods, approaches such as approximate dynamic programming and policy iteration with sampling [AS07; Ji+07; WZC10] have been proposed. These methods offer additional practical solutions for complex control problems but fall outside the scope of this paper.

Reinforcement learning [SB18] provides a powerful framework for approximating value functions and deriving near-optimal policies, especially in cases where exact methods are impractical. RL methods focus on learning optimal policies through iterative interactions with the environment, making them particularly effective in scenarios with partially unknown or highly complex models.

This section offers a high-level overview of these approaches, covering their theoretical underpinnings and practical applications without delving into algorithmic details. Specifically, Section 1.3.1 introduces dynamic programming-based algorithms, which provide exact solutions; Section 1.3.2 discusses Monte Carlo Planning algorithms, which are designed to handle problems of higher complexity, and finally Section 1.3.3 presents RL approaches, tailored for frameworks with partially unknown models.

1.3.1 Dynamic Programming Algorithms

MDPs with finite state and action spaces and explicit models may be solved exactly via dynamic programming. These iterative methods compute the optimal value function and policy by defining and solving simpler embedded subproblems.

Backward iteration The backwards induction algorithm (see Algorithm 1) consists of systematically solving decision-making problems over a fixed number of time steps by working backwards from the final stage. Unlike the infinite-horizon case, where the goal is to optimize decisions over an indefinite period, in the finite-horizon case, decisions are optimized over a fixed number of stages. The process begins with the definition of the terminal value function, where no further decisions can be made. For each preceding time step, the value function is updated using Bellman recursion, which calculates the expected cumulative cost based on the

current state, available actions, immediate costs, and the value of future states. This backward induction process continues until we reach the initial time step $t = 0$, at which point the value function for each state has been determined for every stage. Once the value function has been calculated for all stages, an optimal policy can be derived by selecting, at each time step, the action that minimizes the expected cost for each state. This approach ensures that the resulting policy is optimal for the finite-horizon problem, as it takes into account both immediate and future costs at each decision point. In finite-horizon MDPs, the optimal decision rule generally varies with the time step t , making the optimal policy non-stationary.

Algorithm 1 Backward Iteration Algorithm

```

1: for  $s \in \mathbb{S}$  do
2:    $V_H(s) = C(s)$ 
3: end for
4: for  $t$  from  $H - 1$  to  $0$  do
5:   for  $s \in \mathbb{S}$  do
6:      $V_t(s) \leftarrow \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a)[c_t(s, a, s') + V_{t+1}(s')]$ 
7:      $\pi_t^*(s) \leftarrow \arg \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a)[c_t(s, a, s') + V_{t+1}(s')]$ 
8:   end for
9: end for

```

Example 1.3.1: Resolution with backward iteration

We consider the same squirrel toy model MDP as in Example 1.1.1. Let's determine the optimal policy and value function using the backward iteration algorithm.

We start from the terminal condition $V_H(s) = 0$ and recursively compute the value function. Because the cost $c(s, a)$ is minimized when the action a matches the current squirrel location s , the optimal policy is to always place the camera where the squirrel currently is.

Under this optimal policy, the cost $c(s, \pi^*(s)) = 0$ at every step, and since the dynamics do not depend on the action, the future cost is unaffected. Thus, the optimal value function is $V_h(s) = 0$ for all h and s .

For the full code implementation and extended examples, please refer to the associated Jupyter notebook.

Value iteration The value iteration algorithm (Algorithm 2) is a dynamic programming method used to solve MDPs in infinite-horizon problems. The algorithm updates the values $V(s)$ at each iteration, for every states $s \in \mathbb{S}$, converging to the optimal value function $V^*(\cdot)$. It begins with arbitrary initial values $V_0(s), \forall s \in \mathbb{S}$ and improves them by repeatedly applying the Bellman operators (see Theorem 1.1.1). The stopping criterion is based on the maximum change between two consecutive steps of the algorithm. After n steps it returns a value function $V_n(\cdot)$, from which an optimal policy $\pi^*(s)$ can be derived (even though V_n may not equal V^*). Note that the optimal policy obtained is stationary. The algorithm convergence to the optimal

policy is guaranteed for a discounted factor $0 < \gamma < 1$ and a maximal allowed change between to consecutive algorithm steps, ϵ .

Algorithm 2 Value Iteration Algorithm

Require: Discount factor $\gamma \in (0, 1)$, convergence tolerance ϵ

```

1: for  $s \in \mathbb{S}$  do
2:    $V_0(s) \leftarrow 0$ 
3: end for
4:  $n \leftarrow 0, \delta \leftarrow +\infty$ 
5: while  $\delta > \epsilon(1 - \gamma)\gamma$  do
6:   for  $s \in \mathbb{S}$  do
7:      $V_{n+1}(s) \leftarrow \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a)[c(s, a, s') + \gamma V_n(s')]$ 
8:      $n \leftarrow n + 1$ 
9:   end for
10:   $\delta \leftarrow \max_{s \in \mathbb{S}} |V_{n+1}(s) - V_n(s)|$ 
11: end while
12: for  $s \in \mathbb{S}$  do
13:   $\pi^*(s) \leftarrow \arg \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a)[c(s, a, s') + \gamma V_n(s')]$ 
14: end for
    
```

Policy iteration The policy iteration algorithm (Algorithm 3) is closely related to the value iteration algorithm described. While value iteration focuses on iterating over value functions, policy iteration directly updates the policies themselves. After initialization with an arbitrary initial policy, the algorithm operates by iteratively improving the policy as follows: (i) Policy Evaluation: this step computes the value of each state under the current policy by solving the associated system of linear equations or iteratively estimating the value function; (ii) Policy Improvement: this step updates the policy by deriving a "greedy" policy with respect to the current value function. This involves selecting actions that maximize the expected return for each state.

These steps are repeated, with the newly updated policy being evaluated and improved in turn. The process continues until the policy converges to the optimal policy, which remains unchanged in subsequent iterations. Policy iteration is guaranteed to compute the optimal policy in a finite number of steps, since the number of different deterministic policies is bounded. The number of iterations required is polynomial in the number of states and actions of the MDP, making it an efficient algorithm for solving MDPs.

Limitations and extensions The three algorithms discussed above rely on dynamic programming equations, which represent one of the most efficient methods for exploring the policy space. While dynamic programming is guaranteed to yield an optimal policy in theory, it often becomes computationally infeasible for real-world applications. This limitation arises due to the curse of dimensionality, a phenomenon identified by [Bel58], where the computational complexity grows exponentially with the dimensionality of the state and action spaces, especially if those are continuous.

One approach to mitigate this complexity is to employ discrete piecewise policies, enabling

Algorithm 3 Policy Iteration Algorithm

Require: Discount factor $\gamma \in (0, 1)$, initial policy π_0

- 1: Initialize policy π arbitrarily
- 2: **repeat**
- 3: **Policy Evaluation:**
- 4: Solve the following linear program in variables $V^\pi(s), s \in \mathbb{S}$:

$$V^\pi(s) = \sum_{s' \in \mathbb{S}} P(s'|s, \pi(s)) [c(s, \pi(s), s') + \gamma V^\pi(s')], \quad \forall s \in \mathbb{S}$$

- 5: **Policy Improvement:**
- 6: Compute greedy policy π' :

$$\pi'(s) \leftarrow \arg \min_{a \in \mathbb{A}} \sum_{s' \in \mathbb{S}} P(s'|s, a) [c(s, a, s') + \gamma V^\pi(s')], \quad \forall s \in \mathbb{S}$$

- 7: **until** $\exists s \in \mathbb{S}, V^\pi(s) \neq V^{\pi'}(s)$
-

the application of dynamic programming techniques (see, for example, [Ber75]). Discretization simplifies the problem by drastically reducing the complexity of the state space, making computations more manageable. However, this reduction comes at the cost of precision, often resulting in suboptimal policies (see [Fen+04]). Additionally, the process of discretizing a state space is itself challenging. It can be time-consuming, difficult to implement effectively, and prone to introducing inaccuracies, especially as the dimensionality of the problem increases.

Given these limitations, alternative approaches such as approximation methods and RL algorithms are often preferred. These methods provide practical solutions for handling high-dimensional discrete or continuous spaces more effectively, frequently outperforming discretized dynamic programming in terms of both policy quality and computational efficiency.

For simplicity, the remainder of this discussion will focus on finite-horizon problems to streamline the analysis and facilitate the exploration of these alternative methods.

1.3.2 Monte-Carlo Simulations-Based Algorithms

For finite-horizon MDPs with a known starting state, solving the MDP involves exploring a min-expectation tree. In these trees, nodes alternate between nodes where an expectation (over successor states) is computed and nodes where a minimum value (over actions) is computed. The root of this tree corresponds to the initial state, s_0 . Each action $a_0 \in \mathbb{A}$ generates a successor action node. Each action node a_0 further branches into successor state nodes for all s_1 such that $P(s_1|s_0, a_0) > 0$. This process continues across time steps, with the tree's leaves representing the final states s_H .

Solving the MDP using exact dynamic programming involves fully constructing this tree and applying backward induction. Backward induction labels each state node s_t with its optimal value $V_t^*(s_t)$, which represents the return obtained by following the optimal policy π^* from s_t

until the horizon H .

However, backward induction is computationally intensive because it requires exploring the entire tree. Even storing the complete tree is infeasible in many cases, though depth-first exploration with pruning of dominated actions can help mitigate this storage issue.

To address these computational challenges, Monte Carlo Planning methods, and in particular the family of Monte Carlo Tree Search (MCTS) algorithms, offer an efficient alternative. These heuristic-free approaches, such as the Upper Confidence bound applied to Trees (UCT) algorithm [KS06], focus on exploring the tree through simulations and do not rely on problem-specific heuristic functions to guide the search. MCTS algorithms prioritize exploration of the most promising parts of the tree—areas that show high potential or require further evaluation to refine their estimated values. By doing so, they avoid exhaustive exploration, making them well-suited for large or complex MDPs (see, *e.g.*, [Cou06]).

There exist many variants of MCTS [Bro+12; Świ+23], however most of them iteratively update a search tree (which is not exactly the same as the dynamic programming tree) until a computation budget is exhausted, at which point an action, computed for the root node, is returned. A tuple $\langle s, a, \nu, \rho \rangle$ is attached to each node v . Component $v.s = s \in \mathbb{S}$ is a state, $v.a = a \in \mathbb{A}$ is the action that led to state $v.s$, $v.\nu = \nu \in \mathbb{N}$ is the number of visits to node v following the current edge (*i.e.* the number of times $v.a$ was applied in $v'.s$ and led to $v.s$, if $v' = \text{Par}(v)$ is the parent node of v), $v.\rho = \rho \in \mathbb{R}$ is an estimate of the value function $V^*(v.s)$ at state $v.s$. Initially, the tree consists of a single root node v_0 with attached tuple $\langle v_0.s = s_0, v_0.a = \emptyset, v_0.\nu = 1, v_0.\rho = 0 \rangle$. At any step, the algorithm loops over the following four successive substeps to update the current tree, see Figure 1.7 for an illustration.

1. **Selection.** Starting from the root node, child nodes are selected recursively.

- If v is the current node, we check that it has a child node v^a for every $a \in \mathbb{A}$, *i.e.* that all available actions have been tried in state $v.s$. Else, Selection phase is over (condition I).
- Then, an action a is selected, using a tree policy and a successor state $s' \sim P(\cdot | v.s, a)$ is simulated. If there exist v' such that $v'.s = s'$ and $v'.a = a$, then v' is selected and the Selection phase goes on from v' . Else, Selection phase is over (condition II).

2. **Expansion.** When Selection phase is over, the tree is expanded. A new child node of v (selected node at the end of the Selection phase) is added. It is obtained

- either (if condition I was met) by playing a yet unplayed action $a \in \mathbb{A}$ and simulating $s' \sim P(\cdot | v.s, a)$,
- or (if condition II was met) corresponding to the newly simulated state s' .

Let v' be this node and $\langle v'.s = s', v'.a = a, v'.\nu = 1, v'.\rho = 0 \rangle$ the attached tuple.

3. **Simulation.** When a node v' has been added in the expansion phase, a single trajectory is simulated, starting from $v'.s$, using a default policy (for example, successive actions may

be selected uniformly at random). Let $\mathcal{C}$ be the total (discounted in case of a discounted MDP) cost collected along the simulated trajectory. Then, $v'.\rho \leftarrow \mathcal{C}$.

4. **Backpropagation.** The values of the ancestor nodes of v' are updated recursively: For every ancestor nodes v of v' , selected in backward order,

$$v.\rho \leftarrow \min_{a \in \mathbb{A}} \frac{\sum_{\tilde{v} \in \text{Succ}(v), \tilde{v}.a=a} \tilde{v}.\nu \times (c(v.s, a, \tilde{v}.s) + \gamma \tilde{v}.\rho)}{\sum_{\tilde{v} \in \text{Succ}(v), \tilde{v}.a=a} \tilde{v}.\nu}.$$

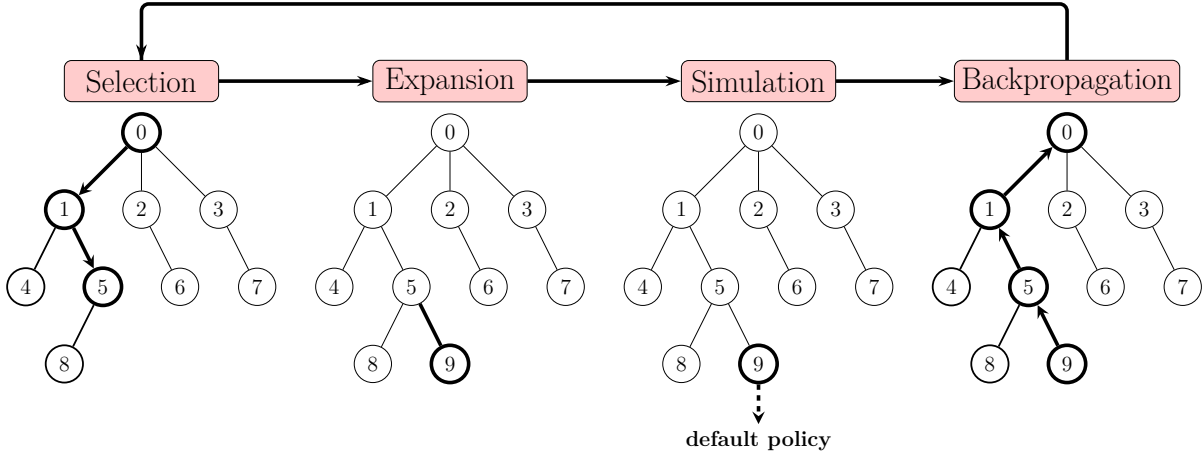


Figure 1.7: **The four steps of Monte-Carlo Tree Search.** A child node is selected (here node 5), then the tree is expanded with a new child node (here node 9) and a trajectory is simulated from this new node with default policy. Finally, the tree is backpropagated to update the Q-values. From [Cle+25]

Once the Backpropagation step is over, the algorithm goes back to the Selection step, until the computation budget is over. At this point, an action a^* is returned for the initial state s_0 :

$$a^* \leftarrow \arg \min_{a \in \mathbb{A}} \sum_{v' \in \text{Succ}(v_0), v'.a=a} \frac{\sum_{v'' \in \text{Succ}(v'), v''.a=a} v''.\nu \times (c(v.s, a, v''.s) + \gamma v''.\rho)}{\sum_{v'' \in \text{Succ}(v'), v''.a=a} v''.\nu}.$$

An MCTS algorithm instance is determined by the choice of a tree policy and a default policy. Typically, the tree policy is derived from a UCT criterion [KS06], balancing exploitation of the best action so far and exploration of under-used actions. The default policy is often uniformly random. However many choices of tree policies and default policies are possible, leading to different algorithms. The reader may refer to [Świ+23] for a recent overview.

Note that, contrarily to exact dynamic programming, Monte-Carlo planning does not require complete knowledge of the MDP model, (P, c) , but only the faculty to simulate $P(\cdot|s, a)$ and to record experiences (s, a, s', c) . Even though the backpropagation step uses costs $c(v.s, a, v'.s')$, these costs have been encountered during the expansion step and may be added to the tree data structure. This feature is common with RL approaches, described in the next section.

Example 1.3.2: Resolution with Monte-Carlo Tree Search

We consider again the squirrel toy model POMDP introduced in Example 1.2.1. Our goal here is to approximate the optimal policy and value function using the Partially Observable Upper Confidence bounds for Trees (POUCT) algorithm [SV10]. This algorithm is an extension of Monte Carlo Tree Search tailored for POMDPs. It maintains a belief over hidden states and balances exploration and exploitation through an upper confidence bound mechanism, making it suitable for planning under uncertainty.

After a single trajectory, POUCT achieved a cumulative cost of 2.00 ± 1.00 . Figure 1.8 illustrates an example of trajectory under the POUCT policy.

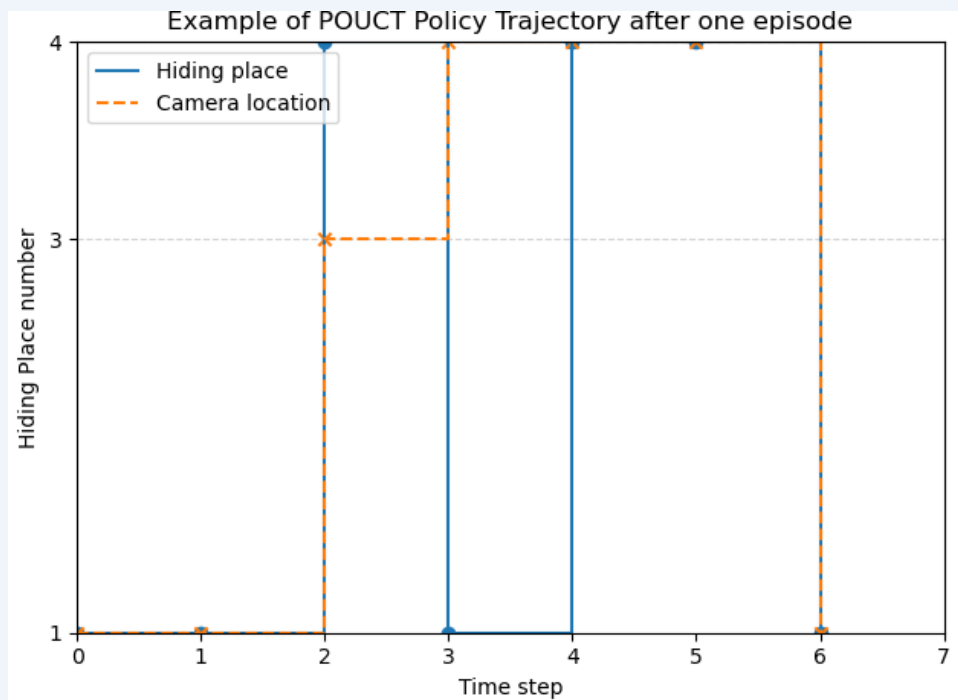


Figure 1.8: An example of trajectory under POUCT policy.

For full code implementation details [ZT20], additional episodes, and extended analysis, please refer to the accompanying Jupyter notebook provided in the supplementary material.

1.3.3 Reinforcement-Learning Algorithms

Reinforcement Learning (RL) is a computational framework that enables agents to learn optimal mappings from situations to actions in order to minimize a numerical cost. In RL, the environment represents everything beyond the agent's direct control. In the context of MDPs, the environment corresponds to the mathematical structure that defines state transitions and costs, as described in Section 1.1.1. Unlike traditional learning methods, RL does not require complete knowledge of this structure. Instead, it allows agents to learn directly through interactions with the environment.

The RL process involves a cycle of interaction: the agent observes the current state, selects an action, and receives feedback in the form of a cost, which reflects the quality of its decision (see Figure 1.9). This action also influences the next state of the environment. Through repeated iterations, the agent refines its behavior, ultimately developing a policy that optimizes long-term costs, as outlined in Section 1.1.3.

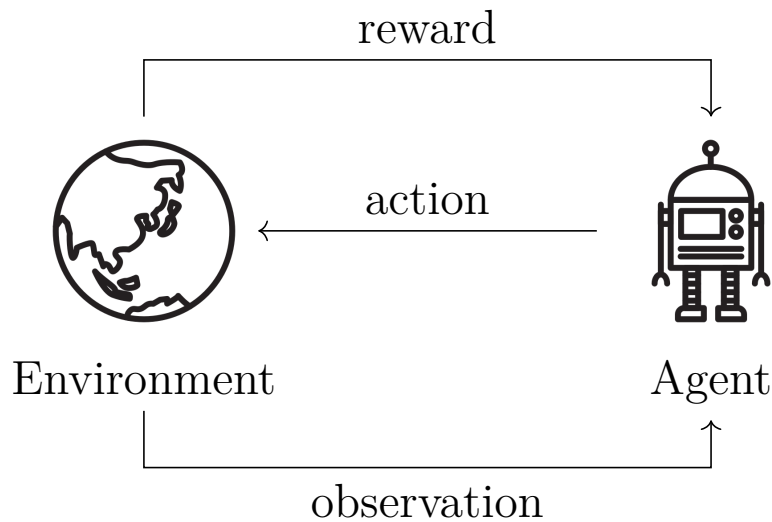


Figure 1.9: **Reinforcement learning schematics.** The learning process involves an agent observing a state, taking an action that affects the state, and receiving feedback through costs.

One of the fundamental challenges in RL is balancing the trade-off between exploration and exploitation. To minimize costs, an RL agent must exploit actions that have proven effective in the past while also exploring new actions to uncover potentially better strategies. Striking this balance is essential to prevent the agent from becoming stuck in suboptimal behaviors and to enable continuous policy refinement through exploration.

The process of learning optimal actions in RL is approached from multiple perspectives, each characterized by distinct methodologies and challenges. This section begins with model-free methods, which do not rely on a model of the environment’s dynamics. Within this category:

- Value-based approaches focus on estimating the value function to derive optimal policies.
- Policy-based methods directly search for the optimal policy within the policy space.

Following model-free approaches, we delve into model-based methods, which involve constructing an explicit model of the environment to predict future states and associated costs.

By structuring the discussion in this manner, we aim to provide a comprehensive overview of the key strategies used in RL, emphasizing the strengths and limitations of each approach. This section draws heavily on the seminal work by [SB18], which offers an extensive survey of RL methodologies and their applications.

Recent advancements in deep learning (*e.g.* [LBH15; GBC16]) have significantly enhanced RL capabilities, particularly in addressing challenges posed by high-dimensional state and action spaces. These developments have given rise to deep reinforcement learning (deep RL) algorithms, which are discussed throughout this section. These algorithms show considerable promise for advancing the field.

For practitioners, practical implementation can begin by encoding their environment using the Gymnasium library introduced by [Bro+16]. This library integrates seamlessly with RLlib ([Lia+18a]), which offers a suite of pre-implemented deep RL algorithms ready for use.

Model-free approaches: value-based algorithms

Model-free value-based RL is a key area within artificial intelligence that focuses on learning optimal policies by estimating value functions, without the need for an explicit model of the environment. This approach is especially valuable in complex environments where creating an accurate model is either difficult or impractical.

Tabular algorithms. We begin with the simplest cases, where the state and action spaces are small enough for the value functions to be approximated using arrays. These methods are known as tabular algorithms (see [SB18]).

- **Q-learning.** [Wat89] introduced Q-learning, one of the foundational algorithms in this category, an off-policy method that allows an agent to learn a policy different from the one it follows during training. This distinction enables the agent to learn the optimal policy while adhering to a more exploratory behavior policy. In Q-learning, the agent interacts with the environment and updates its Q-values based on received costs and the minimum expected sum of future costs, as defined by the Bellman equation. The iterative process is continued until a stopping criterion is reached. [WD92b] show that provided the action space is finite, the Q-values converge to the optimal action values with probability 1. Through this process, the agent identifies the best possible action for any given state, as outlined in Algorithm 4.

Q-learning is widely recognized for its simplicity and effectiveness in the case of finite state / action spaces. However, [WD92b] also show that it can be computationally expensive and slow to converge, particularly in large state spaces.

- **SARSA and TD.** In [RN94], the authors introduce the State-Action-Reward-State-Action (SARSA) algorithm, which is also model-free and value-based. The name SARSA reflects the sequence of events the algorithm uses to update its knowledge. Unlike Q-learning, SARSA updates its Q-values based on the action actually taken by the agent, rather than the optimal action, making it an on-policy algorithm in contrast to Q-learning's off-policy approach. [Sin+00] show that the SARSA algorithm is guaranteed to converge if every state-action pair is visited infinitely often, but its convergence can be slow, especially in environments with large state spaces. Another alternative to Q-learning is Temporal-Difference (TD) learning, introduced by [Sut88], which combines Monte Carlo methods with dynamic programming.

Over the last few decades, extensions to these algorithms have been developed to address

Algorithm 4 Q-learning

```

1: Initialize  $Q(s, a), \forall s \in \mathbb{S}, a \in \mathbb{A}$ , arbitrarily
2: for each episode do
3:   Initialize  $s$  randomly
4:   for each step of episode do
5:     Choose  $a \leftarrow \arg \max_{a \in \mathbb{A}} Q(s, a)$  with probability  $\epsilon$  or uniformly in  $\mathbb{A}$  with probability
        $(1 - \epsilon)$  ( $\epsilon$ -greedy choice)
6:     Apply action  $a$ , observe  $c, s'$ 
7:      $Q(s, a) \leftarrow Q(s, a) + \alpha [c + \gamma \min_{a'} Q(s', a') - Q(s, a)]$ 
8:      $s \leftarrow s'$ 
9:   end for
10: end for
    
```

their limitations. For instance, an on-policy Q-learning extension was created to achieve better computational complexity bounds ([Str+06]), perform in multi-agent environments ([SPG07]), and address the issue of overestimated action values ([Has10]).

Function approximation techniques. While tabular algorithms and their extensions work well in finite and relatively small state spaces, many real-world problems involve larger or even continuous state spaces. In cases where a tabular representation of the Q-function is impractical, [Tsi03] proposed to approximate the Q-function using function approximation techniques. Here, the value function is not represented as a table but as a parameterized function with a weight vector $\mathbf{w} \in \mathbb{R}^d$. The approximate value of a state s given weight vector $\mathbf{w}$ is denoted by $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$. $\hat{v}$ could take forms such as a linear function, a decision tree, or others. [SB18] extended the theoretical results for methods using function approximation to cases involving partial observability.

Value-based deep RL extends function approximation techniques by using a multi-layer artificial neural network to approximate the value function $\hat{v}$. This approach provides a powerful method for function approximation, albeit without theoretical guarantees ([SB18]). Many Deep RL algorithms are built upon foundational tabular methods. A notable example is the Deep Q-Network (DQN), introduced by [Mni+13] and illustrated in Figure 1.10. Subsequent advancements, such as Double DQN ([HGS16]), Dueling DQN ([Wan+16]), and Rainbow ([Hes+18]), have further enhanced the efficiency and performance of DQN-based approaches.

As depicted in Figure 1.10, when dealing with partially observable environments, the neural network receives observations rather than full state information. In such cases, the optimal policy depends on the entire history of observations, as discussed in Section 1.2.1. Addressing this challenge, [HS15] introduced R2D2, which uses recurrent neural networks (RNNs) instead of convolutional networks. RNNs are capable of retaining information from past observations, making them particularly well-suited for partially observable Markov decision processes (POMDPs).

These algorithms collectively demonstrate the flexibility and potential of model-free value-based RL methods in tackling complex decision-making challenges across various domains.

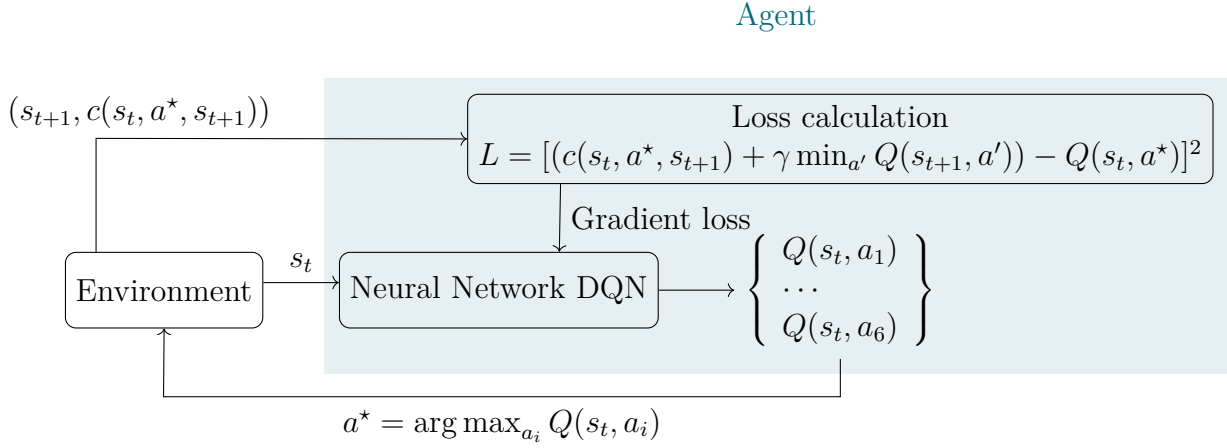


Figure 1.10: **Deep Q-Network algorithms.** Deep Q-Network (DQN) algorithms send the current state of the environment to a neural network, which calculates the Q values for each possible action in the MDP. The algorithm then selects the action that minimizes these Q-values. Once the action has been chosen, the environment moves to the next state and the associated cost is calculated. This cost is used to update the loss function, which measures the difference between the predicted Q-values and the actual values. Finally, the network weights are adjusted by backpropagation to improve future predictions. In practice, neural networks process batches of transitions rather than a single state at a time, making the actual implementation significantly more efficient.

Model-free approaches: policy-based algorithms

While value-based RL algorithms generally learn a Q-function, from which a (nearly-) optimal policy can be obtained, policy-based algorithms directly learn a policy from experiments. A classic example of a policy-based algorithm in dynamic programming is policy iteration, which alternates between evaluating the current (deterministic) policy and improving it (see Section 1.3.1).

Policy gradient approaches. When the model of the MDP is unknown (RL context) and/or when state and action spaces are too large or even continuous, policy iteration is no longer applicable. In this context, Policy Gradient approaches (*e.g.* [DNP13; Sil+14]) suggest to use continuously parameterized policies and replace policy improvement steps with gradient descent steps, to improve the current policy (defined by its continuous parameters). With parameterized policies, the value of the policy in an arbitrary step can be considered itself as a function of the parameters of the policy. As such, the gradient of the value function can be estimated using stochastic policy gradient algorithms (the value function in state s is an expectation of the sum of costs obtained along possible trajectories).

More precisely, model-free policy gradient algorithms exploit the gradient theorem from [Sut+99] which states that:

$$\nabla_{\theta} V(\pi_{\theta}, s_0) = \mathbb{E}_{s \sim \mu_{s_0}^{\pi_{\theta}}, a \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) \tilde{Q}(s, a) \right]$$

Here, $\pi_{\theta}(\cdot|\cdot)$ is any differentiable parameterized representation of a stochastic policy (with

parameter vector θ) and $\tilde{Q}$ is an estimation of the true Q-function (which can be computed, *e.g.* by simulation). This policy gradient expression makes it easy to estimate the gradient of the current policy by simulation, since the gradient of π_θ may be known explicitly, and simulations can be used to estimate $\tilde{Q}$ as well as the global expectation ($\mu_{s_0}^{\pi_\theta}$ is the discounted occupation measure of the Markov chain followed by starting in s_0 and applying π_θ). Note also that policy-gradient algorithms naturally handle continuous state and action spaces.

One limitation of these approaches is the need to arbitrarily choose a parameterized policy, something that may not always be ideal. Moreover, estimating the gradient through simulations often requires substantial computational resources, even if the model is known. The challenge becomes more significant when the model is unknown, as the approach then depends on datasets of observed MDP trajectories.

Parametrized policy approaches. On the other hand, policy iteration approaches are highly versatile and capable of addressing remarkably complex MDP problems. This is especially true with recent advancements handling neural network representations of policies. Proximal Policy Optimization (PPO, [Sch+17]), illustrated in Figure 1.11, is a popular example of a deep RL algorithm with policy iteration introduced as a practical and more stable alternative to Trust Region Policy Optimization (TRPO) [Sch+15]. It computes the Advantage function, which is simply the difference between the Q-value for a given state–action pair and the value function of the state, defined as $A(s, a) = Q(s, a) - V(s)$. In PPO, instead of approximating the value function as in deep Q networks (DQNs), the policy itself is parameterized by a neural network. This network directly associates states (or observations in the case of partial observability) with actions, optimizing the policy to minimize expected costs. In PPO, the algorithm alternates between collecting data from the environment using the current policy and updating the policy on the basis of the data collected. As with DQN, PPO can be extended to handle partial observability by incorporating recurrent neural networks (RNNs) instead of feedforward networks [Hee+15; Igl+18]. This allows the policy to retain past observations and improve decision making in environments with hidden states, such as POMDPs.

Unlike DQN, which generates a deterministic policy by selecting the action with the highest Q value, PPO generates a stochastic policy by sampling actions from a probability distribution over actions, typically modeled by a neural network. This stochastic nature allows the agent to explore more diverse behaviors, which makes it particularly useful in environments where there is uncertainty or partial observability, as it can prevent the policy from getting stuck in sub-optimal solutions by occasionally trying different actions.

Model-based algorithms

This class of algorithms operates in an iterative loop: the agent begins by collecting data to update its model of the environment’s dynamics. Using the refined model, the agent learns and optimizes its policy within a simulated environment. Once a new policy is derived, the agent applies it in the real world, gathering additional data. This new data is then used to further refine the model.

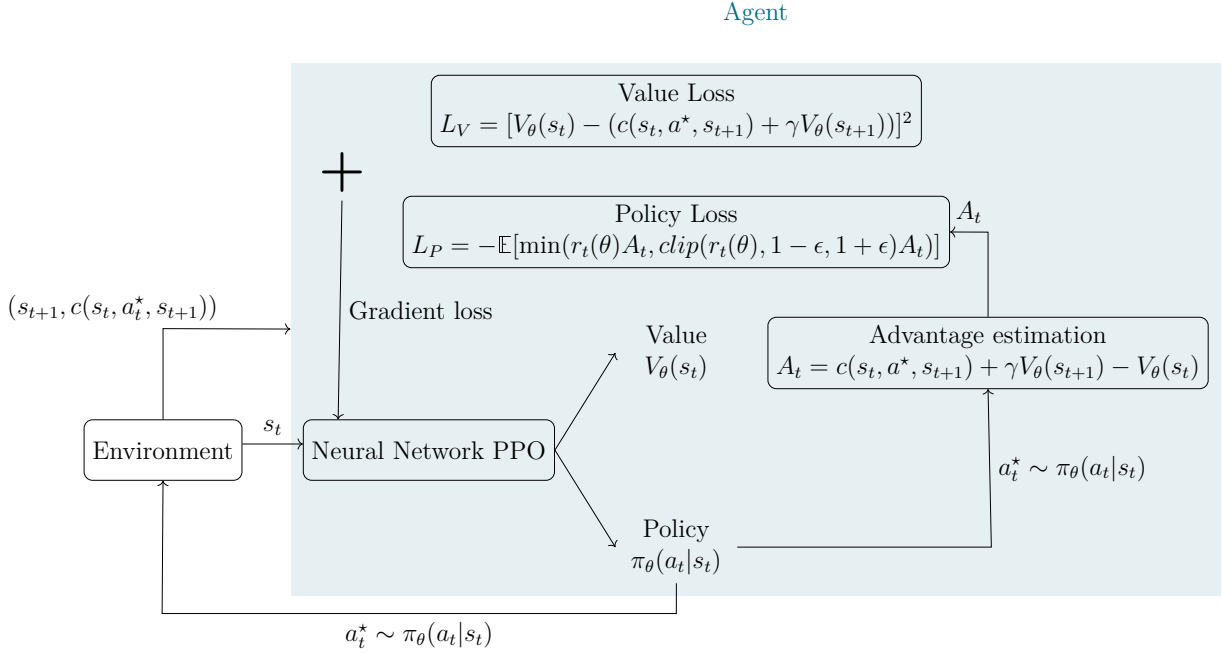


Figure 1.11: **Proximal Policy Optimization algorithms.** Proximal Policy Optimization (PPO) algorithms send the current state of the environment to a neural network, which calculates a probability distribution over actions (the policy) and an estimated value function. An action is sampled from the policy and executed in the environment, leading to a new state and a received cost. The collected data is then used to compute two losses: a policy loss that encourages improvement of the action distribution while constraining large policy updates with $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{OLD}}(a_t|s_t)}$ and the *clip* function, and a value loss that minimizes the error in value estimation. Both losses are combined, and the network weights are updated via backpropagation to progressively refine the policy and value function. In practice, neural networks process batches of transitions rather than a single state at a time, making the actual implementation significantly more efficient.

The cycle of data collection, model updating, and policy optimization repeats continuously, enabling the agent to improve its performance over time. This iterative process is depicted in Figure 1.12.

[AS97] show that this provides better results than model-free approaches with less data. For the sake of simplicity, we only focus on Bayesian Reinforcement Learning (BRL) in this survey. However, readers interested in model-based methods are encouraged to explore the comprehensive survey on model-based RL provided in [Luo+22], as well as the state-of-the-art DreamerV3 algorithm proposed by [Haf+24], a model-based deep RL algorithm.

Introduced in [MT15], BRL is a model-based RL approach that leverages methods from Bayesian inference to model uncertainty in the environment's dynamics. The Bayesian framework, described in section 1.2.2, allows for principled incorporation of prior knowledge and can help in balancing the exploration-exploitation tradeoff effectively. The implementation aspects of this framework are studied in [CP07] who provide empirically efficient algorithms to solve BAMDP. An excellent review of algorithmic challenges and state of the art of the domain is

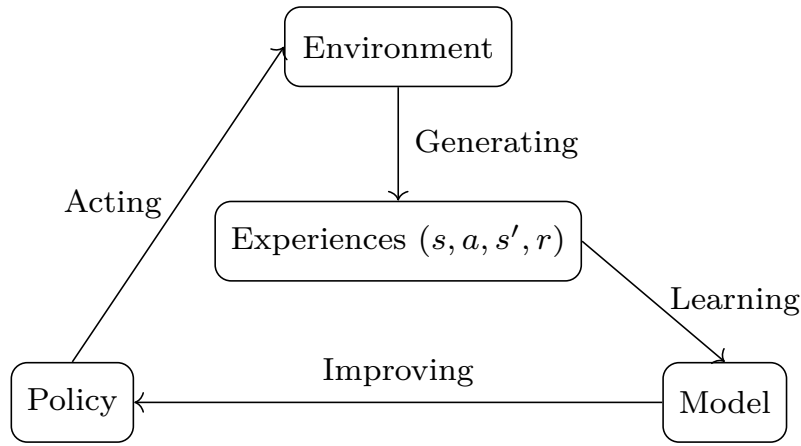


Figure 1.12: **Model-based reinforcement learning.** Data is collected to update the environment’s model knowledge, which is then used to learn and optimize the policy in a simulated environment. This updated policy is applied, yielding new data.

given in [AS22]. [AS22] provide an overview of the literature on algorithmic approaches to solve BAMDP, as well as a complexity analysis of the full problem.

A Bayesian approach to dynamic programming was introduced by [Str00], where a model is sampled from the posterior distribution and dynamic programming is applied to the sampled model. To improve accuracy, the model is periodically resampled to account for new information. While effective, this method can be computationally slow, especially in complex environments. In [Duf02], a method is proposed that utilizes Finite State Controllers (FSCs) to represent Bayesian policies in a more compact form. FSCs are graph-based structures where the nodes represent memory states and the edges correspond to transitions between states, defined by the tuple (s, a, s') . The algorithm then searches the space of possible FSCs to identify the optimal controller, effectively reducing the complexity of finding the best policy. The BEETLE algorithm ([Pou+06]) builds upon the PERSEUS algorithm from [SV05], originally developed for solving continuous POMDPs. The key innovation of BEETLE is to reformulate the Bayesian Adaptive Markov Decision Process (BAMDP) as a POMDP that operates over the space of states and transition functions. This approach leverages the strengths of POMDP techniques to address the complexities inherent in BAMDPs.

Due to the high complexity associated with model-based BRL, most approaches have been limited to very small state-spaces, typically comprising only 10-20 states. As the number of states increases, a large amount of data is required to accurately learn a robust model. In addition, planning methods become difficult to implement in larger state spaces, as they require consideration of the full range of possible posterior distributions, which significantly complicates the decision-making process.

Online tree search methods have become a promising solution for solving BAMDPs by evaluating potential future action sequences from the current state, building a decision tree to guide actions. Unlike offline methods, online approaches focus on finding effective policies for

the current belief state, making them more feasible for complex problems. A notable example is Bayesian Sparse Sampling, introduced in [Wan+05], which non-uniformly explores the tree of future states and actions, reducing computational complexity by concentrating on the most promising paths. Another well-known approach is Monte Carlo Tree Search (MCTS, [Bro+12], see above) algorithms, which balance exploration and exploitation through random sampling.

Building on MCTS, [GSD12] developed Bayesian Adaptive Monte Carlo Planning (BAMCP) to extend tree search for BAMDPs, incorporating Bayesian updates to inform action exploration. Despite improvements over simpler methods, BAMCP remains computationally demanding, especially in large state and action spaces, and lacks function approximation, limiting its use in very large or continuous state spaces.

Example 1.3.3: Resolution with BAMCP

We consider the same squirrel toy model BAMDP as in Example 1.2.2. Let's determine the approximate optimal policy and value function using the BAMCP algorithm.

The BAMCP algorithm quickly finds that the optimal policy is to always move the camera to the location where the squirrel is actually hiding, achieving a cost of zero. Since we are working with a very simple MDP where the optimal strategy is trivial, BAMCP identifies the correct behavior from the very first simulated trajectory. As shown on Figure 1.13, the optimal policy found by BAMCP algorithm is to follow the squirrel.

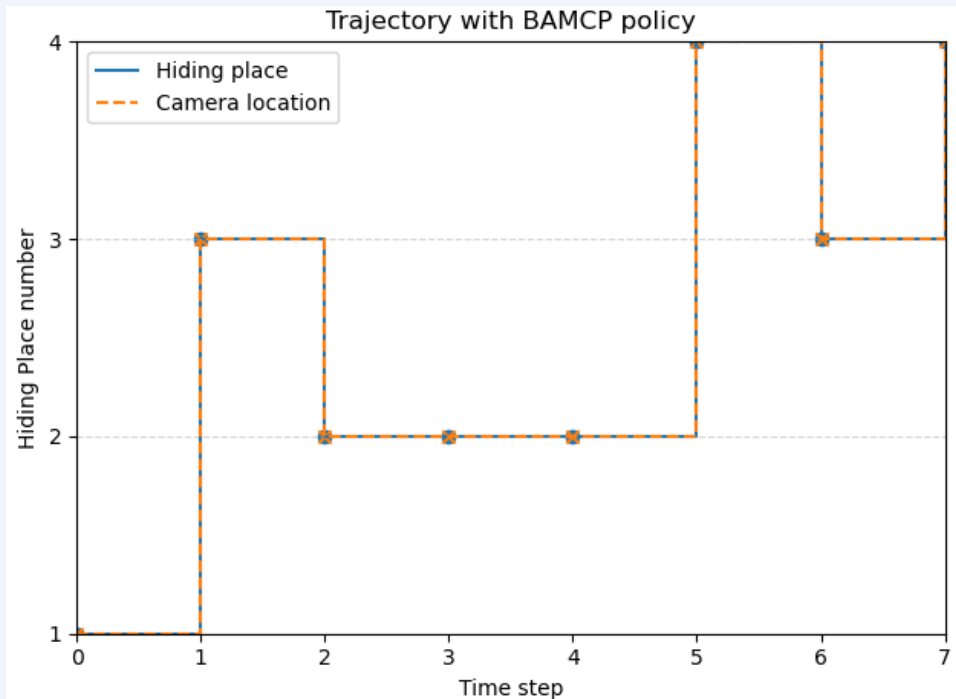


Figure 1.13: An example of trajectory under BAMCP policy.

Comment: In general, BAMCP is an online planning algorithm and its initial decisions can be suboptimal because they are based on limited sampled trajectories. However, as more trajectories are simulated, the planning becomes increasingly accurate, progressively

converging toward the optimal strategy.

For the full code implementation and extended examples, please refer to the associated Jupyter notebook available in the supplementary material.

In BRL, significant progress has been made with algorithms designed to handle continuous state spaces and complex environments. BayesianExplore (BE) ([KL23]) is a standout algorithm, leveraging posterior sampling of value functions for efficient exploration. Another key advancement is Deep Bayesian Reinforcement Learning (Deep BRL), which has been particularly effective in dealing with continuous state spaces. Methods such as BRL-DSS from [Gro20] combine deep learning with sparse sampling to approximate value functions in high-dimensional continuous spaces where other methods often falter.

1.4 Conclusion

In this chapter, we introduced the fundamental principles of MDPs. Through the simple example of a squirrel hiding in different places, we illustrated the core concepts of MDPs—states, actions, transition probabilities, and rewards. This toy model served as an intuitive entry point to help understand how decision-making under uncertainty can be formalized. We then reviewed the main approaches for solving MDPs, including exact resolution methods, approximate resolution methods, and RL.

The central objective of this manuscript is to address impulse control problems for Piecewise Deterministic Markov Processes (PDMPs), which require a broader mathematical framework than the discrete MDPs discussed so far. In particular, we have until now focused on discrete state spaces and discrete time as a foundation. In the next chapter, we will move into continuous state spaces and continuous time and formally introduce impulse control for PDMPs. We will also demonstrate how these problems can be reformulated as extended MDPs in Chapter 5 and 6. This reformulation paves the way for applying RL algorithms.

Chapter 2

Impulse Control of Piecewise Deterministic Markov Processes

2.1	PDMP, Definition and Examples	51
2.1.1	Generic Definition	51
2.1.2	Semi-Markov Dynamics	55
2.1.3	Simulation	57
2.1.4	Embedded Chains	58
2.2	Impulse Control for PDMPs	60
2.2.1	Continuous versus Impulse Control	61
2.2.2	Definition of an Impulse Control Problem	61
2.2.3	Resolution of Impulse Control Problems for PDMP	65
2.3	Conclusion	66

Piecewise Deterministic Markov Processes (PDMPs) are a general class of non-diffusion processes introduced by M. Davis in the 80's [Dav84]. These continuous-time processes extend the class of point processes [Dav93; Jac06; Coc21] and are characterized by deterministic motions punctuated by random jumps. They are especially suitable to model complex systems with interacting random phenomena, or large population asymptotics.

PDMPs are constructed using a small set of intuitive and easily interpretable local characteristics, making them a highly versatile and powerful modeling tool. Their ability to capture both continuous dynamics and discrete transitions makes them a powerful tool for modeling complex real-world phenomena. They extend the framework of discrete-time and discrete-state space Markov or semi-Markov chains, enabling the representation of a broad spectrum of complex phenomena across various domains. Examples from the PDMP literature cover a wide range of application domains, from workshop optimization and queuing theory [Dav93], internet networks *e.g.* [Bar+13], reliability [SDZ16], insurance and finance [BR11], neurosciences

[RTW12; RT15], population dynamics or genomics [Dou+15; Cos16; Her+17], or medical treatment [CS18; Pas+18] to name just a few.

Research on PDMPs has focused on several key aspects, including their long-term behavior, large population asymptotics, Monte Carlo methods, and statistical inference. A fundamental question in the study of stochastic processes is their long-term behavior, particularly the existence and convergence to an invariant or ergodic distribution. Necessary and sufficient conditions for PDMPs to exhibit such properties have been established using an embedded discrete-time Markov chain approach [CD08], leading to further studies on explicit ergodic distributions and convergence rates [FGM12; Ben+12; Clo+17]. Another active research area involves large population asymptotics, where PDMPs serve as models for interacting populations evolving on different time scales or as limiting processes in large-scale population dynamics [RTW12; GT14; FHC15; Clo+17; CSJ20; PS24].

PDMPs also play a significant role in computational methods. They provide a unifying framework for continuous-time Monte Carlo methods, such as Markov Chain Monte Carlo and Sequential Monte Carlo, and are particularly valued for their efficient simulation properties [Fea+18; Bie+18; MGM23]. Furthermore, statistical inference for PDMPs remains an active but challenging research direction, particularly in estimating jump intensities and transition kernels. Various parametric and non-parametric estimation techniques have been proposed, including methods for jump rate estimation [Kre16; ADG14; KS21] and transition kernel approximation [CL08; Aza14; AG19]. The diversity of these approaches highlights the adaptability of PDMPs across different application domains, reinforcing their importance in both theoretical and applied research.

In this manuscript, we focus specifically on the control of PDMPs, with a particular emphasis on impulse control. Impulse control refers to optimization strategies where decisions are made at discrete intervention times, leading to instantaneous changes in the system's state. In this chapter we formally define the mathematical framework of PDMPs in section 2.1 and present the impulse control problem along with existing resolution methods in section 2.2, illustrating key concepts through examples. This chapter is an adapted version of a submitted survey written with Alice Cleynen, Benoîte de Saporta, Régis Sabbadin and Amélie Vernay [Cle+25].

2.1 PDMP, Definition and Examples

In this section, we introduce PDMPs by defining their key components and properties. The discussion extends to handling semi-Markov dynamics, where the inter-jump times depend on both the state and the elapsed time since the last jump. We explain how to simulate PDMPs and describe embedded chains that capture the discrete dynamics of these processes.

2.1.1 Generic Definition

We start with a formal definition of a PDMP, its state space and its local characteristics.

Definition 2.1.1: Piecewise Deterministic Markov Process

A Piecewise Deterministic Markov Process (PDMP) $X = (X_t)_{t \geq 0}$ is defined by a tuple $\langle E, \Phi, \lambda, Q \rangle$, where

- The state space E is a finite union of Borel subsets of $\mathbb{R}^d$, for some $d \geq 1$ and $\mathcal{B}(E)$ its Borel σ -field. It is the set of all possible states of the process. Denote $\bar{E}$ the closure of E and $\mathcal{B}(\bar{E})$ the Borel σ -field on $\bar{E}$.
- The flow Φ is a continuous function from $E \times \mathbb{R}_+$ onto $\bar{E}$ satisfying a semi-group property, *i.e.* $\Phi(\cdot, t + s) = \Phi(\Phi(\cdot, t), s)$ for all $s, t \in \mathbb{R}_+$. The flow prescribes the deterministic motion between jumps.
- The jump intensity, or hazard rate, or risk function λ is a measurable function from $\bar{E}$ onto $\mathbb{R}_+$ that determines the occurrence of random jumps.
- The jump kernel Q is a Markov kernel on $(\mathcal{B}(E), \bar{E})$ that selects the new location of the process after each jump.

The flow Φ , the jump rate λ , and the Markov kernel Q are called the local characteristics of the PDMP.

Remark 2.1 *The flow may be defined by an explicit function or simply described as the solution to a system of ordinary differential equations (ODE).*

It is particularly convenient, when dealing with PDMPs, to separate the state space E and states x into discrete and Euclidean components. In this case the state space is called hybrid. Let $\mathbf{M}$ be a finite set, and for all $\mathbf{m} \in \mathbf{M}$, let $\mathbf{E}_{\mathbf{m}}$ be some Borel subset of $\mathbb{R}^{d_{\mathbf{m}}}$, which dimension $d_{\mathbf{m}}$ is allowed to change with $\mathbf{m}$. The state space is thus

$$E = \bigcup_{\mathbf{m} \in \mathbf{M}} \{\mathbf{m}\} \times \mathbf{E}_{\mathbf{m}}.$$

Any state $x \in E$ can be written as $x = (\mathbf{m}, \mathbf{x})$ with $\mathbf{m} \in \mathbf{M}$ and $\mathbf{x} \in \mathbf{E}_{\mathbf{m}}$. The discrete component $\mathbf{m}$ is called the mode or regime and $\mathbf{x}$ is called the Euclidean component. Define on E the following distance, for $x = (\mathbf{m}, \mathbf{x})$ and $x' = (\mathbf{m}', \mathbf{x}') \in E$:

$$|x - x'| = |\mathbf{x} - \mathbf{x}'| \mathbb{1}_{\mathbf{m}=\mathbf{m}'} + \infty \mathbb{1}_{\mathbf{m} \neq \mathbf{m}'}.$$

In the hybrid setting, we set

$$\Phi(x, t) = (\mathbf{m}, \Phi_{\mathbf{m}}(\mathbf{x}, t)),$$

for all $x = (\mathbf{m}, \mathbf{x}) \in E$ and $t \in \mathbb{R}_+$.

The jump intensity can be written as $\lambda(x) = \lambda_{\mathbf{m}}(\mathbf{x})$ for all $x = (\mathbf{m}, \mathbf{x}) \in E$ and $t \in \mathbb{R}_+$ and

is such that for any x in E , there exists $\epsilon > 0$ such that

$$\int_0^\epsilon \lambda(\Phi(x, t)) dt < +\infty,$$

forbidding instantaneous jumps. Let ∂E be the boundary of E : $\partial E = \cup_{\mathbf{m} \in \mathbf{M}} \{\mathbf{m}\} \times \partial \mathbf{E}_{\mathbf{m}}$, and $t^*(x)$ be the deterministic time the flow takes to reach the boundary ∂E of E when it starts from x

$$t^*(x) = t_{\mathbf{m}}^*(\mathbf{x}) = \inf\{t > 0 : \Phi(x, t) \in \partial E\}.$$

For convenience, we also define

$$\Lambda(x, t) = \Lambda_{\mathbf{m}}(\mathbf{x}, t) = \int_0^t \lambda(\Phi(x, s)) ds,$$

the cumulative risk up to t starting from x .

Finally, the Markov kernel Q on $(\mathcal{B}(E), \bar{E})$ selects the new location after each jump. We also set

$$Q_{\mathbf{m}}(\cdot | \mathbf{x}) = Q(\cdot | x),$$

for $x = (\mathbf{m}, \mathbf{x})$ in accordance with other notations. It satisfies $Q(\{x\} | x) = 0$ so that a jump has to change the state of the process (either the mode, or the Euclidean variable or both).

Remark 2.2 *In between jumps, mode $\mathbf{m}$ remains constant and only $\mathbf{x}$ evolves through the flow. At jump times, both the mode and the Euclidean component may be allowed to change through the Markov kernel Q .*

Figure 2.1 provides a generic representation of a PDMP, offering a visual illustration of its dynamics. The PDMP dynamics can be described informally as follows: starting from some initial point $x \in E$, the motion of the process follows the deterministic flow $t \mapsto \Phi(x, t)$ until a first jump time T_1 , which may be deterministically triggered by the process reaching the boundary ∂E of E (e.g. time T_2 on Figure 2.1), or may occur randomly with hazard rate $t \mapsto \lambda(\Phi(x, t))$ (e.g. time T_1 on Figure 2.1). Thus, starting from x at time 0, the first jump time T_1 has the following distribution

$$\mathbb{P}_x(T_1 > t) = \mathbb{P}(T_1 > t | X_0 = x) = \exp\left(-\int_0^t \lambda(\Phi(x, s)) ds\right) \mathbb{1}_{t < t^*(x)}, \quad t \geq 0.$$

At T_1 the process jumps to a new point $x' = X_{T_1}$ selected with probability $Q(dx' | \Phi(x, T_1))$ (conditional on T_1), and the motion restarts from this new point as before and follows the flow until the next jump time T_2 . The sojourn time $T_2 - T_1$ until the next jump is drawn from the jump intensity and time to reach the boundary as before. At the jump time T_2 , the post-jump location is selected by the Markov kernel Q and so on iteratively, as described in

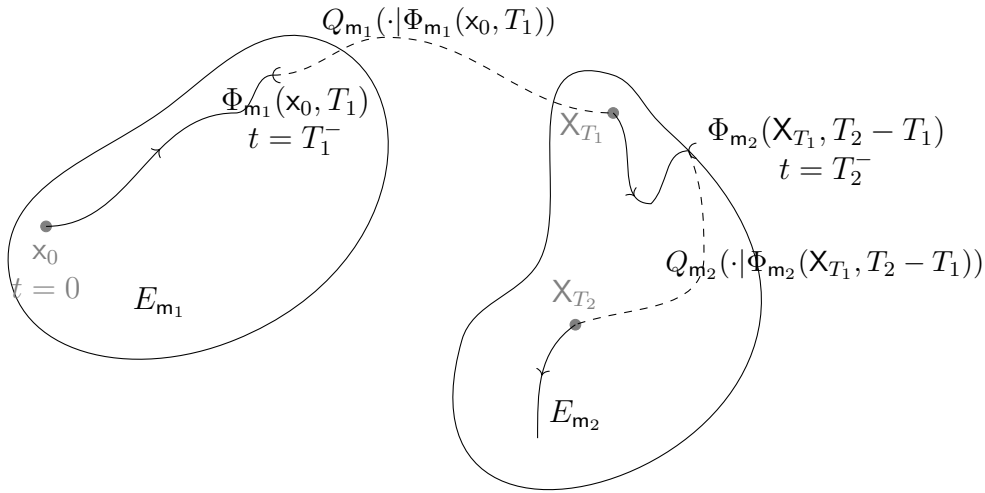


Figure 2.1: **Trajectory of a generic PDMP.** Starting from an initial value x_0 at time $t = 0$, the process follows a deterministic trajectory until a jump occurs, either at a random time (as at T_1) or because the process reaches the state boundary (as at T_2). At jump times, the process jumps to a new location drawn from kernel Q . *Adapted from [Cle+25]*

Algorithm 5. Note that Algorithm 5 can only simulate the process up to time $T_\infty = \lim_{n \rightarrow \infty} T_n$ that might be finite. This phenomenon is called explosion. To avoid any technical problems due to the possible explosion of the process, we make the following standard assumptions [Dav93, Section 24].

Assumption. The mean number of jumps up to $t \in \mathbb{R}_+$ is finite, whatever the initial position of the process: for all $x \in E$ and $t \in \mathbb{R}_+$, $\mathbb{E}_x[\sum_{n=1}^{\infty} \mathbb{1}_{T_n \leq t}] < \infty$. This implies in particular that $T_k \rightarrow \infty$ almost surely.

Algorithm 5 Iterative construction of a trajectory of a PDMP starting from state $x_0 = (m_0, x_0)$ at time 0 and up to the N -th jump time

- 1: $x \leftarrow x_0$
 - 2: $T \leftarrow 0$
 - 3: **for** $n \leftarrow 0$ to $N - 1$ **do**
 - 4: $S \sim \lambda(x)$
 - 5: $S \leftarrow \min\{S, t^*(x)\}$
 - 6: $X_t \leftarrow \Phi(x, t - T)$ for $T \leq t < T + S$
 - 7: $x' \sim Q(\cdot | \Phi(x, S))$
 - 8: $x \leftarrow x'$
 - 9: $T \leftarrow T + S$
 - 10: **end for**
-

Remark 2.3 A process X defined by a flow Φ , an intensity function λ and a kernel Q verifying the above properties is well defined and is strong Markov, as is shown, for instance, in [Dav93] or [Jac06].

Example 2.1.1: Squirrel toy model as a PDMP

Over the course of a week, you observe a squirrel moving from hiding place to hiding place as described in Example 1.1.1. The squirrel now transitions between hiding places in continuous time. Let $m \in \{1, \dots, 4\}$ be the set of hidings places. The state of the squirrel is its location $x = m$. This example describes a PDMP with a purely discrete state space and no Euclidean variable where $\mathbf{m} = m$. The PDMP associated to this example is decribed by

- $E = \{1, \dots, 4\}$
- $\Phi(m, t) = m$
- $\lambda(m) = \sum_{m' \neq m} f(m, m')$
- $Q(m'|m) = \frac{f(m, m')}{\lambda(m)}$

Here $f(m, m')$ corresponds to the intensity to jump from hiding place m to hiding place m' .

Remark 2.4 *When the flow is constant, as in the previous example 2.1.1, the jump intensity also has to be constant, i.e. it corresponds to a memory-less exponential distribution. In this case, as in many other practical examples, see e.g. [BL08], the exponential distribution may not be realistic to model the underlying phenomenon. However allowing for more general distributions breaks the Markov property and falls into the class of semi-Markov processes.*

2.1.2 Semi-Markov Dynamics

It is especially easy to encompass semi-Markov dynamics into the PDMP framework by enlarging the state space. This new process is sometimes called the time-augmented process, see [Dav93, Section 31.5] for a slightly different definition in the same spirit.

Definition 2.1.2: Time-augmented PDMP

A time-augmented Piecewise Deterministic Markov Process is a PDMP defined by the tuple $\langle E, \Phi, \lambda, Q \rangle$, where

- The state space E has the specific form

$$E = \bigcup_{\mathbf{m} \in \mathbf{M}} \{\mathbf{m}\} \times E_{\mathbf{m}} \times \mathbb{R}_+,$$

for some finite mode set $\mathbf{M}$, and where for all $\mathbf{m} \in \mathbf{M}$, $E_{\mathbf{m}}$ is some Borel subset of $\mathbb{R}^{d_{\mathbf{m}}}$.

- The flow Φ has the specific form

$$\Phi(x, t) = (\mathbf{m}, \Phi_{\mathbf{m}}(\mathbf{x}), u + t),$$

for $x = (\mathbf{m}, \mathbf{x}, u) \in E$, where $\Phi_{\mathbf{m}}$ are flows from $E_{\mathbf{m}} \times \mathbb{R}_+$ onto $E_{\mathbf{m}}$ satisfying the semi-group property.

- The jump intensity just satisfies the conditions from definition 2.1.1 and is allowed to depend on all coordinates of $x \in E$.
- The jump kernel Q has the specific form

$$Q(\{m'\} \times A \times B | x) = \mathbb{1}_{0 \in B} Q_{\mathbf{m}}(A | \mathbf{x}, u),$$

for $x = (\mathbf{m}, \mathbf{x}, u) \in E$, A any Borel subset of $E_{\mathbf{m}}$, B any Borel subset of $\mathbb{R}_+$, and where $Q_{\mathbf{m}}$ is a Markov kernel on $(\mathcal{B}(E_{\mathbf{m}}), \bar{E}_{\mathbf{m}})$.

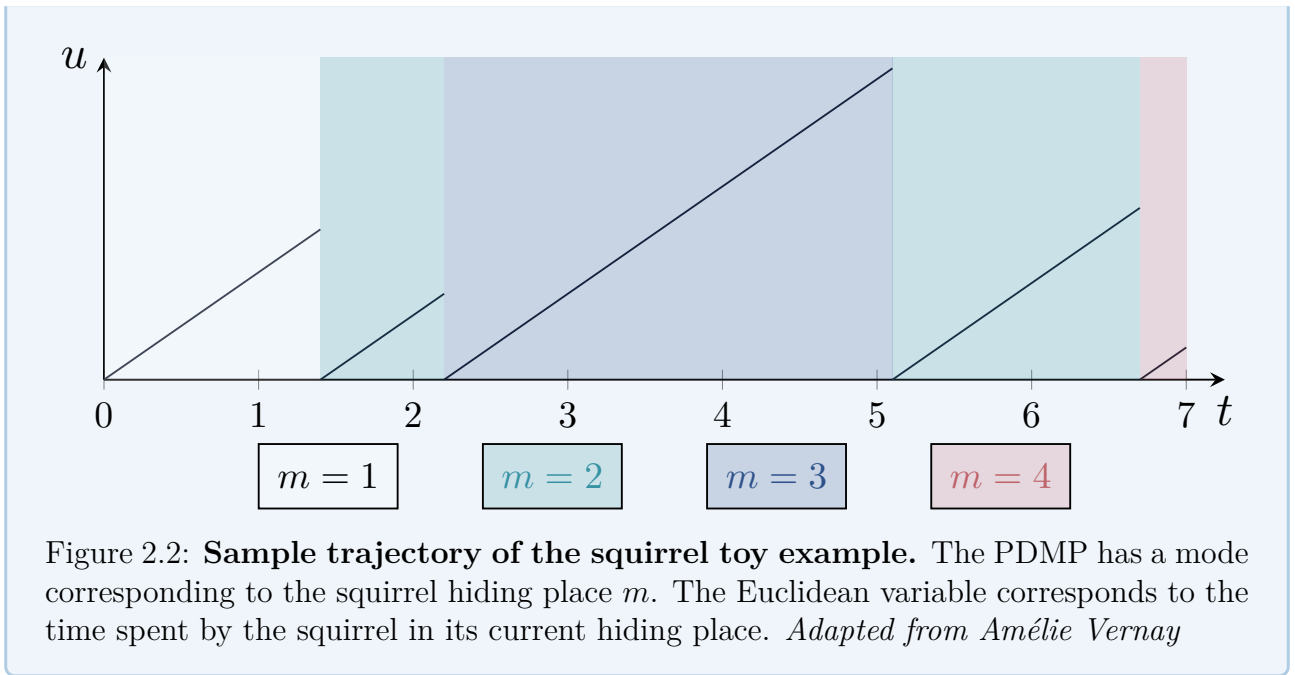
The additional component u can be interpreted as the time elapsed since the last jump. It moves along the flow at speed 1 and is reset to 0 at every jump.

Example 2.1.2: Squirrel toy model as a time-augmented PDMP

The setting builds on the example 2.1.1. Over the course of a week, you observe a squirrel moving from hiding place to hiding place. Let $u \in [0, 7]$ be the time spent by the squirrel in its current hiding place. The state of the squirrel is now composed of two components $x = (m, u)$. The PDMP associated to this example is described by

- $E = \{1, \dots, 4\} \times [0, 7]$
- $\Phi(x = (m, u), t) = (m, u + t)$
- $\lambda(m, u) = \sum_{m' \neq m} f(m, m', u)$
- $Q(\{m'\} \times A | x = (m, u)) = \mathbb{1}_{0 \in A} \frac{f(m, m', u)}{\lambda(m, u)}$

Here $f(m, m', u)$ corresponds to the intensity to jump from hiding place m to hiding place m' if the squirrel has spent time u in m . A trajectory of the associated PDMP is illustrated in Figure 2.2.



Remark 2.5 One could define more sophisticated time-augmented processes where either u is never reset to 0 (it is thus the time since the beginning, as in [Dav93, section 31.5]) or resets less frequently than at each jump.

2.1.3 Simulation

A very interesting property of PDMPs is that they are particularly easy to simulate due to their intrinsic iterative construction as seen in Algorithm 5.

Remark 2.6 Computing the trajectory along the flow (Alg. 5, line 6) is straightforward if the flow is explicit. If it is only known as the solution to a system of ODEs, one just needs to use any suitable ODE solver.

Remark 2.7 Computing the post-jump location from the kernel Q (Alg. 5, line 7) is also usually straightforward. In most applications, the kernel only changes the mode and leaves the Euclidean variable unchanged, or changes the Euclidean variable through some known mapping depending only on its value just before the jump.

The only challenging step is then to simulate the jump dates from the non constant intensity (Alg. 5, line 4). We present here an exact simulation method called the Stochastic Simulation Algorithm (SSA), or Gillespie algorithm [Gil77] that is especially suitable for PDMPs. This algorithm will be used in all the examples presented to simulate jump dates. The interested reader may consult [Coc97; SDZ16] for an alternative algorithm to simulate only sojourn-times and post-jump locations instead of full trajectories, and [Rie13; Vel15] and references therein for a broader discussion of alternative exact and approximate simulation methods.

The SSA is based on the Markov property and the following property of the minimum of independent exponential distributions.

Lemma 2.1.1 *Let S_1 and S_2 be two independent random variables with respective (non-constant) intensity functions λ_1 and λ_2 . Set $S = \min\{S_1, S_2\}$ and $I = \arg \min\{S_1, S_2\}$. Then the intensity of S is $\lambda_1 + \lambda_2$ and conditionally to $S = t$, the distribution of I is $(\frac{\lambda_1(t)}{\lambda_1(t) + \lambda_2(t)}, \frac{\lambda_2(t)}{\lambda_1(t) + \lambda_2(t)})$.*

Proof. First, notice that if S_i has intensity λ_i , then its density function is $\lambda_i(t)e^{-\int_0^t \lambda_i(s)ds}$. Using the independence between S_1 and S_2 , one obtains the joint distribution of (S, I) as follows

$$\begin{aligned} \mathbb{P}(S > t, I = 1) &= \mathbb{P}(S_1 > t, S_2 > S_1) \\ &= \int_t^\infty \lambda_1(x) e^{-\int_0^x \lambda_1(s)ds} \int_x^\infty \lambda_2(y) e^{-\int_0^y \lambda_2(s)ds} dy dx \\ &= \int_t^\infty \lambda_1(x) e^{-\int_0^x \lambda_1(s)ds} e^{-\int_0^x \lambda_2(s)ds} dx \\ &= \int_t^\infty \frac{\lambda_1(x)}{\lambda_1(x) + \lambda_2(x)} (\lambda_1(x) + \lambda_2(x)) e^{-\int_0^x (\lambda_1(s) + \lambda_2(s))ds} dx. \end{aligned}$$

Hence the density of S is $(\lambda_1(x) + \lambda_2(x))e^{-\int_0^x (\lambda_1(s) + \lambda_2(s))ds}$, and

$$\mathbb{P}(I = 1 | S = t) = \frac{\lambda_1(t)}{\lambda_1(t) + \lambda_2(t)},$$

and the result follows. $\square$

The principle underlying the SSA algorithm is as follows. The aim is to sample from a random variable S_1 with (non-constant) intensity λ_1 . Suppose that λ_1 is bounded from above by some constant $\bar{\lambda}$ and let S be random variable with (regular) exponential distribution with parameter $\bar{\lambda}$, so that is is very easy to sample from S . In view of Lemma 2.1.1, S can be interpreted as the minimum of S_1 and some virtual extra random variable S_2 with intensity $\lambda_2 = \bar{\lambda} - \lambda_1$. Let t be a sample value from S , then t can be considered as a value sampled from S_1 with probability $\frac{\lambda_1}{\bar{\lambda}}$. This simple fact leads to a rejection algorithm where one repeatedly samples from S until the result is accepted as a sample from S_1 . In a Markov context, it is even more efficient as by construction $S_1 \geq S$. Hence all samples are used, some are accepted as jump times (boundary jump: Alg. 6 line 7 or random jump: Alg. 6 line 15), the others just move the process forward without creating a jump (Alg. 6 line 17). The procedure then restarts from the new location, as detailed in Algorithm 6.

Remark 2.8 *The choice of the upper bound $\bar{\lambda}$ can have a strong impact on the speed of the algorithm. If it is too high, it will keep proposing very short times and move forward extremely slowly. If at all possible, one should choose a tight upper bound for the intensity.*

2.1.4 Embedded Chains

As for any continuous-time Markov process, one can define many different discrete-time Markov chains embedded in a PDMP. Some of them are particularly interesting either for theory or applications and are specified below.

Algorithm 6 Stochastic Simulation Algorithm for a PDMP starting from state $x_0 = (\mathbf{m}_0, \mathbf{x}_0)$ at time 0 and up to time horizon H

```

1: Input:  $\bar{\lambda}$ 
2:  $x \leftarrow x_0$ 
3:  $t \leftarrow 0$ 
4: while  $t < H$  do
5:    $S \sim \text{Exp}(\bar{\lambda})$ 
6:    $X_{t+s} \leftarrow \Phi(x, s)$  for  $0 \leq s < \min\{S, t^*(x)\}$ 
7:   if  $S > t^*(x)$  then
8:      $x' \sim Q(\cdot | \Phi(x, t^*(x)))$ 
9:   else
10:     $U \sim U[0, 1]$ 
11:    if  $S \geq H - t$  then
12:       $x' \leftarrow \Phi(x, H)$ 
13:       $t' \leftarrow H$ 
14:    else
15:      if  $U \leq \frac{\lambda(S)}{\bar{\lambda}}$  then
16:         $x' \sim Q(\cdot | \Phi(x, S))$ 
17:      else
18:         $x' \leftarrow \Phi(x, S)$ 
19:      end if
20:    end if
21:  end if
22:   $x \leftarrow x'$ 
23:   $t \leftarrow t + \min\{S, t^*(x)\}$ 
24: end while
    
```

Canonical chain The canonical chain is the most natural Markov chain embedded in a PDMP. It corresponds to the post-jump-locations and inter-jump-times chains. Let us define $Z_0 = X_0$ and $S_0 = 0$, and then iteratively, for all $n \in \mathbb{N}^*$, $Z_n = X_{T_n}$ the value of the PDMP at the n^{th} jump time and $S_n = T_n - T_{n-1}$ the inter-jump time between the $(n-1)^{\text{th}}$ and n^{th} jumps. Then $\Theta_n = (Z_n, S_n)_{n \in \mathbb{N}}$ is a Markov chain called the canonical embedded chain associated to the PDMP process X . Its transition kernel $P^{Z,S}$ is defined by, for any Borel subsets A of E and B of $\mathbb{R}_+$,

$$\begin{aligned}
 P^{Z,S}(A \times B | x, t) &= P^{Z,S}(A \times B | x) \\
 &= \int_0^{t^*(x)} \lambda(\Phi(x, s)) e^{-\Lambda(x,s)} Q(A | \Phi(x, s)) \mathbb{1}_B(s) ds + e^{-\Lambda(x, t^*(x))} \\
 &\quad Q(A | \Phi(x, t^*(x))) \mathbb{1}_B(t^*(x)),
 \end{aligned}$$

where the first part of the righthand side corresponds to random jumps, while the second part corresponds to deterministic jumps.

As the motion of the PDMP is deterministic between jumps, it is straightforward to reconstruct the full trajectory of the continuous-time PDMP from the knowledge of $(\Theta_n)_{n \in \mathbb{N}}$

as

$$X_t = \Phi \left(Z_n, t - \sum_{i=0}^n S_i \right) \quad \text{if } t \in \left[T_n = \sum_{i=0}^n S_i, T_{n+1} = \sum_{i=0}^{n+1} S_i \right),$$

with the convention $T_0 = S_0 = 0$. Thus, the canonical chain contains all the information of the process and is especially useful to obtain recursive formulations based on the Markov property. It is also at the heart of several numerical approximations for PDMPs, see *e.g.* [SDZ16] and will be largely used in Chapter 4.

For some applications, especially in inference and MCMC frameworks, this canonical chain can be extended with the pre-jump locations $\Phi(Z_{n-1}, S_n)$. In some numerical procedures, it can also be extended with the jump times T_n . All these chains are especially interesting as they capture all the randomness of the process and have explicit transition kernels with a relatively simple form depending on the local characteristics of the PDMP.

Skeleton chains Other common embedded chains are the skeleton chain defined by sampling the PDMP on a deterministic (but not necessarily homogeneous) grid $(t_1, \dots, t_n)$. We simply denote $X_n = X_{t_n}$, which once again is a (time inhomogeneous) Markov chain with kernels $P_{t_{n+1}-t_n}$.

While kernels $P_{t_{n+1}-t_n}$ might still have explicit forms, unless constraints are added on the PDMP there is no a priori limit in the number of jumps occurring between two time-points t_n and t_{n+1} , which prevents a generic formulation of the kernels. Moreover, unless jump-points are observed, it is not possible to reconstruct the original PDMP process from the observation of X_n . However, in many application settings, X_n might be the only available information from the process, see for instance the running medical example developed in Chapter 3.

Augmented canonical chain Interestingly, in [CD08], to improve comparisons of the canonical chain to the original process when jumps are scarce, the authors combine the canonical chain with a skeleton chain where the grid is randomly chosen. In this setting, the observations of the skeleton chain are constrained to occur in between jump times on a time-frame defined by an independent Poisson process of rate 1. Among other properties, this allows to translate necessary and sufficient conditions on the PDMP regarding reducibility, recurrence, existence and convergence to invariant measures to the resulting discrete-time Markov chain for which assumptions are much easier to verify.

2.2 Impulse Control for PDMPs

In this section, we delve into the control of PDMPs. A stochastic control problem involves influencing the dynamics of a process through control variables or decisions to either maximize a reward or minimize a cost. The primary questions of interest include the characterization and regularity properties of the value function—representing the optimal reward or cost achievable across all possible decisions or decision sequences—and the existence and properties of optimal decision sequences, called strategies. We briefly outline the two principal classes of control

problems for PDMPs in Section 2.2.1. Our focus then shifts to impulse control problems.

2.2.1 Continuous versus Impulse Control

There are two types of control for PDMPs: continuous control and impulse control, see [Dav93, Chapter 4 and 5].

Definition 2.2.1: Continuous control

Continuous control describes situations in which the control variable is selected at all times and acts on the process through the characteristics $(\Phi, \lambda, \text{ and } Q)$ by influencing the deterministic motion and the jump mechanism.

In continuous control problems, the local characteristics of the PDMP, the flow Φ , the jump intensity λ and the transition kernel Q , depend on an additional control parameter that is selected by the controller at each instant of time. Such problems have been studied in the literature mostly under the stationarity condition by considering two strategies, the resolution of an integro-differential optimality inequality, the so called Hamilton-Jacobi-Bellman (HJB) equation (see for instance [DY92; DY96]), or the translation into a discrete-time Markov Decision Process (MDP) on a functional space (see [Alm01; BR11; FSS04; Sch98]). Continuous control problems where the process is not required to reach stationarity have also received some attention (see *e.g.* [CD10]) but remain seldom studied.

Definition 2.2.2: Impulse control

An impulse control problem involves a controller making discrete interventions at chosen times to move the process to a new controller-chosen point in the state space.

The simplest form of impulse control is optimal stopping where then controller selects a single date at which the process is stopped. Impulse control problems for PDMPs have been studied from the theoretical point of view: characterization of the value function and optimal policies through dynamic programming in *e.g.* [Gug86; CD88; CSJ20] for optimal stopping and [CD89; SDG17] for the general impulse control problem. Numerical schemes to approximate the value function and compute explicit strategies close to optimality have also been proposed in [CD89; SDZ16].

2.2.2 Definition of an Impulse Control Problem

The formal probabilistic apparatus necessary to rigorously define the impulse control problem for PDMPs is rather technical, and will not be used in the sequel, therefore, for the sake of simplicity, we only present an informal description of the problem. The interested reader is referred to [CD89] or [DHP16] for a proper definition.

Definition 2.2.3: Impulse strategy

A general impulse strategy $\mathcal{S} = (\tau_n, \chi_n)_{n \geq 1}$ is a sequence of non-anticipative $\mathbb{R}_+$ -valued intervention times τ_n and $\mathbb{U} \subset E$ -valued non-anticipative random variables χ_n .

The set of admissible strategies is denoted by $\mathbb{S}$.

The trajectory of the PDMP controlled by strategy $\mathcal{S}$ is described in Algorithm 7. Between intervention dates, the PDMP follows its natural trajectory. At the n -th impulsions (line 12), the process is instantaneously moved to the new starting point χ_n and restarts from there.

Algorithm 7 Simulation of a trajectory of a PDMP controlled by strategy $\mathcal{S}$ starting from state x_0 up to the n -th intervention

```

1: Input:  $\mathcal{S} = (\tau_n, \chi_n)_{n \geq 1}$ 
2:  $x \leftarrow x_0$ 
3:  $T \leftarrow 0$ 
4: for  $n \leftarrow 1$  to  $N$  do
5:   while  $T < \tau_n$  do
6:      $S \sim \lambda(x)$ 
7:      $S \leftarrow \min\{S, t^*(x)\}$ 
8:      $X_t \leftarrow \Phi(x, t - T)$  for  $T \leq t < T + S$ 
9:     if  $S < \tau_n - T$  then
10:       $x \sim Q(\cdot | \Phi(x, S))$ 
11:       $T \leftarrow T + S$ 
12:   else
13:      $x \leftarrow \chi_n$ 
14:      $T \leftarrow \tau_n$ 
15:   end if
16: end while
17: end for
    
```

Definition 2.2.4: Cost of an impulse strategy

The cost of strategy $\mathcal{S}$ for a PDMP starting at x at time 0 and up to infinity is defined by

$$\mathcal{V}(\mathcal{S}, x) = \mathbb{E}_x^{\mathcal{S}} \left[\int_0^{+\infty} e^{-\gamma t} c_R(X_t) dt + \sum_{n=1}^{\infty} c_I(X_{\tau_n}, X_{\tau_n^+}) \right],$$

where $\gamma \geq 0$ is a discount factor, c_R is the running cost and c_I is the impulse cost.

Definition 2.2.5: Value function

The value function of strategy $\mathcal{S}$ for a PDMP starting at x at time 0 is defined by

$$\mathcal{V}^*(x) = \inf_{\mathcal{S} \in \mathbb{S}} \mathcal{V}(\mathcal{S}, x).$$

In most impulse control problems for PDMPs, there is no optimal strategy as the infimum

may not be reached. We define instead ϵ -optimal policies.

Definition 2.2.6: ϵ -optimal strategy

An ϵ -optimal strategy $\mathcal{S}_\epsilon$ is an admissible strategy satisfying

$$\mathcal{V}(\mathcal{S}_\epsilon, x) \leq \mathcal{V}^*(x) + \epsilon,$$

for all $x \in E$.

Under suitable assumptions (see again [CD89] for details), the value function can be characterized as the unique solution of some dynamic programming equations.

Theorem 2.2.1 (Dynamic programming) *The optimal value function $\mathcal{V}^*$ is limit of the following recursion. Set $\mathcal{V}_0^*(x) = h(x)$ and $\mathcal{V}_{n+1}^*(x) = \mathcal{L}(\mathcal{V}_n^*)(x)$ for all $n \geq 0$, then for all x in E , one has*

$$\mathcal{V}^*(x) = \lim_{n \rightarrow +\infty} \mathcal{V}_n^*(x),$$

where $h(x)$ is the cost associated to the no-impulse strategy $\mathcal{S}_\emptyset = (\tau_1 = +\infty)$ starting from x

$$h(x) = \mathbb{E}_x^{\mathcal{S}_\emptyset} \left[\int_0^\infty e^{-\gamma t} c_R(X_t) dt \right].$$

For $v, w \in \mathbf{L}_\Phi(E)^2$ and φ defined on $\mathbb{U}$ set $\mathcal{L}$ the single-jump-or-intervention operator defined as

$$\mathcal{L}w(x) = L(Mw, w)(x),$$

with

$$\begin{aligned} L(v, w)(x) &= \inf_{t \in [0, t^*(x)]} J(v, w)(x, t) \wedge Kw(x), \\ M\varphi(x) &= \inf_{\chi \in \mathbb{U}} \{c_I(x, \chi) + \varphi(\chi)\}, \\ J(v, w)(x, t) &= \int_0^{t \wedge t^*(x)} e^{-\gamma s - \Lambda(x, s)} [c_R(\Phi(x, s)) + \lambda Qw(\Phi(x, s))] ds \\ &\quad + e^{-\gamma(t \wedge t^*(x)) - \Lambda(x, t \wedge t^*(x))} v(\Phi(x, t \wedge t^*(x))) \\ &= F(x, t) + \mathbb{E}_x[e^{-\gamma(t \wedge t^*(x))} v(\Phi(x, t \wedge t^*(x))) \mathbb{1}_{S_1 \geq t \wedge t^*(x)} + e^{-\gamma S_1} w(Z_1) \mathbb{1}_{S_1 < t \wedge t^*(x)}] \\ Kw(x) &= \int_0^{t^*(x)} e^{-\gamma s - \Lambda(x, s)} [c_R(\Phi(x, s)) + \lambda Qw(\Phi(x, s))] ds \\ &\quad + e^{-\gamma t^*(x) - \Lambda(x, t^*(x))} Qw(\Phi(x, t^*(x))) \\ &= F(x, t^*(x)) + \mathbb{E}_x[e^{\gamma S_1} w(Z_1)]. \\ F(x, t) &= \int_0^{t \wedge t^*(x)} e^{-\gamma s - \Lambda(x, s)} c_R(\Phi(x, s)) ds \\ &= \mathbb{E}_x \left[\int_0^{T_1 \wedge t} e^{-\gamma s} c_R(\Phi(x, s)) ds \right] \end{aligned}$$

Operator M selects the best starting point χ in the control set $\mathbb{U}$ after an intervention. Note that the control set could also depend on the position x before the intervention. Operator $\inf_t J$ selects the best intervention date along the flow, while operator K accounts for cases when it is best to wait for the next natural jump of the process before triggering an intervention. The iterations in the dynamic programming equation thus correspond either to natural jumps of the process or to interventions, hence the name single-jump-or-intervention operator. Dynamic programming is based on the Markov property for the canonical embedded chain, and operators can be reformulated as expectations.

The iterate $\mathcal{V}_n^*$ can be interpreted as the optimal value function of the impulse control problem where at most n jumps or interventions are allowed, and then strategy $\mathcal{S}_0$ is applied. Theorem 2.2.1 is also valid when replacing $h(x)$ by any function g such that $h(x) \leq g(x)$ for all $x \in E$.

Finally, we have stated the results for an infinite horizon only. If one is interested in controlling the process up to a finite time horizon H , one just needs to kill the process, or send it to a cemetery state, when time H is reached, and define a null cost on this state to fall back to the infinite horizon framework. The discount factor γ can be set to 0 in this context as the interval of integration is finite. Hence, the cost of strategy $\mathcal{S}$ can typically be defined, in the fixed horizon H , by

$$\mathcal{V}_H(\mathcal{S}, x) = \mathbb{E}_x^{\mathcal{S}} \left[\int_0^H c_R(X_t) dt + \sum_{\tau_n < H} c_I(X_{\tau_n}, X_{\tau_n}^+) + c_T(X_H) \right],$$

where c_T corresponds to some terminal cost function.

Example 2.2.1: Squirrel toy model as a impulse control problem

We revisit the squirrel example 2.1.2. Now, you can intervene and relocate the squirrel to a new hiding place and reset its local state. The objective is to manage the squirrel's movements to reduce risks of predator exposure and maintain its safety by strategically deciding when and where to intervene.

We extend the model to include an absorbing captured state, $m = 5$, representing the case where the squirrel is caught by a predator. The PDMP associated with this example is defined by

- The state space is defined as $E = \{1, \dots, 4\} \times [0, 7] \cup \{5\}$.
- $\Phi(x, t) = (m, u + t)$ if $m \neq 5$ and $\Phi(x, t) = 5$ if $m = 5$.
- $\lambda(m, u) = \sum_{m'=1}^4 f(m, m', u) + f_{m'=5}(m, u)$, where $f(m, m', u)$ is the intensity of the jump from m to m' and $f_{m'=5}(m, u)$ is the intensity of being caught by a predator.
- $Q(m' \times A | x) = \mathbb{1}_{0 \in A} \mathbb{1}_{m' \neq 5} \frac{f(m, m', u)}{\lambda(m, u)} + \mathbb{1}_{m'=5} \frac{f(m, u)}{\lambda(m, u)}$.

A strategy $\mathcal{S} = (\tau_n, \chi_n)_{n \geq 1}$ is then a sequence of (continuous) dates at which you change the squirrel location: an impulsion moves the process from $x = (m, u)$ to $\chi = (m', 0)$. A simple strategy is to move the squirrel every 24 hours to the next closest hiding place. We illustrate a sample of controlled trajectory on Figure 2.3.

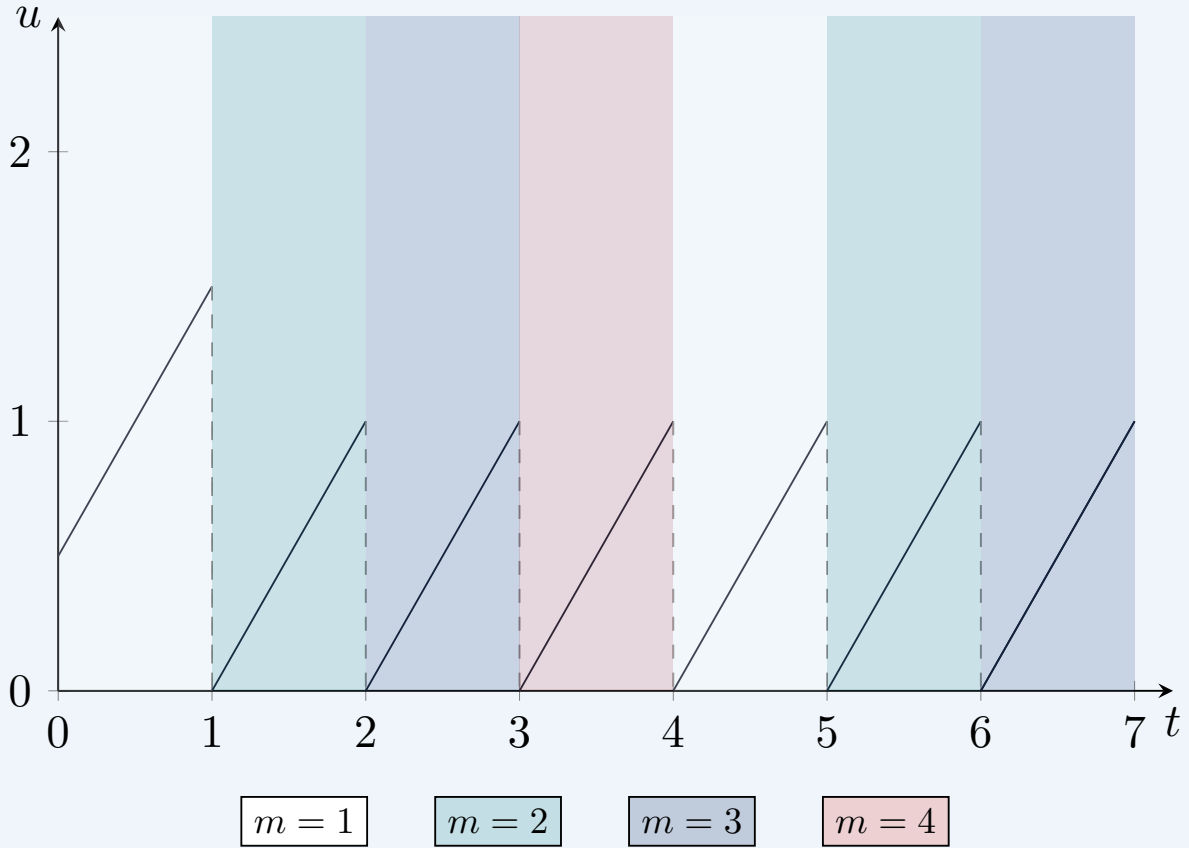


Figure 2.3: **Sample of controlled trajectory of the squirrel toy example.** The PDMP has a mode corresponding to the squirrel's hiding place m . The Euclidean variable represents the time spent by the squirrel in its current hiding place. This figure shows an example trajectory simulated with the strategy of moving the squirrel every 24 hours to the next closest hiding place on the left.

2.2.3 Resolution of Impulse Control Problems for PDMP

Optimal impulse control of PDMPs attracted attention almost as soon as the concept of PDMP was formalized by Davis. Contributions focusing only on the estimation of the value function are usually based on variational inequalities or value improvement and do not provide explicit strategies (see for instance [Len89; DY95; Gat92; Dem91; Ban18]). Theoretical results on the existence of optimal strategies not relying on the discretization of kernels also have been derived in the case of infinite horizon impulse control, but they do not provide explicit policies (see for instance [CD10; CD13]). In this work, we focus specifically on deriving explicit optimal policies. In the simpler context of optimal stopping (the decision-maker can only decide to

stop the process without restarting it from a new location) the literature exhibits strategies based on either the discretization of the PDMP Markov jump kernel Q [CD88], or on the discretization of the Markov kernel of the canonical chain [CD08; SDG10] (see Section 2.1.4 for the definition of the canonical chain). After discretization, the value function can be estimated via dynamic programming [Gug86] using the computable discretized kernels. Such approaches have been extended to the more general impulse control setting. For example, [Cos93] considers a discretization of the state space and time to obtain uniform convergence of the discretized problem to the original one. In Chapter 4, we introduce a new discretization-based numerical method tailored to our framework.

A promising direction of recent research is to recast PDMPs within the framework of Markov Decision Processes (MDPs), embedding impulse control problems into this setting [Cle+25]. This reformulation leverages the well-established MDP theory, offering new avenues for analyzing and solving complex PDMP control problems. To bridge the two frameworks, we restrict the class of impulse strategies available to the PDMP decision-maker. More general and less restrictive approaches, involving richer state spaces, are also possible; see [DHP16] for example. In Chapter 5 and Chapter 6, we introduce a new discretization-based numerical method tailored to our framework.

A very natural extension to the classical impulse control problem for PDMPs is to consider partial observation, ie to include hidden layers in the process. The literature is much scarcer for this problem. Constructive resolution approaches have been proposed in the case where jumps are still perfectly observed, but the post-jump locations are only measured through noise. In [BSD12], the authors consider the equivalent fully observed process on the filter space and apply the quantization approach of [SD12] on the filtered process. [BL18] reduce the problem to a discrete-time MDP and prove the existence of optimal policies, but provide no numerical approximation of the value function or optimal strategies. In the most challenging case where jump dates are unobserved, A. Cleynen and B. de Saporta have proposed a framework to construct an explicit strategy close to optimality in the optimal stopping context [CS18] and for more general impulse problems [CS21; Sap+24]. Those approaches are based on a conversion of the impulse problem into a Partially Observed MDP (POMDP), as detailed in Chapter 5.

2.3 Conclusion

In this chapter, we introduced the impulse control problem for PDMPs and laid the groundwork for its analysis and resolution. PDMPs provide a powerful framework for modeling continuous-time decision problems with a relatively simple structure, as they require few parameters while still capturing realistic system dynamics. Given the complexity of impulse control for PDMPs, our objective is not only to find a solution but also to compute an ϵ -optimal strategy that can be implemented in practice.

To achieve this, we propose several complementary numerical methods, which will be presented in the following chapters:

- In Chapter 4, we introduce a discretization-based numerical method that extends existing approaches in the literature. This method is designed for fully observed PDMPs with known dynamics.
- In Chapter 5, tackles the more complex setting of partially observed PDMPs. Here, we reformulate the impulse control problem within the POMDP framework to account for noisy observations available to the decision-maker.
- In Chapter 6, explores scenarios involving uncertainty in PDMP dynamics. We address this by adopting the Bayes Adaptive framework.

All these solution methods will be illustrated through a medical case study presented in Chapter 3.

Chapter 3

Case Study: Myeloma Cancer Patient Follow-Up

3.1	Myeloma Cancer Patient	69
3.1.1	Disease Dynamics	70
3.1.2	Decisions	70
3.2	Modeling Patient Dynamics	71
3.2.1	Construction of a Controlled Process	71
3.2.2	Local Characteristics	73
3.2.3	Transition Kernel	74
3.3	Conclusion	76

Multiple myeloma is a cancer that affects the antibody-producing plasma cells in the bone marrow. The disease often progresses in cycles of remission and relapse. Effective management requires a dedicated balance between the intensity of treatment and the frequency of medical examinations. Over-aggressive treatment can lead to drug resistance and, ultimately, death. Clinicians primarily monitor the progression of multiple myeloma using blood biomarkers, which provide crucial indicators of disease status and response to treatment [Soz+01; HHB07; Ger+16]. Through careful interpretation of these biomarkers, healthcare providers can adjust treatment strategies to manage disease more effectively over time.

Additionally, disease progression varies from patient to patient, making personalized treatment strategies invaluable [Sch+03; Sha09; Nag+16]. Personalized treatment strategies adapt follow-up to the dynamics of each patient's disease [KL19]. By tailoring care to individual characteristics, this approach seeks to reduce unnecessary treatments and support early relapse detection. However, it demands a detailed understanding of disease dynamics and reliable predictive algorithms for timely adjustments.

Cancer monitoring and treatment should ideally be based on empirical evidence. Clinical trials are the gold standard for assessing the effectiveness of cancer treatments. They are

complex to set up and require considerable human, material and financial resources. In addition, they cannot test all possible combinations of treatments and scenarios. This limitation hinders the development of personalized treatments, adapted to individual variations in the disease. Moreover, it can be complicated to fully capture the long-term effects of treatments on patients' lives, which limits the ability of trials to provide an overview of the efficacy and implications of treatments over time. In recent years, mathematical models have been developed to simulate the dynamics of cancer with treatment [ALM15; Yin+19; XC22; Wei23].

Game theory has become an increasingly popular framework for modelling cancer dynamics. Game-theoretic models approach cancer as a competitive process, where cancer cells compete for resources, space, and survival, often evolving strategies that resist treatment and evade the immune response [Zha+19; Sta+19; Wöl+21; JGN23]. This approach holds particular promise for personalized medicine, where treatment can be tailored based on the unique strategies that a patient's cancer cells employ. However, a key limitation of many game-theoretic models is their deterministic nature, assuming fixed outcomes based on specific strategies without accounting for the inherent randomness in cancer progression and treatment response.

In reality, the dynamics of cancer are stochastic, with unpredictable changes in cancer cell development, mutation rates and responses to treatment. To better capture these uncertainties, we propose the use of Piecewise Deterministic Markov Processes (PDMPs), which can model both the deterministic and stochastic elements of disease progression. PDMP modelling also allows the construction of a universal model with patient-specific parameters to comprehensively capture the outcomes of clinical decisions.

Our case study is based on the analysis of a large cohort of Multiple Myeloma (MM) patient data from the Intergroupe Francophone du Myélome (IFM) 2009 clinical trial [ALa17]. The data includes regular clinical and lab measurements to monitor disease progression and treatment response. We use these data to model patient outcomes. Although some disease mechanisms are simplified, such as treatment options and stochasticity sources, data allows parameter estimation and simulation of realistic disease trajectories.

This chapter has two main sections. First, the mechanisms of the disease are presented in detail, along with monitoring and treatment options though section 3.1. We then present the PDMP model associated with this problem in section 3.2. The construction of this model is based on previous work [Sap+24]. This structure serves as a foundation for the applications and analyses developed throughout the thesis.

3.1 Myeloma Cancer Patient

In the clinical trial from which our data originates, patients were monitored for up to 6 years. Medical visits are set up to measure the concentration of clonal immunoglobulins by a blood test. On the basis of this noisy biomarker measurements, the practitioner decides the date of the next visit and the treatment to apply. We begin by describing the dynamics of the disease, before detailing the possible decisions made by the practitioner. For further details on the

clinical dataset, interested readers can refer to [Sap+24; CSV25].

3.1.1 Disease Dynamics

The disease is considered to have four phases:

- the remission phase, during which the patient is cancer-free;
- the relapse phase, when the cancer develops but is responsive to treatment;
- the incurable relapse phase, during which the cancer develops and treatment is ineffective;
- the death of the patient.

In remission, the level of the biomarker remains at its baseline value. Without treatment the patient may randomly transition to one or other of the two relapses. The probability of this transition depends on the duration of remission and the number of past treatments. When treatment is applied, the level of the biomarker also remains at its nominal value, but the patient can only enter a state of therapeutic escape. The probability of escaping treatment increases with the number of treatments.

During a curable relapse without treatment, the level of the biomarker increases exponentially and can reach a critical level associated with death. Under treatment, however, the biomarker decreases exponentially and may return to its baseline level. In both cases, the patient may enter the therapeutic escape phase, this probability being influenced by the number of treatments and the level of the biomarker. In the incurable relapse phase, the level of the biomarker increases exponentially, with no effective treatment able to slow its progression. If it reaches the death threshold, the patient dies and the follow-up ends. This dynamics is summarized in Figure 3.1.

Note that the different phases of disease are hidden, except when the patient dies. Moreover, biomarker measurements are corrupted by noise and measurements are only collected at visit dates. An example of patient follow-up data is presented in Figure 3.2.

3.1.2 Decisions

For context, we assume that the study begins at time $t_0 = 0$, when the patient first goes into remission. The practitioner will plan the dates of visits over a period of six years, or until the patient dies, whichever comes first. The interval between visits is not fixed, so the number of visits and dates may vary from patient to patient. More specifically, at each visit, the practitioner can set the date for the next visit in 15, 30 or 60 days. This decision can be based on current and previous biomarker measurements.

In addition to setting the date of the next visit, the practitioner can choose to adjust the patient's treatment, by selecting the available option, which in this study is chemotherapy. However, certain constraints limit these choices. The practitioner cannot schedule visits beyond

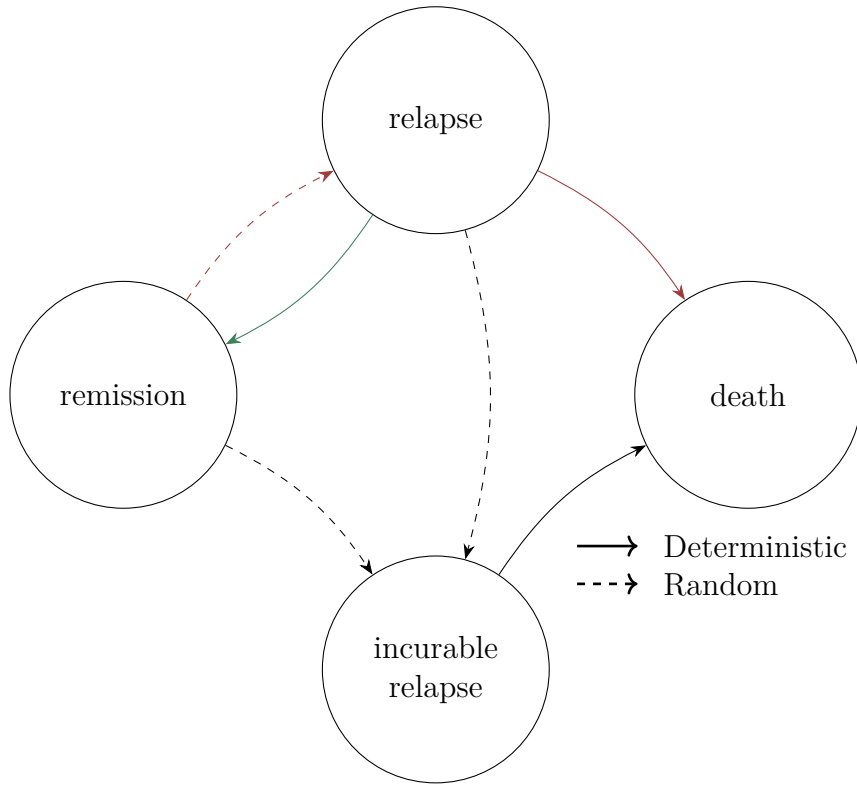


Figure 3.1: **Transitions between the different phases of the disease.** Dashed arrows represent transitions that occur at random times, while filled arrows indicate transitions that happen at deterministic times. Green arrows indicate transitions that can occur under treatment, while red arrows indicate transitions that can occur without treatment. Black arrows indicate transitions that can occur regardless of treatment status.

the six-year follow-up period or schedule a visit if the patient has died. There are also constraints on changing the treatment plan. At the first visit, the practitioner cannot initiate treatment, and after treatment has begun, he or she must wait a minimum of 45 days before making any further changes.

3.2 Modeling Patient Dynamics

A variation of this problem has previously been formulated as an impulse-control PDMP in previous work [Sap+24; CS21]. Here, we present the time-augmented controlled PDMP, as presented in definition 2.1.2, corresponding to the patient model introduced in the previous section.

All PDMP parameters were defined based on clinical considerations and selected to match the observed clinical characteristics of the patient cohort, rather than being directly estimated from it.

3.2.1 Construction of a Controlled Process

The state space E is defined as $E = \{x = (\mathbf{m}, \mathbf{x})\}$ where the mode is $\mathbf{m} = (h, \ell, k)$ with:

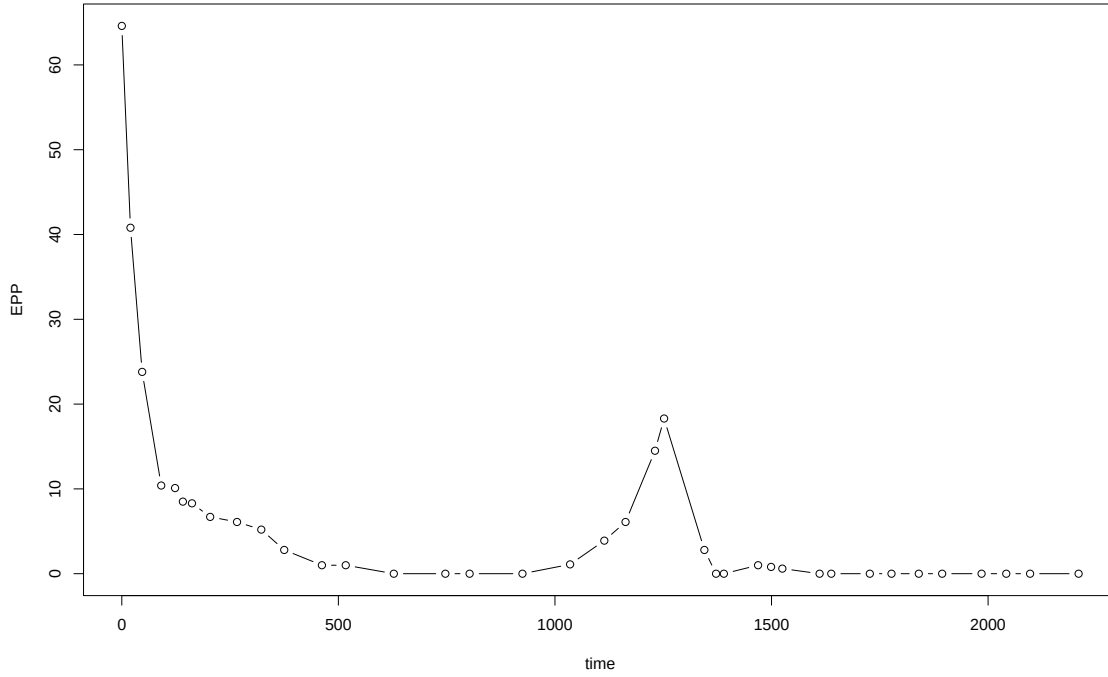


Figure 3.2: Example of patient follow-up data. Marker values are measured at each patient visits over a certain period of time. Data from the *Intergroupe Francophone du Myélome 2009 clinical trial*, courtesy of the *Centre de Recherche en Cancérologie de Toulouse*.

- $h \in \mathcal{H} = \{0, 1, 2, 3\}$ represents the health status ($h = 0$: remission, $h = 1$: relapse, $h = 2$: therapeutic escape, $h = 3$: death of the patient),
- $\ell \in L = \{0, 1\}$ is the patient current treatment ($\ell = 0$: no treatment, $\ell = 1$: chemotherapy),
- $k \in \{1, 2, \dots, K\}$ is the number of past treatments. The minimal lapse between visits is set to 15 days, hence the number of applied treatments is bounded by $K = \lfloor H/15 \rfloor$, where H is the follow-up time for a patient, 2400 days in our scenario.

The Euclidean variable is $\mathbf{x} = (\zeta, u, \tau) \in E_m$ where ζ is the biomarker level, u is the time spent in the current health status and τ the time since a treatment is applied. In addition we note ζ_0 the marker level of a patient in remission and when the marker level reaches D , the patient dies.

Remark 3.1 *The state space E_m does not depend on the time t since the beginning of the follow-up.*

Subsets E_m are defined as follows:

- $E_{(0,\ell,k)} = \{\zeta_0\} \times [0, H]^2$, $\forall \ell, k$: a patient in remission (health status $h = 0$) has a marker level of ζ_0 , whatever his current treatment and number of past treatments.

- $E_{(1,0,k)} = [\zeta_0, D) \times [0, H]^2$, $\forall k$: without treatment, a patient suffering a relapse (neither dead nor in remission) has a marker level $\zeta_0 \leq \zeta < D$, whatever the number of past treatments.
- $E_{(1,1,k)} = (\zeta_0, D) \times [0, H]^2$, $\forall k$: with treatment, a patient suffering a relapse (neither dead nor in remission) has a marker level $\zeta_0 < \zeta < D$, whatever the number of past treatments.
- $E_{(2,\ell,k)} = [\zeta_0, D) \times [0, H]^2$, $\forall \ell, k$: a patient suffering an incurable relapse (neither dead nor in remission) has a marker level $\zeta_0 \leq \zeta < D$, whatever the treatment and number of past treatments.
- $E_{(3)} = \{\partial\}$: a patient dies when the marker level reaches D (and reaches an arbitrary absorbing state, ∂), whatever the treatment and number of past treatments.

3.2.2 Local Characteristics

We explicit the three local characteristics (Φ, λ, Q) associated to the process $\{X_t\}$.

The flow. It depends on control applied and in particular on the treatment and number of treatments: $\Phi(x, t) = (\mathbf{m}, \Phi_{\mathbf{m}}(\zeta, t), u + t, (\tau + t)\mathbb{1}_{\ell=1})$, where $\Phi_{\mathbf{m}}(\zeta, t)$ describes only the trajectory of the biological marker between jumps. The biomarker evolution is summarized in Table 3.1. The case $m = 3$ is omitted since the process has reached the absorbing state ∂ .

Table 3.1: Flow $\Phi_{\mathbf{m}}$ of the controlled continuous time PDMP. For initial state $x = (h, \ell, k, \zeta, u, \tau)$ and duration t , $\Phi_{\mathbf{m}}(\zeta, t)$ is the patient biomarker level after a time t starting from state x at time 0 if no change of condition. Note that during a relapse under treatment ($h = 1$ and $\ell = 1$), the decrease in biomarker levels depends on k : the more treatments the patient has received, the less effective the current treatment becomes.

	$\ell = 0$	$\ell = 1$
$h = 0$	ζ_0	ζ_0
$h = 1$	$\zeta e^{v_1 t}$	$\zeta e^{-\frac{v_1}{k} t}$
$h = 2$	$\zeta e^{v_2 t}$	$\zeta e^{v_2 t}$

The jump intensity. It is also influenced by the treatment and $\lambda(x) = \lambda_{\mathbf{m}}(\mathbf{x})$ is summarized in Table 3.2. Notably, there are two distinctive types of relapse scenarios considered: standard relapses occurring during remission phases and relapses indicative of therapeutic escape. From remission, the probability of relapse occurrence increases with the duration of time spent in remission and the number of previous treatments. From standard relapse, the risk of relapses associated with therapeutic escape is influenced by the biomarker level and the number of previous treatments.

In light of these considerations, we choose Weibull distributions of the form: $\mu_{(0,\ell,k)}^{h'}(u) = (\alpha_m^{h'} u)^{\beta_m^{h'}}$ and $\mu_{(1,\ell,k)}^2(\zeta) = (\alpha^k \zeta)^{\beta}$. Details of jump intensity are available in Table 3.2.

Let $t^*(x)$ be the deterministic time the flow takes to reach the boundary of the state space E . Let $\partial E = \{1, 2\} \times \mathbb{N} \times \{\zeta_0, D\} \times (0, H]^2$ be the boundary on E . The time $t^*(x)$ also depends

Table 3.2: Jump intensity $\lambda_{\mathbf{m}}(\mathbf{x})$ of the controlled continuous time PDMP.

	$\ell = 0$	$\ell = 1$
$h = 0$	$(\mu_{\mathbf{m}}^1 + \mu_{\mathbf{m}}^2)(u)$	$\mu_{\mathbf{m}}^2(u)$
$h = 1$	$\mu_{\mathbf{m}}^2(\zeta)$	$\mu_{\mathbf{m}}^2(\zeta)$
$h = 2$	0	0

on the treatment, the number of past treatment and the disease regimen: $t_{\mathbf{m}}^*(\zeta) = \inf\{t > 0 : \Phi_{\mathbf{m}}(\zeta, t) \in \partial E\}$ (detailed in table 3.3).

Table 3.3: Deterministic time to reach the boundary of the state space $t_{\mathbf{m}}^*(\zeta)$ of the controlled continuous time PDMP.

	$\ell = 0$	$\ell = 1$
$h = 0$	$+\infty$	$+\infty$
$h = 1$	$\frac{1}{v_1} \log(\frac{D}{\zeta})$	$\frac{k}{v_1} \log(\frac{\zeta}{\zeta_0})$
$h = 2$	$\frac{1}{v_2} \log(\frac{D}{\zeta})$	$\frac{1}{v_2} \log(\frac{D}{\zeta})$

The Markov kernel. In remission, the patient may transition to either a standard relapse in the absence of chemotherapy or an incurable relapse. In the case of relapse and without treatment, the biomarker increases to a critical value D , leading to the patient's death. When chemotherapy is administered, the biomarker decreases to ζ_0 and returns to remission. Regardless of treatment chosen, therapeutic escape may occur at any time. In the case of therapeutic escape, the biomarker increases, regardless of the administered treatment, toward the D threshold, ultimately resulting in the patient's death. We define the Markov kernel Q in table 3.4. Case $h = 3$ is omitted as no jumps are allowed when patients are dead.

Table 3.4: Markov kernel $Qg(x)$ for $x = (\mathbf{m} = (h, \ell, k), \zeta, u, \tau) \in E$ and for all real-valued bounded measurable functions g on E .

	$\ell = 0$
$h = 0$	$g(1, 0, 0, \zeta_0, 0, 0) \frac{\mu_{\mathbf{m}}^1(u)}{(\mu_{\mathbf{m}}^1 + \mu_{\mathbf{m}}^2)(u)} + g(2, 0, k, \zeta_0, 0, 0) \frac{\mu_{\mathbf{m}}^2(u)}{(\mu_{\mathbf{m}}^1 + \mu_{\mathbf{m}}^2)(u)}$
$h = 1$	$g(2, 0, k, \zeta, 0, 0) \mathbb{1}_{D > \zeta} + g(3, \partial) \mathbb{1}_{\zeta = D}$
$h = 2$	$g(3, \partial) \mathbb{1}_{\zeta = D}$
	$\ell = 1$
$h = 0$	$g(2, 1, k, \zeta_0, 0, \tau)$
$h = 1$	$g(2, 1, k, \zeta, 0, \tau) \mathbb{1}_{\zeta > \zeta_0} + g(0, 1, k, \zeta_0, 0, \tau) \mathbb{1}_{\zeta = \zeta_0}$
$h = 2$	$g(3, \partial) \mathbb{1}_{\zeta = D}$

3.2.3 Transition Kernel

The transition kernel $\mathcal{P}$ can be written as a combination of the deterministic flow, the jump intensity and the Markov kernel such that for a time period r and all real-valued bounded measurable functions g on E , $\mathcal{P}g(x) = \mathbb{E}[g(X_r) | X_0 = x]$. It is assumed that in the event

of a relapse from the remission state ($m = 0$), the patient takes more than 60 days to die ($t_{(1,0,k)}^*(\zeta_0) > 60$ and $t_{(2,\ell,k)}^*(\zeta_0) > 60$), this implies $v_1 < 0.061$ and $v_2 < 0.061$. As this form of $\mathcal{P}$ is complex, we work case by case for $x = (\mathbf{m} = (h, \ell, k), \mathbf{x} = (\zeta, u, \tau)) \in E$.

- If $\mathbf{m} = (3)$, then $\mathcal{P}g(x) = g(3, \partial)$, the patient is dead.
- If $\mathbf{m} = (2, \ell, k)$, over a time step r the patient may either die or remain in therapeutic escape.

$$\mathcal{P}g(x) = g(3, \partial)\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) \leq r} + g(\Phi_{\mathbf{m}}(\mathbf{x}, r))\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > r}.$$

- If $\mathbf{m} = (1, 0, k)$, the biomarker increases. Over a time step r , the patient may die, remain in relapse, transition to a therapeutic escape and stay there, or transition to a therapeutic escape and die.

$$\begin{aligned} \mathcal{P}g(x) &= g(3, \partial)e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, t_{\mathbf{m}}^*(\zeta))}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) \leq r} + g(\Phi_{\mathbf{m}}(\mathbf{x}, r))e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, r)}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > r} \\ &\quad + \int_0^r g(2, 0, k, \Phi_{(2,0,k)}(\Phi_{\mathbf{m}}(\zeta, s), r-s), r-s, 0) \\ &\quad \times \mu_{\mathbf{m}}^2(\Phi_{\mathbf{m}}(\zeta, s))e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, s)}\mathbb{1}_{t_{(2,0,k)}^*(\Phi_{\mathbf{m}}(\zeta, s)) > r-s}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > s} ds \\ &\quad + \int_0^r g(3, \partial)\mu_{\mathbf{m}}^2(\Phi_{\mathbf{m}}(\zeta, s))e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, s)}\mathbb{1}_{t_{(2,0,k)}^*(\Phi_{\mathbf{m}}(\zeta, s)) \leq r-s} ds \end{aligned}$$

- If $\mathbf{m} = (1, 1, k)$ the biomarker decreases. Over a time step r , the patient may fall in remission and remain there, remain in relapse, transition to a therapeutic escape and remain there, fall into remission and transition to a therapeutic escape, transition to a therapeutic escape and die.

$$\begin{aligned} \mathcal{P}g(x) &= g(0, 1, k, \zeta_0, r - t_{\mathbf{m}}^*(\zeta), \tau + r)e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, t_{\mathbf{m}}^*(\zeta))}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) \leq r} \\ &\quad + g(1, 1, k, \Phi_{\mathbf{m}}(\zeta, r), u + r, \tau + r)e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, r)}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > r} \\ &\quad + \int_0^r g(2, 1, k, \Phi_{(2,1,k)}(\Phi_{\mathbf{m}}(\zeta, s), r-s), r-s, \tau + r) \\ &\quad \times \mu_{\mathbf{m}}^2(\Phi_{\mathbf{m}}(\zeta, s))e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, s)}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > s}\mathbb{1}_{t_{(2,1,k)}^*(\Phi_{\mathbf{m}}(\zeta, s)) > r-s} ds \\ &\quad + \int_0^{r-t_{\mathbf{m}}^*(\zeta)} g(2, 1, k, \Phi_{(2,1,k)}(\zeta_0, r-s-t_{\mathbf{m}}^*(\zeta)), r-s-t_{\mathbf{m}}^*(\zeta), \tau + r) \\ &\quad \times \mu_{(0,1,k)}^2(s)e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, t_{\mathbf{m}}^*(\zeta))}e^{-\Lambda_{(0,1,k)}(\mathbf{x}, s)}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) \leq r} ds \\ &\quad + \int_0^r g(3, \partial)\mu_{\mathbf{m}}^2(\Phi_{\mathbf{m}}(\zeta, s))e^{-\Lambda_{\mathbf{m}}(\mathbf{x}, s)}\mathbb{1}_{t_{\mathbf{m}}^*(\zeta) > s}\mathbb{1}_{t_{(2,1,k)}^*(\Phi_{\mathbf{m}}(\zeta, s)) \leq r-s} ds \end{aligned}$$

- If $\mathbf{m} = (0, 0, k)$, the biomarker remains at its nominal value. Over a time step r , the patient may stay in remission, transition to a relapse, transition to a therapeutic escape

or transition to a **relapse before transitioning to a therapeutic escape**.

$$\begin{aligned}
\mathcal{P}g(x) = & g\left(\Phi_{(0,0,k)}(x, r)\right) e^{-\Lambda_m(x, r)} \\
& + \int_0^r g\left(1, 0, k, \Phi_{(1,0,k)}(\zeta_0, r-s), r-s, 0\right) \mu_m^1(u+s) \\
& \times e^{-\Lambda_m(x, s)} e^{-\Lambda_{(1,0,k)}(\Phi_{(1,0,k)}(\zeta_0, r-s), r-s)} ds \\
& + \int_0^r g\left(2, 0, k, \Phi_{(2,0,k)}(\zeta_0, r-s), r-s, 0\right) \mu_m^2(u+s) e^{-\Lambda_m(x, s)} ds \\
& + \int_0^r \int_0^{r-s} g\left(2, 0, k, \Phi_{(2,0,k)}\left(\Phi_{(1,0,k)}(\zeta_0, s'), r-s'-s\right), r-s'-s, 0\right) \\
& \times \mu_m^1(u+s) e^{-\Lambda_m(x, s)} \mu_{(1,0,k)}^2\left(\Phi_{(1,0,k)}(\zeta_0, s')\right) e^{-\Lambda_{(1,0,k)}((\zeta_0, 0), s')} ds' ds
\end{aligned}$$

- If $m = (0, 1, k)$, the biomarker remains at its nominal value. Over a time step r , the patient may **stay in remission** or transition to **a therapeutic escape**.

$$\begin{aligned}
\mathcal{P}g(x) = & g\left(\Phi_{(m)}(x, r)\right) e^{-\Lambda_m(x, r)} \\
& + \int_0^r g\left(2, 1, k, \Phi_{(2,1,k)}(\zeta_0, r-s), r-s, \tau+r\right) \mu_m^2(u+s) e^{-\Lambda_m(x, s)} ds
\end{aligned}$$

Remark 3.2 Although the PDMP transition kernel appears lengthy to write out, its structure allows for straightforward simulation.

3.3 Conclusion

In this chapter, we have presented our main case study, which will serve as a basis for the analyses carried out throughout this manuscript. We have also presented the controlled PDMP as a coherent framework for modelling. However, as the study progresses, this PDMP model will evolve. We have delayed the inclusion of partial observability discussed in Section 3.1.1.

To keep the problem manageable, we consider a simplified version of reality. Even in this simplified form, the problem remains complex and difficult. In the following chapters, we will build on this foundation and present different methods for controlling this type of PDMP model, adding layers of complexity as we go. This case study will go through different stages, including real impulsions control scenarios in Chapter 4, partial observability in Chapter 5 and cases where a process parameter is unknown in Chapter 6.

Chapter 4

Construction of ϵ -Optimal Strategies for Fully-Observed Controlled PDMPs

4.1	Theoretical Foundations	79
4.1.1	Assumptions	79
4.1.2	Principles of Quantization	81
4.1.3	Numerical Method to Approximate the Value Function	84
4.2	Numerical Construction of a Controlled Trajectory	87
4.2.1	Definition of the Candidate Policy Components	87
4.2.2	Numerical Construction of a Candidate Strategy	88
4.2.3	Cost of the Candidate Strategy	89
4.2.4	Candidate Strategy Optimality	96
4.3	Numerical Application	101
4.3.1	Application to Cancer Treatment Optimization	101
4.3.2	Practical Implementation	103
4.3.3	Numerical Results	103
4.4	Conclusion	104

As discussed in Section 2.2.3, constructing an ϵ -optimal strategy for controlling a Piecewise Deterministic Markov Process (PDMP) is in general numerically intractable. A method is available if the value function can be computed over the entire state space, but this requires the state space to be very small and stationary [CD89]. The main contribution of this chapter is to fill this gap by providing an explicitly computable ϵ -optimal strategy in an idealized setting where the model is fully known and the PDMP is continuously and perfectly observed. Our approach is based on quantization and dynamic programming, allowing for a fully explicit offline computation of the control strategy.

The chapter is organized as follows. We begin with a brief review of the theoretical background, including the assumptions satisfied by our PDMP that ensure convergence guarantees for our method in section 4.1.1. Information relative to the numerical method are given in the next section, where we recall the principles of optimal quantization through section 4.1.2 and describe the algorithm used to approximate the value function for controlled PDMPs developed in [SD12] in section 4.1.3. Building on this procedure and the construction of an ϵ -optimal strategy from [SDG17], we define a candidate strategy and the associated controlled PDMP in sections 4.2.1 and 4.2.2. We then prove that the cost, defined in Section 4.2.3, of following this strategy can be made arbitrarily close to the value function, thus establishing its ϵ -optimality in section 4.2.4. Finally, we illustrate the practical performance of our method through a numerical application based on the medical case study presented in Chapter 3 in section 4.3.

4.1 Theoretical Foundations

In this section, we begin by stating a set of assumptions for the impulse control problem of PDMPs. These assumptions are essential for establishing the convergence of our proposed approximation procedures. We then briefly recall the key ideas behind quantization algorithms, which play a central role in our numerical methods. Finally, we introduce the operators and approximation techniques developed in [SD12] which form the foundation of our approach.

4.1.1 Assumptions

Inspired from [SD12], we introduce some technical regularity assumptions on the local characteristics of the PDMP. Let us introduce first some standard notation. Let M be a metric space. $B(M)$ is the set of real-valued, bounded, measurable functions defined on M . The Borel σ -field of M is denoted by $\mathcal{B}(M)$. Let Q be a Markov kernel on $(M, \mathcal{B}(M))$ and $w \in B(M)$, $Qw(x) = \int_M w(y)Q(x, dy)$ for $x \in M$. Let E be an open subset of $\mathbb{R}^d$, ∂E its boundary and $\bar{E}$ its closure. For $(a, b) \in \mathbb{R}^2$, $a \wedge b = \min(a, b)$ and $a \vee b = \max(a, b)$.

Assumption. The exit time t^* is bounded and Lipschitz-continuous on E : there exist $[t^*]$ such that for any $(x, y) \in E^2$, $|t^*(x) - t^*(y)| \leq [t^*] |x - y|$.

In most practical applications, the system's physical properties ensure that either t^* is bounded or the problem has a natural deterministic time horizon T . In the latter case, there is no loss of generality in considering that t^* is bounded by T leading to replacing $[t^*]$ by T .

There is no assumption on the flow as we will rather assume that functions of interest are Lipschitz continuous along the flow.

Assumption. The jump rate λ is bounded and there exists $[\lambda] \in \mathbb{R}_+$ such that for any $(x, y) \in E^2$, $u \in [0, t^*(x) \wedge t^*(y))$, $|\lambda(\Phi(x, u)) - \lambda(\Phi(y, u))| \leq [\lambda] |x - y|$.

Define $\mathbf{L}_\Phi(E)$ as the set of functions w , that are Lipschitz continuous along the flow, i.e., the real-valued, bounded, measurable functions defined on E satisfying

1. For all $x \in E$, the map $w(\Phi(x, \cdot)) : [0, t^*(x)) \rightarrow \mathbb{R}$ is continuous, the limit $\lim_{t \rightarrow t^*(x)} w(\Phi(x, \cdot))$ exists and is denoted by $w(\Phi(x, t^*(x)))$,
2. there exists $[w] \in \mathbb{R}_+$ such that for any $(x, y) \in E^2$, $t \in [0, t^*(x) \wedge t^*(y)]$ one has,

$$|w(\Phi(x, t)) - w(\Phi(y, t))| \leq [w]_1 |x - y|,$$

3. there exists $[w]_2 \in \mathbb{R}_+$ such that for any $x \in E$, $(t, s) \in [0, t^*(x)]^2$, one has

$$|w(\Phi(x, t)) - w(\Phi(x, s))| \leq [w]_2 |t - s|,$$

4. there exists $[w]_* \in \mathbb{R}_+$ such that for any $(x, y) \in E^2$, one has

$$|w(\Phi(x, t^*(x))) - w(\Phi(y, t^*(y)))| \leq [w]_* |x - y|.$$

Finally we need some global Lipschitz continuity properties for the Markov kernel Q .

Assumption. The Markov kernel Q is Lipschitz in the following sense: there exists $[Q] \in \mathbb{R}_+$ such that for any function $w \in \mathbf{L}_\Phi(E)$ the Markov kernel Q satisfies:

1. For any $(x, y) \in E^2$, and $t \in [0, t^*(x) \wedge t^*(y)]$ we have

$$|Qw(\Phi(x, t)) - Qw(\Phi(y, t))| \leq [Q][w]_1 |x - y|,$$

2. for any $(x, y) \in E^2$, we have,

$$|Qw(\Phi(x, t^*(x))) - Qw(\Phi(y, t^*(y)))| \leq [Q][w]_* |x - y|.$$

In the sequel, for any function f in B^c , we denote by C_f its bound

$$C_f = \sup_{x \in E} |f(x)|,$$

and for any Lipschitz-continuous function f in $B(E)$ or $B(\bar{E})$, we denote by $[f]$ its Lipschitz constant

$$[f] = \sup_{x \neq y \in E} \frac{|f(x) - f(y)|}{|x - y|}.$$

To prove the convergence of our approximation scheme, the assumption that the running cost c_R has suitable Lipschitz-continuity properties is required.

Assumption. c_R is a non negative function in $\mathbf{L}_\Phi(E)$.

Let's define $\mathcal{U} = \{y^i, 1 \leq i \leq u\} \subset E$ as the finite set of points the process can restart from after an intervention. Finally, we assume that the impulse cost c_I is Lipschitz continuous along the flow.

Assumption. The function c_I is continuous on $\bar{E} \times \mathbb{U}$ and there exists $[c_I] \in \mathbb{R}_+$, $[c_I]_2 \in \mathbb{R}_+$ and $[c_I]_\star \in \mathbb{R}_+$ such that

1. for any $(x, y) \in E^2$, $u \in [0, t^\star(x) \wedge t^\star(y)]$,

$$\max_{z \in \mathbb{U}} |c_I(\Phi(x, u), z) - c_I(\Phi(y, u), z)| \leq [c_I]_1 |x - y|,$$

2. for any $x \in E$, and $(t, s) \in [0, t^\star(x)]^2$,

$$\max_{z \in \mathbb{U}} |c_I(\Phi(x, t), z) - c_I(\Phi(x, s), z)| \leq [c_I]_2 |t - s|,$$

3. for any $(x, y) \in E^2$,

$$\max_{z \in \mathbb{U}} |c_I(\Phi(x, t^\star(x)), z) - c_I(\Phi(y, t^\star(y)), z)| \leq [c_I]_\star |x - y|,$$

4. for any $(x, y) \in \bar{E} \times \mathbb{U}$,

$$0 \leq c_0 \leq c_I(x, y) \leq c_C,$$

5. for any $(x, y, z) \in \bar{E} \times \mathbb{U} \times \mathbb{U}$,

$$c_I(x, y) + c_I(y, z) \geq c_I(x, z).$$

The last assumption implies that the cost of taking two or more interventions instantaneously will not be higher than taking a single intervention.

4.1.2 Principles of Quantization

Optimal quantization refers to the process of approximating a probability distribution with an optimized discrete representation. We do not aim to provide an exhaustive review of the available methods. For further details, readers are referred to [GN98], [Pag98], [PPP04], and the website <http://www.quantize.maths-fi.com>. The exposition here is inspired by Section 3 of [SD12] and Chapter 3 of the book [SDZ15]. We start with the definition of optimal quantization for a single random variable before turning to the extension to Markov chains.

Random variable quantization. Let X be an $\mathbb{R}^d$ -valued random variable with finite L_2 -norm: $\|X\|_2 = \mathbb{E}[|X|^2]^{1/2} < \infty$ and let K be a fixed integer. The goal of optimal L_2 -quantization is to get a L_2 -approximation of X by a discrete random vector $\hat{X}$ taking values in a finite grid of size at most K values: $\hat{X} \in \{x^1, \dots, x^K\}$. The asymptotic properties of the optimal L_2 -quantization are given by Zador's theorem 4.1.1.

Theorem 4.1.1 *If $\mathbb{E}[|X|^{2+\eta}] < +\infty$ for some $\eta > 0$ then one has*

$$\lim_{K \rightarrow \infty} K^{2/d} \min_{|\Gamma| \leq K} \|X - \hat{X}^\Gamma\|_2^2 = J_{2,d} \int |h|^{d/(d+2)}(u) du,$$

where the law of X can be decomposed with respect to the Lebesgue measure λ_d on $\mathbb{R}^d$ as $P_X(du) = h(u)\lambda_d(du) + \nu(du)$, h being the density of its absolutely continuous part, ν its singular part, and $J_{2,d}$ is a universal constant.

Remark 4.1 X needs to have finite moments up to the order $2 + \eta$ to ensure the above convergence.

In practice, the quantization procedure involves two main steps: (1) finding a finite grid $\Gamma \in \mathbb{R}^d$ with $\Gamma = \{x^1, \dots, x^K\}$ and (2) defining $\hat{X} = \hat{X}^\Gamma = p_\Gamma(X)$ with p_Γ is the closest-neighbor projection on Γ . The quality of the approximation depends on the dimension of X . It is given by Theorem 4.1.1.

The Competitive Learning Vector Quantization algorithm (CLVQ), described in details in [BP03] and in Algorithm 8, provides an optimal grid $\Gamma^* = \{x_1^*, \dots, x_K^*\}$ for the L_2 -quantization of a random variable. The weights for each point of the optimal grid Γ^* can be obtained by Monte Carlo simulations of X and projection on Γ^* , see Algorithm 9.

Algorithm 8 CLVQ algorithm to compute an optimal quantization grid for X . *Adapted from [SDZ15].*

- 1: Input: number of points K , number of runs NR , learning rate sequence (γ_m)
 - 2: Initialize grid Γ^0 with K points
 - 3: **for** $m \leftarrow 0$ to $NR - 1$ **do**
 - 4: Simulate x according to law ν
 - 5: Competitive phase: select y as the closest neighbour of x in Γ^m
 - 6: Learning phase: set $y' = y - \gamma_{m+1}(y - x)$
 - 7: Update grid: $\Gamma^{m+1} \leftarrow \Gamma^m \cup \{y'\} \setminus \{y\}$
 - 8: **end for**
-

Algorithm 9 CLVQ algorithm to compute the weights of the quantization grid Γ for X . *Adapted from [SDZ15].*

- 1: Input: number Monte-Carlo runs NMC
 - 2: Initialize a nul count vector of size K .
 - 3: **for** $n \leftarrow 1$ to NMC **do**
 - 4: Simulate x according to law ν
 - 5: Select i in $\{1, \dots, K\}$ such that x_i is the closest neighbour of x in Γ
 - 6: Count(i) $\leftarrow$ count(i) + 1
 - 7: **end for**
 - 8: count $\leftarrow$ count / NMC
-

Figure 4.1 shows the corresponding quantization grid with the Voronoi tessellation obtained with the CLVQ algorithm for the two dimensional standard Gaussian distribution.

Markov chain quantization. Optimal quantization of a Markov chain involves approximating its distribution with a process taking values on finite grids. As described in [PPP04], two main approaches exist: marginal quantization, which quantizes each time step separately,

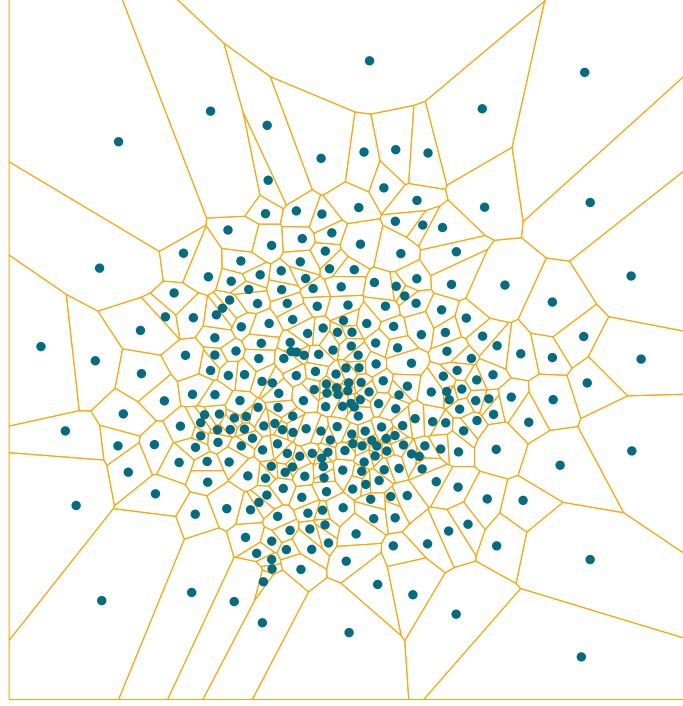


Figure 4.1: **Voronoi tessellation with 300 points for standard Gaussian distribution.**

and Markovian quantization, which maintains the Markov property. This chapter focuses on the marginal approach, while both are detailed in the cited reference.

The Algorithm 10 describes the adaption of CLVQ algorithm to the context of optimal quantization of Markov chains. Let us denote $\Gamma_n = \{x_1^n, \dots, x_K^n\}$ the grid at time step n . The marginal quantization of the Markov chain $(X_n)_{n \in \mathbb{N}}$ is defined by the quantized Markov chain $(\hat{X}_n)_{n \in \mathbb{N}}$ with transition probabilities $\hat{p}_n(i, j)$ and initial distribution by $\hat{X}_0$. Computation of weights and transition matrices are equivalent to Algorithm 9.

Algorithm 10 Extended CLVQ algorithm to quantize a Markov chain. *Adapted from [SDZ15].*

- 1: Input: number of points K , horizon T , number of runs NR , learning rate sequence (γ_m)
 - 2: Initialize grids $(\Gamma_n^0)_{n \in \mathbb{N}}$ with K points each
 - 3: **for** $m \leftarrow 0$ to $NR - 1$ **do**
 - 4: Simulate trajectory $x_0, x_1, \dots, x_N$ according to law of Markov chain
 - 5: **for** $n \leftarrow 0$ to N **do**
 - 6: Competitive phase: select y as the closest neighbour of x_n in Γ_n^m
 - 7: Learning phase: set $y' = y - \gamma_{m+1}(y - x_n)$
 - 8: Update grid: $\Gamma_n^{m+1} \leftarrow \Gamma_n^m \cup \{y'\} \setminus \{y\}$
 - 9: **end for**
 - 10: **end for**
-

Remark 4.2 *The grids depend on the dynamics of the process and not a specific functional or optimization problem we may want to solve or compute. Although it may take a lot of time to compute them, this computation can be done offline and the same grids may be used to solve*

different problems related to the same process.

4.1.3 Numerical Method to Approximate the Value Function

We now present key elements from [SD12], which introduced a numerical method to approximate the value function of a controlled PDMP. The approach is based on quantizing the associated canonical Markov chain $\Theta_n = (Z_n, S_n)_{n \in \mathbb{N}}$ as defined in Section 2.1.4. This method serves as the foundation for the rest of the chapter and underpins the numerical scheme used to define an ϵ -optimal strategy in Section 4.2.

First, they define the path-adapted discretization grids as follows.

Definition 4.1.1: Path-adapted discretization grids

For $x \in E$, set $\Delta(x) \in (0, t^*(x))$. Define $n(x) = \text{int}(\frac{t^*(x)}{\Delta(x)}) - 1$, where $\text{int}(x)$ denotes the greatest integer smaller than or equal to x . The set of points $(t_i)_{i \in \{0, \dots, n(x)\}}$ with $t_i = i\Delta(x)$ is denoted by $G(x)$. This is the grid associated to the time interval $[0, t^*(x)]$.

Then, an approximation of operator L , defined in section 2.2.2, is defined, where the continuous minimization is replaced by a discrete-time minimization on the path adapted grids.

Definition 4.1.2: Discrete-time minimization of operator L

For $(v, w) \in \mathbf{L}_\Phi(E)^2$ and $x \in E$, set

$$L^d(v, w)(x) = \min_{t \in G(x)} J(v, w)(x, t) \wedge Kw(x)$$

Recall that $\mathbb{U} = \{y^i, 1 \leq i \leq u\}$ is the set of points the process can restart from after an intervention. The quantization procedure generates two families of finite grids for the chain (Z_n, S_n) , depending on whether the starting point arises from the original uncontrolled process or from the post-intervention process: $\Gamma_n^{x_0} \cup E \times \mathbb{R}_+$ for $0 \leq n \leq N$, constructed when it starts from the starting point $Z_0 = x_0$ and $\Gamma_n^\mathbb{U} \cup E \times \mathbb{R}_+$ for $0 \leq n \leq N$ construct from starting point $Z_0 \sim \mathcal{U}(\mathbb{U})$. Each grid is equipped with associated weights for its points and transition probabilities between successive grids. Define $p_n^{x_0}$ and $p_n^\mathbb{U}$ as the nearest-neighbour projections from $E \times \mathbb{R}_+$ onto $\Gamma_n^{x_0}$ and $\Gamma_n^\mathbb{U}$, respectively.

The quantization of Θ_n is defined by: $\hat{\Theta}_n = (\hat{Z}_n, \hat{S}_n) = p_n^{x_0}(Z_n, S_n)$ or $\hat{\Theta}_n = (\hat{Z}_n, \hat{S}_n) = p_n^\mathbb{U}(Z_n, S_n)$ depending on the starting point of the chain. The context will make clear which quantization is considered. We will also denote $\Gamma_n^{x_0, Z}$ and $\Gamma_n^{\mathbb{U}, Z}$ the projection of $\Gamma_n^{x_0}$ and $\Gamma_n^\mathbb{U}$, respectively on E and $\Gamma_n^{x_0, S}$ and; $\Gamma_n^{\mathbb{U}, S}$ the projection of $\Gamma_n^{x_0}$ and $\Gamma_n^\mathbb{U}$, respectively on $\mathbb{R}_+$. Quantized approximation of operators introduced in Theorem 2.2.1 depend on indices j , the number of past natural jumps.

Definition 4.1.3: Quantized approximation of operators

For $\forall \quad 0 \leq j \leq N-1$, $x \in E$, $z \in \Gamma_j^{x_0, Z}$, $v \in \mathbf{L}_\Phi(E)$, w defined on $\Gamma_{j+1}^{x_0, Z}$ consider:

$$\begin{aligned}\hat{K}_{j+1}^{x_0} w(z) &= \mathbb{E}[F(\hat{Z}_j^{x_0}, t^*(\hat{Z}_j^{x_0})) + e^{-\gamma \hat{S}_{j+1}} w(\hat{Z}_{j+1}^{x_0}) | \hat{Z}_j^{x_0} = z]. \\ \hat{J}_{j+1}^{x_0}(v, w)(z, t) &= \mathbb{E}[F(\hat{Z}_j^{x_0}, t) + e^{-\gamma \hat{S}_{j+1}} w(\hat{Z}_{j+1}^{x_0}) \mathbb{1}_{\hat{S}_{j+1} < t \wedge t^*(\hat{Z}_j^{x_0})} | \hat{Z}_j^{x_0} = z] \\ &\quad + \mathbb{E}[e^{-\gamma(t \wedge t^*(\hat{Z}_j^{x_0}))} v(\Phi(\hat{Z}_j, t \wedge t^*(\hat{Z}_j^{x_0}))) \mathbb{1}_{\hat{S}_{j+1} \geq t \wedge t^*(\hat{Z}_j^{x_0})} | \hat{Z}_j^{x_0} = z] \\ \hat{L}_{j+1}^{x_0, d}(v, w)(z) &= \min_{t \in G(z)} \hat{J}_{j+1}^{x_0}(v, w)(z, t) \wedge \hat{K}_{j+1}^{x_0} w(z).\end{aligned}$$

For $\forall \quad 0 \leq j \leq N-1$, $x \in E$, $z \in \Gamma_j^{\cup, Z}$, $v \in \mathbf{L}_\Phi(E)$, w defined on $\Gamma_{j+1}^{\cup, Z}$ consider:

$$\begin{aligned}\hat{K}_{j+1}^{\cup} w(z) &= \mathbb{E}[F(\hat{Z}_j^{\cup}, t^*(\hat{Z}_j^{\cup})) + e^{-\gamma \hat{S}_{j+1}} w(\hat{Z}_{j+1}^{\cup}) | \hat{Z}_j^{\cup} = z]. \\ \hat{J}_{j+1}^{\cup}(v, w)(z, t) &= \mathbb{E}[F(\hat{Z}_j^{\cup}, t) + e^{-\gamma \hat{S}_{j+1}} w(\hat{Z}_{j+1}^{\cup}) \mathbb{1}_{\hat{S}_{j+1} < t \wedge t^*(\hat{Z}_j^{\cup})} | \hat{Z}_j^{\cup} = z] \\ &\quad + \mathbb{E}[e^{-\gamma(t \wedge t^*(\hat{Z}_j^{\cup}))} v(\Phi(\hat{Z}_j, t \wedge t^*(\hat{Z}_j^{\cup}))) \mathbb{1}_{\hat{S}_{j+1} \geq t \wedge t^*(\hat{Z}_j^{\cup})} | \hat{Z}_j^{\cup} = z] \\ \hat{L}_{j+1}^{\cup, d}(v, w)(z) &= \min_{t \in G(z)} \hat{J}_{j+1}^{\cup}(v, w)(z, t) \wedge \hat{K}_{j+1}^{\cup} w(z).\end{aligned}$$

Remark 4.3 In what follows, we write $\hat{Z}_j$ to denote either $\hat{Z}_j^{x_0}$ or $\hat{Z}_j^{\cup}$, depending on the chosen starting point. The intended meaning will be clear from the context. This convention is adopted to simplify the notation.

The approximation scheme goes backwards in time.

Definition 4.1.4: Approximation scheme

Let i denotes to the step of the last impulse and j denotes to the number of natural jumps since either the beginning or the last impulse, $\hat{V}_0(x_0)$ denotes the approximation of $\mathcal{V}(x_0)$, and N denote the maximum number of natural jumps and permitted impulses.

For $1 \leq i \leq N$, for $z \in \Gamma_{N-i}^{\cup, Z}$, $\hat{V}_{N-i}^i(z) = h(z)$, with $h(x)$ the cost associated to the no-impulse strategy defined in Theorem 2.2.1. For $0 \leq j \leq N-i-1$, for $z \in \Gamma_j^{\cup, Z}$, $\hat{V}_j^i(z) = \hat{L}_{j+1}^{\cup, d}(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(z)$.

For $z \in \Gamma_N^{x_0, Z}$, $\hat{V}_N(z) = h(z)$. For $0 \leq j \leq N-1$, for $z \in \Gamma_j^{x_0, Z}$, $\hat{V}_j(z) = \hat{L}_{j+1}^{x_0, d}(M\hat{V}_0^{j+1}, \hat{V}_{j+1})(z)$.

A graphical illustration of this numerical procedure is available in Figure 4.2.

Remark 4.4 This approach described above is numerically demanding and works for low dimensional PDMPs with limited complexity and limited number of natural jumps before absorption. It also requires a perfect knowledge of the model and its parameters, together with a perfect observation of the process at all times (or equivalently at jumps times).

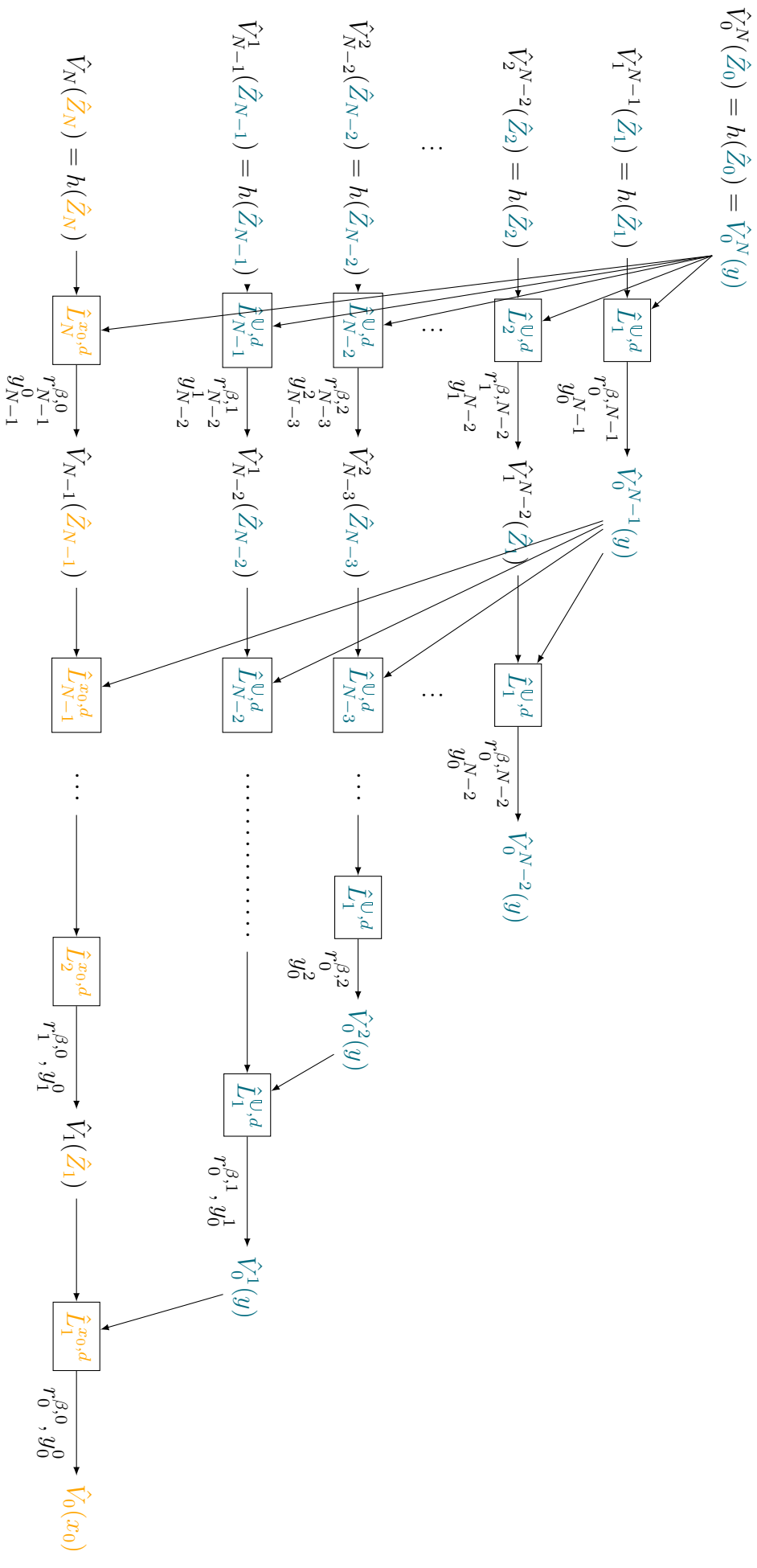


Figure 4.2: Step by step representation of the numerical method to approximate the value function. Blue nodes indicate results obtained from $\Gamma_n^{\cup}$ quantization grids, while orange indicate results obtained from $\Gamma_n^{x_0}$ quantization grids. Each row represents the successive natural jumps of the process starting from a given initial point. A change of row corresponds to a new impulse, which resets the process at a new starting point. The first (bottom) row corresponds to the natural trajectory of the process without any impulse. *Adapted from [SD12].*

4.2 Numerical Construction of a Controlled Trajectory

Using the iterative scheme from [SDG17] combined with the discretized operators introduced in [SD12], we define the candidate policy components. Next, we define an auxiliary process to explicitly build a trajectory controlled by our candidate strategy. Then we compute the cost associated with this strategy. Finally, we prove that the candidate strategy is ϵ -optimal.

4.2.1 Definition of the Candidate Policy Components

[SDG17] builds a ϵ -optimal strategy family based on true operators, while [SD12] builds quantized approximation of these operators. Hence, to construct a family of computable ϵ -optimal strategies, one may replace the true operators in [SDG17] construction by their quantized approximation. This process is summarized in Figure 4.3. In this framework, we recall that $N \in \mathbb{N}$ represents the maximum number of natural jumps and impulses allowed, serving as a finite budget that constrains the evolution of the controlled process. The process consists of N steps. At each step, there is either a natural jump or an impulse. At stage n , the remaining budget is thus $N_0 = N - n$.

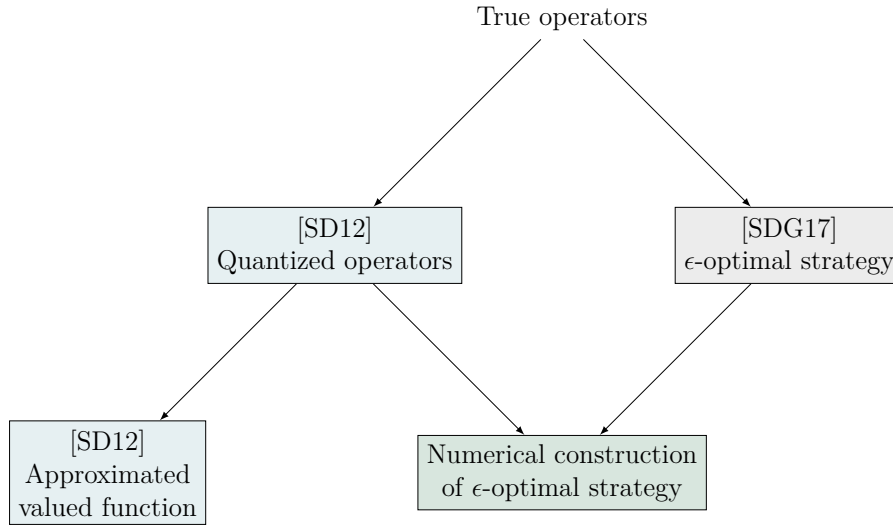


Figure 4.3: Graphical illustration of how this chapter contributes to the numerical construction of a controlled trajectory. Blue boxes indicate results from [SD12], the grey box highlights findings from [SDG17], and the green box represents the original contributions of this chapter.

Recall that $p_n^{x_0}$ and $p_n^\cup$ be the nearest-neighbour projections from $E \times \mathbb{R}_+$ onto $\Gamma_n^{x_0}$ with $(\hat{z}_j, \hat{s}_j) = p_j^{x_0}(z, s)$ and $\Gamma_n^\cup$ with $(\hat{z}_j, \hat{s}_j) = p_j^\cup(z, s)$, respectively. Note that $\hat{z}_j$ and $\hat{s}_j$ depend on both z and s . Now, for all $1 \leq i \leq N$, $0 \leq j \leq N - i - 1$ and $\hat{z}_j = \hat{z}_j^{x_0} \in \Gamma_j^{x_0, Z}$ if $i = 0$, else $\hat{z}_j = \hat{z}_j^\cup \in \Gamma_j^{\cup, Z}$, define the best possible intervention time that can be chosen before the next jump and that is available from the computation of the approximation value functions:

$$s_{i,j}^*(z, s) = \min_{t \in G(\hat{z}_j)} \{ \arg \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^\cup(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t) \}. \quad (4.1)$$

Then, by induction, we define, for a given $\beta > 0$, the next best intervention time, either waiting for the next natural jump or intervening before it. For all $0 \leq j \leq N-1$, $i = 0$ and $\hat{z}_j \in \Gamma_j^{x_0, Z}$:

$$\hat{r}_j^\beta(z, s) = \begin{cases} t^*(z) & \text{if } \hat{K}_j^{x_0} \hat{V}_{j+1}(\hat{z}_j) < \min_{t \in G(\hat{z}_j)} \hat{J}_j^{x_0}(M \hat{V}_0^{j+1}, \hat{V}_{j+1})(\hat{z}_j, t) \\ s_{0,j}^*(z, s) \mathbb{1}_{s_{0,j}^*(z, s) < t^*(z)} + (t^*(z) - \beta) \mathbb{1}_{s_{0,j}^*(z, s) \geq t^*(z)} & \text{otherwise.} \end{cases} \quad (4.2)$$

For all $1 \leq i \leq N$, $0 \leq j \leq N-i-1$ and $\hat{z}_j \in \Gamma_j^{\cup, Z}$:

$$\hat{r}_j^{\beta, i}(z, s) = \begin{cases} t^*(z) & \text{if } \hat{K}_j^{\cup} \hat{V}_{j+1}^i(\hat{z}_j) < \min_{t \in G(\hat{z}_j)} \hat{J}_j^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t) \\ s_{i,j}^*(z, s) \mathbb{1}_{s_{i,j}^*(z, s) < t^*(z)} + (t^*(z) - \beta) \mathbb{1}_{s_{i,j}^*(z, s) \geq t^*(z)} & \text{otherwise.} \end{cases} \quad (4.3)$$

If the optimal decision is to intervene before the next natural jump ($r_j^{\beta, i}(z, s) < t^*(z)$), we must also determine the best new restart point for the process. For $0 \leq i \leq N$, $0 \leq j \leq N-i-1$:

$$y_j^i(z, s) = \arg \min_{y \in \mathbb{U}} \{c(\Phi(x, \hat{r}_j^{\beta, i}(z, s)), y) + \hat{V}_0^{i+1+j}(y)\}. \quad (4.4)$$

Remark 4.5 We have already computed the values of $\hat{K}_j^{x_0} \hat{V}_{j+1}^i$, $\hat{K}_j^{\cup} \hat{V}_{j+1}^i$, $\hat{J}_j^{x_0} \hat{V}_{j+1}^i$ and $\hat{J}_j^{\cup} \hat{V}_{j+1}^i$ on the grids. One just has to store an indicator that either $\hat{K}_j^{x_0} \hat{V}_{j+1}^i$ or $\hat{J}_j^{x_0} \hat{V}_{j+1}^i$ or $\hat{K}_j^{\cup} \hat{V}_{j+1}^i$ or $\hat{J}_j^{\cup} \hat{V}_{j+1}^i$ won the minimization and, if needed, the point where the minimum value of $\hat{J}_j^{x_0} \hat{V}_{j+1}^i$ or $\hat{J}_j^{\cup} \hat{V}_{j+1}^i$ is reached and where $M \hat{V}_0^{i+1+j}$ is reached.

4.2.2 Numerical Construction of a Candidate Strategy

Following the approach introduced in [SDG17], we define a new process $\tilde{X}_t$ on an enlarged state space. This process represents a controlled version of the original PDMP $\{X_t\}$, with the same deterministic flow but a modified jump mechanism to incorporate control actions. It keeps the memory of past jumps and impulses.

A key challenge arises from the fact that the strategy was constructed using discretized operators, yet we aim to apply it to the original PDMP. We must enrich the state space to carry additional information: the number of natural jumps j since the beginning of the last impulse, the step of the last impulse i and the time since the last jump θ , so that the candidate strategy can be properly implemented in the continuous-time setting: $\tilde{X}_t = (\tilde{\mathbf{m}}_t = (\mathbf{m}_t, j_t, i_t), \tilde{\mathbf{x}}_t = (\mathbf{x}_t, \theta_t))$ for $t \geq 0$. The state space of this process is defined by

$$\tilde{E} = \{(\mathbf{m}, \mathbf{x}, j, i, \theta) : (\mathbf{m}, \mathbf{x}) \in E, 0 \leq i \leq N, 0 \leq j \leq N-i, \theta \in [0, \hat{r}_j^{\beta, i}(x) \wedge t^*(x)]\}.$$

We define a distance on $\tilde{E}$ in the same way as on E . For $\tilde{x} = (\mathbf{m}, \mathbf{x}, j, i, \theta)$ and $\tilde{x}' =$

$(\mathbf{m}', \mathbf{x}', j', i', \theta')$, we define

$$|(\tilde{\mathbf{m}}, \tilde{\mathbf{x}}) - (\tilde{\mathbf{m}}', \tilde{\mathbf{x}}')| = |\tilde{\mathbf{x}} - \tilde{\mathbf{x}}'| \mathbb{1}_{\tilde{\mathbf{m}}=\tilde{\mathbf{m}}'} + \infty \mathbb{1}_{\tilde{\mathbf{m}} \neq \tilde{\mathbf{m}}'}.$$

Denote by π the projection of space $E \times \mathbb{N}^2 \times \mathbb{R}$ on space E . Denote by $(\tilde{S}_n)_{n \in \mathbb{N}_*}$ the sojourn times between two consecutives jumps and $(\tilde{Z}_n)_{n \in \mathbb{N}_*}$ the post jump locations of the process $\{\tilde{X}_t\}$.

Let $(\mathbf{m}, \mathbf{x}, j, i, \theta) \in \tilde{E}$. We set $r_j^{\beta, i}(\tilde{z}, \tilde{s}) = \hat{r}_j^{\beta, i}(z, s)$. Set for all $0 \leq j \leq N-1$ and $r_j^{\beta, i}(\tilde{z}, \tilde{s}) < t^*(x)$, $\tilde{y}_j^i(\tilde{z}, \tilde{s}) = (y_j^i(z, s), i+1, j, 0)$.

Consider $\tilde{x} \in \tilde{E}$, the process $\{\tilde{X}_t\}$ can be interpreted as a controlled version of $\{X_t\}$ where at most N natural jumps or impulses are allowed. Let's define the control candidate strategy $\tilde{\mathcal{S}} = (\tau_n, y_n)_{0 \leq n \leq N}$, where y_n and τ_n are defined in Algorithm 11 that explicits the dynamics of the PDMP $\{\tilde{X}_t\}$.

4.2.3 Cost of the Candidate Strategy

In this subsection, we once again rely on the approach from [SDG17], this time to compute the cost of the candidate strategy on the original PDMP X_t . For simplicity, in this section, we define everything assuming the process starts post-intervention, with initial state in $\mathbb{U}$ and grids $\Gamma_n^{\mathbb{U}}$. When $i = 0$, however, the operators are computed using the grids $\Gamma_n^{x_0}$. First we define the cost associated with the controlled process defined by Algorithm 11 for the candidate strategy $\tilde{\mathcal{S}}$. When there is no remaining budget of natural jumps or impulses, we have for $x \in E$ and $i + j = N$:

$$\mathcal{J}_N^{\tilde{\mathcal{S}}(N-i-j)}(x, j, i) = \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \quad (4.5)$$

and for $n = i + j \leq N-1$:

$$\mathcal{J}_{N-i-j}^{\tilde{\mathcal{S}}(n)}(x, j, i) = \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + \sum_{k=1}^{N-j-i} e^{-\gamma r_j^{\beta, k}} \tilde{c}_I(\tilde{X}_{r_j^{\beta, k}-}, \tilde{X}_{r_j^{\beta, k}}) \right], \quad (4.6)$$

where functions $\tilde{c}_R$ and $\tilde{c}_I$ are defined by

$$\forall t \in \mathbb{R}_+, \quad \begin{cases} \tilde{c}_I(\tilde{X}_{t-}, \tilde{X}_t) &= c_I(\pi(\tilde{X}_{t-}), \pi(\tilde{X}_t)) \\ \tilde{c}_R(\tilde{X}_t, \tilde{X}_t) &= c_R(\pi(\tilde{X}_t)) \end{cases} \quad (4.7)$$

recalling π is the projection of space $\tilde{E}$ on space E .

We now present the cost associated with the controlled PDMP $\{X_t\}$ in a recursive form. To establish this formulation, we proceed by induction. The proof is divided into three parts: Theorem 4.2.1 addresses the cost at the terminal step ($n = N$) of the trajectory; Theorem 4.2.3 establishes the recursive cost relation between two generic consecutive steps; and Theorem 4.2.2 covers an intermediate case, between steps N and $N-1$, included to clarify the overall argument, though it may be skipped without loss of generality.

Algorithm 11 Iterative construction of a controlled trajectory

```
1: Initialize starting point  $x_0$ 
2:  $z \leftarrow x_0$ 
3:  $i \leftarrow 0$ 
4:  $j \leftarrow 0$ 
5:  $s \leftarrow 0$ 
6:  $\Theta \leftarrow 0$ 
7:  $T \leftarrow 0$ 
8: for  $n \leftarrow 0$  to  $N$  do
9:    $\tau_n \leftarrow \hat{r}_j^{\beta,i}(z, s)$ 
10:   $S \sim \mathbb{P}_z(S > t) = e^{-\int_0^t \lambda \Phi(z,w) dw} \mathbb{1}_{t < t^*(z)}$ 
11:   $\forall T \leq t \leq T + S \wedge \tau_n, \tilde{X}_t = (\Phi(z, t - T), i, j, \Theta + t)$ 
12:   $T \leftarrow T + S \wedge \tau_n$ 
13:  if  $S < \tau_n$  then
14:     $s \leftarrow S$ 
15:     $\Theta \leftarrow 0$ 
16:     $j \leftarrow j + 1$ 
17:     $z \sim Q(\cdot, \Phi(z, s))$ 
18:  else
19:     $Z \leftarrow \tilde{y}_j^i$ 
20:     $s \leftarrow 0$ 
21:     $\Theta \leftarrow 0$ 
22:     $i \leftarrow n + 1$ 
23:     $j \leftarrow 0$ 
24:     $y_n \leftarrow (y_j^i(z, s), i, j, \Theta)$ 
25:  end if
26: end for
27: for  $n \geq N + 1$  do
28:   $S \sim \mathbb{P}_z(S > t) = e^{-\int_0^t \lambda \Phi(z,w) dw} \mathbb{1}_{t < t^*(z)}$ 
29:   $\forall T \leq t < T + S \tilde{X}_t = (\Phi(z, t - T), j, i)$ 
30:   $T \leftarrow T + S$ 
31:   $z \sim Q(\cdot, \Phi(z, s))$ 
32:   $\Theta \leftarrow 0$ 
33:   $j \leftarrow j + 1$ 
34: end for
```

Theorem 4.2.1 Set $x \in E$, $0 \leq i \leq N$ and $j = N - i$, so that there is no remaining budget for impulses. We have the following equality:

$$\mathcal{J}_N^{\tilde{S}}(x, j, i) = h(x) = \mathbb{E}_x \left[\int_0^\infty e^{-\gamma s} c_R(X_s) ds \right].$$

Proof. Set $x \in E$, $0 \leq i \leq N$ and $j = N - i$. By definition, we have :

$$\mathcal{J}_N^{\tilde{S}}(x, j, i) = \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^\infty e^{-\gamma s} c_R(\pi(\tilde{X}_s)) ds \right].$$

It is obvious that $\{\pi(\tilde{X}_t)\}$ starting at $\tilde{x} = (x, j, i, 0)$ is a PDMP which has the same local characteristics as the uncontrolled process $\{X_t\}$ starting at x , as no more impulses are allowed (see Algorithm 11). $\square$

Theorem 4.2.2 Set $x \in E$, $0 \leq i \leq N - 1$, $j = N - 1 - i$ and $\hat{z}_j \in \Gamma_j^{\cup, Z}$ if $i > 0$, and $\hat{z}_j \in \Gamma_j^{x_0, Z}$ if $i = 0$. We have the following equality:

$$\mathcal{J}_{N-1}^{\tilde{S}}(x, j, i) = \begin{cases} K \mathcal{J}_N^{\tilde{S}}(x, j+1, i), & \text{if } \hat{K}_{j+1}^{\cup} \hat{V}_{j+1}^i(\hat{z}_j) \\ < \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t), \\ J(c_I(\cdot, y_j^i) + \mathcal{J}_N^{\tilde{S}}(y_j^i, 0, N), \mathcal{J}_N^{\tilde{S}(0)}(\cdot, j+1, i))(x, \tau_n). \end{cases}$$

Proof. Construction of this proof follows [SDG17] Theorem 9 proof. Set $x \in E$ and $\tilde{x} := (x, j, i, 0)$, with $0 \leq i \leq N - 1$, $j = N - i$ and $\hat{z}_j \in \Gamma_j^{\cup, Z}$ if $i > 0$ or $\hat{z}_j \in \Gamma_j^{x_0, Z}$ if $i = 0$. There are two cases to consider.

- If $\hat{K}_{j+1}^{\cup} \hat{V}_{j+1}^i(\hat{z}_j) < \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t)$, the last jump is a natural jump.

$$\begin{aligned} \mathcal{J}_{N-1}^{\tilde{S}}(x, j, i) &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^{\tilde{S}} e^{-\gamma s} c_R(\Phi(x, s)) ds \right. \\ &\quad \left. + e^{-\gamma \tilde{S}} \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^\infty e^{-\gamma(t+\tilde{s})} \tilde{c}_R(\tilde{X}_{t+\tilde{s}}) dt \mid \tilde{\mathcal{F}}_{\tilde{S}} \right] \right] \end{aligned} \quad (4.8)$$

As $\{\tilde{X}_t\}$ is a Markov process and S corresponds to a natural jump, with $\tilde{s}$ is defined in line 10 in Algorithm 11, of the process and no impulse is allowed after S , we have

$$\begin{aligned} \mathcal{J}_{N-1}^{\tilde{S}}(x, j, i) &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^{\tilde{S}} e^{-\gamma s} c_R(\Phi(x, s)) ds \right] + \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[e^{-\gamma \tilde{S}} \tilde{\mathbb{E}}_{\tilde{Z}} \left[\int_0^\infty e^{-\gamma t} \tilde{c}_R(\tilde{X}_t) dt \right] \right] \\ &= \mathbb{E}_x \left[\int_0^S e^{-\gamma s} c_R(\Phi(x, s)) ds \right] + \tilde{\mathbb{E}}_x [e^{-\gamma S} \tilde{\mathbb{E}}_{(Z, j+1, 0)} \left[\int_0^\infty e^{-\gamma t} c_R(\tilde{X}_t) dt \right]] \\ &= F(x, t^*(x)) + \mathbb{E}_x [e^{-\gamma S} \mathcal{J}_N^{\tilde{S}}(x, j+1, i)] \\ &= K \mathcal{J}_N^{\tilde{S}}(x, j+1, i). \end{aligned}$$

- If $\hat{K}_{j+1}^U \hat{V}_{j+1}^i(\hat{z}_j) \geq \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^U(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t)$,

By definition, $\tilde{S}$ might be either a natural jump ($\tilde{S} = S$) or an impulse ($\tilde{S} = \tau_n$). In both cases, no more impulses are allowed after $\tilde{S}$.

$$\begin{aligned} \mathcal{J}_{N-1}^{\tilde{S}}(x, j, i) &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^{S \wedge \tau_n} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\ &\quad + \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\mathbb{1}_{S < \tau_n} \int_S^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\ &\quad + \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\mathbb{1}_{S \geq \tau_n} \int_{\tau_n}^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + e^{-\gamma \tau_n} c_I(\Phi(x, \tau_n), y_j^i) \right] \\ &= \gamma_1 + \gamma_2 + \gamma_3. \end{aligned}$$

Let us study these three terms separately.

- First consider the running cost until the next natural jump or impulsion.

$$\begin{aligned} \gamma_1 &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^{S \wedge \tau_n} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\ &= \mathbb{E}_x \left[\int_0^{S \wedge \tau_n} e^{-\gamma s} c_R(\Phi(x, s)) ds \right] \\ &= F(x, \tau_n). \end{aligned}$$

- Then, consider, the running cost from the natural jump until the end of process

$$\begin{aligned} \gamma_2 &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\mathbb{1}_{S < \tau_n} \int_S^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\ &= \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} \tilde{\mathbb{E}}_{(x, j, i, 0)} \left[\int_0^\infty e^{-\gamma(t+\tilde{s})} \tilde{c}_R(\tilde{X}_{t+S}) dt \middle| \mathcal{F}_S \right] \right] \\ &= \mathbb{E}_x \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} \mathbb{E}_Z \left[\int_0^\infty e^{-\gamma t} c_R(X_t) dt \right] \right] \\ &= \mathbb{E}_x \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} h(Z) \right]. \end{aligned}$$

- Finally, consider, the cost of the impulsion and the cost along the end of the trajec-

tory

$$\begin{aligned}
 \gamma_3 &= \tilde{\mathbb{E}}_{(x,j,i,0)} \left[\mathbb{1}_{S \geq \tau_n} \int_{\tau_n}^{\infty} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + e^{-\gamma \tau_n} c_I(\Phi(x, \tau_n), y_j^i) \right] \\
 &= \tilde{\mathbb{E}}_{(x,j,i,0)} \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} (h(y_j^i) + c_I(\Phi(x, \tau_n), y_j^i)) \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} (h(y_j^i) + c_I(\Phi(x, \tau_n), y_j^i)) \right].
 \end{aligned}$$

Now combine all above

$$\begin{aligned}
 \mathcal{J}_{N-1}^{\hat{S}}(x, j, i) &= \gamma_1 + \gamma_2 + \gamma_3 \\
 &= J \left(c_I(\cdot, y_j^i) + \mathcal{J}_N^{\hat{S}}(y_j^i, 0, N), \mathcal{J}_N^{\hat{S}}(\cdot, j+1, i) \right) (x, \tau_n),
 \end{aligned}$$

hence the result. □

Theorem 4.2.3 Set $x \in E$, $0 \leq n \leq N-1$, $0 \leq i \leq n$, $j = n-i$ and $\hat{z}_j \in \Gamma_j^{\cup, Z}$ if $i > 0$, and $\hat{z}_j \in \Gamma_j^{x_0, Z}$ if $i = 0$. We have the formula:

$$\mathcal{J}_{n-1}^{\hat{S}}(x, j, i) = \begin{cases} K \mathcal{J}_n^{\hat{S}}(x, j, i), & \text{if } \hat{K}_{j+1}^{\cup} \hat{V}_{j+1}^i(\hat{z}_j) \\ & < \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t), \\ J \left(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\hat{S}}(y_j^i, 0, n), \mathcal{J}_n^{\hat{S}}(\cdot, j+1, i) \right) (x, \tau_n). \end{cases} \quad (4.9)$$

Proof. Some details are similar to the previous proof and are therefore omitted. We proceed by backward induction. The case $n = N-1$ was showed before in Theorem 4.2.2. For all $0 \leq n \leq N-1$, $0 \leq i \leq n$, $j = n-i$ and $x \in E$.

$$\mathcal{J}_{n-1}^{\hat{S}}(x, j, i) = \begin{cases} K \mathcal{J}_n^{\hat{S}}(x, j, i), & \text{if } \hat{K}_{j+1}^{\cup} \hat{V}_{j+1}^i(\hat{z}_j) \\ & < \min_{t \in G(\hat{z}_j)} \hat{J}_{j+1}^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t), \\ J \left(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\hat{S}}(y_j^i, 0, n), \mathcal{J}_n^{\hat{S}}(\cdot, j+1, i) \right) (x, \tau_n). \end{cases}$$

Once again, we distinguish two cases.

- If $\hat{K}_j^{\cup} \hat{V}_j^i(\hat{z}_{j-1}) < \min_{t \in G(\hat{z}_{j-1})} \hat{J}_j^{\cup}(M \hat{V}_0^{i+j}, \hat{V}_j^i)(\hat{z}_{j-1}, t)$, the next jump corresponds to a natural jump.

$$\begin{aligned}
 \mathcal{J}_{n-1}^{\tilde{S}}(x, j, i) &= \tilde{\mathbb{E}}_{(x, i, j)} \left[\int_0^{\tilde{S}} e^{-\gamma s} \tilde{c}_R(\tilde{\Phi}(x, i, j), s) ds \right] \\
 &\quad + \tilde{\mathbb{E}}_{(x, i, j)} \left[\int_{\tilde{S}}^{\infty} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) + \sum_{k=1}^{N-i-j} e^{-\gamma \tau_k} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) ds \right] \\
 &= F(x, t^*(x)) \\
 &\quad + \tilde{\mathbb{E}}_{(x, i, j)} \left[e^{-\gamma \tilde{S}} \tilde{\mathbb{E}}_{(x, i, j)} \left[\int_0^{\infty} e^{-\gamma(t+\tilde{s})} \tilde{c}_R(\tilde{X}_{t+\tilde{s}}) dt \right. \right. \\
 &\quad \left. \left. + \sum_{k=2}^{N-i-j} e^{-\gamma(\tau_k - \tilde{S}_k)} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) | \mathcal{F}_{\tilde{S}} \right] \right] \\
 &= F(x, t^*(x)) + \mathbb{E}_x \left[e^{-\gamma S} \mathbb{E}_{(Z, i, j)} \left[\int_0^{\infty} e^{-\gamma t} c_R(X_t) dt \right. \right. \\
 &\quad \left. \left. + \sum_{k=1}^{N-i-j} e^{-\gamma \tau_k} c_I(X_{\tau_k-}, X_{\tau_k}) \right] \right] \\
 &= F(x, t^*(x)) + \mathbb{E}_x \left[e^{-\gamma S} \mathcal{J}_{n-1}^{\tilde{S}}(Z, j+1, i) \right] \\
 &= K \mathcal{J}_n^{\tilde{S}}(x, j+1, i).
 \end{aligned}$$

- If $\hat{K}_j^{\cup} \hat{V}_j^i(\hat{z}_{j-1}) \geq \min_{t \in G(\hat{z}_{j-1})} \hat{J}_j^{\cup}(M \hat{V}_0^{i+j}, \hat{V}_j^i)(\hat{z}_{j-1}, t)$

By definition, $\tilde{S}$ might be either a natural jump ($\tilde{S} = S$) or an impulse ($\tilde{S} = \tau_n$).

$$\begin{aligned}
 \mathcal{J}_{n-1}^{\tilde{S}}(x, j, i) &= \tilde{\mathbb{E}}_{(x, i, j)} \left[\int_0^{S \wedge \tau_n} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\
 &\quad + \tilde{\mathbb{E}}_{(x, i, j)} \left[\mathbb{1}_{S < \tau_n} \left\{ \int_S^{\infty} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + \sum_{k=1}^{N-i-j} e^{-\gamma \tau_k} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) \right\} \right] \\
 &\quad + \tilde{\mathbb{E}}_{(x, i, j)} \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} \tilde{c}_I \left(\tilde{\Phi}((x, j, i), \tau_n), (y_j^i, 0, n) \right) \right] \\
 &\quad + \tilde{\mathbb{E}}_{(x, i, j)} \left[\mathbb{1}_{S \geq \tau_n} \left\{ \int_{\tau_n}^{\infty} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + \sum_{k=2}^{N-i-j} e^{-\gamma \tau_k} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) \right\} \right] \\
 &= \psi_1 + \psi_2 + \psi_3 + \psi_4.
 \end{aligned}$$

Let us study these four terms separately.

- First consider the running cost until the next natural jump or impulsions

$$\begin{aligned}
 \psi_1 &= \tilde{\mathbb{E}}_{(x, i, j)} \left[\int_0^{S \wedge \tau_n} e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds \right] \\
 &= F(x, r_j^{\beta, i}).
 \end{aligned}$$

- Then consider the running cost from the natural jump and all impulsion cost until the end of process

$$\begin{aligned}
 \psi_2 &= \tilde{\mathbb{E}}_{(x,j,i,0)} \left[\mathbb{1}_{S < \tau_n} \left\{ \int_S^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + \sum_{k=1}^{N-i-j} e^{-\gamma \tau_k} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) \right\} \right] \\
 &= \tilde{\mathbb{E}}_{(x,j,i,0)} \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} \tilde{\mathbb{E}}_{(x,i,j)} \left[\int_0^\infty e^{-\gamma(t+\tilde{s})} \tilde{c}_R(\tilde{X}_{s+S}) dt \right. \right. \\
 &\quad \left. \left. + \sum_{k=2}^{N-i-j} e^{-\gamma(\tau_k - S_k)} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) | \mathcal{F}_S \right] \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} \tilde{\mathbb{E}}_{(Z,i,j)} \left[\int_0^\infty e^{-\gamma t} c_R(X_s) dt + \sum_{k=1}^{N-i-j} e^{-\gamma \tau_k} c_I(X_{\tau_k-}, X_{\tau_k}) \right] \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S < \tau_n} e^{-\gamma S} \mathcal{J}_{n-1}^{\tilde{S}}(Z, j+1, i) \right].
 \end{aligned}$$

- Then, consider the cost of the impulsion

$$\begin{aligned}
 \psi_3 &= \tilde{\mathbb{E}}_{(x,i,j)} \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} \tilde{c}_I \left(\tilde{\Phi}((x, j, i), \tau_n), (y_j^i, 0, i+1) \right) \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} c_I \left(\Phi(x, \tau_n), y_j^i \right) \right].
 \end{aligned}$$

- Finally, consider the current cost from an impulsion and all impulsion cost until the end of process

$$\begin{aligned}
 \psi_4 &= \tilde{\mathbb{E}}_{(x,j,i)} \left[\mathbb{1}_{S \geq \tau_n} \left\{ \int_{\tau_n}^\infty e^{-\gamma s} \tilde{c}_R(\tilde{X}_s) ds + \sum_{k=2}^{N-i-j} e^{-\gamma \tau_k} \tilde{c}_I(\tilde{X}_{\tau_k-}, \tilde{X}_{\tau_k}) \right\} \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} \tilde{\mathbb{E}}_{(x,i,j)} \left[\int_0^\infty e^{-\gamma(t+\tau_n)} c_R(X_{t+\tau_n}) dt \right. \right. \\
 &\quad \left. \left. + \sum_{k=2}^{N-i-j} e^{-\alpha \tau_k} c_I(X_{\tau_k-}, X_{\tau_k}) | \mathcal{F}_{\tau_n} \right] \right] \\
 &= \mathbb{E}_x \left[\mathbb{1}_{S \geq \tau_n} e^{-\gamma \tau_n} \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n) \right].
 \end{aligned}$$

Now combine all above

$$\begin{aligned}
 \mathcal{J}_{n-1}^{\tilde{S}}(x, j, i) &= \psi_1 + \psi_2 + \psi_3 + \psi_4 \\
 &= J \left(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i) \right) (x, \tau_n),
 \end{aligned}$$

hence the result. $\square$

4.2.4 Candidate Strategy Optimality

In this section, we establish the ϵ -optimality of the candidate strategy introduced in Section 4.2.2. Specifically, we prove that the cost of the corresponding controlled trajectory, as defined in Section 4.2.3, converges to the true value function, thereby showing that the candidate strategy is an ϵ -optimal strategy. Once again, for simplicity, in this section, we define everything assuming the process starts post-intervention, with initial state in $\mathbb{U}$ and grids $\Gamma_n^\mathbb{U}$. When $i = 0$, however, the operators are computed using the grids $\Gamma_n^{x_0}$.

First, let's recall some previous results. For $\tilde{x} \in \tilde{E}$, $\mathcal{J}_N^{\tilde{S}}(\tilde{x}) = \mathcal{J}_N^{\tilde{S}}(x) = h(x)$, and for $N \in \mathbb{N}_*$, $0 \leq n \leq N$, $0 \leq i \leq n$, $0 \leq j \leq n - i$ and $\hat{z}_j \in \Gamma_n^{\mathbb{U}, Z}$ if $i \geq 1$ or $\hat{z}_j \in \Gamma_n^{x_0, Z}$ if $i = 0$, we have the following result:

$$\mathcal{J}_n^{\tilde{S}}((z, s), i, j) = \begin{cases} K \mathcal{J}_{n+1}^{\tilde{S}}(z, j+1, i), & \text{if } \hat{K}_j^\mathbb{U} \hat{V}_{j+1}^i(\hat{z}_j) \\ & < \min_{t \in G(\hat{z}_j)} \hat{J}_j^\mathbb{U}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{z}_j, t), \\ J\left(c_I(\cdot, y_j^i) + \mathcal{J}_{n+1}^{\tilde{S}}(y_j^i, 0, n+1), \mathcal{J}_{n+1}^{\tilde{S}}(\cdot, j+1, i)\right)(z, \tau_n), \end{cases} \quad (4.10)$$

where

$$Kw(z, j+1, i) = F(z, t^*(z)) + \mathbb{E}_z[e^{\gamma S} w(Z)],$$

and

$$J(v, w)(z, t) = F(z, t) + \mathbb{E}_z[e^{\gamma(t \wedge t^*(z))} v(\Phi(z, t \wedge t^*(z))) \mathbb{1}_{S \geq t \wedge t^*(z)} + w(Z) \mathbb{1}_{S < t \wedge t^*(z)}].$$

We need technical results to show that $\mathcal{J}_n^{\tilde{S}}$ is Lipschitz continuous along the flow.

Lemma 4.2.4 *For all $0 \leq n \leq N$, $0 \leq i \leq n$, $0 \leq j \leq n - i$ and for all $\tilde{x} \in \tilde{E}$, we have $\mathcal{J}_{i+j}^{\tilde{S}} \in \mathbf{L}_\Phi(E)$. Let's denote $[\mathcal{J}_{i+j}^{\tilde{S}}]$ its Lipschitz constant.*

Proof.

The proof is based on the same framework as in [BdD13], together with the Lipschitz continuity results from [Sap+12].

□

Now, let's demonstrate the candidate strategy is an ϵ -optimal strategy. The convergence proof builds on Theorem 5.11 from [SD12], which establishes the convergence of the approximate value function to the true value function. Using intermediate results from [SD12], we show that the cost of the trajectory controlled by the candidate strategy converges to this approximate value function. The proof follows the structure of [SD12], decomposing the trajectory cost and

the value function into corresponding components and comparing them individually to establish convergence. Different proof steps are summed up into Figure 4.4.

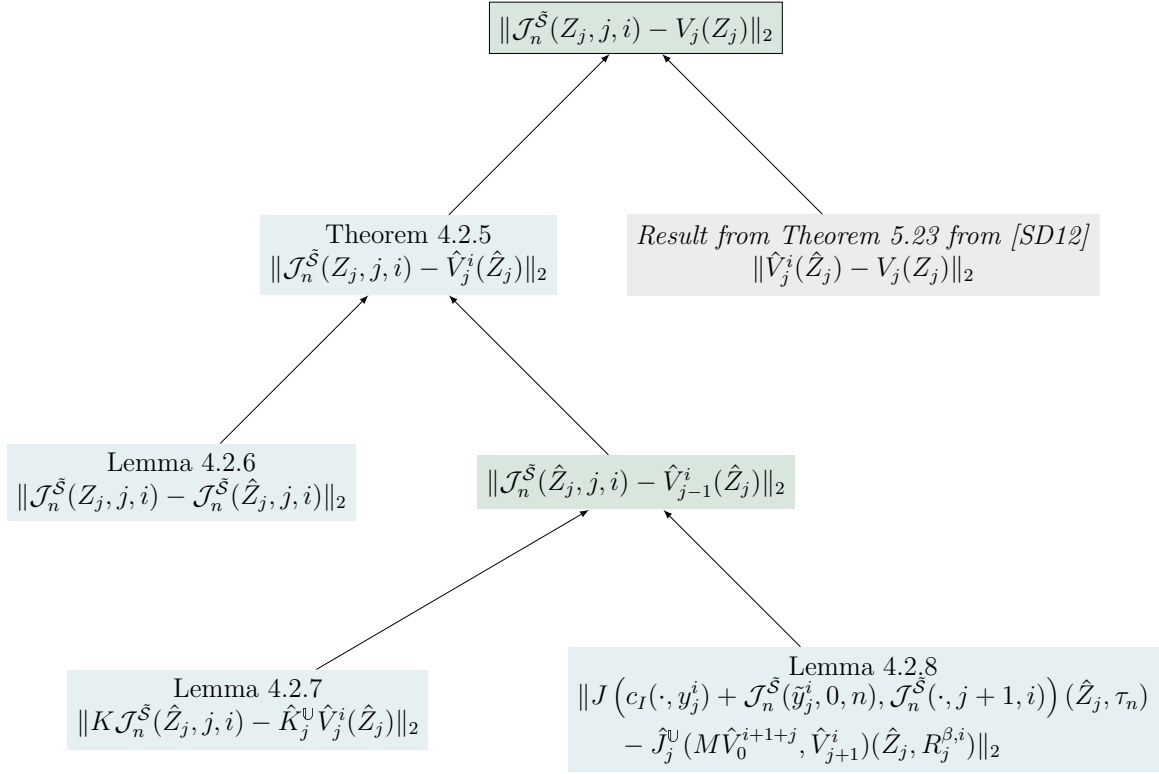


Figure 4.4: The main result (top green box) is proved progressively by decomposing each term into more elementary components. Green boxes correspond to statements that are established using the decomposition of the blue and grey results. Grey boxes refer to known results from the literature, specifically from the 2012 paper [SD12]. Blue boxes present intermediate results that are derived and detailed throughout this chapter.

Remark 4.6 *The use of ϵ in the term ϵ -optimal strategy is somewhat misleading. In practice, the strategy is only arbitrarily close to the true value function, depending on the chosen parameters (e.g., the number of points in the quantization grids).*

In this part, for notation convenience, we denote

$$\mathbb{1}_{\hat{K} < j} = \mathbb{1}_{\hat{K}_j^\cup \hat{V}_{j+1}^i(p_n^\cup(\tilde{Z}_n, \tilde{S}_n)) < \min_{t \in G(\hat{z}_j)} \hat{J}_j^\cup(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(p_n^\cup(\tilde{Z}_n, \tilde{S}_n), t)}$$

$$\mathbb{1}_{\hat{K} \geq j} = \mathbb{1}_{\hat{K}_j^\cup \hat{V}_{j+1}^i(p_n^\cup(\tilde{Z}_n, \tilde{S}_n)) \geq \min_{t \in G(\hat{z}_j)} \hat{J}_j^\cup(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(p_n^\cup(\tilde{Z}_n, \tilde{S}_n), t)}.$$

Theorem 4.2.5 *For all $0 \leq n \leq N$, $0 \leq i \leq n$, $0 \geq j \geq n-i$ and $z \in E$ and $\hat{z}_j = p_n^\cup(z) \in \Gamma_j^{\cup, Z}$:*

$$\|\mathcal{J}_{n-1}^S(Z_j, j, i) - \hat{V}_j^i(\hat{Z}_j)\|_2$$

The proof is going to be detailed in the following sections. We first split the error into two terms. For all $0 \leq n \leq N$, $0 \leq i \leq n$, $0 \leq j \leq n - i$ and $z \in E$ and $\hat{z}_j = p_n^\cup(z) \in \Gamma_j^{\cup, Z}$, we have

$$\|\mathcal{J}_{n-1}^{\tilde{S}}(z_j, j, i) - \hat{V}_j^i(\hat{z}_j)\|_2 \leq \gamma_1 + \gamma_2$$

where

$$\begin{aligned} \gamma_1 &= \|\mathcal{J}_{n-1}^{\tilde{S}}(Z_j, j, i) - \mathcal{J}_{n-1}^{\tilde{S}}(\hat{Z}_j, j, i)\|_2 \\ \gamma_2 &= \|\mathcal{J}_{n-1}^{\tilde{S}}(\hat{Z}_j, j, i) - \hat{V}_j^i(\hat{Z}_j)\|_2. \end{aligned}$$

Lemma 4.2.6 For all $0 \leq n \leq N$, $0 \leq i \leq n$, $0 \leq j \leq n - i$ and $z \in E$ and $\hat{z}_j = p_n^\cup(z) \in \Gamma_j^{\cup, Z}$, an upper bound for γ_1 is

$$\gamma_1 = \|\mathcal{J}_{n-1}^{\tilde{S}}(Z_j, j, i) - \mathcal{J}_{n-1}^{\tilde{S}}(\hat{Z}_j, j, i)\|_2 \leq [\mathcal{J}_{n-1}^{\tilde{S}}] \|Z_j - \hat{Z}_j\|_2$$

Proof.

The first term is easy to handle thanks to Lemma 4.2.4 and follows from Lemma 5.13 in [SD12]. $\square$

Notice that γ_2 is by definition

$$\begin{aligned} \gamma_2 &= \|K \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) \mathbb{1}_{\hat{K} < j} \\ &\quad + J(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \mathbb{1}_{\hat{K} \geq j} \\ &\quad - \hat{K}_j^\cup \hat{V}_{j+1}^i(\hat{Z}_j) \mathbb{1}_{\hat{K} < j} - \hat{J}_j^\cup(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{Z}_j, \tau_n) \mathbb{1}_{\hat{K} \geq j}\|_2 \\ &= \psi_1 \mathbb{1}_{\hat{K} < j} + \psi_2 \mathbb{1}_{\hat{K} \geq j}, \end{aligned}$$

where

$$\psi_1 = \|K \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{K}_j^\cup \hat{V}_{j+1}^i(\hat{Z}_j)\|_2$$

and

$$\begin{aligned} \psi_2 &= \|J(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \\ &\quad - \hat{J}_j^\cup(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{Z}_j, \tau_n)\|_2. \end{aligned}$$

Lemma 4.2.7 For all $0 \leq i \leq N - 1$, $0 \leq j \leq N - 1 - i$ and $\hat{Z}_j \in \Gamma_j^{\cup, Z}$, an upper bound for ψ_1 is

$$\|K \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{K}_j^\cup \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \leq \psi_{1,1} + \psi_{1,2},$$

where $\psi_{1,1}$ is null and $\psi_{1,2} = C_k$.

Proof. Once again, ψ_1 can be split into two parts.

- First, one has

$$\psi_{1,1} = \|K \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{K}_j^\cup \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j, i+1)\|_2.$$

By definition one can write:

$$\begin{aligned} \psi_{1,1} = & \left\| \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{S}_{\bar{n}+1}^k} \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) \mid \hat{Z}_j \right] \right. \\ & \left. - \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{S}_{\bar{n}+1}^k} \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) \mid \hat{Z}_j \right] \right\|_2 = 0. \end{aligned}$$

- Secondly, one has

$$\psi_{1,2} = \|\hat{K}_j^{\cup} \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{K}_j^{\cup} \hat{V}_{j+1}^i(\hat{Z}_j)\|_2.$$

We proceed by backward induction. For $N_0 = 0$, we have by definition $\mathcal{J}_N^{\tilde{S}(0)}(\hat{Z}_j, j, i) = h(\hat{Z}_j)$ and $V_N(\hat{Z}_j) = h(\hat{Z}_j)$. For $N_0 > 0$, our induction hypothesis is:

$$\|\mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \leq C_K$$

By definition, we have

$$\begin{aligned} \psi_{1,2} = & \left\| \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{S}_{\bar{n}+1}^k} \mathcal{J}_{\bar{n}+1}^{\tilde{S}(N_0-1)}(\hat{Z}_j, j+1, i) \mid \hat{Z}_j \right] \right. \\ & \left. - \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{S}_{\bar{n}+1}^k} \hat{V}_{j+1}^i(\hat{Z}_j) \mid \hat{Z}_j \right] \right\|_2 \\ \leq & \|F(\hat{Z}_j, t^*(\hat{Z}_j)) - F(\hat{Z}_j, t^*(\hat{Z}_j))\|_2 + \|\mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j, i+1) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \\ \leq & \|\mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j, i+1) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \\ \leq & C_K. \end{aligned}$$

Hence the result follows from the induction hypothesis:

$$\|K \mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, j+1, i) - \hat{K}_j^{\cup} \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \leq C_k$$

□

Lemma 4.2.8 For all $0 \leq i \leq N-1$, $0 \leq j \leq N-1-i$ and $\hat{Z}_j \in \Gamma_j^{\cup, Z}$, an upper bound for ψ_2 is

$$\begin{aligned} & \|J(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \\ & - \hat{J}_j^{\cup}(M \hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{Z}_j, \tau_n)\|_2 \leq \psi_{2,1} + \psi_{2,2}, \end{aligned}$$

where $\psi_{2,1}$ is null and $\psi_{2,2} = C_J$

Proof. Once again, ψ_2 can be split into two parts.

- First, one has

$$\begin{aligned} \psi_{2,1} = & \left\| J(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \right. \\ & \left. - \hat{J}_j^{\cup}(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \right\|_2. \end{aligned}$$

By definition one can write:

$$\begin{aligned}
\psi_{2,1} &= \left\| \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{s}_j} \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i) \mathbb{1}_{\hat{s}_j < t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \right. \\
&\quad + \mathbb{E} \left[e^{-\gamma(t \wedge t^*(\hat{Z}_j))} c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n) \cdot \mathbb{1}_{\hat{s}_j \geq t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \\
&\quad - \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{s}_j} \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i) \mathbb{1}_{\hat{s}_j < t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \\
&\quad \left. - \mathbb{E} \left[e^{-\gamma(t \wedge t^*(\hat{Z}_j))} c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n) \cdot \mathbb{1}_{\hat{s}_j \geq t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \right\|_2 \\
&= 0.
\end{aligned}$$

- Secondly, one has

$$\begin{aligned}
\psi_{2,2} &= \|\hat{J}_j^{\mathbb{U}}(c_I(\cdot, y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n), \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i))(\hat{Z}_j, \tau_n) \\
&\quad - \hat{J}_j^{\mathbb{U}}(M\hat{V}_0^{i+1+j}, \hat{V}_{j+1}^i)(\hat{Z}_j, \tau_n)\|_2
\end{aligned}$$

We proceed by backward induction. For $N_0 = 0$, we have by definition $\mathcal{J}_N^{\tilde{S}(0)}(\hat{Z}_j, j, i) = h(\hat{Z}_j)$ and $V_N(\hat{Z}_j) = h(\hat{Z}_j)$. For $N_0 > 0$, our induction hypothesis is :

$$\|\mathcal{J}_n^{\tilde{S}}(\hat{Z}_j, 0, n) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \leq C_J$$

By definition, we have :

$$\begin{aligned}
\psi_{2,2} &= \left\| \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{s}_j} \mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i) \mathbb{1}_{\hat{s}_j < t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \right. \\
&\quad + \mathbb{E} \left[e^{-\gamma(t \wedge t^*(\hat{Z}_j))} c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n) \mathbb{1}_{\hat{s}_j \geq t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \\
&\quad - \mathbb{E} \left[F(\hat{Z}_j, t^*(\hat{Z}_j)) + e^{-\gamma \hat{s}_j} \hat{V}_{j+1}^i(\hat{Z}_j) \mathbb{1}_{\hat{s}_j < t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \\
&\quad \left. - \mathbb{E} \left[e^{-\gamma(t \wedge t^*(\hat{Z}_j))} M\hat{V}_0^{i+1+j}(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j))) \mathbb{1}_{\hat{s}_j \geq t \wedge t^*(\hat{Z}_j)} \mid \hat{Z}_j \right] \right\|_2 \\
&\leq \mathbb{E} \left[\|F(\hat{Z}_j, t^*(\hat{Z}_j)) - F(\hat{Z}_j, t^*(\hat{Z}_j))\|_2 \mid \hat{Z}_j \right] \\
&\quad + \mathbb{E} \left[\mathbb{1}_{\hat{s}_j \geq t \wedge t^*(\hat{Z}_j)} e^{-\alpha(t \wedge t^*(\hat{Z}_j))} \right. \\
&\quad \times \left\| c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y_j^i) + \mathcal{J}_n^{\tilde{S}}(y_j^i, 0, n) \right. \\
&\quad \left. - \inf_{y \in \mathbb{U}} \{c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y) + \mathcal{J}_n^{\tilde{S}}(y, 0, n)\} \right\|_2 \mid \hat{Z}_j \Big] \\
&\quad + \mathbb{E} \left[\mathbb{1}_{\hat{s}_j < t \wedge t^*(\hat{Z}_j)} e^{-\gamma \hat{s}_j} \|\mathcal{J}_n^{\tilde{S}}(\cdot, j+1, i) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2 \mid \hat{Z}_j \right]
\end{aligned}$$

By definition of y_j^i , we have:

$$\begin{aligned} & \inf_{y \in \mathbb{U}} \left\{ c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y) + \mathcal{J}_n^{\tilde{\mathcal{S}}}(y, 0, n) \right\} \\ &= c_I(\Phi(\hat{Z}_j, \tau_n \wedge t^*(\hat{Z}_j)), y_j^i) + \mathcal{J}_n^{\tilde{\mathcal{S}}}(y_j^i, 0, n) \end{aligned}$$

We finally have

$$\psi_{2,2} \leq e^{-\gamma \hat{S}_j} \|\mathcal{J}_n^{\tilde{\mathcal{S}}}(\cdot, j+1, i) - \hat{V}_{j+1}^i(\hat{Z}_j)\|_2$$

□

4.3 Numerical Application

We now apply our procedure to our medical PDMP presented in section 3.2, beginning with a detailed description of the procedure and the associated parameters. We then present the numerical results.

4.3.1 Application to Cancer Treatment Optimization

The structure of the PDMP employed in this example is described in detail in Chapter 3, and all model parameters are summarized in Table 4.1. The time horizon of the process is bounded by the patient's follow-up period of six years (2400 days), so the discount factor $\gamma = 0$.

A cost function was specifically designed for this experiment. An initial approach was to penalize the biomarker level, as in previous work [CS21]. However, this approach leads to a major drawback: treating too frequently keeps the biomarker low, but it increases the long-term risk of an incurable relapse, which typically occurs late in the trajectory. We then considered also penalizing the number of treatments directly, but this made the cost function overly restrictive, implicitly favoring a predefined policy.

We ultimately defined a cost function composed of three terms:

$$\int_0^T c_R(X_t) dt + \sum_{n=1}^N c_I(X_{\tau_n}, X_{\tau_n+}) + c_T(X_T),$$

with:

- a running cost, penalizing the duration of treatment: $c_R(x) = c_\ell \mathbb{1}_{\ell=1}$,
- an impulse cost, penalizing each treatment change: $c_I(x = (m, \ell, k, \mathbf{x}), y = (m, \ell', k', \mathbf{x})) = c_a \mathbb{1}_{\ell \neq \ell'}$,
- a terminal cost, penalizing patient death: $c_T(x) = c_D \mathbb{1}_{h=3}$.

Table 4.1: **Controlled PDMP parameter values used in experiments**

Name	Notation	Value
State Space		
Biomarker nominal level	ζ_0	1
Biomarker death level	D	40
Horizon	T	2400
Flow		
Slope parameter in relapse	v_1	$e^{\frac{-6.5+0.5^2}{2}}$
Slope parameter in non curable relapse	v_2	0.03005
Intensity		
Scale parameter of μ_{01}^k	α_1^k	$0.0002/k$
Shape parameter of μ_{01}^k	β_1^k	3.5
Scale parameter of μ_{02}^k	α_2^k	0.00001
Shape parameter of μ_{02}^k	β_2^k	3.5
Scale parameter of $\mu_{12}^{k,0}$	$\alpha_2^{k,0}$	$1.116e^{-20}k/4$
Shape parameter of $\mu_{12}^{k,0}$	$\beta_2^{k,0}$	4.5
Scale parameter of $\mu_{12}^{k,1}$	$\alpha_2^{k,0}$	$5.58e^{-7}k$
Shape parameter of $\mu_{12}^{k,1}$	$\beta_2^{k,0}$	-0.8
Cost function		
Impulse cost	c_a	1
Treatment cost	c_ℓ	0.05
Death cost	c_D	10^6

As defined in Chapter 3, the medical example involves specific constraints that our numerical method is not directly designed to handle. The first constraint requires each treatment to last at least 45 days; to enforce this, we assign an infinite cost to any impulse occurring before this duration has elapsed. The second constraint, related to the time horizon, is addressed within the set of starting points after intervention $\mathbb{U}$, as explained below.

In this medical example, the set of starting points after intervention is given by $\mathbb{U} = E$, which is not finite. This is a real issue, since our method is only guaranteed to work when $\mathbb{U}$ is finite. To address this, we manually discretize $\mathbb{U}$ by including only the admissible values of (ζ, t) associated with all modes (m, ℓ, k) , while allowing at most nine natural impulse jumps. Here, ζ (the biomarker level) is the key variable, while t is introduced to handle constraints: since no impulse can be applied after the end of the follow-up, we must track elapsed time to identify when this horizon is reached. The admissible starting points $\mathbb{U}$ are summarized in Table 4.2.

Table 4.2: **Authorized values of ζ and t used to construct the admissible starting points grid $\mathbb{U}$.** The resulting grid yields a total of 1680 new starting points.

ζ values	1, 2, 5, 10, 15, 20, 30
t values	300, 600, 900, 1200, 1500, 1800, 2100

4.3.2 Practical Implementation

The first step is to set the computational horizon N , which determines the maximum number of allowed natural jumps and impulses. In our scenario, starting from any initial state $x_0 = (\mathbf{m} = (0, 0, 0), \mathbf{x} = (1, 0, 0))$, at most four jumps are required to reach the death mode (i.e., the termination of the process). Accordingly, we fix $N = 9$.

The second step consists in computing the quantized approximations of the sequences (Z_n, S_n) , starting from x_0 , and (Z_n^i, S_n^i) , starting from $\mathbb{U}$. Although the quantization algorithm can be computationally demanding, it is important to note that this step depends on the optimization procedure only through the control set $\mathbb{U}$, and not on the cost function. Therefore, only two families of quantization grids are required: one for the uncontrolled process and one initialized from the admissible control set $\mathbb{U}$. Each grid was constructed with 500 points.

To study the behavior of our controlled PDMP, we estimate the Monte Carlo cost of each strategy using 10^6 controlled trajectories and its 95%-confidence interval. Two constraints are imposed: each treatment must last at least 45 days, and no impulse can be applied at the very end of the follow-up period (2400 days). These constraints are directly incorporated into the benchmark strategies:

- *Nothing*: the process evolves naturally without any intervention;
- *Threshold*: administer the treatment as soon as the biomarker level reaches 5 and the patient is in a treatable relapse and stop the treatment when the patient falls into untreatable relapse or remission, provided it has been applied for at least 45 days; otherwise, continue until the 45-day threshold is reached;
- *Random*: apply or stop the treatment at random, with the constraint that once started, the treatment must be maintained for at least 45 days;
- *Abusive*: after the first 15 days of follow-up, start the treatment and never stop it;
- *Periodic*: administer the treatment every 100 days, each time for a duration of 45 days;
- *Dynamic Programming*: the ϵ -optimal strategy identify with our numerical method.

4.3.3 Numerical Results

Table 4.3 presents the Monte Carlo expected costs of the previously described strategies, providing reference values for comparison. From the results, we observe that the *Threshold* strategy achieves the lowest total cost, primarily due to its minimal terminal cost, indicating that very few patients die under this strategy. The *Nothing* strategy incurs no running or impulse costs but results in a high terminal cost, reflecting poor patient outcomes. The *Random* strategy performs worse than the *Perfect* strategy, with higher running, impulse, and terminal costs. This reflects the inefficiency of unplanned interventions, as both the number of past treatments and elevated biomarker levels increase the risk of an untreatable relapse, leading to a higher

terminal cost and, consequently, higher mortality. The *Abusive* strategy generates extremely high running costs due to continuous treatment, while its terminal cost remains relatively low, showing that excessive treatment reduces mortality but at a substantial cost. Finally, the *Periodic* strategy results in the highest total cost, mainly driven by large impulse costs from frequent scheduled treatments, despite relatively low terminal costs due to low biomarker level.

Our numerical method allows us to obtain an ϵ -strategy that outperforms other predefined strategies. In terms of running cost, the time spent under treatment is shorter than with the threshold strategy (2.61 ± 0.01 vs. 3.51 ± 0.01). Regarding terminal cost, we observe a very low percentage of deaths at the end of the trajectory (0.04 ± 0.01).

Table 4.3: **Expected cost with 95%-confidence intervals of the considered strategies**, computed from 10^6 Monte Carlo controlled trajectories.

Strategy	Running cost	Impulse cost	Terminal cost	Total cost
<i>Nothing</i>	0.0 ± 0.0	0.0 ± 0.0	12.01 ± 0.21	12.01 ± 0.21
<i>Threshold</i>	3.51 ± 0.01	0.60 ± 0.0	0.04 ± 0.00	4.15 ± 0.10
<i>Random</i>	9.10 ± 0.02	1.31 ± 0.0	4.68 ± 0.13	15.10 ± 0.13
<i>Abusive</i>	23.85 ± 0.0	1.0 ± 0.0	0.83 ± 0.06	25.68 ± 0.06
<i>Periodic</i>	7.20 ± 0.0	32.0 ± 0.0	0.38 ± 0.04	39.58 ± 0.04
<i>Dynamic Programming</i>	2.61 ± 0.01	0.60 ± 0.0	0.04 ± 0.01	3.25 ± 0.96

Further work is needed to provide a detailed explanation of the obtained ϵ -strategy. In addition, a sensitivity analysis should be carried out to thoroughly assess the impact of parameter choices on the quality of the results.

4.4 Conclusion

In this chapter, we proposed a numerical method for constructing an explicit control strategy for PDMPs, building on the value function approximation technique introduced in [SD12] and the ϵ -optimal strategy construction developed in [SDG17]. We showed that the resulting candidate strategy is indeed ϵ -optimal. However, this approach relies on strong assumptions: it requires full knowledge of the model and its parameters, as well as perfect observation of the process at all times (or at least at each jump time). Moreover, the method is computationally intensive and applicable only to low-dimensional PDMPs with limited complexity.

This line of research is still ongoing, and further sensitivity analyses are required to assess the robustness of the method with respect to algorithmic parameters, such as the number of grid points or the maximum number of allowed jumps or interventions. The approach could also be employed to calibrate the cost function, thereby excluding those that lead to suspicious ϵ -optimal strategies.

Because such idealized conditions are rarely met in practice, the following chapters explore alternative approaches better suited to more realistic scenarios. In Chapter 5, we address the challenge of partial observation. In Chapter 6, we go further by considering the case where the

model itself is only partially known. However, this idealized framework could be used as an oracle to benchmark the performance of more realistic strategies presented in Chapter 5 and Chapter 6.

Chapter 5

Planning for Partially Observed Impulse Control of PDMPs

5.1	Controlled PDMP as a MDP	107
5.1.1	Formulation as a MDP	107
5.1.2	Application to Cancer Treatment Optimization	110
5.2	Partially Observed Controlled PDMP as a POMDP	112
5.2.1	Formulation as a POMDP	112
5.2.2	Application to Cancer Treatment Optimization	113
5.3	Resolution of a POMDP	114
5.3.1	Experimental Design	114
5.3.2	Numerical Results	116
5.4	Conclusion	119

Obtaining an ϵ -optimal strategy for a controlled Piecewise Deterministic Markov Process (PDMP) requires full knowledge of the underlying model and perfect observation of the system, including exact jump times as shown in Chapter 4. These assumptions are rarely met in real-world applications, where both the system dynamics and observations are often uncertain or incomplete.

A promising direction of recent research is to recast PDMPs within the framework of Markov Decision Processes (MDPs), embedding impulse control problems into this setting [Cle+25]. This reformulation leverages the well-established MDP theory, offering new avenues for analyzing and solving complex PDMP control problems. To make this connection tractable, the class of impulse strategies is typically restricted. More general formulations, involving larger state spaces, are also possible, see [DHP16] for example.

When the process is only partially observed, jump times are hidden, approximating solutions becomes even more challenging. In such settings, previous work [CS18; CS23] propose to formulate the controlled PDMP as a Partially Observable Markov Decision Process (POMDP).

Then they resort to discretizing the state space and employing dynamic programming to approximate the value function, effectively addressing problems of continuous state space and partial observability. While effective, this method is constrained by its reliance on explicit model knowledge and the discretization process.

To relax these requirements, alternative strategies have been developed. Approximation algorithms were developed for POMDP with discrete state space and an unknown model (e.g. [BB01]) but these approaches generally require some knowledge of a good policy class, as well as substantial amounts of data. More recently, simulation-based approaches using Monte Carlo planning have been explored [Sap+24], allowing one to learn decision strategies from simulated trajectories rather than relying on limited or costly real-world data, assuming the model can be simulated.

In this chapter, we revisit the formulation of impulse-controlled PDMPs as POMDPs and propose a new resolution method based on deep reinforcement learning (RL). This approach leverages recent advances in deep learning to approximate optimal policies in high-dimensional, partially observed settings. Crucially, it only requires access to a simulator of the environment, without any need for an explicit model of the underlying dynamics or observation process. This makes it particularly suitable for realistic applications where the system is complex and difficult to model accurately.

The chapter is organized as follows. We begin by recalling, in section 5.1, how a controlled PDMP can be reformulated as a MDP, following the framework introduced in recent literature. To illustrate the transformation, we start with a simplified version of the squirrel example presented through Chapters 1 and 2. We then extend the procedure more formally to the patient monitoring case study introduced in Chapter 3. Next, we address the partially observed setting in section 5.2. We explain how the controlled PDMP can be translated into a POMDP, building on the methodology proposed in [CS23]. Finally, we introduce a resolution method based on deep RL in section 5.3, showing how it can be used to learn control strategies for the resulting POMDP using simulated trajectories. This part of the work is based on the contributions presented in [Cle+24b] and [Cle+24a].

5.1 Controlled PDMP as a MDP

In this section, based on the more general survey written with Alice Cleynen, Benoîte de Saporta, Régis Sabbadin and Amélie Vernay [Cle+25], we show how to convert a controlled PDMP into an MDP, first through the squirrel example from Chapter 2, then by applying the same approach to the patient monitoring model from Chapter 3.

5.1.1 Formulation as a MDP

To formalize control in the restricted setting of impulse controlled PDMPs, where impulse delays lie in a fixed set $\mathbb{T}$ and control is limited to mode switching, we introduce the mode-augmented

PDMP framework. This framework distinguishes between two types of modes: those under the decision-maker's control and that can only change at impulse dates, denoted by L , and those that are not, denoted by M , that can freely change at natural jumps of the process. Formally, we decompose the discrete component $\mathbf{M}$ of the PDMP state space as $\mathbf{M} = L \times M$, so that each mode $\mathbf{m}$ can be written as $\mathbf{m} = (\ell, m) \in L \times M$.

Definition 5.1.1: Mode-augmented controlled PDMP

A mode-augmented controlled Piecewise Deterministic Markov Process is a PDMP defined by the tuple $\langle E, \Phi, \lambda, Q \rangle$, where

- The state space E has the specific form

$$E = \{(\ell, m, \mathbf{x}), \ell \in L, m \in M, \mathbf{x} \in E_m^\ell\},$$

for some mode product set $\mathbf{M} = L \times M$, and where $E_m^\ell = E_{\mathbf{m}=(\ell, m)}$.

- The flow Φ and intensity λ satisfy the conditions from definition 2.1.1.
- The jump kernel Q satisfies the conditions from definition 2.1.1 with the additional constraint that Q cannot change the value of ℓ , as ℓ is intended to be decision-maker chosen.

In line with notation E_m^ℓ , we denote $\Phi_m^\ell = \Phi_{\mathbf{m}=(\ell, m)}$, $\lambda_m^\ell = \lambda_{\mathbf{m}=(\ell, m)}$, and $Q_m^\ell = Q_{\mathbf{m}=(\ell, m)}$.

Example 5.1.1: Squirell toy model as a mode-augmented controlled PDMP

We revisit the squirrel example 2.1.2. Consider a controlled PDMP with very different decisions from example 2.2.2. Now, you have one camera in a location ℓ and you can move it to a new location ℓ' at any time and you can not change the squirrel location. We consider a new mode and time augmented PDMP. Its state space is defined from the modes sets $L = M = \{1, \dots, 4\}$ and the time set $E_m^\ell = [0, 7]$, for all $\ell \in L$. Local characteristics remains the same as presented in example 2.1.2 since the camera location does not influence the squirrel behaviour.

A strategy $\mathcal{S} = (\tau_n, \chi_n)_{n \geq 1}$ is then a sequence of discrete delays in $\mathbb{T} = \{1, \dots, 7\}$ at which you change the camera location and the new location of the process after the change. Here only the camera location can change: an impulsion moves the process from $x = (m, \ell, u)$ to $\chi = (m, \ell', u)$. The cost functions are defined by

$$\begin{aligned} c_R(x = (m, \ell, u)) &= \mathbf{1}_{\ell \neq m}, \\ c_I(x, x') &= c. \end{aligned}$$

The terminal cost is null. If the impulse cost is null (*i.e.* $c = 0$), an ϵ -optimal strategy is to move the camera to the new hiding place of the squirrel as soon as possible (with an infinitesimally small delay ϵ) after the jump. If the impulse cost is not zero, the optimal delay before moving the camera may be longer and may depend on the remaining time until the horizon.

We want to define a MDP with finite action set that models the mode-augmented controlled PDMP. First, we need to separate the controller-chosen modes from the state space. Set

$$E_M = \bigcup_{m \in M} \{m\} \times \left(\bigcup_{\ell \in L} E_m^\ell \right).$$

In the MDP framework, the time steps correspond to the decision dates of the impulse-controlled PDMP. While, by definition, these decision dates can form an infinite set, existing works typically assume that the delay between consecutive impulses is restricted to a finite set $\mathbb{T}$. Unlike in the typical MDP framework, the next decision date is part of the control for PDMPs. In particular, for a fixed horizon H of the impulse control task, different strategies may result in varying numbers of impulse dates, meaning different numbers of decisions. To deal with this difficulty, we add an absorbing cemetery state Δ to the state space. When the real-time horizon H is reached, the process transitions to Δ . Note that the number of decisions of any trajectory is bounded by $H = T/\delta$ where δ is the minimum delay between impulses. Therefore, we will be facing a MDP with an absorbing state Δ , which is reached in bounded time. This allows us to write a stationary infinite horizon MDP which is completely equivalent to the finite horizon controlled PDMP, as follows.

Definition 5.1.2: MDP model of a mode-augmented controlled-PDMP

Let $\langle E, \Phi, \lambda, Q \rangle$ be a mode-augmented controlled PDMP. We define the corresponding MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$ with the following characteristics.

- The state space is $\mathbb{S} = (E_M \times [0, T]) \cup \{\Delta\}$. A state $s \neq \Delta$ is denoted $s = (m, \mathbf{x}, t)$, where t corresponds to the (deterministic) time since the beginning.
- The action space is $\mathbb{A} = L \times \mathbb{T}$. An action is denoted $a = (\ell, r)$, where $r \in \mathbb{T}$ is the chosen delay until the next decision.
- The set of constraints $\mathbb{K} \subset \mathbb{S} \times \mathbb{A}$, defines the available actions in state s . Note that $\mathbb{K}(s) \neq \emptyset$, except $s = \Delta$, $\mathbb{K}(s) = \emptyset$ since Δ is an absorbing state.
- The transition kernel $P(\cdot | s, a)$ specifies the transitions of the MDP. Choosing $a = (\ell, r)$ in state $s = (m, \mathbf{x}, t)$ specifies that decision mode ℓ will be applied from now on, until date $t' = t + r$.

- The cost function $c: \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow \mathbb{R}_+$ expresses the cost incurred during a transition of the MDP model, with $c(\Delta) = 0$.

Remark 5.1 *While the stochastic dynamics of the resulting MDP is quite complex to describe analytically, it can be efficiently simulated when the controlled PDMP model is known.*

Example 5.1.2: Squirrel toy model as a MDP model

Based on the example 5.1.1 we now turn the mode-augmented controlled PDMP into a MDP model. We assume that decisions are taken at discrete time steps t_n , where the time lapse between two visits is 1 day. The associated MDP is defined by $\langle \mathbb{S}, \mathbb{A}, P, c \rangle$.

- The state space corresponds to the state of a squirrel $s \in \mathbb{S} = \{1, \dots, 4\} \times [0, 7]$,
- The action space $\mathbb{A} = \{1, \dots, 4\}$ is the camera location choice,
- The transition function P can be written as a combination of the PDMP flow, jump intensity and Markov kernel. Detailed analytic formulas are omitted here, but notice that the MDP transition function is set according to the PDMP parameters and is easily simulated,
- The cost function c as introduced in Example 5.1.1 is approximated by $c(s, a) = \mathbb{1}_{m \neq \ell} + c$ where c corresponds to a location change
- The horizon $H = 7$.

Solving a mode-augmented controlled PDMP $\langle E, \Phi, \lambda, Q \rangle$ corresponds to computing an optimal strategy of the corresponding stationary, total-cost MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$.

Remark 5.2 *Even though π^* is a stationary strategy of the MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$, it is not stationary, in general, when translated in the controlled PDMP $\langle E, \Phi, \lambda, Q \rangle$, since states $s \in \mathbb{S}$ include the time variable.*

5.1.2 Application to Cancer Treatment Optimization

We now describe how the patient monitoring model, initially formulated as a PDMP in Section 3.2, is converted into MDP framework. As a reminder from Chapter 3, Figure 6.2 illustrates a fully observed patient trajectory, highlighting the dynamics of the PDMP flow.

We assume that visits take place at discrete dates, and that the time lapse between two visits can be 15, 30 or 60 days. Thus at most $H = \frac{H}{15}$ (recall that H is assumed to be a multiple of 15) visits can occur. Moreover, decision-related constraints appear as we impose that the last visit must take place at the end H of the follow-up.

The associated MDP is $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$. Its characteristics are defined as follows.

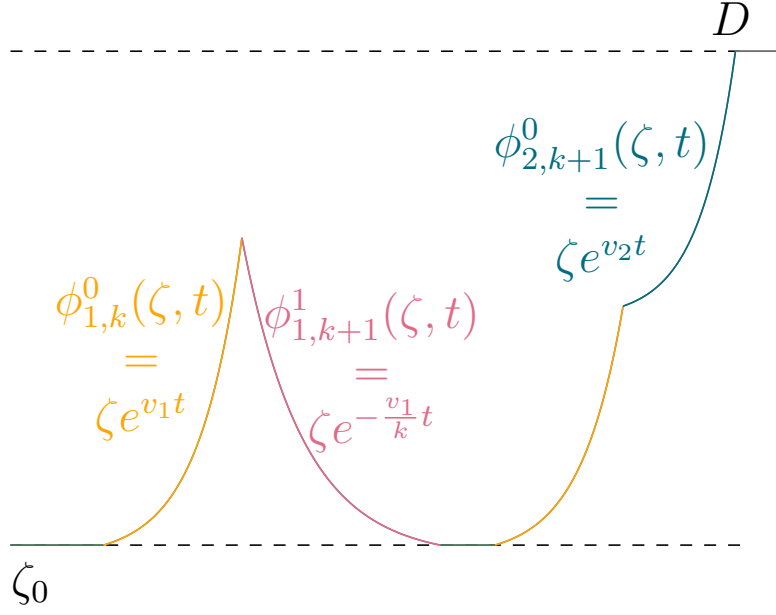


Figure 5.1: **Example of a simulated patient trajectory, highlighting the flow functions.** Green curves indicate the remission state, orange curves represent relapse without treatment, pink curves indicate relapse under treatment, and blue curves represent untreated relapse. Grey curves indicate the patient is dead. Flow under remission and when the patient is dead are omitted as they are constant.

- The state space $\mathbb{S}$ corresponds to the state of a patient $s = (h, k, \zeta, u, \tau, t)$, where $s \in E_m \times [0, T]$, along with an absorbing state Δ . Here, t denotes the time elapsed since the start of follow-up.
- The action space $\mathbb{A}$ collects the decision pairs $a = (\ell, r)$, where $r \in \mathbb{T} = \{15, 30, 60\}$ is the delay until the next visit and $\ell \in L = \{0, 1\}$ is the treatment choice.
- The set of constraints $\mathbb{K}(s) \subseteq \mathbb{A}$:

$$\mathbb{K}(s) = \begin{cases} \emptyset & \text{if } s = \Delta \\ (1, r), r \in \mathbb{T} & \text{such that } t + r \leq H \text{ and if } \tau \in (0, 45) \\ (\ell, r) \in L \times \mathbb{T} & \text{such that } t + r \leq H \end{cases}$$

- The transition kernel P can be written from the PDMP transition kernel $\mathcal{P}$, defined in Section 3.2.3. For all real-valued bounded measurable function g on $\mathbb{S}$, let $Pg(s) = \mathbb{E}[g(S_r)|S_0 = s]$. Then one has

$$Pg(s, a) = \begin{cases} g(\Delta) & \text{if } s = \Delta \\ \mathcal{P}g(h, \ell, k, \zeta, u, \tau, t + r) & \text{else.} \end{cases}$$

- The cost function c is $c(s, a, s') = r\ell C_\ell + C_V + C_D \mathbf{1}_{s=\Delta}$ where $a = (\ell, r)$, *i.e.* the cost is

composed of a fixed visit cost C_V , a cost of treatment $r\ell C_\ell$ proportional to the treatment duration and a death cost C_D .

- The horizon $H = \lfloor H/15 \rfloor$.

5.2 Partially Observed Controlled PDMP as a POMDP

In this section, we build on the survey by [Cle+25] and the results from [CS23] to reformulate partially observed controlled PDMPs as POMDPs. We then extend the patient monitoring case formulated as a MDP introduced in Section 5.1.2 by incorporating the partial observation issues detailed in Chapter 3.

5.2.1 Formulation as a POMDP

The state $(\ell, m, \mathbf{x})$ of a mode-augmented controlled PDMP may not be fully observed, apart from ℓ , which is controlled by the decision-maker. In particular, the mode m may be hidden, or the Euclidean part $\mathbf{x}$ observed with noise.

Since partial observability does not impact the PDMP transition model, it is easy to incorporate it directly into the MDP model of the now partially-observed controlled PDMP. This requires the addition of an observation space Ω , which can be discrete, continuous or hybrid. We then define the observation kernel O of the PDMP as a Markov kernel on $(\mathcal{B}(\Omega), E_M \times L)$ corresponding to the conditional probability distribution of the next observation $\omega \in \Omega$ given that action $\ell \in L$ was chosen and led to state $(m', \mathbf{x}')$ in the controlled PDMP. We assume, for sake of simplicity, that the observation function does not depend on time, but this assumption can be easily relaxed. Solving a partially-observed controlled PDMP problem thus amounts to solve a POMDP $\langle \mathbb{S}, \mathbb{A}, \Omega, \mathbb{K}, P, O, c, H \rangle$ where the observation space Ω and the observation kernel $O : (\mathcal{B}(\Omega), \mathbb{S} \times \mathbb{A})$ are simple extensions of that of the PDMP, obtained by assuming full-observability of the time variable.

Definition 5.2.1: POMDP model of a mode-augmented controlled-PDMP

Let $\langle E, \Phi, \lambda, Q \rangle$ be a mode-augmented controlled PDMP. We define the corresponding POMDP $\langle \mathbb{S}, \mathbb{A}, \Omega, \mathbb{K}, P, O, c, H \rangle$ with the following characteristics.

- The state space $\mathbb{S}$, the action space $\mathbb{A}$, the transition kernel P , the cost function c and the horizon H are the same as those defined in 5.1.1.
- The observation space is $\Omega = \Omega \times [0, T]$.
- The set of constraints $\mathbb{K} \subset \Omega \times \mathbb{A}$, defines the available actions in observation ω . Note that for $\omega = \Delta$, $\mathbb{K}(\omega) = \{\emptyset\}$ since Δ corresponds to an observation in an absorbing state.

- The observation kernel is $Og(s' = (m', \mathbf{x}', t'), a = (\ell, r)) = \int_{\Omega} g(\omega) O(m', \mathbf{x}', \ell, d\omega)$ for any real-valued bounded measurable function g on Ω .

Remark 5.3 *This POMDP has a hybrid observation space in addition to a hybrid state space.*

Remark 5.4 *Optimal policies of partially-observed controlled PDMP are history dependent (i.e. optimal action choices depend on the sequence of past actions and observations) in general, as for usual POMDP.*

5.2.2 Application to Cancer Treatment Optimization

As detailed in Chapter 3, marker measurements are intrinsically subject to variations independent of the medical condition. These fluctuations can be attributed to measurement errors, natural variations and external influences. The biomarker is thus observed through an additive noise. Let $y = \zeta + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$ be the noisy biomarker. In addition, the patient's overall health h is hidden, together with the time u since the last change of health status, except when the patient is deceased, with z the death indicator. The time since the current treatment started τ and the number of treatment k are perfectly observed.

The associated POMDP $\langle \mathbb{S}, \mathbb{A}, \Omega, \mathbb{K}, P, O, c, H \rangle$, is building on the MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$ described in 5.1.2. Its characteristics are defined as follows.

- The state space $\mathbb{S}$, the action space $\mathbb{A}$, the transition kernel P , the cost function c and the horizon H are the same as those defined in 5.1.2.
- The observation space Ω corresponds to the set of all possible observation ω of a patient $\omega = (k, z, y, \tau, t)$, with $z = \mathbb{1}_{h=3}$ and $y = \zeta + \epsilon$ where ϵ are real-valued independent and identically distributed random variables with density f independent from the controlled PDMP. Here f correspond to the Gaussian distribution.
- The set of constraints $\mathbb{K}(\omega) \subseteq \mathbb{A}$:

$$\mathbb{K}(\omega) = \begin{cases} \emptyset & \text{if } \omega = \Delta \\ (1, r), r \in \mathbb{T} & \text{such that } t + r \leq H \text{ and if } \tau \in (0, 45) \\ (\ell, r) \in L \times \mathbb{T} & \text{such that } t + r \leq H \end{cases}$$

- The observation kernel O does not rely on the action a . For all real-valued bounded measurable function q for $s' = (m' = (h', k'), \mathbf{x}', t') \in E_M \times [0, T]$

$$Oq(s', \omega) = \begin{cases} q(\Delta) & \text{if } s' = \Delta, \\ \int q(k, 0, y, \tau, t) f(y - \zeta') dy & \text{otherwise.} \end{cases}$$

5.3 Resolution of a POMDP

We now present how the resulting POMDP can be solved using deep RL algorithms. We begin by detailing the experimental setup, including the basic algorithms and their variants that will be benchmarked on the medical model introduced in section 5.2.2. Next, we analyse the results of these numerical experiments and compare them with the results discussed in Section 4.3. Finally, for the best performing algorithm, we attempt to interpret and explain the learned policy.

5.3.1 Experimental Design

We implemented a Gymnasium [Tow+24] compatible RL environment in Python to simulate the trajectories of the cancer treatment follow-up POMDP model, presented in Section 5.2.2. We conducted numerical experiments using two baseline algorithms: Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN), both of which are widely used in the deep RL community and more detailed in Chapter 1, section 1.3.3. For the PPO and DQN algorithms, we utilized the implementations available in RLlib [Lia+18b; Wu+21], an open-source Python library specialized in deep RL algorithms. Code are available at GitLab¹.

Reminder: Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN)

DQN is a *value-based* deep RL algorithm that approximates the value function. For each seen transition, it computes the Q-values for each possible action. Then it selects the action that minimizes the Q-values. This approach provides deterministic policies.

PPO is a *policy-based* deep RL algorithm that does not directly approximate the value function. It associates each seen transition with actions, optimizing the parametrized policy to maximize expected rewards. This approach provides stochastic policies.

Both PPO and DQN were originally designed for fully observable MDPs. To address partial observability, we first computed all terms based on the current observation rather than the true state. Subsequently, we integrated Long Short-Term Memory (LSTM) networks into PPO to capture temporal dependencies. LSTMs are capable of maintaining an internal memory of past observed transitions, which allows the policy to infer hidden states and make more informed decisions even when the current observation does not provide complete information about the environment.

Additionally, to handle problem-related constraints, we test two different methods. First, constraints are directly implemented in the cost function with prohibitive cost:

$$c_K(s, a, s') = c(s, a, s') + C_H \mathbb{1}_{t>H} + C_T \mathbb{1}_{\ell=0} \mathbb{1}_{0<\tau<45},$$

¹https://forge.inrae.fr/orlane.rossini/controlled_pdmp_po

where C_H is a cost for exceeding the horizon and C_T penalizes stopping treatment too early. Second action masking is applied to the action space, which prevents the agent from selecting actions that violate the constraints and the cost function is simply $c(s, a, s')$. All the parameter values of the underlying PDMP model are present in Table B.1 and the values specific to the POMDP are in Table 5.1. All experiments that will be conducted are summarized in Figure 5.2.

Table 5.1: **POMDP parameter values used in experiments**

Name	Notation	Value
Observation		
Noise	$\epsilon \sim \mathcal{N}(\mu, \sigma)$	$(\mu = 0, \sigma = 1)$
Cost function		
Treatment cost	C_ℓ	0.1
Visit cost	C_v	1
Death cost	C_D	300
Horizon constraint cost	C_H	1000
Treatment constraint cost	C_T	1000

It is important to note that both PPO and DQN are inherently designed to optimize a reward function rather than a cost function. Therefore, our analysis focuses on the expected reward rather than the expected cost. However, since our reward function is simply the negation of the cost function, i.e., $r(s, a, s') = -c(s, a, s')$, maximizing the expected reward is equivalent to minimizing the expected cost. This ensures that our results remain aligned with our objective, even though they are expressed in terms of rewards.

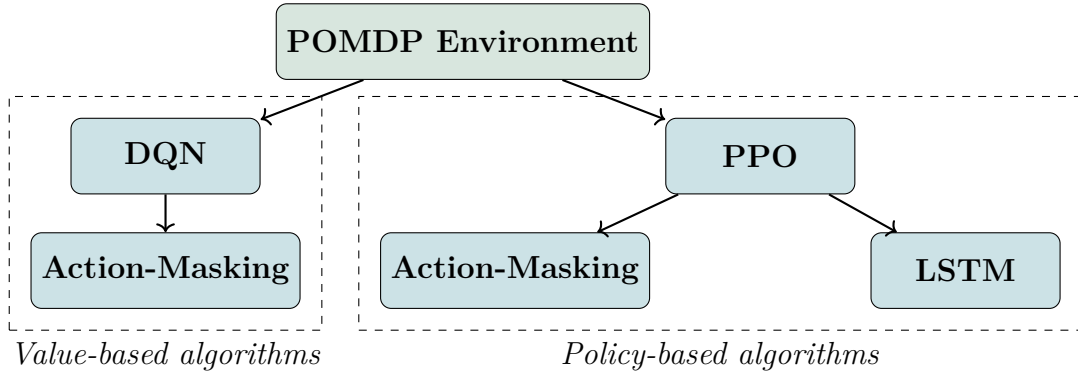


Figure 5.2: **Overview of the numerical experiments.** The green box represents the environment, modelled as a POMDP, which generates trajectories for training the algorithms. Each blue box corresponds to a distinct experiment, defined by a specific algorithm (PPO, DQN), whether or not an LSTM network is incorporated, and whether or not action masking is applied to manage constraints.

First, we performed a hyperparameter search for each algorithm variant with trajectories generated from POMDP simulator. This tuning process covered parameters such as the discount factor, learning rate, batch size, and gradient clipping threshold, aimed at improving training stability and efficiency. We also explored different network architecture choices, including fully

connected layer sizes and activation functions, to enhance representation learning. In addition, we investigated the use of generalized advantage estimation (GAE), various observation filters, and value function loss coefficients to improve policy optimization. The specific values for these hyperparameters are provided in Table B.3.

Hyperparameter were optimised using **Ray Tune** [Lia+18b; Wu+21], exploring 1 000 different configurations. Each configuration corresponds to a specific set of hyperparameters and is evaluated in the selected environment. Training for each configuration is stopped as soon as it reaches 100 000 simulation timesteps or 1 000 training iterations, whichever occurs first. The performance of each configuration is measured using the average reward per episode. The best configuration is defined as the configuration achieving the highest value among all tested settings.

Then, using the best configuration, each algorithm was run with 5 random seeds. Training was stopped once it reached 100 000 simulation timesteps or 1 000 training iterations, whichever occurs first. Final hyperparameter values are provided in Table B.4. Every 5 training iterations the current policy is evaluated in the environment without exploration noise. These evaluation runs use 10 episodes per evaluation point to compute the average reward per episode under the current policy. Tracking this metric over time allows us to assess how quickly each algorithm improves its performance and reaches a high-reward policy.

We evaluated the effectiveness of each resulting policy obtained from the different configurations. Each of the five algorithms was trained with five different random seeds, resulting in five policies per algorithm. For every trained policy, we randomly selected one of the five training samples and performed 10,000 Monte Carlo simulations of full trajectories, using the learned policy to act in the environment. We then computed the average cumulative reward and constructed confidence intervals to assess both the performance and variability of each policy.

Finally, we attempt to interpret the best-performing policy. First, we identified the best algorithm, defined as the one achieving the highest average cumulative reward with stable performance. Then, within this algorithm, we selected the best policy among the five trained with different seeds. To do so, we compute several indicators such as survival rates, the average number of treatments, the average time spent under treatment, the mean visit frequency and the average delay between two visits. These indicators are derived from 10 000 Monte Carlo simulations of full trajectories, providing an initial insight into the policy’s behavior.

5.3.2 Numerical Results

Performance during training. We evaluate performance of each algorithm, with the best configuration, by measuring the mean episode reward as a function of the number of training experiences. Figures 5.3 illustrate these results. Note that the plots do not all start at the same point on the x-axis because different configurations may begin generating meaningful rewards at different stages of training.

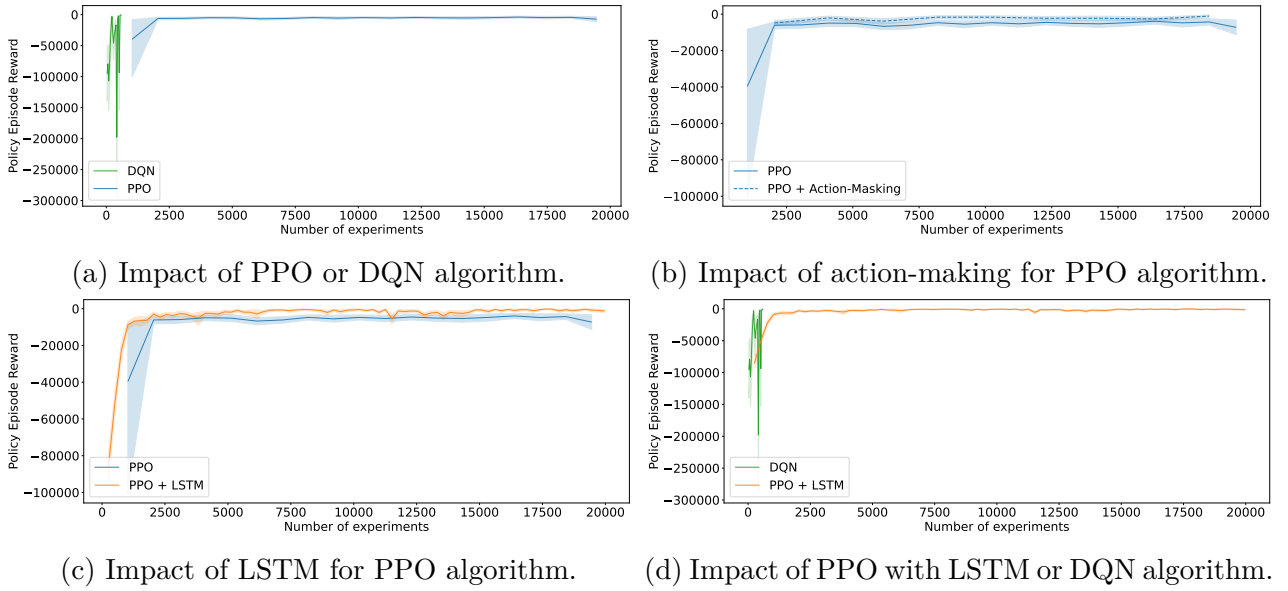


Figure 5.3: Impact of algorithm choice on the performance in terms of mean episode reward per episode with respect to the number of training experiences used on POMDP model.

- Impact of algorithm type (value-based vs. policy-based). Figure 5.3a compares DQN and PPO, both using action penalty, to assess the effect of algorithm type. PPO shows performance degradation after approximately 20,000 training iterations, indicating over-fitting or training collapse; training should therefore be stopped at this point. DQN performances are slightly better than PPO at the end of the training. However, DQN exhibits high variability, with large reward fluctuations and limited stability, whereas PPO demonstrates a more stable training process.
- Impact of action masking. Figure 5.3b compares handling invalid actions with action masking versus action penalties in PPO. We observe that PPO with an action penalty requires significantly more training interactions to learn to avoid invalid actions, whereas action masking enables the agent to incorporate this constraint directly and learn faster. Any comparison with DQN with AM is not presented, as the masked version produced too few evaluation points to allow for a meaningful analysis.
- Impact of memory in PPO. Figure 5.3c examines the impact of adding memory (LSTM) to PPO. PPO-LSTM exhibits substantially higher performance during the initial training iterations before converging to slightly better results.

Overall, the results highlight the more conservative nature of PPO updates: the clipping mechanism limits abrupt policy changes, which contributes to lower variance. Additionally, incorporating LSTM or using action-masking appears to improve the modeling of temporal correlations, enabling more informed and consistent policy updates.

Performance of policies. To determine which algorithm should be used, we evaluated the resulting policies in the POMDP environment consistent across all simulated patients. The

Table 5.2: **Policy interpretation performance on 10 000 Monte-Carlo simulations.**

Configuration	Expected Reward	Confidence Interval
<i>Policy-based algorithms</i>		
PPO	-9.09	± 0.02
PPO with AM	-8.02	± 0.02
PPO with LSTM	-11.73	± 0.03
<i>Value-based algorithms</i>		
DQN	-7.75	± 0.02
DQN with AM	-9.38	± 0.02

Table 5.3: **Summary of policy behavior indicators based on 10 000 Monte-Carlo simulations.**

Indicator	DQN	PPO with AM
Survival rates	$63.51\% \pm 0.01$	$99.90\% \pm 0.00$
Average number of treatment	0.00 ± 0.00	1.74 ± 0.02
Average time spend under treatment	0.00 ± 0.00	$2\,293.64 \pm 0.15$
Average number of visit	35.45 ± 0.12	39.95 ± 0.02
Average delay between two visits	60 ± 0.00	58.62 ± 0.02
Rate of visits occurring within 15 days	0.00 ± 0.00	0.21 ± 0.01
Rate of visits occurring within 30 days	0.00 ± 0.00	4.34 ± 0.06
Rate of visits occurring within 60 days	100 ± 0.00	95.45 ± 0.06

results are summarized in Table 5.2. Those results highlight that DQN seems to be better-suited to our problem than PPO.

Policies interpretation. DQN achieve the best performance in terms of expected reward (-7.75 ± 0.02), followed closely by PPO with AM (-8.02 ± 0.02). An important distinction is that the DQN policy is deterministic, whereas the PPO with AM policy is stochastic. Moreover, PPO with AM provides the additional guarantee that no forbidden actions are ever selected during execution, ensuring compliance with the environment’s constraints. Among the five trained policies, the best is chosen by highest average cumulative reward. For DQN, the best policy reaches -4.98 ± 0.02 and for PPO with AM, it reaches -6.27 ± 0.01 . Reported indicators, available in Table 5.3, provide an explicit characterization of the policies obtained with DQN and PPO with AM.

The two policies display very different behaviors in how they manage treatment and follow-up. The DQN policy never prescribes chemotherapy and schedules longer visit delays (60 days). This behavior results in relatively few visits and a moderate survival rate ($\approx 63\%$). In contrast, the PPO policy with AM prescribes treatments much more frequently (1.74 ± 0.02 on average), but also schedules longer visit delays (60 days) resulting in a long cumulative time under chemotherapy ($\approx 2\,294$ days). This leads to more intensive monitoring and treatment, reflected in higher survival rates ($\approx 99\%$), but also a substantially treatment burden. In other words, DQN behaves like a minimal-intervention policy, while PPO with AM behaves like an aggressive treatment strategy.

5.4 Conclusion

In this chapter, we introduced a novel approach to solving a PDMP that has been transformed into a POMDP. This approach relies on deep RL, which allows policies to be learned from simulated trajectories without requiring an explicit model of the underlying dynamics or the observation process.

In our experiments, DQN achieve the best performance in terms of expected reward, followed closely by PPO with action-masking. While the calculated performance metrics provide information about the overall behavior of these strategies, they do not provide an explicit and interpretable policy. Therefore, although we can infer trends from the metrics, the precise decision rules underlying the learned policies remain opaque.

When selecting an algorithm, it is important not to base the choice solely on raw performance measurements. For example, PPO produces a stochastic policy, whereas DQN produces a deterministic one. Depending on the application, it may be preferable to have a deterministic policy, for example in medical treatment protocols where consistency and reproducibility are essential. In addition, DQN’s use of a replay buffer allows learning from previously collected data. Although this feature was not implemented in our work, it offers the possibility of exploiting real patient trajectories collected as part of standard treatment protocols, making better use of available clinical data and improving policy learning.

However, the method introduced in this Chapter assumes access to a simulator of the environment, or at least sufficient model knowledge to construct one, which is not always feasible in real-world applications. In Chapter 6, we will explore how deep RL can be applied to control problems involving PDMPs when some parameters are ill-known.

Chapter 6

Planning under Model Uncertainty for Impulse Control of PDMPs

6.1	Controlled PDMP as a BAMDP	122
6.1.1	Controlled PDMP with ill-known parameters	122
6.1.2	Formulation as a BAMDP	122
6.1.3	Application to Cancer Treatment Optimization	125
6.2	Partially-Observed Controlled PDMP as a BAPOMDP	126
6.2.1	Formulation as a BAPOMDP	127
6.2.2	Application to Cancer Treatment Optimization	129
6.3	Solving a BAPOMDP with Deep RL	129
6.3.1	Experimental Design	129
6.3.2	Numerical Results	131
6.4	Comparative Study of Modeling Frameworks	134
6.4.1	Experimental Design	134
6.4.2	Numerical Results	135
6.5	Conclusion	137

To run deep reinforcement learning (RL) algorithms, a large amount of data is usually required before the agent converges to a reasonably good policy. In practice, however, such large datasets are rarely available, and it is often infeasible or unsafe to let the agent explore random actions in the real world, especially during the early stages of learning. One common solution is to build a simulator that replicates the real-world environment. A simulator allows the generation of unlimited artificial data and enables the agent to explore various scenarios without any real-world consequences. This is the approach we took in Chapter 5.

However, in many real-world applications, the true dynamics of the environment are unknown, making it impossible to build an accurate simulator from scratch. In such cases, one

can try to estimate the unknown parameters from real data. Yet, when the available data is limited, the quality of these estimators can be poor. More importantly, any simulator built from these estimates should account for the uncertainty associated with the estimated parameters. The Bayesian inference framework provides a natural way to incorporate this uncertainty. In particular, the controlled PDMP framework relies on a flow Φ , a jump intensity λ , and a probability kernel Q , which may be ill-known in practice. To address this, we assume that Φ , λ , and Q are parameterized and that their parameters are unknown and we seek to account for their associated uncertainty in a Bayesian setting. In this work, we focus on the case where a single parameter in the parametric flow is unknown, while all other aspects of the dynamics are assumed to be fully known, though the general principles of the approach remain unchanged.

[Duf02] initially proposed a Bayesian approach to represent and solve finite horizon Markov Decision Processes (MDPs) with an unknown transition model. [Ros+11] later extended this approach to Partially Observable MDPs (POMDPs) by incorporating uncertainty about state transition and observability function. This framework is described as Bayes-Adaptive Partially Observable MDPs (BAPOMDPs). In a BAPOMDP, the agent must not only make decisions based on partial observations of the environment, but also adapt to the current knowledge about the unknown parameters that affect the system dynamics. These parameters are treated as random variables with associated prior distributions, which are updated over time using Bayesian inference as new observations are acquired. This allows the agent to maintain and refine its belief about the environment while interacting with the system, improving decision-making in the face of both partial observability and uncertainty about the system’s underlying dynamics.

Katt et al. [Kat+22] proposed a deep reinforcement learning (RL) framework for solving general BAPOMDP problems, in which the prior-to-posterior transition is modeled by a neural network. In this chapter, we study a setting where the parametric flow of a controlled PDMP depends on a single unknown parameter, while all other aspects of the dynamics are fully known. We reformulate this partially observed, impulse-controlled PDMP with an unknown parameter as a BAPOMDP. This transformation makes it possible to build a simulator with completely known dynamics, thereby enabling the use of deep RL to address the original control problem. However, in contrast to [Kat+22], we retain an explicit prior transition function in our BAPOMDP formulation and apply deep RL solely to solve the corresponding POMDP.

The chapter is supported by an ongoing paper [Ros+25] and structured as follows. We begin with a didactic introduction to the transformation of a fully observed PDMP with ill-known parameters into a fully-known Bayes-Adaptive MDP (BAMDP) in section 6.1. We then extend this framework to partial observability, introducing the BAPOMDP model in section 6.2, which is the core focus of this chapter. Special attention is given to the challenges arising from the continuous-time nature of PDMPs, particularly in modeling parameter uncertainty. The methodology is illustrated through the patient monitoring case study from Chapter 3. Next, we present how to solve the resulting BAPOMDP using deep RL in section 6.3. To conclude, we assess the added value of the BAPOMDP framework compared to the POMDP framework

with an estimated parameter in section 6.4.

6.1 Controlled PDMP as a BAMDP

We leverage the BAMDP framework to effectively update the agent’s beliefs about this parameter over time, improving decision-making despite uncertainty about the system’s parametric flow. Then, we illustrate the transformation of a PDMP into a BAMDP using the medical example.

6.1.1 Controlled PDMP with ill-known parameters

The controlled PDMP framework relies on a flow Φ , a jump intensity λ and a probability kernel Q which may be ill-known in practice. In the following, we assume that Φ is parameterized and that one of its parameters is unknown.

BAMDP, introduced in [Duf02], precisely extend the MDP framework by incorporating uncertainty about state transition and observability function. In a BAMDP, the agent must adapt to the current knowledge about the unknown parameters that affect the system dynamics. These parameters are treated as random variables with associated prior distributions, which are updated over time using Bayesian inference as new observations are acquired. This allows the agent to maintain and refine its belief about the environment while interacting with the system, improving decision-making in the face of uncertainty about the system’s underlying dynamics.

Note that the BAMDP framework is not an online reinforcement learning framework. Instead, updates to the MDP model based on future transitions are predetermined before taking any action and are already incorporated into the BAMDP model. Thus, while the original MDP model is ill-known, the constructed BAMDP model is fully-known at planning time.

6.1.2 Formulation as a BAMDP

Consider that the distribution of the unknown parameter in the parametric flow can be estimated with Bayesian inference.

Definition 6.1.1: BAMDP model of a mode-augmented controlled-PDMP

Let $\langle E, \Phi, \lambda, Q \rangle$ be a mode-augmented controlled PDMP with parametric flow, where one flow parameter v is unknown but supposed to have a probability density $f_{v|\theta}$ parameterized by a vector of hyper-parameters θ . The corresponding BAMDP is defined by $\langle \tilde{\mathbb{S}}, \mathbb{A}, \mathbb{K}, \tilde{P}, c, H \rangle$ with the following characteristics:

- The hyper state space is $\tilde{\mathbb{S}} = \mathbb{S} \times \Theta$, where $\mathbb{S}$ is the same as in Definition 5.1.1 and $\Theta \subset \mathbb{R}^d$ is the set of hyper parameters’ possible values.

- The action space $\mathbb{A}$, the cost function c and the horizon H are the same as in Definition 5.1.1. They do not depend on Θ .
- The set of constraints $\mathbb{K} \subset \tilde{\mathbb{S}} \times \mathbb{A}$, defines the available actions in the hyperstate $\tilde{s} = (s, \theta)$, with $\mathbb{K}(\tilde{s}) \neq \emptyset, \forall \tilde{s} \in \tilde{\mathbb{S}}$.
- The transition kernel $\tilde{P}(\cdot | \tilde{s} = (s, \theta), a)$ specifies the transitions of the BAMDP. Choosing $a = (\ell, r)$ in state $\tilde{s} = (s, \theta)$ specifies that decision mode ℓ will be applied from now on, until date $t' = t + r$.

$$\tilde{P}(\tilde{s}' = (s', \theta') \in B_{\mathbb{S}} \times B_{\Theta} | \tilde{s} = (s, \theta), a) = \int_{B_{\mathbb{S}}} \mathbb{1}_{B_{\Theta}} \mathcal{U}(\theta, s, a, s') P(ds' | s, a, \theta)$$

for any Borel subsets $B_{\mathbb{S}}, B_{\Theta}$ of $\mathbb{S}$ and Θ , where $\mathcal{U}(\theta, s, a, s')$ is the posterior update function.

Remark 6.1 *The only constraints we consider are those related to exceeding the horizon, meaning they are independent of history. However, depending on the specific problem, additional constraints may need to be incorporated.*

Typically authors assume that the prior over the transition function follows a Dirichlet distribution [Duf02; Ros+11]. In contrast, our setting places the prior on a specific parameter of the transition function, and this parameter has continuous support. As a result, the posterior update does not directly depend on the transition $\langle s, a, s' \rangle$, as in the classical case, but instead requires inferring the unknown parameter from the observed transition:

$$\mathcal{U}(\theta, s, a, s') = \mathcal{W}(\theta, \mathcal{V}(s, a, s')),$$

where $\mathcal{V}(s, a, s')$ denotes the inference function for the unknown parameter, and $\mathcal{W}(\theta, v')$ represents the prior update function. To align with the medical example illustrating this paper, we assume that the flow parameter follows a log-normal distribution. Consequently, we model the prior distribution over the log-normal parameters using a Normal-Gamma prior [Fin97], though other prior distributions could also be considered. In Theorem 6.1.1, we provide the update formulas for the Normal-Gamma distribution, which will be used in the next section to update the hyperparameters of the BAMDP. The complete transition function between hyperstates is summarized in Figure 6.1.

Theorem 6.1.1 *Let $v = (v^1, \dots, v^n)$ be an i.i.d. sample drawn from a log-normal distribution:*

$$v^i | \mu, \sigma^2 \sim \text{Log-}\mathcal{N}(\mu, 1/\sigma^2),$$

where the mean μ and the variance σ^2 are unknown.

The conjugate prior for $(\mu, 1/\sigma^2)$ is assumed to be a normal-gamma distribution:

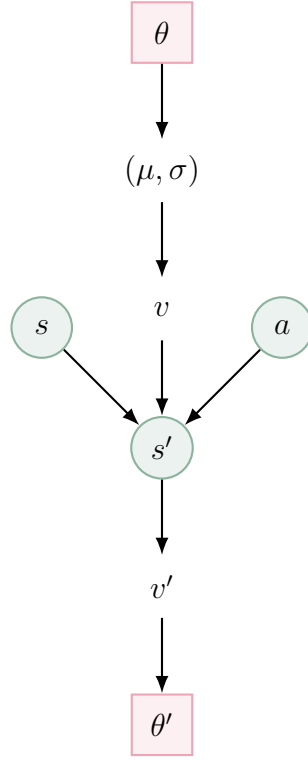


Figure 6.1: **Transition model in a BAMDP framework for cancer application.** White nodes represent latent (simulated) variables, green nodes denote the state and action, and the pink node corresponds to the hyperparameter being updated.

$$(\mu, 1/\sigma^2) \sim \mathcal{N}\Gamma(\mu_0, \kappa_0, \alpha_0, \beta_0),$$

where $(\mu_0, \kappa_0, \alpha_0, \beta_0)$ are the prior parameters.

Assume that, at time n , the posterior distribution of $(\mu, 1/\sigma^2)$ follows a normal-gamma distribution:

$$(\mu, 1/\sigma^2) \sim \mathcal{N}\Gamma(\mu_n, \kappa_n, \alpha_n, \beta_n).$$

Then, after a new observation v^{n+1} , the posterior parameters are given by:

$$\begin{aligned} \mu_{n+1} &= \frac{\kappa_n \mu_n + \log(v^{n+1})}{\kappa_n + 1} \\ \kappa_{n+1} &= \kappa_n + 1 \\ \alpha_{n+1} &= \alpha_n + 1/2 \\ \beta_{n+1} &= \beta_n + \frac{\kappa_n (\log(v^{n+1}) - \mu_n)^2}{2(\kappa_n + 1)}. \end{aligned}$$

Proof. The proof is derived from [Mur07; Fin97] and available in Appendix ??.

□

Remark 6.2 In a BAMDP, policies $\pi : \tilde{S} \rightarrow \mathbb{A}$ choose actions as functions of the current state

s and of the current prior θ .

Let us make clear that the Bayes-Adaptive framework handles parameters' uncertainty in two phases:

1. The first phase is completely off-line and does not require to observe any transition of the MDP. In this phase, we use the prior knowledge about transitions in order to build a BAMDP with augmented state space $\tilde{S} = S \times \Theta$.
2. In the second phase, the agent uses the optimal BAMDP policy $\tilde{\pi} : S \times \Theta \rightarrow \mathbb{A}$ provided by the first phase to the "real-life" problem, in which only transitions (s, a, s') are perfectly observed and a prior estimate can be obtained.

6.1.3 Application to Cancer Treatment Optimization

The full medical example is described in Chapter 3. As a reminder, Figure 6.2 illustrates a fully observed patient trajectory, highlighting the dynamics of the PDMP flow. In the following, it is assumed that all PDMP parameters are known, except in relapse where v_1 is unknown. We assume that $v_1 \sim \text{Log-}\mathcal{N}(\mu, 1/\sigma^2)$, where the mean μ and variance σ^2 are unknown.

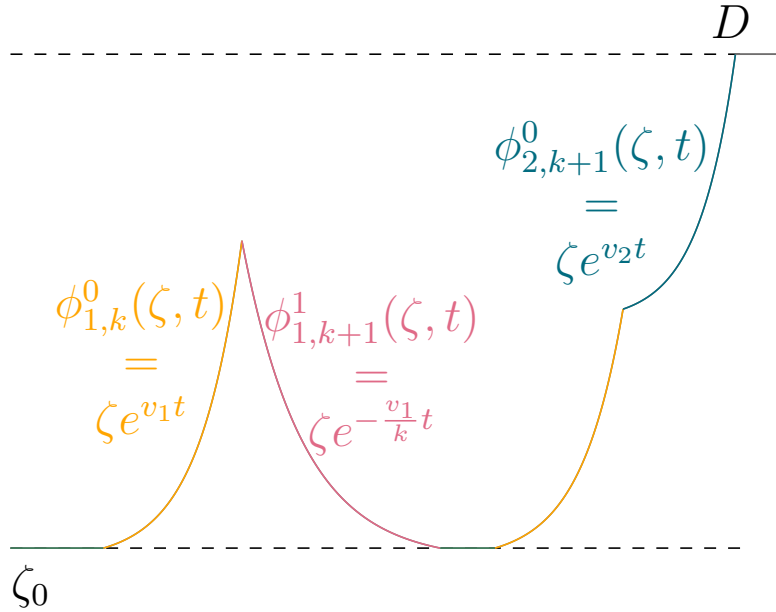


Figure 6.2: **Example of a simulated patient trajectory, highlighting the flow functions.** Green curves indicate the remission state, orange curves represent relapse without treatment, pink curves indicate relapse under treatment, and blue curves represent untreated relapse. Grey curves indicate the patient is dead. Flow under remission and when the patient is dead are omitted as they are constant.

The associated BAMDP $\langle \tilde{S}, \mathbb{A}, \mathbb{K}, \tilde{P}, c, H \rangle$, is building on the MDP $\langle \mathbb{S}, \mathbb{A}, \mathbb{K}, P, c, H \rangle$ described in Section 5.1. Its characteristics are defined as follows:

- The state space $\tilde{\mathbb{S}} = S \times \Theta$ with S from the MDP model in Section 5.1 and $\Theta \subset \mathbb{R}^4$ the space of hyperparameters. Model uncertainty only concerns the value of the flow parameter v_1 , which follows a log-normal distribution with unknown parameters μ and σ . According to Theorem 6.1.1, a conjugate prior for the log-normal distribution is a normal-gamma distribution. Therefore, the set of hyperparameters Θ consists of the set of vectors $(\mu_n, \kappa_n, \alpha_n, \beta_n) \subseteq \mathbb{R}^4$.
- The action space $\mathbb{A}$, the set of constraints $\mathbb{K}$ and the cost function c are the same as those defined in Section 5.2.
- The transition kernel $\tilde{P}(\cdot | \tilde{s} = (s, \theta), a)$ is defined in Definition 6.1.2. At the start of a patient's trajectory, the unknown parameters μ and σ are assumed to follow a Normal-Gamma distribution with hyperparameters $(\mu_0, \kappa_0, \alpha_0, \beta_0)$. Unlike standard BAMDP cases where the transition update only depends on (s, a, s') , here the update of the function $\mathcal{U}$ is different because we are not estimating the transition probabilities directly. Instead, we estimate the distribution of the flow parameter v_1 . From each observed transition (s, a, s') , we obtained, with function $\mathcal{V}$, the value of v_1 , as detailed in Table 6.1. This estimate allows us to update the posterior distribution of v_1 using the Bayesian update formulas for a normal-gamma prior, available in Theorem 6.1.1.

Table 6.1: Function $\mathcal{V}$ extracts the current value of the slope parameter v_1 from the observed transition $\langle s, a, s' \rangle$.

	$h' = 0$	$h' = 1$	$h' = 2$
$h = 0$ and $\ell = 0$		$\frac{1}{u'} \log(\frac{\zeta'}{\zeta_0})$	
$h = 1$ and $\ell = 0$		$\frac{1}{r} \log(\frac{\zeta}{\zeta'})$	$\frac{1}{r-u'} (\log(\frac{\zeta'}{\zeta}) - v_2 u')$
$h = 1$ and $\ell = 1$	$\frac{k}{r-u'} \log(\frac{\zeta}{\zeta'})$	$\frac{k}{r} \log(\frac{\zeta}{\zeta'})$	$\frac{k}{r-u'} (\log(\frac{\zeta}{\zeta'}) - v_2 u')$

Remark 6.3 *The BAMDP model is included for illustrative purposes only, as our primary focus is on the partially observable problem.*

6.2 Partially-Observed Controlled PDMP as a BAPOMDP

We introduce the BAPOMDP framework to handle the case of partial observability, where the agent does not have access to the full state of the PDMP but only to partial observations. Unlike for fully observed controlled PDMP with ill-known model, the constructed BAPOMDP model is fully-known at planning time. Then, we illustrate the transformation of a partially observable PDMP into a BAPOMDP using the medical example.

6.2.1 Formulation as a BAPOMDP

The overall approach remains conceptually the same as before: the system dynamics are augmented with hyperparameters that evolve according to the agent's belief updates. In general, when dealing with partial observability, one may also consider the observation function as being unknown. This source of uncertainty can be incorporated within the BAPOMDP framework by extending the hyperstate to include an additional set of hyperparameters representing the prior over the observation function. The observation at each step is then generated according to this current observation prior.

Definition 6.2.1: BAPOMDP model of a mode-augmented controlled-PDMP

Let $\langle E, \Phi, \lambda, Q \rangle$ be a mode-augmented controlled PDMP under partial observations and parametric flow, where one flow parameter v is unknown but supposed to have a probability density $f_{v|\theta}$ parameterized by a vector of hyper-parameters θ . The observation function is both random and parametric, where one random variable ϵ has a probability density $g_{\epsilon|\psi}$ parameterized by a vector of hyper-parameters ψ . The corresponding BAPOMDP is defined by $\langle \tilde{\mathbb{S}}, \mathbb{A}, \Omega, \mathbb{K}, \tilde{T}, c, H \rangle$ with the following characteristics:

- The hyper state space is $\tilde{\mathbb{S}} = \mathbb{S} \times \Theta \times \Psi$, where $\mathbb{S}$ is the same as in Definition 5.1.1, $\Theta \subset \mathbb{R}^d$ is the same as Definition 6.1.2 and $\Psi \subset \mathbb{R}^k$ is the set of hyper parameters' possible values.
- The action space $\mathbb{A}$, the cost function c and the horizon H are the same as in Definition 5.1.1.
- The observation state space Ω is the same as in Definition 5.2.1.
- The set of constraints $\mathbb{K} \subset \Omega \times \mathbb{A}$, defines the available actions in the observation ω .
- The transition kernel $\tilde{T}(\cdot | \tilde{s} = (s, \theta, \psi), a)$ specifies the transitions of the BAPOMDP. Choosing $a = (\ell, r)$ in state $\tilde{s} = (s, \theta, \psi)$ specifies that decision mode ℓ will be applied from now on, until date $t' = t + r$.

$$\begin{aligned} \tilde{T}(\tilde{s}' = (s', \theta', \psi') \in B_{\mathbb{S}} \times B_{\Theta} \times B_{\Psi}, \omega' | \tilde{s} = (s, \theta, \psi), a) \\ = \int_{B_{\mathbb{S}}} \int_{\Omega} \mathbb{1}_{B_{\Theta}} \mathcal{U}_1(\theta, s, a, s') \mathbb{1}_{B_{\Psi}} \mathcal{U}_2(\psi, s', a, \omega) P(ds' | s, a, \theta) O(d\omega, s, a, \psi) \end{aligned}$$

for any Borel subsets $B_{\mathbb{S}}$, B_{Θ} , B_{Ψ} of $\mathbb{S}$, Θ and Ψ , where $\mathcal{U}_1(\theta, s, a, s')$ and $\mathcal{U}_2(\psi, s', a, \omega)$ are the posterior update functions of θ and ψ .

As for the prior over transition function, authors assume that the prior over the observation function follows a Dirichlet distribution [Ros+11]. In contrast, our setting places also this prior on a specific parameter of the observation function, and this parameter has typically a continuous support. As for BAMDP, the posterior update does not directly depend on

the transition $\langle s', a, \omega \rangle$, as in the classical case, but instead requires inferring the unknown parameter. The posterior update function of θ and ψ can be written:

$$\mathcal{U}_1(\theta, s, a, s') = \mathcal{W}_1(\theta, \mathcal{V}(s, a, s'))$$

$$\mathcal{U}_2(\psi, s', a, \omega) = \mathcal{W}_2(\psi, \mathcal{E}(s', a, \omega)),$$

where $\mathcal{V}(s, a, s')$ denotes the inference function for the unknown parameter in the transition function, $\mathcal{W}_1(\theta, v')$ represents the prior update function for θ , $\mathcal{E}(s', a, \epsilon)$ denotes the inference function for the unknown parameter in the observation function, $\mathcal{W}_2(\psi, \epsilon')$ represents the prior update function for ψ . The transition kernel of the BAPOMDP is illustrated in Figure 6.3.

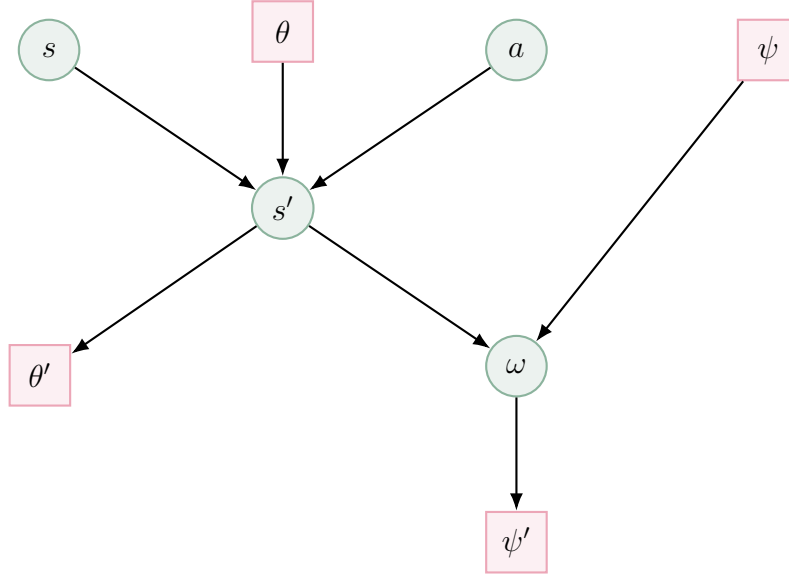


Figure 6.3: Transition model in a generic BAPOMDP framework. Green nodes denote the state and action, and the pink node corresponds to the hyperparameter being updated. Latent (simulated) variables are omitted for clarity.

Remark 6.4 *We could have a partially observed process with a known observation function. This is the case we explore in the following example.*

Remark 6.5 *In a BAPOMDP, policies $\pi : \mathcal{H}_t \rightarrow \mathbb{A}$ choose actions as functions of the current history h_t only.*

Let us make clear that the Bayes-Adaptive framework handles parameters' uncertainty in two phases:

1. The first phase is completely off-line and does not require to observe any transition of the POMDP. In this phase, we use the prior knowledge about transitions in order to build a BAPOMDP with augmented state space $\tilde{S} = S \times \Theta \times \Psi$ and generate observation $\omega \in \Omega$.

2. In the second phase, the agent uses the optimal BAPOMDP policy $\tilde{\pi} : \mathcal{H}_t \rightarrow \mathbb{A}$ provided by a reinforcement learning algorithm to the "real-life" problem, in which only observations $\omega \in \Omega$ are available. There is no need to obtained prior estimates to apply the policy.

6.2.2 Application to Cancer Treatment Optimization

In the following, from Chapter 3, we assume the state is not fully observed. The associated BAPOMDP $\langle \tilde{\mathbb{S}}, \mathbb{A}, \Omega, \mathbb{K}, \tilde{P}, O, c, H \rangle$, is then a special case of the BAPOMDP described in Definition 6.2.1. It is built on the POMDP described in section 5.2.2 and BAMDP described in section 6.1.3. Its characteristics are defined as follows:

- The state space $\tilde{\mathbb{S}}$, the action space $\mathbb{A}$, the transition kernel $\tilde{P}$ and the cost function c are the same as those defined in Section 6.1.3.
- the observation space Ω , the set of constraints $\mathbb{K}$, the observation kernel O are the same as those defined in Section 5.2.2.

Remark 6.6 *We could easily create the complete example with unknown observation function as follows. Consider the additive noise $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon)$ with σ_ϵ an unknown parameter. Then, the state space of the associated BAPOMDP is now $\tilde{\mathbb{S}} = \mathbb{S} \times \Theta \times \Psi$ with $\Psi \subset \mathbb{R}^d$ the space of hyperparameters of the observation function. According to [Mur07], a conjugate prior for the normal distribution with unknown variance is an inverse-gamma distribution. Therefore, the set of hyperparameters Ψ consists of the set of vectors $(\alpha_\epsilon^n, \beta_\epsilon^n) \in \mathbb{R}^2$. For sake of simplicity, we do not consider this case in the following.*

6.3 Solving a BAPOMDP with Deep RL

We now describe how the obtained BAPOMDP can be solved using deep RL algorithms. We begin by detailing the experimental setup, including the baseline algorithms and their variants that will be benchmarked on the medical model introduced in Section 6.2.2. Next, we analyse the results of these numerical experiments, focusing on the performance of the trained policies. Finally, for the best performing algorithm, we attempt to interpret and explain the learned policy.

6.3.1 Experimental Design

The experimental design closely follows the approach detailed in Chapter 5 in section 5.3.1, with the key difference being that the environment is now modeled as a BAPOMDP instead of a POMDP. All parameter of BAPOMDP model are listed in Tables 6.2.

Table 6.2: **BAPOMDP** parameter values used in experiments

Name	Notation	Value
State and Observation		
Prior	θ	$(-6.785, 5.001, 3.51, 1)$
Noise	$\epsilon \sim \mathcal{N}(\mu, \sigma)$	$(\mu = 0, \sigma = 1)$
Cost function		
Treatment cost	C_ℓ	0.1
Visit cost	C_v	1
Death cost	C_D	300
Horizon constraint cost	C_H	1000
Treatment constraint cost	C_T	1000

Reminder: Experimental Design

- **Algorithms.** We benchmarked five algorithms: PPO, PPO with LSTM, PPO with action masking (AM), DQN and DQN with AM.
- **Hyperparameter tuning.** We performed an extensive hyperparameter search.
- **Training.** Each algorithm was trained with final set of hyperparameters listed in Table B.5.
- **Evaluation.** The goal was to assess whether the network trained with BAPOMDP is robust to a specific patient distribution modeled with POMDP. Trajectories were generated from the POMDP model described in Chapter 5, where v_1 follows a log-normal distribution with parameters $\mu = -6.40$ and $\sigma = 0.5$.
- **Policy interpretation.** We attempt to interpret the best-performing policy.

The steps related to training and evaluation are illustrated in Figure 6.4.

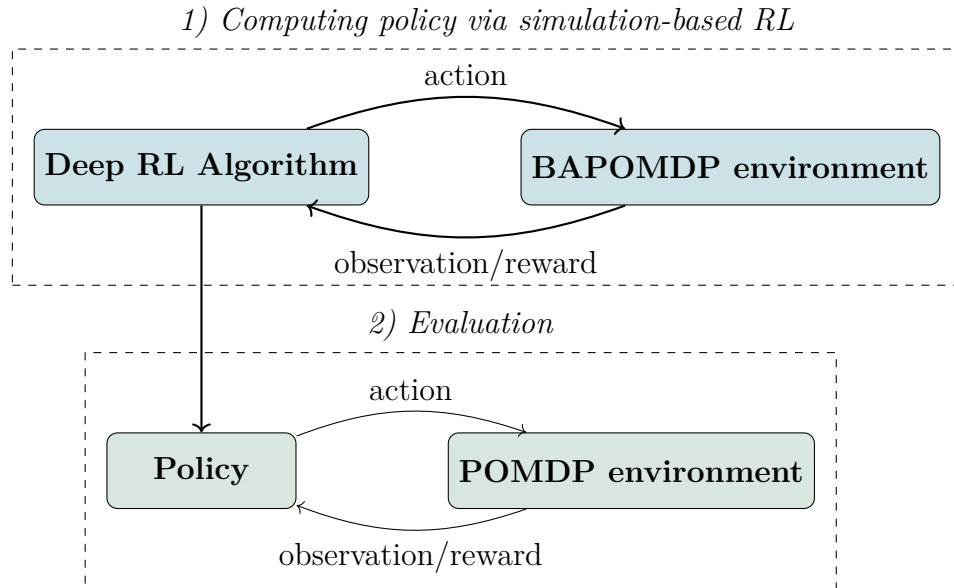


Figure 6.4: **Schematic of solving and evaluating the BAPOMDP model.** *Adapted from [Lia+18b; Wu+21].*

6.3.2 Numerical Results

Performance during training. We evaluate performance of each algorithm, with the best configuration, by measuring the mean episode reward as a function of the number of training experiences. Figures 6.5 illustrate these results. Note that the plots do not all start at the same point on the x-axis because different configurations may begin generating meaningful rewards

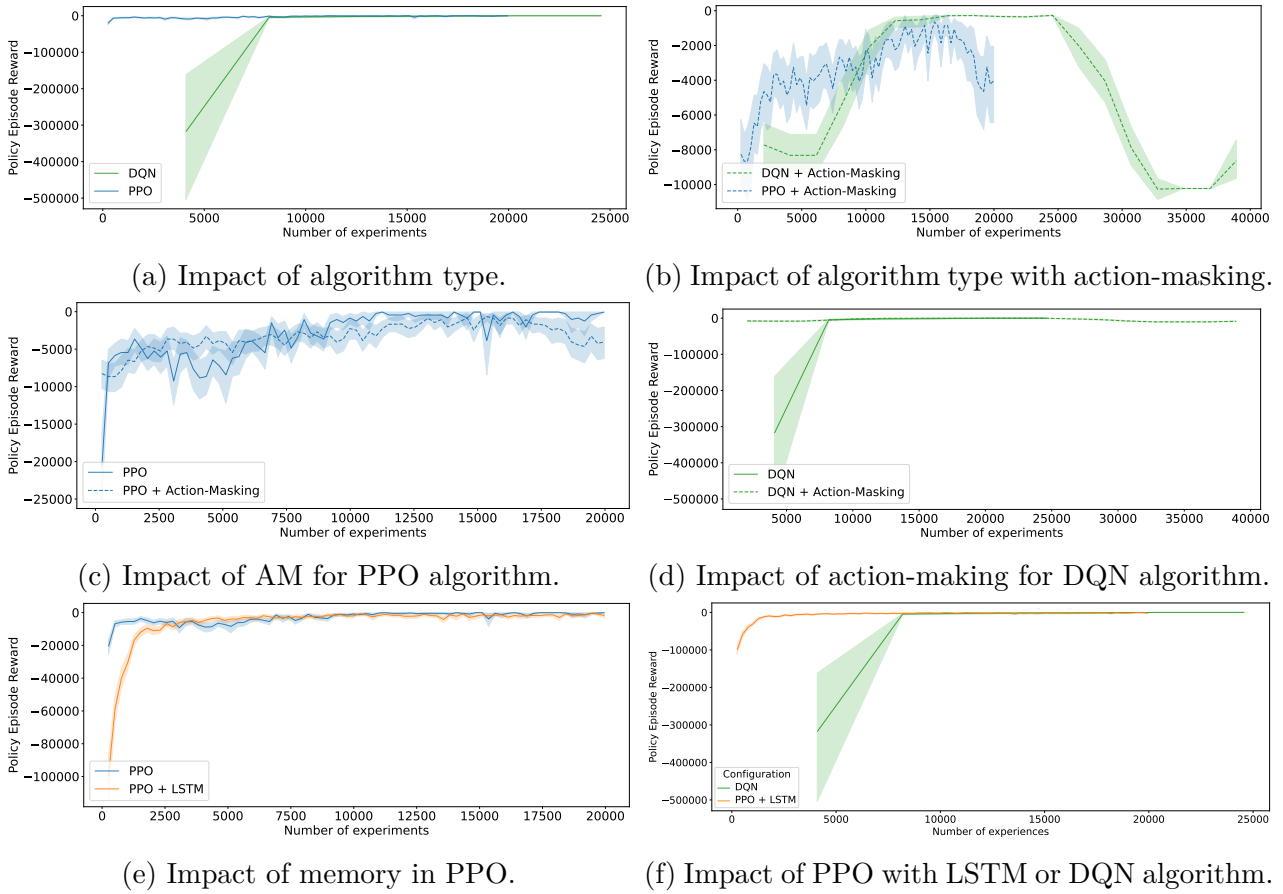


Figure 6.5: **Impact of algorithm choice on the performance** in terms of mean episode reward per episode with respect to the number of training experiments used on BAPOMDP model.

at different stages of training.

- Impact of algorithm type (value-based vs. policy-based). Figure 6.5b compares DQN and PPO, both using action masking, to assess the effect of algorithm type. Both algorithms show performance degradation after approximately 15,000 training iterations, indicating overfitting or training collapse; training should therefore be stopped at this point. Up to this stage, their performances are comparable. However, PPO with action masking exhibits high variability, with large reward fluctuations and limited stability, whereas DQN with action masking demonstrates a more stable training process.
- Impact of action masking. Figures 6.5d and 6.5c compares handling invalid actions with action masking versus action penalties in DQN and PPO, respectively. In Figure 6.5d, we observe that DQN with an action penalty requires significantly more training interactions to learn to avoid invalid actions, whereas action masking enables the agent to incorporate this constraint directly and learn faster. In Figure 6.5c, the difference for PPO is less pronounced, but the action-masking variant still adapts to invalid actions more quickly than the action-penalty counterpart.
- Impact of memory in PPO. Figure 6.5e examines the impact of adding memory (LSTM) to

Table 6.3: **Policy evaluation robustness on 25 000 Monte-Carlo simulations.**

Configuration	Expected Reward	Confidence Interval
<i>Policy-based algorithms</i>		
PPO	−13.50	±0.01
PPO with AM	−8.64	±0.02
PPO with LSTM	−9.24	±0.02
<i>Value-based algorithms</i>		
DQN	−11.92	±0.04
DQN with AM	−8.73	±0.02

PPO. Incorporating LSTM does not significantly alter the overall learning curve compared to standard PPO, as both ultimately achieve similar reward levels. However, PPO-LSTM exhibits substantially lower performance during the initial training iterations before converging to comparable results. Based on the above results, the remainder of our evaluation focuses on PPO with action masking and DQN with action masking, while excluding the PPO variant with memory and the action-penalty approach.

Robustness of policies. We evaluated the robustness of policies in the POMDP environment consistent across all simulated patients. The results are summarized in Table 6.3.

- DQN and PPO with action penalty has lower performances. DQN with action-masking shows better stability than DQN with action penalty.
- Incorporating LSTM significantly improves PPO performances.
- DQN with action-penalty has greater performances than PPO with action penalty. However PPO achieves slightly stabler results.
- PPO with action-masking shows slightly better results than DQN with action-masking.

The robustness evaluation therefore highlights that, although PPO with AM learns in a more stable way, DQN with AM generalizes better when facing the original POMDP setting.

Policies interpretation. Since DQN with AM (-8.73 ± 0.02) and PPO with AM (-8.64 ± 0.02) achieve comparable rewards, we focus on both. Among the five trained policies, the best is chosen by highest average reward. For DQN with AM, the best policy reaches -5.08 ± 0.00 , while PPO with AM achieves -5.94 ± 0.01 . Reported indicators in Table 6.4 provide further insight into these policies.

The two obtained policies show similar survival rates (99.8% for PPO with AM and 99.7% for DQN with AM), and both administer an average of 19.99 treatments per trajectory. However, they slightly differ in their management of patient visits and treatment schedules. Notably, PPO with AM results in a higher number of patient visits (58.99 on average) with shorter intervals between them (40 days), while DQN with AM reduces the number of visits (38.99)

Table 6.4: **Summary of policy behavior indicators based on 5 000 Monte-Carlo simulations.**

Indicator	PPO with AM	DQN with AM
Survival rates	99.80% $\pm$ 0.00	99.70% $\pm$ 0.00
Average number of treatment	19.99 $\pm$ 0.00	19.99 $\pm$ 0.01
Average time spend under treatment	1199.63 $\pm$ 00.04	1199.56 $\pm$ 0.05
Average number of visit	58.99 $\pm$ 0.01	38.99 $\pm$ 0.01
Average delay between two visits	40.00 $\pm$ 0.00	60.00 $\pm$ 0.00
Rate of visits occurring within 15 days	0.01 $\pm$ 0.00	0.00 $\pm$ 0.00
Rate of visits occurring within 30 days	66.66 $\pm$ 0.17	0.00 $\pm$ 0.00
Rate of visits occurring within 60 days	33.33 $\pm$ 0.17	100 $\pm$ 0.00

but increases the delay between them (60 days). Despite these differences in visit frequency, the time spent under treatment remains nearly identical for both policies (around 1120 days).

6.4 Comparative Study of Modeling Frameworks

We have used the BAPOMDP framework to overcome the lack of knowledge about the PDMP model, in particular the unknown parameter v_1 . Alternatively, one could infer the value of v_1 directly from real data and subsequently build a POMDP model using this estimated parameter, as described in Chapter 5. In this section, we compare the two modelling approaches: the BAPOMDP framework and the non-adaptative POMDP framework. We begin by detailing the experimental design for this comparative study, followed by a presentation of the numerical results

6.4.1 Experimental Design

First, using real data, one can obtain a point estimate of the parameter v_1 based on the number of days a patient survives after relapse. In our simulation scenario, we assume that, on average, patients live 80 days after relapse without treatment, leading to an estimate of $\hat{v}_1^m = 0.046$. This estimate, may be biased because, among the few available data points, not all patients die during the observation period. Those who die despite receiving chemotherapy could be experiencing untreatable relapse, which would lead to biased estimation. To account for this potential bias, we consider two extreme scenarios to assess the impact of slope estimation on the performance of the POMDP model:

- Very slow progression: average time to death of 120 days, giving $\hat{v}_1^s = 0.03$,
- Very fast progression: average time to death is 40 days, giving $\hat{v}_1^f = 0.09$.

To mitigate the limitations of relying on a single, potentially biased point estimate, one could instead adopt a Bayesian inference approach to represent uncertainty in the POMDP model. In this setting, the parameter v_1 is treated as a random variable drawn from a Normal

Table 6.5: **Parameter values of Normal-Gamma priors used in model evaluation.**

Name	Notation	Value
Weak prior	θ_0	(1, 0.001, 1.01, 1)
Moderate prior	θ_1	(−6.785, 5.001, 3.51, 1)
Strong prior	θ_2	(−6.23, 10, 6.01, 1)

distribution with mean μ and standard deviation σ . The hyperparameters (μ, σ) themselves are drawn from a Normal–Gamma prior at the start of each simulated patient trajectory. This approach allows the simulator to generate a variety of trajectories, reflecting patients who differ in their progression rates and survival times.

In our work, we place this POMDP with prior model in direct comparison with the BAPOMDP framework. We hypothesise that the adaptive nature of BAPOMDP is particularly advantageous when available information is scarce. To test this, we evaluated different priors (Table 6.5), covering a range from very uninformative (θ_0) to highly informative (θ_2).

As shown in Section 6.3.2, DQN with action-masking achieved the best performance in solving the BAPOMDP model, while PPO with action-masking demonstrated greater robustness. We therefore selected these two algorithms for the comparative study. The experimental design still closely follows the approach detailed in Chapter 5 in section 5.3.1.

Reminder: Experimental Design

- **Algorithms.** We benchmarked two algorithms: PPO with action masking (AM) and DQN with AM.
- **Models.** Each algorithm was trained with nine different models: POMDP with three different fixed estimates, POMDP with three different priors and BAPOMDP with three different priors.
- **Training.** Each algorithm was trained with final set of hyperparameters listed in Table B.5.
- **Evaluation.** The goal was to assess whether the network trained with a specific model is robust to a specific patient distribution modeled with POMDP. Trajectories were generated from the POMDP model described in Chapter 5, where v_1 follows a log-normal distribution with parameters $\mu = -6.40$ and $\sigma = 0.5$.

6.4.2 Numerical Results

Performance during training. In Figure 6.6, we present the learning curves of the best-performing algorithm (DQN with AM) trained with the BAPOMDP model (Figure 6.6a) and with the POMDP model (Figure 6.6b). It can be observed that, regardless of whether the model is trained on the BAPOMDP or the POMDP, the algorithm stabilizes and achieves a

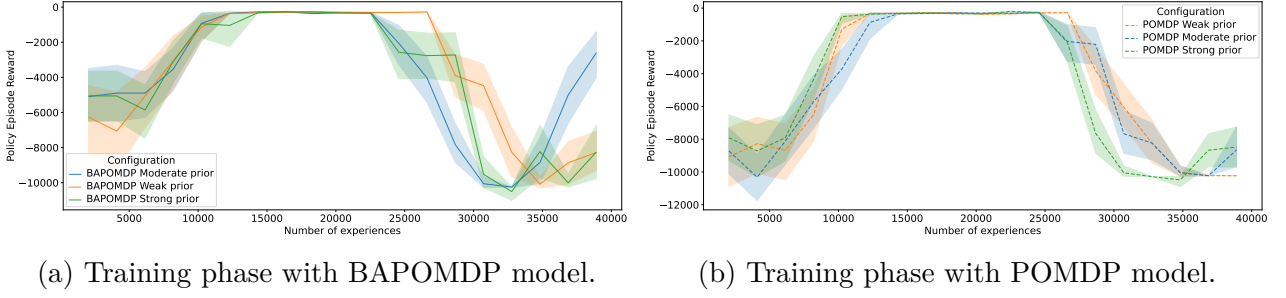


Figure 6.6: Impact of algorithm choice on the performance in terms of mean episode reward per episode with respect to the number of training experiments used on BAPOMDP model.

satisfactory average reward on the training model before suddenly experiencing a sharp drop in performance. Therefore, we stop training after approximately 17 500 episodes, at which point the learning curve is stable and the average reward is promising. So, DQN with AM, associated to its optimal set of hyperparameters (Table B.5) was trained with each framework with 5 different random seeds up to 50 000 simulation timesteps.

We observed that PPO exhibits a more stable learning phase in Section 6.3.2, and its performance, are comparable to that of DQN with action-masking. For this reason, we also investigate the comparative framework with PPO.

Performance with fixed slope estimate. Let's, first, analyse the performance of the policies obtained with the POMDP model ($\mu = -6.40, \sigma = 0.5$) using the three different point estimates of v_1 . The simulation results (Table 6.6) show that performance remains largely consistent across the three different point estimates of v_1 :

- DQN with AM: training with the slow slope estimate ($\hat{v}_1^s$) yields to the worst result in terms of average reward of -5.75 ± 0.00 ,
- DQN with AM: training with the fast slope ($\hat{v}_1^f$) has a slightly better average reward than training with the mean slope ($\hat{v}_1^m$),
- PPO with AM: The fast slope estimate ($\hat{v}_1^f$) yields the best performance (-7.45 ± 0.02), outperforming both the slow slope ($\hat{v}_1^s$, -7.65 ± 0.02) and the mean slope ($\hat{v}_1^m$, -8.02 ± 0.02).

These results suggest that the choice of estimates have a impact on the performance of the policy, but this impact remains limited, suggesting that the learned policies are robust to variations in this parameter.

Performance with bayesian slope estimate. We now turn to the Bayesian inference approach for representing uncertainty in the POMDP model. We initially hypothesized that the BAPOMDP would outperform the POMDP model when the prior is weakly informative, while the POMDP with prior would progressively improve and eventually match or surpass BAPOMDP as the prior becomes stronger. However, the simulation results only partially confirm this hypothesis (Table 6.7):

Table 6.6: **Policy evaluation robustness on 25 000 Monte-Carlo simulations.**

Parameter	Expected Reward	Confidence Interval
<i>Result obtained with DQN with AM</i>		
Slow slope $\hat{v}_1^s$	-5.75	± 0.00
Mean slope $\hat{v}_1^m$	-5.68	± 0.00
Fast slope $\hat{v}_1^f$	-5.67	± 0.00
<i>Result obtained with PPO with AM</i>		
Slow slope $\hat{v}_1^s$	-7.65	± 0.02
Mean slope $\hat{v}_1^m$	-8.02	± 0.02
Fast slope $\hat{v}_1^f$	-7.45	± 0.02

Table 6.7: **Policy evaluation robustness on 25 000 Monte-Carlo simulations.**

Parameter	BAPOMDP	POMDP with prior
<i>Result obtained with DQN with AM</i>		
Weak prior θ_0	-5.74 ± 0.00	-5.70 ± 0.00
Moderate prior θ_1	-5.66 ± 0.00	-5.97 ± 0.00
Strong prior θ_2	-5.76 ± 0.00	-5.45 ± 0.00
<i>Result obtained with PPO with AM</i>		
Weak prior θ_0	-7.54 ± 0.02	-7.04 ± 0.01
Moderate prior θ_1	-7.87 ± 0.02	-7.80 ± 0.02
Strong prior θ_2	-8.53 ± 0.02	-8.21 ± 0.02

- with a weak prior θ_0 , the training with the fixed-prior POMDP framework slightly outperforms BAPOMDP (-5.70 ± 0.00 vs. -5.74 ± 0.00),
- for the medium prior θ_1 , BAPOMDP significantly outperforms the fixed-prior POMDP (-5.66 ± 0.00 vs. -5.97 ± 0.00),
- under a strong prior θ_2 , the fixed-prior POMDP achieves the best performance (-5.45 ± 0.00).

These results indicate that BAPOMDP provides robustness in settings with weak or moderate prior knowledge, but fixed-prior models may be preferable when accurate priors are available. For PPO algorithm with action-masking, training with BAPOMDP consistently leads to lower performance compared to POMDP also as refining the prior.

Overall, these results show that the BAPOMDP, a principled framework grounded in Bayesian inference, can be implemented in a tractable way achieving performance comparable to currently used non-adaptive methods when facing models with ill-known parameters.

6.5 Conclusion

In this chapter, we have provided a framework for the optimal control of PDMPs under partial state and model observability. A Bayes-adaptive formulation was developed to handle control problems where uncertainty about the PDMP's dynamics is explicitly modeled. We have

considered the case where the parameters of the PDMP flow are ill-known and a probability distribution over their values is maintained. These limitations could be relaxed, resulting in a more complex BAPOMDP model. A key practical finding of this chapter is to have demonstrated that the problem of optimally controlling PDMPs under parameter uncertainty and partial observability can be modeled as a hybrid-state POMDP and, therefore, can be solved using readily available deep RL software.

The BAPOMDP framework operates in a simulation-based RL setting, meaning it does not require interaction with real-world data. In this setup, the agent learns solely through interactions with the simulated BAPOMDP environment. Our experiments show that DQN and PPO, both with action-masking (AM), are the most effective algorithms for solving BAPOMDPs. Incorporating action masking is generally preferred, as it enforces constraints by design and ensures that the learned policy never selects invalid actions.

Once a policy has converged, it can be deployed in the real-world environment, which in our case is represented by a POMDP. Our study indicates that the best performance is achieved when training with a point estimate of the missing parameters. However, this approach may be highly sensitive to the choice of estimator: it is unclear how it would perform when multiple parameters need to be inferred simultaneously. Thus, the strong results observed here could simply reflect the specifics of our example rather than a general property of the method. By contrast, the Bayesian approach offers greater robustness to parameter uncertainty, though it does not consistently outperform simpler models and remains strongly dependent on how prior information is encoded. Finally, the Bayes-adaptive framework does not yield systematic improvements over the POMDP with prior, especially when the prior is accurate.

As more real data becomes available, the agent’s knowledge could be updated by creating a new BAPOMDP model with an improved initial prior. The agent could be then retrained on this updated model using the more informative prior. This iterative procedure is based on the Bayesian-Adaptive Partially Observable Monte Carlo Planning (BAPOMCP) algorithm principle [KOA17] and its principle is illustrated on Figure 6.7.

The field of deep RL research for hybrid POMDPs is experiencing rapid growth. Emerging algorithms, such as those based on transformer models [Vas+23], are being investigated for POMDP solutions. We believe that the unique characteristics of Bayes-adaptive control of PDMPs make this problem a valuable new benchmark for exploring the capabilities of cutting-edge deep RL methods. Our work paves the way for controlling more complex PDMPs, where future research could explore scenarios in which the jump intensity or Markov kernel also contain multiple unknown parametric components, going beyond the uncertainty of a single flow parameter considered in this study.

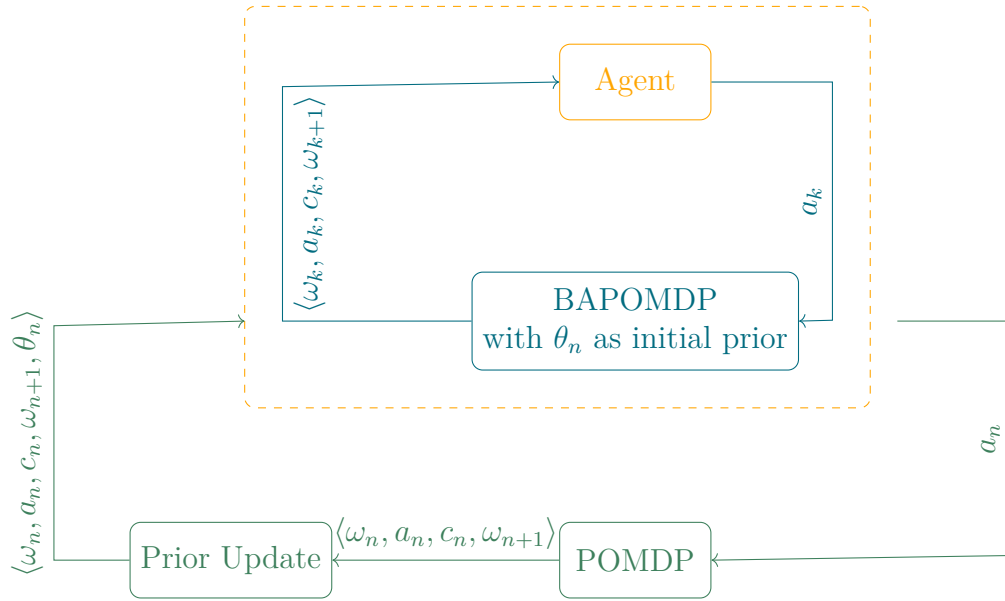


Figure 6.7: **Overview of the proposed deep BAPOMCP approach.** The agent learns through simulation with the BAPOMDP model and can be deployed in a real-world POMDP environment. The orange box highlights the core components implemented to solve the BAPOMDP, blue elements relate to BAPOMDP modeling, and green elements represent the realistic patient modelled as a POMDP with unknown dynamics.

Chapter 7

Stress-responsive Epitranscriptome of Translated RNA in Yeast

7.1	Algorithm Specification	141
7.2	Experimental Results	143
7.3	Conclusion	145

This chapter presents ongoing collaborative work initiated during my second PhD year, which lies outside the main scope of the thesis. The project emerged through collaborations with Alice Cleynen, Nikolay Shirokikh¹ and Shafi Mahmud², and focuses on RNA biology in the budding yeast *Saccharomyces cerevisiae*. Although thematically distinct from the central analysis in the rest of this manuscript, it illustrates the methodological and statistical approaches I contributed to in a different biological context. In the first task I contributed most of the methodological aspects developing a segmentation-based algorithm to improve gene annotation in yeast. In the second study, the contribution focused on analyzing the impact of mRNA modifications induced by glyceimic stress on the efficiency of protein translation.

To place this work in context, we briefly review the molecular biology underpinning RNA research, drawing in part from Alice Cleynen’s Chapter 2 HDR thesis [Cle24]. Genetic information is encoded in DNA, a double-stranded helix of complementary nucleotides (A, C, T, G). DNA contains coding regions (genes) and non-coding regions (out of scope for this work).

Genes are transcribed into RNA, a single-stranded and oriented, from 5’ end to 3’ end, sequence of nucleotides (A, C, U and G). There exist 8 types of RNA, classified by function and by length, but here we focus only on.

- messenger RNAs (mRNA) carry the genetic information necessary to make proteins. Let’s focus on two aspect of the maturation of mRNAs before its exist from cytoplasm. The first is the splicing of introns and exons, sub compartments of the gene that the mRNA

¹Researcher in molecular biology from The John Curtin School of Medical Research, Australian National University

²PhD student under Nikolay Shirokikh supervision

may retain or not for translation into different proteins. Such splicing allows to create different isoforms from a same original gene on the DNA. The second is the addition of long sequence of A nucleotides called a poly-A tail at the downstream extremity of the mRNA (3' end). Once they are mature, they are ready to undergo translation, precisely with the help of ribosomal RNAs.

- Ribosomal RNAs (rRNAs) constitute about 80% of the total RNA produced. They work around mRNAs to decipher the protein code and produce proteins. It is not translated.

While all cells in an organism share the same DNA, their functional differences arise from which genes they express and how those transcripts are translated. Many diseases such as cancer are linked to DNA mutations [Sin23; Kel+25], but understanding healthy cell diversity requires examining RNA.

Over the past three decades, advances in high-throughput sequencing have revolutionised transcriptome analysis, enabling unprecedented insight into gene expression and RNA modifications. Recently, Oxford Nanopore Technology developed a novel sequencing technology: Direct RNA Sequencing (DRS). After extraction from the cell, transcripts are navigated through a nanopore by a protein capturing the poly-A tail of the transcripts. The pore measures an ionic current that is specifically affected by the navigation of nucleotides, and base-calling algorithms such as guppy [WJH19] translate the current measures into sequences of nucleotides (see Figure 7.1). DRS can therefore capture the sequence of whole transcripts yielding long reads that can directly be mapped to the appropriate isoform, allowing the analysis of abundance, modification or polyA length directly at the isoform level.

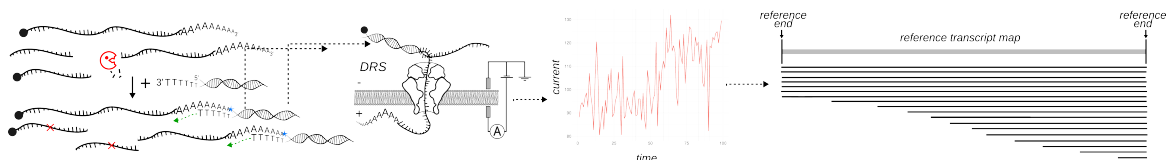


Figure 7.1: **Direct RNA Sequencing**

Regulation of RNA translation and abundance is a rapid adaptative mechanism in response to environmental stress. In yeast, cells adjust mRNA's production (transcription), degradation, and translation to cope with changes such as heat shock, oxidative stress, and osmotic alterations [Cas+11; CP18; Nad+25; Ho+18]. However, the role of RNA modifications in this regulatory network, particularly under nutrient deprivation, remains poorly understood.

7.1 Algorithm Specification

Despite being a widely used model organism, the yeast genome has very limited known annotation, ie the location of the start and end of the genes on the genome are approximative. To allow for a robust integrative analysis of the different RNA features, the algorithm developed here aims to automatically identify gene annotations, from DRS data, in yeast by determining

both the length of the expressed region and the precise position of the alignment start. It takes advantage of the simple structure of yeast genes, which typically consist of a single exon. A major challenge, however, is that yeast genes are closely packed along the genome, meaning that some reads may overlap and be ambiguously assigned to two neighboring genes. The goal of the algorithm is therefore not only to detect the expressed region of a gene but also to exclude reads that originate from adjacent genes. When plotting the number of reads as a function of genomic position, the expected profile shows a characteristic plateau: a sudden rise in signal marking the gene start, followed by a steady plateau corresponding to the expressed region, and finally a gradual decline toward the end. This graphical signature, read from right to left in our analyses, provides the basis for robust detection and reliable annotation of gene regions.

Figure 7.2 shows read profiles for four yeast genes, illustrating the characteristic shapes targeted by our approach. Gene YBL091C (Figure 7.2a) displays a clear plateau, marking a well-defined expressed region. In contrast, gene YFR008W (Figure 7.2b) exhibits a secondary peak at the left boundary, likely caused by transcripts overlapping from a neighboring gene. Similarly, gene YOR110W (Figure 7.2d) shows overlapping at the left boundary, where the signal from the first peak does not return to zero before merging with transcripts of the adjacent gene. Finally, gene YBL005W-A (Figure 7.2c) presents overlaps at both the beginning (right) and end (left) of the profile. These examples highlight the variability of transcript shapes and the importance of a robust algorithm capable of accurately delineating expressed regions despite overlapping signals.

Initially, genes with a maximum transcript count of less than 20 were filtered out, as these genes were insufficiently expressed for further analysis. In the first step, the Segmentor3IsBack algorithm [Cle+14] was applied gene-by-gene to detect breakpoints in the transcript counts. However, the data do not meet the algorithm's initial independence assumptions, resulting in large number of breakpoints. To refine the segmentation, the breakpoints were filtered to retain only those indicating significant changes in transcript counts. This was done by comparing the average transcript counts on either side of each segment and retaining segments where the average difference exceeded 10% of the maximum transcript count. Subsequently, the expressed regions were identified from these refined segments. Given that 1 000 positions were added to both the beginning and end of each gene sequence, a position was considered as expressed if the average transcript count between two segments, within 1 000 to final position minus 1 000 range, was greater than the minimum of 5% of the maximum transcript count or 50.

Additionally, the aim was to detect untranslated regions (UTR). If position 1 000 (or final position minus 1 000) fell within an expressed region, the search was extended sequentially. A position was considered as expressed if the average transcript count between two segments exceeded the minimum of 10% of the maximum transcript count or 50 and is less than the next (or previous) segment.

Final annotations were determined by taking the maximum UTR length observed across all conditions (starved and non-starved) and rRNA types (ribosome and polyribosome)

The final expressed regions detected by our algorithm were compared against the reference

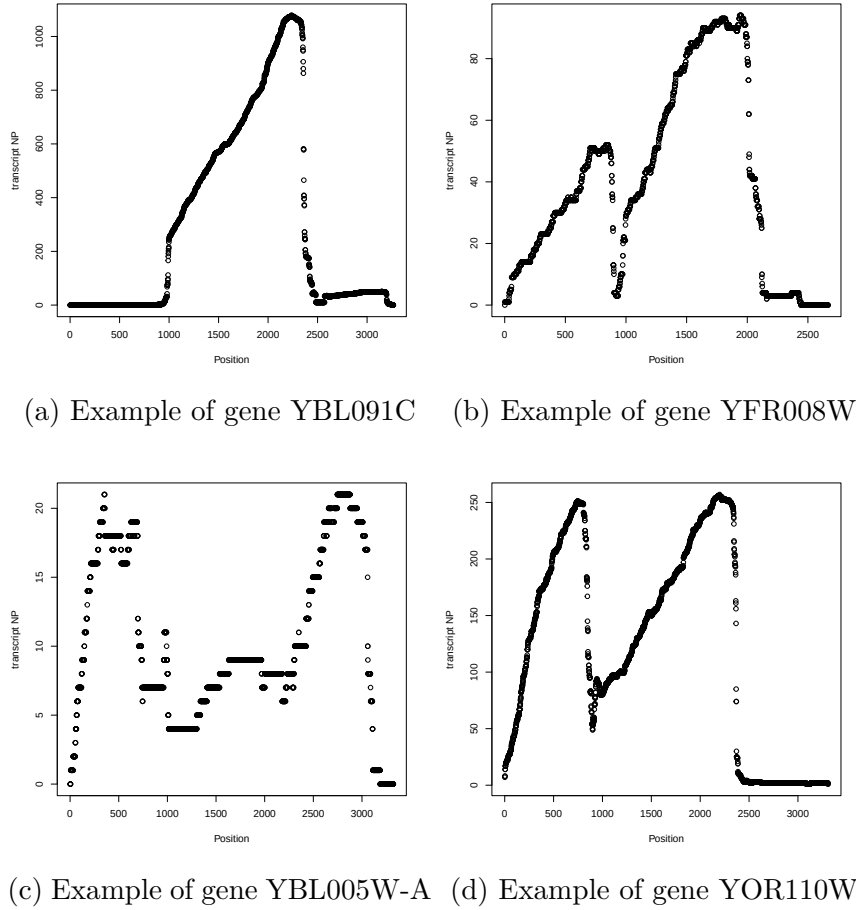


Figure 7.2: **Read profiles for non-starved condition for some gene examples.** It shows the number of read per position obtained from DRS data.

annotation from [Nag+08].

7.2 Experimental Results

Let's consider the gene YBL091C as an example to illustrate the algorithm's steps. First, Segmentor3IsBack algorithm identified 29 breakpoints (Figure 7.3a). The filtering step then reduced this to 6 breakpoints (Figure 7.3b).

Finally, Figure 7.4 illustrates the expressed regions detected for the example genes. The algorithm performs well for YBL091C, representing an ideal case without overlaps. For YFR008W and YOR110W genes, the overlap on the left is correctly handled, and the algorithm successfully identifies a substantial portion of the UTR. For the YBL005W-A gene, the algorithm detects an overlapping gene within the untranslated region at the start of the sequence; this imprecision may be explained by the low number of transcripts.

Additionally, even if it's rare, some genes have two exons. Our algorithm was also tested on these genes, and Figure 7.5 shows the results for two examples. For genes YGR183C the algorithm performs well, accurately identifying the expressed regions for both exons. However,

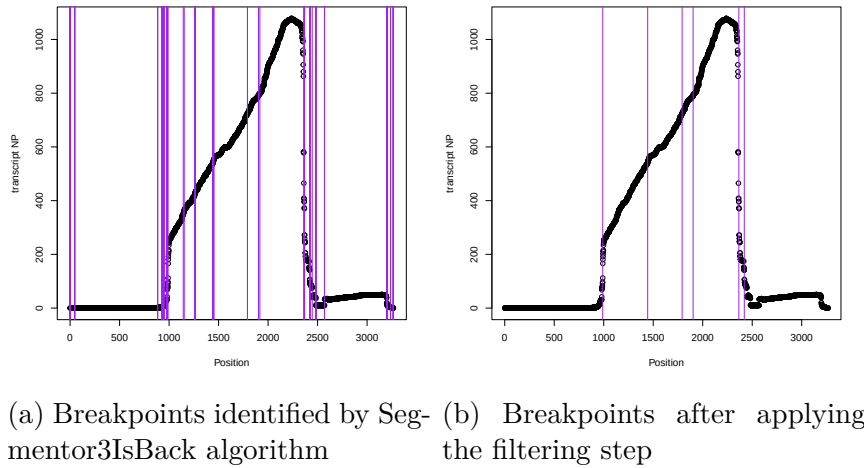


Figure 7.3: Annotation result for gene YBL091C.

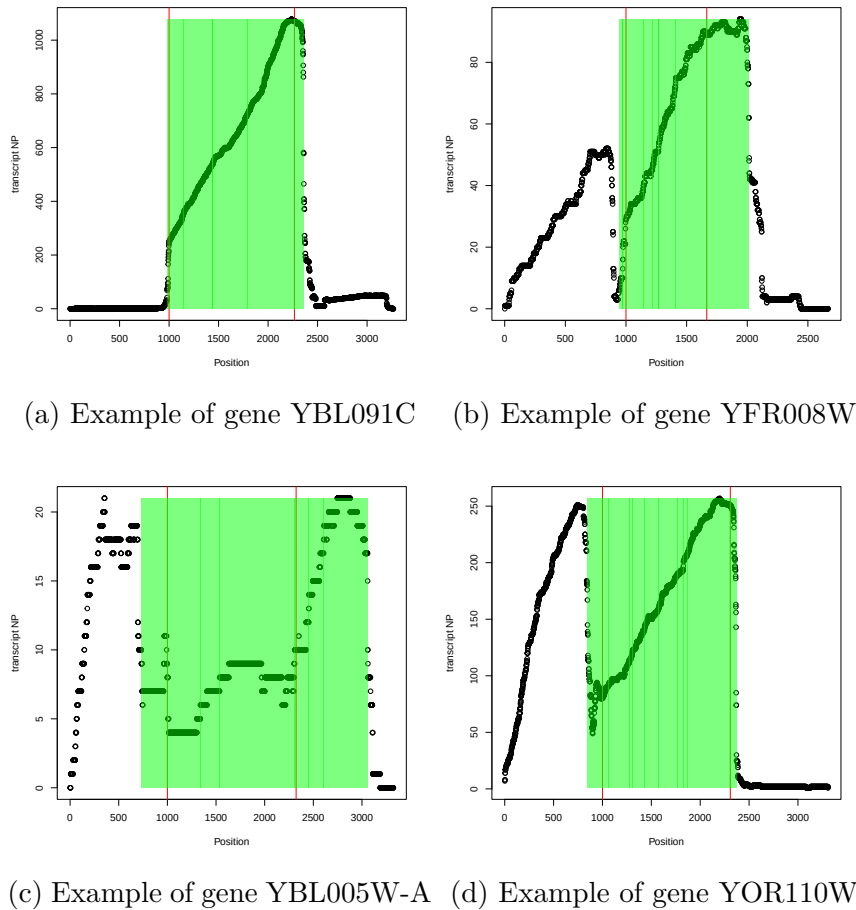


Figure 7.4: **Detected expressed region and untranslated regions some gene examples.** Expressed regions are shown as green areas bounded by red lines, while untranslated regions correspond to the green areas flanking these boundaries.

for gene YKL156W, the algorithm struggles to detect the second exon, likely due to its narrow expressed region.

In comparison with [Nag+08], our algorithm identifies 5' UTRs in 986 additional genes and

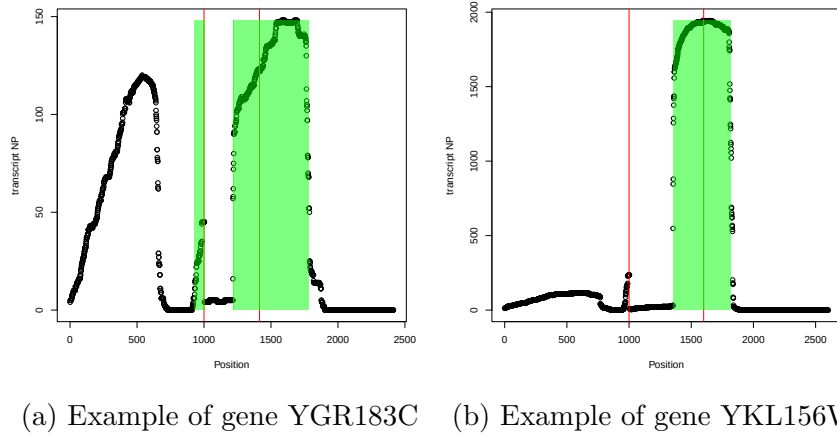


Figure 7.5: **Detected expressed region and untranslated regions some gene with two exons.** Expressed regions are shown as green areas bounded by red lines, while untranslated regions correspond to the green areas flanking these boundaries.

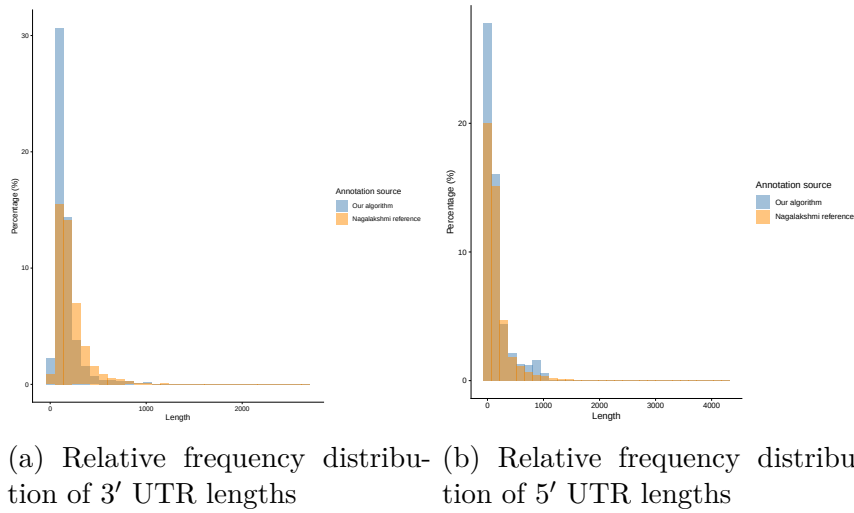


Figure 7.6: Comparison of the relative frequency distributions of UTR lengths detected by our algorithm with the annotation from [Nag+08].

3' UTRs in 962 additional genes. Figure 7.6 presents the relative frequency distributions of 3' and 5' UTR lengths detected by our method versus the reference annotation. Our algorithm detects a larger proportion of shorter 3' UTRs, whereas for 5' UTRs both approaches yield overall shorter lengths with more similar distributions. In both cases, the annotation from [Nag+08] includes unusually long UTRs, which may be questionable.

7.3 Conclusion

The aim of this work was to develop an algorithm for the automatic annotation of *Saccharomyces cerevisiae* genes using direct RNA sequencing (DRS) data. The method, based on the segmentation of transcript count profiles, performs well overall, although some genes remain poorly annotated due to limited transcript coverage. Since the algorithm relies heavily on

prior knowledge of the yeast genome structure, it may not be robust for genes with atypical architectures, such as those containing multiple exons, and its applicability to other organisms is therefore expected to be limited. Nevertheless, compared to the annotation reference of [Nag+08], our approach provides UTR information for more than 950 additional genes, captures shorter UTR lengths more effectively, and is less affected by artefactually long UTRs.

This annotation serves as the foundation for identifying expressed genomic regions in subsequent analyses. The statistical analysis, however, is not included in this manuscript, as it is still ongoing due to delays in data acquisition.

Conclusion and Perspectives

Conclusion

This thesis brings together concepts from statistics, process modelling, and artificial intelligence, with a particular focus on Reinforcement Learning (RL), to address complex sequential decision-making problems in healthcare and beyond. The work begins with an extensive state-of-the-art review covering all relevant fields: Markov Decision Processes (MDPs) and their extensions, RL methods, and Piecewise Deterministic Markov Processes (PDMPs). This foundation ensures that readers from diverse scientific backgrounds can follow the developments in the later chapters.

All contributions are illustrated through a detailed case study on the follow-up and monitoring of multiple myeloma patients. This application grounds each method in a concrete real-world problem, while also highlighting the key concepts developed throughout the manuscript and showcasing the potential of the proposed approaches.

The first original contribution is a numerical method for computing a candidate strategy for an impulse control problem in a fully observed PDMP with known dynamics. We show that this strategy is ϵ -optimal, offering both a theoretical guarantee and a benchmark for future approaches. The main limitation of this method is its reliance on full observability and complete knowledge of the system dynamics. Additionally, it is computationally intensive, making it applicable only to low-dimensional PDMPs with limited complexity.

To address more realistic settings, we extend our analysis to partially observed PDMPs, reformulated as Partially Observable MDPs (POMDPs). The second original contribution is to show that these high-dimensional, continuous-space POMDPs can be effectively solved using deep RL methods. Importantly, the choice of algorithm should not rely solely on raw performance metrics, as the appropriate selection may depend on the specific application. Moreover, the resulting policies often lack interpretability, which can limit their practical usability.

Finally, the third contribution extends the framework to incorporate parametric uncertainty in the PDMP dynamics by formulating the problem as a Bayesian Adaptive POMDP (BAPOMDP). This formulation addresses both partial observability and model uncertainty, and we show that the resulting decision problem can be effectively solved with deep RL methods. To assess the value of this approach, we present a benchmark comparing the Bayes-adaptive framework with its non-adaptive counterpart. The results demonstrate that the BAPOMDP

can be implemented in a tractable manner and achieves performance comparable to existing non-adaptive methods when dealing with models with poorly known parameters.

Together, these contributions form a coherent progression from theoretical modelling to practical algorithmic solutions, paving the way for the application of advanced decision-making tools to complex, uncertain dynamical systems.

Although not directly related to the core topic of this thesis, I also present a side project carried out within a collaborative effort led by Alice Cleynen and Nikolay Shirokikh³. This project investigates the impact of glucose stress on mRNA regulation in yeast. My contribution consists of developing numerical methods for processing yeast transcriptome data. The broader objective, still a work in progress, is to perform statistical analyses on how glucose stress affects mRNA modifications and, in turn, influences protein translation efficiency

Perspectives

While the thesis demonstrates the feasibility and potential of the proposed approaches, several limitations must be acknowledged. These limitations highlight that the core question: *How can we obtain effective control strategies when observations are incomplete and the underlying dynamics are poorly known?* is not yet fully resolved, and there remains substantial work to be done to address it.

Short-Term Perspectives

Robustness and cost calibration for our numerical method.

A sensitivity analysis for the numerical method developed in Chapter 4 would help determine how sensitive the method is to our parameter choices. Several parameters must be selected for this method, including the number of allowed impulses and natural jumps, the number of points in the quantization grids, and the number and placement of points in the allowed impulse grids. Such a sensitivity analysis would reveal how robust the method is to these parameter choices

Additionally, this approach could be used to calibrate a reasonable cost function tailored to our application. Because it produces interpretable policies, it is easier to adjust the cost function in a way that excludes undesirable ϵ -optimal strategies for our problem.

Finally, the numerical method relies on the assumption that the set of allowed restarting points is finite, whereas in our medical follow-up application it is in fact infinite. For the moment, we address this by discretizing the PDMP state space to obtain a finite grid of restarting points. The approximation errors introduced by this discretization should be quantified.

Model complexity and framework robustness.

A natural next step would be to increase the complexity of the dynamics in our application

³Researcher in molecular biology at the John Curtin School of Medical Research, Australian National University

example. This would allow us to examine whether the algorithms can still adapt and recover meaningful policies, while also applying the numerical method to a more demanding setting where identifying an ϵ -optimal strategy is less straightforward and more sensitive to the choice of cost function.

In addition, an extended sensitivity analysis of the Bayes-adaptive framework over broader prior ranges, building on the limited exploration already performed, would help delineate the conditions under which the method remains reliable. Finally, introducing a second unknown parameter in the parametric flow could provide a further test of the framework’s ability to accommodate heightened model uncertainty.

Long-Term Perspectives

Deep RL algorithms.

Deep RL approaches come with several challenges. A major difficulty lies in the large number of parameters that must be initialized and tuned, a process that is typically time-consuming, computationally intensive, and problem-specific. In addition, deep RL methods often yield results that lack robustness, particularly when the training data are not sufficiently representative. This limitation may partly explain why the Bayes-adaptive framework does not show a clear improvement over the non-Bayes-adaptive counterpart: as the state space expands and transitions become smoother, the benchmarked algorithms may struggle to maintain robustness in such environments. While recent research [She+20; RX22] has proposed methods to address these issues, substantial progress remains to be made.

Moreover, the learned policies provide no guarantees of convergence to the true optimum and are inherently difficult to interpret, both in terms of decision logic and in the structure of the underlying neural networks. This lack of interpretability makes it difficult to trust the agent’s decisions or to intervene when necessary. In recent years, explaining RL models [Mil+22] has become an active research area, combining techniques from explainable supervised learning with methods specifically designed to account for the sequential nature of RL problems. Our main perspective here is to focus on approaches that explain long-term behavior at the policy level. However, in a medical context, it could also be particularly valuable: understanding which patient features truly drive decision-making would not only improve trust in the method but also provide insights with real clinical relevance.

Finally, although theory suggests that in partially observable problems the optimal policy should depend on the full history of observations and actions, our experiments showed that incorporating LSTM architectures did not improve performance. In our medical setting, the problem may be only weakly partially observable, with observation windows already providing sufficient information for decision-making. However, recurrent networks such as LSTMs are more difficult to optimize than feedforward architectures, which can lead to issues like overfitting or unstable gradients. The type of memory required in our task, linked to occasional, discrete events, may also be poorly captured by the continuous temporal representation of LSTMs,

making simpler strategies, as stacking recent observations, more effective. Further analysis is needed to better understand these results and to determine whether alternative approaches could address this limitation.

Learning while planning.

We propose an approach to managing parameter uncertainty in decision-making. However, our method does not provide estimates of the unknown parameters themselves. Once a policy has converged, it can be deployed in the real environment, but this strategy is highly sensitive to the choice of estimator.

A more sophisticated approach would allow the policy to adapt continuously as real data are observed. In such a setting, the agent could refine its parameter estimates step by step along the trajectory and adjust its policy accordingly, ensuring that decisions remain aligned with the most up-to-date knowledge. One way to achieve this would be to update the agent’s prior beliefs as new data become available, thereby constructing an improved BAPOMDP model. The agent could then be retrained on this updated model or directly adapt its policy online, progressively improving both parameter estimates and decision quality.

Derived applications.

In this work, we addressed impulse control for PDMPs under partial observation and with unknown flow parameters by reformulating the problem as a BAMDP. This approach could be extended to other classes of controlled Markov models. In direct continuity with our study, one could consider tackling control problems for Hidden (semi-)Markov Models (HsMMs) by transforming them into MDPs. Indeed, an HsMM can be viewed as a discretized version of a PDMP, which suggests a natural connection between the two frameworks.

Medical Application.

Finally, while the case study developed throughout the manuscript is grounded in a real medical problem, it remains a substantial simplification of clinical reality. In our model, decisions are based solely on the concentration of clonal immunoglobulin, whereas in practice, clinicians rely on a wide range of clinical indicators. Real-world treatment choices are also considerably more complex than those considered here, involving options such as dose adjustment in chemotherapy, stem cell transplantation, immunotherapy, radiotherapy, and other alternative interventions. These simplifications were introduced deliberately to keep the problem tractable. Including additional clinical parameters would make the discretisation task of the numerical method considerably more complex and time-consuming. In such a scenario, the discretisation might have needed to be coarser, potentially reducing the accuracy of the resulting strategy in order to keep the algorithm computationally feasible. Likewise, for the deep RL approaches, handling a continuous action-state space would make the interpretation of the learned policy even more challenging, if not impossible. Addressing these challenges remains an open avenue for future work: this thesis represents only a step towards solving real-world clinical decision problems,

and much remains to be done before such methods can be deployed more realistic scenarios.

Appendix A

Proof of the a posteriori update formulas

This chapter contains details on the proof of the a posteriori update formulas in the case of the normal-gamma distribution.

We provide the proof of Theorem 6.1.1, stated in Chapter 6. This theorem establishes the posterior update formulas for the Normal-Gamma distribution.

Proof. The proof is derived from [Mur07; Fin97].

Let $y = [y_1, \dots, y_n]$ be an i.i.d. sample drawn from a log normal distribution $\text{Log-}\mathcal{N}(\mu, \lambda)$ where we assume that both the mean μ and the precision $\lambda = \sigma^{-2}$ are unknown. Here we show how to infer those parameters from a Bayesian model with conjugate priors.

The probability density function is defined as

$$\pi(\cdot|\mu, \lambda) = \frac{1}{\cdot} \left(\frac{\lambda}{2\pi} \right)^{\frac{1}{2}} \exp \left(-\frac{\lambda}{2} (\log(\cdot) - \mu)^2 \right),$$

hence the likelihood of sample y is

$$\begin{aligned} \pi(y|\mu, \lambda) &= \prod_{i=1}^n \pi(y_i|\mu, \lambda) \\ &\propto \prod_{i=1}^n \left(\frac{\lambda}{2\pi} \right)^{\frac{1}{2}} \exp \left(-\frac{\lambda}{2} (\log(y_i) - \mu)^2 \right) \\ &= \left(\frac{\lambda}{2\pi} \right)^{\frac{n}{2}} \exp \left(-\frac{\lambda}{2} \sum_{i=1}^n (\log(y_i) - \mu)^2 \right) \\ &= \left(\frac{\lambda}{2\pi} \right)^{\frac{n}{2}} \exp \left(-\frac{\lambda}{2} (n(\mu - \tilde{y})^2 + \sum_{i=1}^n (\log(y_i) - \tilde{y})^2) \right), \end{aligned}$$

where

$$\tilde{y} = \frac{1}{n} \sum_{i=1}^n \log(y_i).$$

The conjugate prior of the log normal distribution is the normal-Gamma distribution, with

density function:

$$\pi(\mu, \lambda | \mu_0, \kappa_0, \alpha_0, \beta_0) = \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \left(\frac{2\pi}{\kappa_0} \right)^{-\frac{1}{2}} \lambda^{\alpha_0 + \frac{1}{2} - 1} \exp \left(-\frac{\lambda}{2} (\kappa_0 (\mu - \mu_0)^2 + 2\beta_0) \right)$$

Note that it is possible to compute the marginals of such distributions in the following way:

$$\begin{aligned} \pi(\lambda) &= \int_{-\infty}^{\infty} \pi(\mu, \lambda) d\mu \\ &\propto \lambda^{\alpha_0 + \frac{1}{2} - 1} \exp(-\lambda\beta_0) \int_{-\infty}^{\infty} \exp \left(-\frac{\lambda\kappa_0}{2} (\mu - \mu_0)^2 \right) d\mu \\ &\propto \lambda^{\alpha_0} \exp(-\lambda\beta_0) \end{aligned}$$

And therefore λ follows a Gamma distribution with parameters $\alpha_0 + 1, \beta_0$. Similarly,

$$\begin{aligned} \pi(\mu) &= \int_0^{\infty} \pi(\mu, \lambda) d\lambda \\ &\propto \int_0^{\infty} \lambda^{\alpha_0 + \frac{1}{2} - 1} \exp \left(-\lambda \left(\beta_0 + \frac{\kappa_0 (\mu - \mu_0)^2}{2} \right) \right) d\lambda \end{aligned}$$

which integrand corresponds to the density of an unnormalized Gamma distribution $G(a = \alpha_0 + \frac{1}{2}, b = \beta_0 + \frac{\kappa_0 (\mu - \mu_0)^2}{2})$. We can therefore write:

$$\begin{aligned} \pi(\mu) &\propto \frac{\Gamma(a)}{b^a} \\ &\propto b^{-a} \\ &= \left(\beta_0 + \frac{\kappa_0 (\mu - \mu_0)^2}{2} \right)^{-\alpha_0 - \frac{1}{2}} \\ &= \left(1 + \frac{1}{2\alpha_0} + \frac{\alpha_0 \kappa_0 (\mu - \mu_0)^2}{\beta_0} \right)^{-\frac{2\alpha_0 + 1}{2}} \end{aligned}$$

That is a multivariate Student's t distribution with mean μ_0 , scale matrix $\beta_0/(\alpha_0 \kappa_0)$ and $2\alpha_0$ degrees of freedom.

The posterior is defined as follows.

$$\begin{aligned} \pi(\mu, \lambda | y) &\propto \pi(\mu, \lambda | \mu_0, \kappa_0, \alpha_0, \beta_0) \pi(y | \mu, \lambda) \\ &\propto \lambda^{\alpha_0 + \frac{1}{2} - 1} \exp \left(-\frac{\lambda}{2} (\kappa_0 (\mu - \mu_0)^2 + 2\beta_0) \right) \times \lambda^{\frac{n}{2}} \exp \left(-\frac{\lambda}{2} \sum_{i=1}^n (\log(y_i) - \mu)^2 \right) \\ &\propto \lambda^{\frac{1}{2}} \lambda^{\alpha_0 + \frac{n}{2} - 1} \exp(-\beta_0 \lambda) \exp \left[-\frac{\lambda}{2} \left(\kappa_0 (\mu - \mu_0)^2 + \sum_{i=1}^n (\log(y_i) - \mu)^2 \right) \right] \end{aligned}$$

On one side we have

$$\sum_{i=1}^n (\log(y_i) - \mu)^2 = n(\mu - \tilde{y})^2 + \sum_{i=1}^n (\log(y_i) - \tilde{y})^2$$

and on the other side

$$\kappa_0(\mu - \mu_0)^2 + n(\mu - \tilde{y})^2 = (\kappa_0 + n)(\mu - \mu_n)^2 + \frac{\kappa_0 n(\tilde{y} - \mu_0)^2}{\kappa_0 + n}$$

where

$$\mu_n = \frac{\kappa_0 \mu_0 + n \tilde{y}}{\kappa_0 + n}.$$

Therefore,

$$\begin{aligned} \pi(\mu, \lambda | y) &\propto \lambda^{\frac{1}{2}} \exp \left(-\frac{\lambda}{2} (\kappa_0 + n)(\mu - \mu_n)^2 \right) \\ &\times \lambda^{\alpha_0 + \frac{n}{2} - 1} \exp(-\beta_0 \lambda) \exp \left(-\frac{\lambda}{2} \sum_{i=1}^n (\log(y_i) - \tilde{y})^2 \right) \exp \left(-\frac{\lambda}{2} \frac{\kappa_0 n(\tilde{y} - \mu_0)^2}{\kappa_0 + n} \right) \end{aligned}$$

which is once again a normal-Gamma distribution with parameters

$$\begin{aligned} \mu_n &= \frac{\kappa_0 \mu_0 + n \tilde{y}}{\kappa_0 + n} \\ \kappa_n &= \kappa_0 + n \\ \alpha_n &= \alpha_0 + n/2 \\ \beta_n &= \beta_0 + \frac{1}{2} \sum_{i=1}^n (\log(y_i) - \tilde{y})^2 + \frac{\kappa_0 n(\tilde{y} - \mu_0)^2}{2(\kappa_0 + n)} \end{aligned}$$

We have

$$\begin{aligned} \pi(\lambda | y) &= G(\lambda | \alpha_n, \beta_n) \\ \pi(\mu | y) &= T_{2\alpha_n}(\mu | \mu_n, \beta_n / (\alpha_n \kappa_n)) \end{aligned}$$

Note that in the special case where $n = 1$ (only one new observation), the updated parameters are simply

$$\mu_{n+1} = \frac{\kappa_n \mu_n + \log(y_n)}{\kappa_n + 1}$$

$$\kappa_{n+1} = \kappa_n + 1$$

$$\alpha_{n+1} = \alpha_n + 1/2$$

$$\beta_{n+1} = \beta_n + \frac{\kappa_n (\log(y_n) - \mu_n)^2}{2(\kappa_n + 1)}.$$

□

Appendix B

Numerical experiment details

In this Chapter, we provide the details of the parameters used in the numerical experiments of Chapters 4, 5, and 6. We also report the optimal configurations obtained for the neural networks employed in Chapters 5 and 6.

B.1 Model Parameters

Table B.1 presents the PDMP parameters that are consistently applied across all chapters and employed throughout the simulations, except where explicitly noted otherwise.

Table B.1: PDMP parameter values used in experiments

Name	Notation	Value
Boundaries		
Biomarker nominal level	ζ_0	1
Biomarker death level	D	40
Horizon	T	2400
Flow		
Slope parameter in relapse	$v_1 \sim \text{Log} - \mathcal{N}(\mu, 1/\sigma^2)$	$(\mu = -6.40, \sigma = \sqrt{2})$
Slope parameter in non curable relapse	$v_2 \sim \mathcal{U}[a, b]$	$[a = 0.0001, b = 0.06]$
Jump function		
Scale parameter of μ_{01}^k	α_1^k	$0.0002/k$
Shape parameter of μ_{01}^k	β_1^k	3.5
Scale parameter of μ_{02}^k	α_2^k	0.00001
Shape parameter of μ_{02}^k	β_2^k	3.5
Scale parameter of $\mu_{12}^{k,0}$	$\alpha_2^{k,0}$	$1.116e^{-20}k/4$
Shape parameter of $\mu_{12}^{k,0}$	$\beta_2^{k,0}$	4.5
Scale parameter of $\mu_{12}^{k,1}$	$\alpha_2^{k,0}$	$5.58e^{-7}k$
Shape parameter of $\mu_{12}^{k,1}$	$\beta_2^{k,0}$	-0.8

Table B.2 presents the POMDP parameters that are consistently applied across all the simulations for Chapters 5 and 6, except where explicitly noted otherwise.

Table B.2: **POMDP** parameter values used in experiments

Name	Notation	Value
Observation		
Noise	$\epsilon \sim \mathcal{N}(\mu, \sigma)$	$(\mu = 0, \sigma = 1)$
Cost function		
Treatment cost	C_ℓ	0.1
Visit cost	C_v	1
Death cost	C_D	300
Horizon constraint cost	C_H	1000
Treatment constraint cost	C_T	1000

B.2 Neural network set hyperparameters

Table B.3 shows the range of hyperparameters tested for the Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) algorithms throughout Chapters 5 and 6.

B.3 Neural network final hyperparameters

Table B.4 presents the optimal configurations selected for each algorithm trained under the POMDP framework in Chapter 5.

Table B.5 presents the optimal configurations selected for each algorithm trained under the BAPOMDP framework in Chapter 6.

Table B.3: Summary of tuned parameters and their possible values.

Parameter	Possible Values
Shared Hyperparameters	
Discount Factor (γ)	{0.95, 0.97, 0.99, 0.999}
Training Batch Size	{32, 65, 256, 512, 1024, 2048, 4096}
Training Batch Mode	{TruncateEpisodes, CompleteEpisodes}
Learning Rate	{3e-5, 0.0001, 0.0003, 0.001}
Fully Connected Layer Sizes	{[32], [32,32], [64], [64,64]}
Activation Function	{linear, relu, tanh}
Observation Filter	{MeanStdFilter, NoFilter}
DQN	
TD Error Loss	{Huber, MSE}
Number of Step	{1, 2, 3}
Double Q-learning	{True, False}
Dueling Architecture	{True, False}
Replay Buffer Capacity	{50000, 100000, 500000, 1000000}
Prioritized Replay Alpha	{0.5, 0.6}
Prioritized Replay Beta	{0.4, 0.5}
Prioritized Replay Epsilon	{1e-6, 3e-6}
Epsilon Timestep	{2, 10000, 50000, 100000, 200000}
Initial Epsilon	{0.9, 1.0, 1.5}
Final Epsilon	{0.0, 0.01, 0.02}
Value Range Min	{-60 000, -50 000, -40 000}
Value Range Max	{-500, -10, 0, 10}
Noisy Nets	{True, False}
Hidden Layers Size	{[64], [128], [256], [512]}
Training Intensity	{1, 4, 16, 32}
Target Network Update Frequency	{500, 1000, 5000, 10000, 20000}
Steps Before Learning Starts	{1000, 10000, 20000}
PPO Without LSTM	
Number of SGD Iterations	{10, 20, 30}
SGD Minibatch Size	{64, 128, 256}
Clipping Parameter	{0.1, 0.2, 0.25, 0.3, 0.4, 0.5}
Value Function Shared Layers	{False, True}
Gradient Clipping	{None, 0.5, 1.0, 2.0, 5.0}
Use Generalized Advantage Estimation (GAE)	{True, False}
Value Function Loss Coefficient	{0.001, 0.01, 0.1, 0.5, 1.0}
PPO With LSTM	
LSTM Cell Size	{64, 256, 512, 1024}
Use Previous Action in LSTM	{False}
Use Previous Reward in LSTM	{False}

Table B.4: Best configuration for each algorithm trained with POMDP framework.

Parameter	Without Action-Masking	with Action-Masking
DQN		
TD Error Loss	Huber	MSE
Discount Factor (γ)	0.95	0.999
Number of Step	3	3
Double Q-learning	True	True
Dueling Architecture	False	False
Replay Buffer Capacity	50 000	500 000
Prioritized Replay Alpha	0.5	0.6
Prioritized Replay Beta	0.5	0.4
Prioritized Replay Epsilon	3×10^{-6}	3×10^{-6}
Epsilon Timestep	50 000	50 000
Initial Epsilon	1.5	1.5
Final Epsilon	0.0	0.0
Value Range Min	-60 000	-50 000
Value Range Max	-500	-500
Training Batch Size	32	4 096
Training Batch Mode	TruncateEpisodes	CompleteEpisodes
Learning Rate	3×10^{-5}	3×10^{-5}
Fully Connected Layer Sizes	[64,64]	[64]
Activation Function	relu	tanh
Observation Filter	MeanStdFilter	NoFilter
Noisy Nets	False	False
Hidden Layers Size	[128]	[]
Training Intensity	1	1
Target Network Update Frequency	1 000	500
Steps Before Learning Starts	20 000	10 000
PPO		
Discount Factor (γ)	0.999	0.999
Number of SGD Iterations	20	20
SGD Minibatch Size	256	64
Training Batch Size	1 024	2 048
Training Batch Mode	CompleteEpisodes	TruncateEpisodes
Learning Rate	0.001	0.001
Clipping Parameter	0.3	0.2
Fully Connected Layer Sizes	[32]	[32]
Activation Function	linear	relu
Value Function Shared Layers	False	False
Observation Filter	NoFilter	NoFilter
Gradient Clipping	2.0	2.0
Use GAE	False	True
Value Function Loss Coefficient	0.5	0.1
PPO with LSTM		
Discount Factor (γ)	0.99	
Number of SGD Iterations	30	
SGD Minibatch Size	128	
Training Batch Size	256	
Training Batch Mode	TruncateEpisodes	
Learning Rate	3×10^{-5}	
Clipping Parameter	0.5	
Fully Connected Layer Sizes	[64]	
Activation Function	tanh	
Value Function Shared Layers	True	
Observation Filter	MeanStdFilter	
Gradient Clipping	Null	
Use GAE	True	
Value Function Loss Coefficient	0.01	
LSTM Cell Size	[64]	
Use Previous Action in LSTM	False	
Use Previous Reward in LSTM	False	

Table B.5: Best configuration for each algorithm trained with BAPOMDP framework.

Parameter	Without Action-Masking	with Action-Masking
DQN		
TD Error Loss	MSE	MSE
Discount Factor (γ)	0.99	0.99
Number of Step	3	3
Double Q-learning	True	False
Dueling Architecture	False	False
Replay Buffer Capacity	500 000	500 000
Prioritized Replay Alpha	0.6	0.6
Prioritized Replay Beta	0.4	0.4
Prioritized Replay Epsilon	3×10^{-6}	3×10^{-6}
Epsilon Timestep	2	50000
Initial Epsilon	1.5	1.5
Final Epsilon	0.0	0.0
Value Range Min	-50 000	-40 000
Value Range Max	10	-500
Training Batch Size	4096	2048
Training Batch Mode	CompleteEpisodes	TruncateEpisodes
Learning Rate	0.0003	0.001
Fully Connected Layer Sizes	[64,64]	[64,64]
Activation Function	relu	tanh
Observation Filter	NoFilter	NoFilter
Noisy Nets	False	False
Hidden Layers Size	[512]	[]
Training Intensity	16	16
Target Network Update Frequency	500	500
Steps Before Learning Starts	10 000	20 000
PPO		
Discount Factor (γ)	0.95	0.97
Number of SGD Iterations	20	20
SGD Minibatch Size	256	128
Training Batch Size	256	256
Training Batch Mode	TruncateEpisodes	TruncateEpisodes
Learning Rate	0.001	0.0003
Clipping Parameter	0.25	0.5
Fully Connected Layer Sizes	[64,64]	[64,64]
Activation Function	tanh	linear
Value Function Shared Layers	True	False
Observation Filter	MeanStdFilter	NoFilter
Gradient Clipping	2.0	0.5
Use GAE	True	True
Value Function Loss Coefficient	0.5	0.01
PPO with LSTM		
Discount Factor (γ)	0.97	
Number of SGD Iterations	10	
SGD Minibatch Size	64	
Training Batch Size	256	
Training Batch Mode	TruncateEpisodes	
Learning Rate	3×10^{-5}	
Clipping Parameter	0.3	
Fully Connected Layer Sizes	[64,64]	
Activation Function	linear	
Value Function Shared Layers	True	
Observation Filter	MeanStdFilter	
Gradient Clipping	2.0	
Use GAE	True	
Value Function Loss Coefficient	0.001	
LSTM Cell Size	[64]	
Use Previous Action in LSTM	False	
Use Previous Reward in LSTM	False	

Bibliography

- [Alm01] Anthony Almudevar. “A Dynamic Programming Algorithm for the Optimal Control of Piecewise Deterministic Markov Processes”. In: *SIAM Journal on Control and Optimization* 40.2 (2001), pp. 525–539. DOI: 10.1137/S0363012999364474.
- [ALM15] Philipp M. Altrock, Lin L. Liu, and Franziska Michor. “The mathematics of cancer: integrating quantitative models”. In: *Nat. Rev. Cancer* 15.12 (Dec. 2015), pp. 730–745. ISSN: 1474-1768. DOI: 10.1038/nrc4029. eprint: 26597528.
- [AK11] David F. Anderson and Thomas G. Kurtz. “Design and Analysis of Biomolecular Circuits: Engineering Approaches to Systems and Synthetic Biology”. In: New York, NY: Springer New York, 2011. Chap. Continuous Time Markov Chain Models for Chemical Reaction Networks, pp. 3–42. DOI: 10.1007/978-1-4419-6766-4_1.
- [AS22] Dilip Arumugam and Satinder Singh. “Planning to the Information Horizon of BAMDPs via Epistemic State Abstraction”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 20482–20497.
- [Åst65] K. J. Åström. “Optimal control of Markov processes with incomplete state information”. In: *Journal of Mathematical Analysis and Applications* 10.1 (1965), pp. 174–205. ISSN: 0022-247X. DOI: [https://doi.org/10.1016/0022-247X\(65\)90154-X](https://doi.org/10.1016/0022-247X(65)90154-X).
- [AS97] C.G. Atkeson and J.C. Santamaria. “A comparison of direct and model-based reinforcement learning”. In: *Proceedings of International Conference on Robotics and Automation*. Vol. 4. Apr. 1997, 3557–3564 vol.4. DOI: 10.1109/ROBOT.1997.606886.
- [AS07] Chris Atkeson and Benjamin Stephens. “Random Sampling of States in Dynamic Programming”. In: *Advances in Neural Information Processing Systems* 20 (2007).
- [ALa17] Michel Attal, Valerie Lauwers-Cances, and et al. “Lenalidomide, Bortezomib, and Dexamethasone with Transplantation for Myeloma”. In: *N. Engl. J. Med.* 376.14 (Apr. 2017), pp. 1311–1320. ISSN: 1533-4406. DOI: 10.1056/NEJMoa1611750. eprint: 28379796.
- [Aza14] Romain Azaïs. “A recursive nonparametric estimator for the transition kernel of a piecewise-deterministic Markov process”. In: *ESAIM: Probability and Statistics* 18 (2014), pp. 726–749. ISSN: 1292-8100, 1262-3318. DOI: 10.1051/ps/2013054.
- [ADG14] Romain Azaïs, François Dufour, and Anne Gégout-Petit. “Non-Parametric Estimation of the Conditional Distribution of the Interjumping Times for Piecewise-Deterministic Markov Processes”. en. In: *Scandinavian Journal of Statistics* 41.4 (Dec. 2014), pp. 950–969. ISSN: 0303-6898, 1467-9469. DOI: 10.1111/sjos.12076.

- [AG19] Romain Azaïs and Alexandre G  nadot. “Estimation of the average number of continuous crossings for non-stationary non-diffusion processes”. en. In: *Journal of Statistical Planning and Inference* 198 (Jan. 2019), pp. 119–138. ISSN: 03783758. DOI: 10.1016/j.jspi.2018.04.002.
- [BP03] Vlad Bally and Gilles Pag  s. “A quantization algorithm for solving multidimensional discrete-time optimal stopping problems”. In: *Bernoulli* 9.6 (Dec. 2003), pp. 1003–1049. ISSN: 1350-7265. DOI: 10.3150/bj/1072215199.
- [Ban18] Elena Bandini. “Optimal control of Piecewise Deterministic Markov Processes: a BSDE representation of the value function”. In: *ESAIM: Control, Optimisation and Calculus of Variations* 24.1 (2018), pp. 311–354.
- [BL08] Vlad Stefan Barbu and Nikolaos Limnios. *Semi-Markov Chains and Hidden Semi-Markov Models toward Applications: Their use in Reliability and DNA Analysis*. en. Vol. 191. Lecture Notes in Statistics. New York, NY: Springer New York, 2008. ISBN: 978-0-387-73171-1 978-0-387-73173-5. DOI: 10.1007/978-0-387-73173-5.
- [Bar+13] J. B. Bardet et al. “Total variation estimates for the TCP process”. In: *Electronic Journal of Probability* 18 (2013), no. 10, 21.
- [BR10] N. B  uerle and U. Rieder. “Optimal control of piecewise deterministic Markov processes with finite time horizon”. In: *Modern Trends in Controlled Stochastic - Processes - Theory and Applications*. Ed.: A. B. Piunovskiy. Luniver, 2010, pp. 123–143. ISBN: 978-1-905986-30-9.
- [BL18] Nicole B  uerle and Dirk Lange. “Optimal Control of Partially Observable Piecewise Deterministic Markov Processes”. en. In: *SIAM Journal on Control and Optimization* 56.2 (Jan. 2018), pp. 1441–1462. ISSN: 0363-0129, 1095-7138. DOI: 10.1137/17M1134731.
- [BR11] Nicole B  uerle and Ulrich Rieder. *Markov Decision Processes with Applications to Finance*. en. Universitext. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011. ISBN: 978-3-642-18323-2 978-3-642-18324-9. DOI: 10.1007/978-3-642-18324-9.
- [BB01] J. Baxter and P. L. Bartlett. “Infinite-Horizon Policy-Gradient Estimation”. en. In: *Journal of Artificial Intelligence Research* 15 (Nov. 2001), pp. 319–350. ISSN: 1076-9757. DOI: 10.1613/jair.806.
- [Bel57] Richard Bellman. “A Markovian Decision Process”. In: *Journal of Mathematics and Mechanics* 6.5 (1957), pp. 679–684.
- [Bel58] Richard Bellman. “Dynamic programming and stochastic control processes”. In: *Inform. And Control (Shenyang)* 1.3 (Sept. 1958). Publisher: Academic Press, pp. 228–239. ISSN: 0019-9958. DOI: 10.1016/S0019-9958(58)80003-0.
- [Bel61] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton, NJ, USA: Princeton University Press, 1961. ISBN: 978-0-69107901-1.
- [Ben+12] Michel Bena  m et al. “Quantitative ergodicity for some switched dynamical systems”. In: *Electronic Communications in Probability* 17.none (Jan. 2012). ISSN: 1083-589X. DOI: 10.1214/ECP.v17-1932.

- [BH61] William Beranek and Ronald A. Howard. “Dynamic Programming and Markov Processes”. In: *Technometrics* 3.1 (1961), pp. 120–121. DOI: 10.1080/00401706.1961.10489934. eprint: <https://doi.org/10.1080/00401706.1961.10489934>. URL: <https://doi.org/10.1080/00401706.1961.10489934>.
- [Ber75] D. Bertsekas. “Convergence of discretization procedures in dynamic programming”. In: *IEEE Transactions on Automatic Control* 20.3 (June 1975). Conference Name: IEEE Transactions on Automatic Control, pp. 415–419. ISSN: 1558-2523. DOI: 10.1109/TAC.1975.1100984.
- [Bie+18] Joris Bierkens et al. “Piecewise deterministic Markov processes for scalable Monte Carlo on restricted domains”. en. In: *Statistics & Probability Letters* 136 (May 2018), pp. 148–154. ISSN: 01677152. DOI: 10.1016/j.spl.2018.02.021.
- [BDH99] C. Boutilier, T. Dean, and S. Hanks. “Decision-Theoretic Planning: Structural Assumptions and Computational Leverage”. In: *Journal of Artificial Intelligence Research (JAIR)* 11 (July 1999). Publisher: AI Access Foundation. ISSN: 1076-9757. DOI: 10.1613/jair.575.
- [BdD13] Adrien Brandejsky, Benoîte de Saporta, and François Dufour. “Optimal stopping for partially observed piecewise-deterministic Markov processes”. In: *Stochastic Processes and their Applications* 123.8 (2013), pp. 3201–3238. ISSN: 0304-4149. DOI: <https://doi.org/10.1016/j.spa.2013.03.006>.
- [BSD12] Adrien Brandejsky, Benoîte de Saporta, and François Dufour. “Numerical method for expectations of piecewise deterministic Markov processes”. en. In: *Communications in Applied Mathematics and Computational Science* 7.1 (Mar. 2012), pp. 63–104. ISSN: 2157-5452, 1559-3940. DOI: 10.2140/camcos.2012.7.63.
- [Bro+16] Greg Brockman et al. “Openai gym”. In: *arXiv preprint arXiv:1606.01540* (2016).
- [Bro+12] Cameron B. Browne et al. “A Survey of Monte Carlo Tree Search Methods”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (Mar. 2012). Conference Name: IEEE Transactions on Computational Intelligence and AI in Games, pp. 1–43. ISSN: 1943-0698. DOI: 10.1109/TCIAIG.2012.2186810.
- [Cas+11] Laia Castells-Roca et al. “Heat Shock Response in Yeast Involves Changes in Both Transcription Rates and mRNA Stabilities”. In: *PLoS One* 6.2 (Feb. 2011), e17272. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0017272.
- [CP07] Pablo Samuel Castro and Doina Precup. “Using Linear Programming for Bayesian Exploration in Markov Decision Processes”. In: *International Joint Conference on Artificial Intelligence*. 2007.
- [Cha+08] Guillaume Chaslot et al. “Progressive Strategies for Monte-Carlo Tree Search”. In: *New Mathematics and Natural Computation* 04 (Nov. 2008), pp. 343–357. DOI: 10.1142/S1793005708001094.

- [CL08] Julien Chiquet and Nikolaos Limnios. “A method to compute the transition function of a piecewise deterministic Markov process with application to reliability”. en. In: *Statistics & Probability Letters* 78.12 (Sept. 2008), pp. 1397–1403. ISSN: 01677152. DOI: 10.1016/j.spl.2007.12.016.
- [Cle24] Alice Cleynen. “Statistical methods for the diagnosis, prognosos and treatment strategies of cancer patients”. HDR Thesis. Université de Montpellier, Nov. 2024.
- [CS18] Alice Cleynen and Benoîte de Saporta. “Change-point detection for piecewise deterministic Markov processes”. en. In: *Automatica* 97 (Nov. 2018), pp. 234–247. ISSN: 00051098. DOI: 10.1016/j.automatica.2018.08.011.
- [CS21] Alice Cleynen and Benoîte de Saporta. *Numerical method to solve impulse control problems for partially observed piecewise deterministic Markov processes*. 2021. DOI: 10.48550/ARXIV.2112.09408.
- [CS23] Alice Cleynen and Benoîte de Saporta. “Numerical method to solve impulse control problems for partially observed piecewise deterministic Markov processes”. In: *arXiv* (2023). eprint: 2112.09408.
- [CSV25] Alice Cleynen, Benoîte de Saporta, and Amélie Vernay. “Estimating relapse time distribution from longitudinal biomarker trajectories using iterative regression and continuous time Markov processes”. In: *arXiv* (Mar. 2025). DOI: 10.48550/arXiv.2503.10448. eprint: 2503.10448.
- [Cle+14] Alice Cleynen et al. “Segmentor3IsBack: an R package for the fast and exact segmentation of Seq-data”. In: *Algorithms Mol. Biol.* 9.1 (Jan. 2014), pp. 1–11. ISSN: 1748-7188. DOI: 10.1186/1748-7188-9-6.
- [Cle+24a] Alice Cleynen et al. “Deep Reinforcement Learning for Controlled Piecewise Deterministic Markov Process in Cancer Treatment Follow-up”. In: *La journée Santé et IA*. Plate-Forme Intelligence Artificielle (PFIA). La Rochelle, France, July 2024. URL: <https://hal.science/hal-04695790>.
- [Cle+24b] Alice Cleynen et al. “Deep reinforcement learning for controlled piecewise deterministic Markov process in cancer treatment follow-up”. In: *55ièmes Journées de Statistique*. SFdS. Bordeaux, France, May 2024. URL: <https://hal.science/hal-04593903>.
- [Cle+25] Alice Cleynen et al. “Bridging Impulse Control of Piecewise Deterministic Markov Processes and Markov Decision Processes: Frameworks, Extensions, and Open Challenges”. In: *arXiv* (Jan. 2025). DOI: 10.48550/arXiv.2501.04120. eprint: 2501.04120.
- [CSJ20] Bertrand Cloez, Benoîte de Saporta, and Maud Joubaud. “Optimal stopping for measure-valued piecewise deterministic Markov processes”. en. In: *Journal of Applied Probability* 57.2 (June 2020), pp. 497–512. ISSN: 0021-9002, 1475-6072. DOI: 10.1017/jpr.2020.18.
- [Clo+17] Bertrand Cloez et al. “Probabilistic and Piecewise Deterministic models in Biology”. In: *ESAIM: Proceedings and Surveys* 60 (2017). Ed. by Jean-François Coeurjolly and Adeline Leclercq-Samson, pp. 225–245. ISSN: 2267-3059. DOI: 10.1051/proc/201760225.

- [Coc97] Christiane Coccozza-Thivent. *Processus stochastiques et fiabilité des systèmes*. fre. Mathématiques & applications 28. Berlin Heidelberg Paris [etc.]: Springer-Verl, 1997. ISBN: 978-3-540-63390-7.
- [Coc21] Christiane Coccozza-Thivent. *Markov Renewal and Piecewise Deterministic Processes*. en. Vol. 100. Probability Theory and Stochastic Modelling. Cham: Springer International Publishing, 2021. ISBN: 978-3-030-70446-9 978-3-030-70447-6. DOI: 10.1007/978-3-030-70447-6.
- [Cos16] Manon Costa. “A piecewise deterministic model for a prey-predator community”. In: *The Annals of Applied Probability* 26.6 (Dec. 2016). ISSN: 1050-5164. DOI: 10.1214/16-AAP1182.
- [Cos93] O. L. V. Costa. “Discretizations for the average impulse control of piecewise deterministic processes”. en. In: *Journal of Applied Probability* 30.2 (June 1993), pp. 405–420. ISSN: 0021-9002, 1475-6072. DOI: 10.2307/3214849.
- [CD88] O. L. V. Costa and M. H. A. Davis. “Approximations for optimal stopping of a piecewise-deterministic process”. en. In: *Mathematics of Control, Signals, and Systems* 1.2 (June 1988), pp. 123–146. ISSN: 0932-4194, 1435-568X. DOI: 10.1007/BF02551405.
- [CD89] O. L. V. Costa and M. H. A. Davis. “Impulse control of piecewise-deterministic processes”. en. In: *Mathematics of Control, Signals, and Systems* 2.3 (Sept. 1989), pp. 187–206. ISSN: 0932-4194, 1435-568X. DOI: 10.1007/BF02551384.
- [CD08] O. L. V. Costa and F. Dufour. “Stability and Ergodicity of Piecewise Deterministic Markov Processes”. en. In: *SIAM Journal on Control and Optimization* 47.2 (Jan. 2008), pp. 1053–1077. ISSN: 0363-0129, 1095-7138. DOI: 10.1137/060670109.
- [CD10] O. L. V. Costa and F. Dufour. “Average Continuous Control of Piecewise Deterministic Markov Processes”. en. In: *SIAM Journal on Control and Optimization* 48.7 (Jan. 2010), pp. 4262–4291. ISSN: 0363-0129, 1095-7138. DOI: 10.1137/080718541.
- [CD22] O. L. V. Costa and F. Dufour. “Optimal Control of Piecewise Deterministic Markov Processes”. In: *Stochastic Analysis, Filtering, and Stochastic Optimization: A Commemorative Volume to Honor Mark H. A. Davis’s Contributions*. Cham: Springer International Publishing, 2022, pp. 53–77. DOI: 10.1007/978-3-030-98519-6_3.
- [CD13] Oswaldo Luiz Do Valle Costa and François Dufour. *Continuous Average Control of Piecewise Deterministic Markov Processes*. en. SpringerBriefs in Mathematics. New York, NY: Springer New York, 2013. ISBN: 978-1-4614-6982-7 978-1-4614-6983-4. DOI: 10.1007/978-1-4614-6983-4.
- [Cou06] Rémi Coulom. “Efficient selectivity and backup operators in Monte-Carlo tree search”. In: *5th International Conference on Computer and Games*. 2006, pp. 72–83.
- [CP18] Robert A. Crawford and Graham D. Pavitt. “Translational regulation in response to stress in *Saccharomyces cerevisiae*”. In: *Yeast (Chichester, England)* 36.1 (Sept. 2018), p. 5. DOI: 10.1002/yea.3349.

- [Dav84] M. H. A. Davis. “Piecewise-Deterministic Markov Processes: A General Class of Non-Diffusion Stochastic Models”. en. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 46.3 (July 1984), pp. 353–376. ISSN: 1369-7412, 1467-9868. DOI: 10.1111/j.2517-6161.1984.tb01308.x.
- [Dav93] M. H. A. Davis. *Markov models and optimization*. 1st ed. Monographs on statistics and applied probability. London ; New York: Chapman & Hall, 1993. ISBN: 978-0-412-31410-0.
- [DNP13] M. P. Deisenroth, G. Neumann, and J. Peters. “A survey on policy search for robotics”. In: *Foundations and Trends in Robotics* 2.1-2 (2013), pp. 1–142. ISSN: 1614-2411.
- [Dem91] M. A. H. Dempster. “Optimal control of piecewise deterministic Markov processes”. In: *Applied stochastic analysis (London, 1989)*. Vol. 5. Stochastics Monogr. 1991, pp. 303–325.
- [DY95] M. A. H. Dempster and J. J. Ye. “Impulse control of piecewise deterministic Markov processes”. In: *The Annals of Applied Probability* 5.2 (1995), pp. 399–423.
- [DY96] M. A. H. Dempster and J. J. Ye. “Generalized Bellman-Hamilton-Jacobi optimality conditions for a control problem with a boundary condition”. en. In: *Applied Mathematics & Optimization* 33.3 (May 1996), pp. 211–225. ISSN: 0095-4616, 1432-0606. DOI: 10.1007/BF01204702.
- [DY92] M.A.H. Dempster and J.J. Ye. “Necessary and sufficient optimality conditions for control of piecewise deterministic markov processes”. en. In: *Stochastics and Stochastic Reports* 40.3-4 (Sept. 1992), pp. 125–145. ISSN: 1045-1129. DOI: 10.1080/17442509208833785.
- [Dou+15] Marie Doumic et al. “Statistical estimation of a growth-fragmentation model observed on a genealogical tree”. In: *Bernoulli* 21.3 (Aug. 2015). ISSN: 1350-7265. DOI: 10.3150/14-BEJ623.
- [DFD20] Jianzhun Du, Joseph Futoma, and Finale Doshi-Velez. “Model-based reinforcement learning for semi-Markov decision processes with neural ODEs”. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS ’20. Vancouver, BC, Canada: Curran Associates Inc., 2020. ISBN: 9781713829546.
- [Duf02] Michael O’Gordon Duff. “Optimal learning: Computational procedures for Bayes -adaptive Markov decision processes”. PhD Thesis. UMassAmherst, Jan. 2002.
- [DHP16] F. Dufour, M. Horiguchi, and A. B. Piunovskiy. “Optimal impulsive control of piecewise deterministic Markov processes”. en. In: *Stochastics* 88.7 (Oct. 2016), pp. 1073–1098. ISSN: 1744-2508, 1744-2516. DOI: 10.1080/17442508.2016.1197925.
- [Fea+18] Paul Fearnhead et al. “Piecewise Deterministic Markov Processes for Continuous-Time Monte Carlo”. In: *Statistical Science* 33.3 (Aug. 2018). ISSN: 0883-4237. DOI: 10.1214/18-STS648.
- [Fen+04] Zhengzhu Feng et al. “Dynamic Programming for Structured Continuous Markov Decision Problems”. en. In: (2004).
- [Fin97] Daniel Fink. *A Compendium of Conjugate Priors*. Tech. rep. Environmental Statistics Group - Department of Biology - Montana State Univeristy, 1997.

- [FGM12] Joaquin Fontbona, Hélène Guérin, and Florent Malrieu. “Quantitative Estimates for the Long-Time Behavior of an Ergodic Variant of the Telegraph Process”. en. In: *Advances in Applied Probability* 44.4 (Dec. 2012), pp. 977–994. ISSN: 0001-8678, 1475-6064. DOI: 10.1239/aap/1354716586.
- [FSS04] Lothar Forwick, Manfred Schäl, and Michael Schmitz. “Piecewise Deterministic Markov Control Processes with Feedback Controls and Unbounded Costs”. en. In: *Acta Applicandae Mathematicae* 82.3 (July 2004), pp. 239–267. ISSN: 0167-8019. DOI: 10.1023/B:ACAP.0000031200.76583.75.
- [FHC15] Coralie Fritsch, Jérôme Harmand, and Fabien Campillo. “A modeling approach of the chemostat”. en. In: *Ecological Modelling* 299 (Mar. 2015), pp. 1–13. ISSN: 03043800. DOI: 10.1016/j.ecolmodel.2014.11.021.
- [Gat92] Dariusz Gatarek. “Optimality conditions for impulsive control of piecewise-deterministic processes”. en. In: *Mathematics of Control, Signals, and Systems* 5.2 (June 1992), pp. 217–232. ISSN: 0932-4194, 1435-568X. DOI: 10.1007/BF01215846.
- [GT14] A. Génadot and M. Thieullen. “Multiscale Piecewise Deterministic Markov Process in infinite dimension: central limit theorem and Langevin approximation”. In: *ESAIM: Probability and Statistics* 18 (2014), pp. 541–569. ISSN: 1292-8100, 1262-3318. DOI: 10.1051/ps/2013051.
- [Ger+16] Christian Gerecke et al. “The Diagnosis and Treatment of Multiple Myeloma”. In: *Dtsch. Arztebl. Int.* 113.27-28 (July 2016), pp. 470–476. ISSN: 1866-0452. DOI: 10.3238/arztebl.2016.0470. eprint: 27476706.
- [Gil77] Daniel T. Gillespie. “Exact stochastic simulation of coupled chemical reactions”. In: *The Journal of Physical Chemistry* 81.25 (1977), pp. 2340–2361. DOI: <https://doi.org/10.1021/j100540a008>.
- [Goa+23] Ethan Goan et al. “Piecewise Deterministic Markov Processes for Bayesian Neural Networks”. In: *arXiv* (2023). eprint: 2302.08724.
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press. 2016.
- [GN98] R.M. Gray and D.L. Neuhoff. “Quantization”. In: *IEEE Transactions on Information Theory* 44.6 (Oct. 1998), pp. 2325–2383. ISSN: 00189448. DOI: 10.1109/18.720541.
- [Gro20] Divya Grover. “Sample Efficient Bayesian Reinforcement Learning”. fr. PhD thesis. CHALMERS UNIVERSITY OF TECHNOLOGY, 2020.
- [GSD12] Arthur Guez, David Silver, and Peter Dayan. “Efficient Bayes-Adaptive Reinforcement Learning using Sample-Based Search”. In: *Advances in Neural Information Processing Systems*. 2012.
- [Gug86] U.S. Gugerui. “Optimal stopping of a piecewise-deterministic markov process”. en. In: *Stochastics* 19.4 (Dec. 1986), pp. 221–236. ISSN: 0090-9491. DOI: 10.1080/17442508608833426.
- [Haf+24] Danijar Hafner et al. *Mastering Diverse Domains through World Models*. en. Apr. 2024.
- [Has10] Hado Hasselt. “Double Q-learning”. In: *Advances in Neural Information Processing Systems*. Vol. 23. Curran Associates, Inc., 2010.

- [HGS16] Hado van Hasselt, Arthur Guez, and David Silver. “Deep Reinforcement Learning with Double Q-Learning”. en. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 30.1 (Mar. 2016). Number: 1. ISSN: 2374-3468. DOI: 10.1609/aaai.v30i1.10295.
- [HS15] Matthew Hausknecht and Peter Stone. “Deep Recurrent Q-Learning for Partially Observable MDPs”. In: *AAAI* (July 2015).
- [Hee+15] Nicolas Heess et al. “Memory-based control with recurrent neural networks”. In: *arXiv preprint arXiv:1512.04455* (2015).
- [Her+17] Ulysse Herbach et al. “Inferring gene regulatory networks from single-cell data: a mechanistic approach”. en. In: *BMC Systems Biology* 11.1 (Dec. 2017), p. 105. ISSN: 1752-0509. DOI: 10.1186/s12918-017-0487-0.
- [Hes+18] Matteo Hessel et al. “Rainbow: combining improvements in deep reinforcement learning”. In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*. AAAI’18/IAAI’18/EAAI’18. New Orleans, Louisiana, USA: AAAI Press, 2018, pp. 3215–3222. ISBN: 978-1-57735-800-8.
- [Ho+18] Yi-Hsuan Ho et al. “Decoupling yeast cell division and stress defense implicates mRNA repression in translational reallocation during stress”. In: *Current biology : CB* 28.16 (Aug. 2018), p. 2673. DOI: 10.1016/j.cub.2018.06.044.
- [Hoe+10] Jesse Hoey et al. “Automated handwashing assistance for persons with dementia using video and a partially observable Markov decision process”. In: *Comput. Vision Image Understanding* 114.5 (May 2010). Publisher: Academic Press, pp. 503–519. ISSN: 1077-3142. DOI: 10.1016/j.cviu.2009.06.008.
- [How62] Ronald A. Howard. “Dynamic Programming and Markov Processes”. In: *The Mathematical Gazette* 46.358 (1962), pp. 340–341. DOI: 10.2307/3611804.
- [HHB07] Sabrina Hundt, Ulrike Haug, and Hermann Brenner. “Blood markers for early detection of colorectal cancer: a systematic review”. In: *Cancer Epidemiol. Biomarkers Prev.* 16.10 (Oct. 2007), pp. 1935–1953. ISSN: 1055-9965. DOI: 10.1158/1055-9965.EPI-06-0994. eprint: 17932341.
- [Igl+18] Maximilian Igl et al. “Deep Variational Reinforcement Learning for POMDPs”. In: *International Conference on Machine Learning*. PMLR. 2018, pp. 2117–2126.
- [Jac06] Martin Jacobsen. *Point Process Theory and Applications*. en. Probability and its Applications. Boston: Birkhäuser-Verlag, 2006. ISBN: 978-0-8176-4215-0. DOI: 10.1007/0-8176-4463-6.
- [JGN23] Mina Jafari, Behnam Ghavami, and Vahid Sattari Naeini. “A Decision Making Approach for Chemotherapy Planning based on Evolutionary Processing”. In: *arXiv* (Mar. 2023). DOI: 10.48550/arXiv.2303.10535. eprint: 2303.10535.

- [Ji+07] Shihao Ji et al. “Point-based policy iteration”. In: *Proceedings of the 22nd National Conference on Artificial Intelligence - Volume 2*. AAAI’07. Vancouver, British Columbia, Canada: AAAI Press, 2007, pp. 1243–1249. ISBN: 9781577353232.
- [KLC98] L.P. Kaelbling, M.L. Littman, and A.R. Cassandra. “Planning and acting in partially observable stochastic domains”. In: *Artificial Intelligence* 101.1-2 (1998), pp. 99–134.
- [KOA17] Sammie Katt, Frans A Oliehoek, and Christopher Amato. “Learning in POMDPs with Monte Carlo Tree Search”. en. In: (2017).
- [Kat+22] Sammie Katt et al. *BADDr: Bayes-Adaptive Deep Dropout RL for POMDPs*. 2022. arXiv: 2202.08884 [cs.LG].
- [Kel+25] Laura N. Kellman et al. “Functional analysis of cancer-associated germline risk variants”. In: *Nat. Genet.* 57 (Mar. 2025), pp. 718–728. ISSN: 1546-1718. DOI: 10.1038/s41588-024-02070-5.
- [KRL16] Sean K. Keneally, Matthew J. Robbins, and Brian J. Lunday. “A markov decision process model for the optimal dispatch of military medical evacuation assets”. In: *Health Care Manag. Sci.* 19.2 (June 2016). Publisher: See full text options at Springer _eprint: 25223847, pp. 111–129. ISSN: 1572-9389. DOI: 10.1007/s10729-014-9297-8.
- [KL23] Ludvig Killingberg and Helge Langseth. “Bayesian Exploration in Deep Reinforcement Learning”. en. In: *he 2023 symposium of the Norwegian AI Society* (2023).
- [KS06] Levente Kocsis and Csaba Szepesvári. “Bandit Based Monte-Carlo Planning”. In: *Machine Learning: ECML 2006*. Ed. by Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 282–293. ISBN: 978-3-540-46056-5.
- [KL19] Michael R. Kosorok and Eric B. Laber. “Precision Medicine”. In: *Annu. Rev. Stat. Appl.* 6:263-286. (Mar. 2019). ISSN: 2326-8298. DOI: 10.1146/annurev-statistics-030718-105251. eprint: 31073534.
- [Kre16] Nathalie Krell. “Statistical estimation of jump rates for a piecewise deterministic Markov processes with deterministic increasing motion and jump mechanism”. In: *ESAIM: Probability and Statistics* 20 (2016), pp. 196–216. ISSN: 1292-8100, 1262-3318. DOI: 10.1051/ps/2016013.
- [KS21] Nathalie Krell and Émeline Schmisser. “Nonparametric estimation of jump rates for a specific class of piecewise deterministic Markov processes”. In: *Bernoulli* 27.4 (Nov. 2021). ISSN: 1350-7265. DOI: 10.3150/20-BEJ1312.
- [LBH15] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. en. In: *Nature* 521.7553 (May 2015). Publisher: Nature Publishing Group, pp. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539.
- [Len89] Suzanne M. Lenhart. “Viscosity solutions associated with impulse control problems for piecewise-deterministic processes”. en. In: *International Journal of Mathematics and Mathematical Sciences* 12.1 (Jan. 1989), pp. 145–157. ISSN: 0161-1712, 1687-0425. DOI: 10.1155/S0161171289000207.

- [LPE98] Esther Levin, R. Pieraccini, and Wieland Eckert. “Using Markov decision process for learning dialogue strategies”. In: June 1998, 201–204 vol.1. ISBN: 978-0-7803-4428-0. DOI: 10.1109/ICASSP.1998.674402.
- [Lia+18a] Eric Liang et al. “RLlib: Abstractions for Distributed Reinforcement Learning”. In: 2018.
- [Lia+18b] Eric Liang et al. “RLlib: Abstractions for Distributed Reinforcement Learning”. In: *International Conference on Machine Learning (ICML)*. 2018.
- [Luo+22] Fan-Ming Luo et al. “A survey on model-based reinforcement learning”. en. In: (2022).
- [Mil+22] Stephanie Milani et al. “A Survey of Explainable Reinforcement Learning”. In: *arXiv* (Feb. 2022). DOI: 10.48550/arXiv.2202.08434. eprint: 2202.08434.
- [Mni+13] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602 [cs]. Dec. 2013. DOI: 10.48550/arXiv.1312.5602.
- [MT15] Joelle Pineau Mohammad Ghavamzadeh Shie Mannor and Aviv Tamar. “Bayesian Reinforcement Learning: A Survey”. In: *Foundations and Trends in Machine Learning* 8 (2015).
- [MGM23] Athina Monemvassitis, Arnaud Guillin, and Manon Michel. “PDMP Characterisation of Event-Chain Monte Carlo Algorithms for Particle Systems”. en. In: *Journal of Statistical Physics* 190.3 (Mar. 2023), p. 66. ISSN: 0022-4715, 1572-9613. DOI: 10.1007/s10955-023-03069-8.
- [Mur07] Kevin Murphy. *Conjugate Bayesian analysis of the Gaussian distribution*. Tech. rep. University of British Columbia, Nov. 2007.
- [Nad+25] Mariona Nadal-Ribelles et al. “Transcriptional heterogeneity shapes stress-adaptive responses in yeast”. In: *Nat. Commun.* 16.2631 (Mar. 2025), pp. 1–17. ISSN: 2041-1723. DOI: 10.1038/s41467-025-57911-6.
- [Nag+08] Ugrappa Nagalakshmi et al. “The transcriptional landscape of the yeast genome defined by RNA sequencing”. In: *Science* 320.5881 (June 2008), pp. 1344–1349. ISSN: 1095-9203. DOI: 10.1126/science.1158441. eprint: 18451266.
- [Nag+16] Madhav Nagpal et al. “Tumor markers: A diagnostic tool”. In: *Natl. J. Maxillofac. Surg.* 7.1 (Jan. 2016), pp. 17–20. ISSN: 0975-5950. DOI: 10.4103/0975-5950.196135. eprint: 28163473.
- [Pag98] Gilles Pagès. “A space quantization method for numerical integration”. en. In: *Journal of Computational and Applied Mathematics* 89.1 (Mar. 1998), pp. 1–38. ISSN: 03770427. DOI: 10.1016/S0377-0427(97)00190-8.
- [PPP04] Gilles Pagès, Huyên Pham, and Jacques Printems. “An optimal Markovian quantization algorithm for multi-dimensional stochastic control problems”. en. In: *Stochastics and Dynamics* 04.04 (Dec. 2004), pp. 501–545. ISSN: 0219-4937, 1793-6799. DOI: 10.1142/S0219493704001231.
- [Pas+18] Chloé Pasin et al. “Controlling IL-7 Injections in HIV-Infected Patients”. en. In: *Bulletin of Mathematical Biology* 80.9 (Sept. 2018), pp. 2349–2377. ISSN: 0092-8240, 1522-9602. DOI: 10.1007/s11538-018-0465-8.

- [PTL13] Caroline Ponzoni Carvalho Chanel, Florent Teichteil-Königsbuch, and Charles Lesire. “Multi-target detection and recognition by UAVs using online POMDPs”. In: July 2013.
- [Pou+06] Pascal Poupart et al. “An analytic solution to discrete Bayesian reinforcement learning”. In: *Proceedings of the 23rd international conference on Machine learning*. ICML ’06. New York, NY, USA: Association for Computing Machinery, 2006, pp. 697–704. ISBN: 978-1-59593-383-6. DOI: 10.1145/1143844.1143932.
- [PS24] Adrien Prodhomme and Edouard Strickler. “Large population asymptotics for a multitype stochastic SIS epidemic model in randomly switching environment”. In: *The Annals of Applied Probability* 34.3 (June 2024). ISSN: 1050-5164. DOI: 10.1214/23-AAP2035.
- [Put94a] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Apr. 1994. ISBN: 978-0-471-61977-2.
- [Put94b] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Apr. 1994. ISBN: 978-0-471-61977-2.
- [RX22] Md Masudur Rahman and Yexiang Xue. “Robust Policy Optimization in Deep Reinforcement Learning”. In: *arXiv* (Dec. 2022). DOI: 10.48550/arXiv.2212.07536. eprint: 2212.07536.
- [RTW12] Martin Riedler, Michèle Thieullen, and Gilles Wainrib. “Limit theorems for infinite-dimensional piecewise deterministic Markov processes. Applications to stochastic excitable membrane models”. In: *Electronic Journal of Probability* 17.none (Jan. 2012). ISSN: 1083-6489. DOI: 10.1214/EJP.v17-1946.
- [Rie13] Martin G. Riedler. “Almost sure convergence of numerical approximations for Piecewise Deterministic Markov Processes”. In: *Journal of Computational and Applied Mathematics* 239 (2013), pp. 50–71. ISSN: 0377-0427. DOI: <https://doi.org/10.1016/j.cam.2012.09.021>.
- [RT15] Martin G. Riedler and Michèle Thieullen. “Spatio-temporal hybrid (PDMP) models: Central limit theorem and Langevin approximation for global fluctuations. Application to electrophysiology”. In: *Bernoulli* 21.2 (May 2015). ISSN: 1350-7265. DOI: 10.3150/13-BEJ583.
- [RCP07] Stephane Ross, Brahim Chaib-draa, and Joelle Pineau. “Bayes-Adaptive POMDPs”. In: *Advances in Neural Information Processing Systems*. Vol. 20. Curran Associates, Inc., 2007.
- [Ros+11] Stéphane Ross et al. “A Bayesian Approach for Learning and Planning in Partially Observable Markov Decision Processes”. In: *The Journal of Machine Learning Research* 12.null (July 2011), pp. 1729–1770. ISSN: 1532-4435.
- [Ros+25] Orlane Rossini et al. “Bayes-Adaptive Impulse Control of Piecewise-Deterministic Markov Processes”. working paper or preprint. May 2025. URL: <https://hal.science/hal-05054392>.

- [RCA18] Moumita Roy, Chandreyee Chowdhury, and Nauman Aslam. “Designing Transmission Strategies for Enhancing Communications in Medical IoT Using Markov Decision Process”. In: *Sensors (Basel)*. 18.12 (Dec. 2018). Publisher: See full text options at MDPI _eprint: 30558343, p. 4450. ISSN: 1424-8220. DOI: 10.3390/s18124450.
- [RN94] Gavin Adrian Rummery and Mahesan Niranjana. “On-line Q-learning using connectionist systems”. In: (1994).
- [SD12] Benoîte de Saporta and François Dufour. “Numerical method for impulse control of piecewise deterministic Markov processes”. en. In: *Automatica* 48.5 (May 2012), pp. 779–793. ISSN: 00051098. DOI: 10.1016/j.automatica.2012.02.031.
- [SDG17] Benoîte de Saporta, François Dufour, and Alizée Geeraert. “Optimal strategies for impulse control of piecewise deterministic Markov processes”. en. In: *Automatica* 77 (Mar. 2017), pp. 219–229. ISSN: 00051098. DOI: 10.1016/j.automatica.2016.11.039.
- [SDG10] Benoîte de Saporta, François Dufour, and Karen Gonzalez. “Numerical method for optimal stopping of piecewise deterministic Markov processes”. In: *The Annals of Applied Probability* 20.5 (Oct. 2010). ISSN: 1050-5164. DOI: 10.1214/09-AAP667.
- [SDZ15] Benoîte de Saporta, François Dufour, and Huilong Zhang. *Numerical Methods for Simulation and Optimization of Piecewise Deterministic Markov Processes*. Dec. 2015. ISBN: 978-1-84821839-0. DOI: 10.1002/9781119145066.
- [SDZ16] Benoîte de Saporta, François Dufour, and Huilong Zhang. *Numerical methods of simulation and optimization of piecewise deterministic Markov processes: application to reliability*. eng. London Hoboken, NJ: ISTE, Ltd. John Wiley & Sons, 2016. ISBN: 978-1-119-14506-6.
- [Sap+12] Benoîte de Saporta et al. “Optimal stopping for the predictive maintenance of a structure subject to corrosion”. en. In: *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability* 226.2 (Apr. 2012), pp. 169–181. ISSN: 1748-006X, 1748-0078. DOI: 10.1177/1748006X11413681.
- [Sap+24] Benoîte de Saporta et al. “A Monte-Carlo planning strategy for medical follow-up optimization: Illustration on multiple myeloma data”. en. In: *PLOS ONE* 19.12 (Dec. 2024). Ed. by Inés P. Mariño, e0315661. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0315661.
- [Sch+04] Andrew J. Schaefer et al. “Modeling Medical Treatment Using Markov Decision Processes”. In: *Operations Research and Health Care: A Handbook of Methods and Applications*. Ed. by Margaret L. Brandeau, François Sainfort, and William P. Pierskalla. Boston, MA: Springer US, 2004, pp. 593–612. ISBN: 978-1-4020-8066-1. DOI: 10.1007/1-4020-8066-2_23.
- [Sch98] Manfred Schäl. “On piecewise deterministic Markov control processes: Control of jumps and of risk processes in insurance”. en. In: *Insurance: Mathematics and Economics* 22.1 (May 1998), pp. 75–91. ISSN: 01676687. DOI: 10.1016/S0167-6687(98)00010-9.
- [Sch+03] Anne-Sofie Schrohl et al. “Tumor markers: from laboratory to clinical utility”. In: *Mol. Cell. Proteomics* 2.6 (June 2003), pp. 378–387. ISSN: 1535-9476. DOI: 10.1074/mcp.R300006-MCP200. eprint: 12813140.

- [Sch+15] John Schulman et al. “Trust region policy optimization”. In: *International conference on machine learning*. PMLR. 2015, pp. 1889–1897.
- [Sch+17] John Schulman et al. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347 [cs]. Aug. 2017. DOI: 10.48550/arXiv.1707.06347.
- [Sha09] S. Sharma. “Tumor markers in clinical practice: General principles and guidelines”. In: *Indian J. Med. Paediatr. Oncol.* 30.1 (Jan. 2009), pp. 1–8. ISSN: 0975-2129. DOI: 10.4103/0971-5851.56328. eprint: 20668599.
- [She+20] Qianli Shen et al. “Deep Reinforcement Learning with Robust and Smooth Policy”. In: *arXiv* (Mar. 2020). DOI: 10.48550/arXiv.2003.09534. eprint: 2003.09534.
- [SPG07] Yoav Shoham, Rob Powers, and Trond Grenager. “If multi-agent learning is the answer, what is the question?” In: *Artif. Intell.* 171.7 (2007), pp. 365–377. ISSN: 0004-3702. DOI: 10.1016/j.artint.2006.02.006.
- [Sil+14] D. Silver et al. “Deterministic policy gradient algorithms”. In: *International Conference on Machine Learning (ICML)*. 2014, pp. 387–395.
- [SV10] David Silver and Joel Veness. “Monte-Carlo Planning in Large POMDPs”. In: *Advances in Neural Information Processing Systems*. Ed. by J. Lafferty et al. Vol. 23. Curran Associates, Inc., 2010.
- [Sin+00] Satinder Singh et al. “Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms”. en. In: *Machine Learning* 38.3 (Mar. 2000), pp. 287–308. ISSN: 1573-0565. DOI: 10.1023/A:1007678930559.
- [Sin23] Musalula Sinkala. “Mutational landscape of cancer-driver genes across human cancers”. In: *Sci. Rep.* 13.12742 (Aug. 2023), pp. 1–14. ISSN: 2045-2322. DOI: 10.1038/s41598-023-39608-2.
- [Son73] Edward J. Sondik. “The Optimal Control of Partially Observable Markov Processes over a Finite Horizon”. In: *Oper. Res.* 21.5 (Oct. 1973), pp. 1071–1088. ISSN: 0030-364X. DOI: 10.1287/opre.21.5.1071.
- [Soz+01] Gabriella Sozzi et al. “Analysis of Circulating Tumor DNA in Plasma at Diagnosis and during Follow-Up of Lung Cancer Patients”. In: *Cancer Res.* 61.12 (June 2001), pp. 4675–4678. ISSN: 0008-5472.
- [SV05] Matthijs T. J. Spaan and Nikos Vlassis. “Perseus: randomized point-based value iteration for POMDPs”. In: *J. Artif. Int. Res.* 24.1 (2005), pp. 195–220. ISSN: 1076-9757.
- [Sta+19] Kateřina Staňková et al. “Optimizing Cancer Treatment Using Game Theory”. In: *JAMA oncology* 5.1 (Jan. 2019), p. 96. DOI: 10.1001/jamaonco.1.2018.3395.
- [Str+06] Alexander L Strehl et al. “PAC Model-Free Reinforcement Learning”. en. In: (2006).
- [Str00] Malcolm J. A. Strens. “A Bayesian Framework for Reinforcement Learning”. In: *Proceedings of the Seventeenth International Conference on Machine Learning*. ICML '00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, pp. 943–950. ISBN: 978-1-55860-707-1.

- [Sut+99] R. S. Sutton et al. “Policy gradient methods for reinforcement learning with function approximation”. In: *Neural Information Processing Systems 12*. 1999, pp. 1057–1063.
- [Sut88] Richard S. Sutton. “Learning to predict by the methods of temporal differences”. en. In: *Machine Learning* 3.1 (Aug. 1988), pp. 9–44. ISSN: 1573-0565. DOI: 10.1007/BF00115009.
- [SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, Oct. 2018. ISBN: 978-0-262-03924-6.
- [Świ+23] Maciej Świechowski et al. “Monte Carlo Tree Search: a review of recent modifications and applications”. In: *Artificial Intelligence Review* 56.3 (Mar. 2023), pp. 2497–2562. ISSN: 1573-7462. DOI: 10.1007/s10462-022-10228-y.
- [Tow+24] Mark Towers et al. “Gymnasium: A Standard Interface for Reinforcement Learning Environments”. In: *arXiv preprint arXiv:2407.17032* (2024).
- [Tsi03] John N. Tsitsiklis. “On the convergence of optimistic policy iteration | The Journal of Machine Learning Research”. In: vol. 3. J. Mach. Learn. Res., 2003, pp. 59–72. DOI: 10.1162/153244303768966102.
- [Vas+23] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL].
- [Vel15] R. Veltz. “A new twist for the simulation of hybrid systems using the true jump method”. 2015.
- [Wan+05] Tao Wang et al. “Bayesian sparse sampling for on-line reward optimization | Proceedings of the 22nd international conference on Machine learning”. In: Association for Computing Machinery, 2005, pp. 956–963. DOI: 10.1145/1102351.1102472.
- [WC23] Weikai Wang and Xian Chen. “Piecewise deterministic Markov process for condition-based imperfect maintenance models”. In: *Reliability Engineering & System Safety* 236 (2023), p. 109271. ISSN: 0951-8320. DOI: 10.1016/j.ress.2023.109271.
- [Wan+16] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. Apr. 2016. DOI: 10.48550/arXiv.1511.06581.
- [Wat89] Christopher J. C. H. Watkins. “Learning from Delayed Rewards”. PhD thesis. University of Cambridge, 1989.
- [WD92a] Christopher J. C. H. Watkins and Peter Dayan. “Q-learning”. en. In: *Machine Learning* 8.3 (May 1992), pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698.
- [WD92b] Christopher J. C. H. Watkins and Peter Dayan. “Q-learning”. en. In: *Machine Learning* 8.3 (May 1992), pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698.
- [Wei23] Hsiu-Chuan Wei. “Mathematical modeling of tumor growth and treatment: Triple negative breast cancer”. In: *Math. Comput. Simul.* 204.C (Feb. 2023), pp. 645–659. ISSN: 0378-4754. DOI: 10.1016/j.matcom.2022.09.005.
- [WJH19] Ryan R. Wick, Louise M. Judd, and Kathryn E. Holt. “Performance of neural network basecalling tools for Oxford Nanopore sequencing”. In: *Genome Biol.* 20.1 (Dec. 2019), pp. 1–10. ISSN: 1474-760X. DOI: 10.1186/s13059-019-1727-y.

- [Wöl+21] Benjamin Wölfl et al. “The contribution of evolutionary game theory to understanding and treating cancer”. In: *medRxiv* (Apr. 2021), p. 2020.12.02.20241703. DOI: 10.1007/s13235-021-00397-w.
- [WZC10] Feng Wu, Shlomo Zilberstein, and Xiaoping Chen. “Rollout sampling policy iteration for decentralized POMDPs”. In: *Proceedings of the Twenty-Sixth Conference on Uncertainty in Artificial Intelligence*. UAI’10. Catalina Island, CA: AUAI Press, 2010, pp. 666–673. ISBN: 9780974903965.
- [Wu+21] Zhanghao Wu et al. “RLlib Flow: Distributed Reinforcement Learning is a Dataflow Problem”. In: *Conference on Neural Information Processing Systems (NeurIPS)*. 2021.
- [XC22] Changfa Xia and Wanqing Chen. “The NCC mathematical modeling framework for decision-making of six major cancers”. In: *J. Natl. Cancer Cent.* 3.1 (Nov. 2022), pp. 35–47. ISSN: 2667-0054. DOI: 10.1016/j.jncc.2022.11.002. eprint: 39036317.
- [Yin+19] Anyue Yin et al. “A Review of Mathematical Models for Tumor Dynamics and Treatment Resistance Evolution of Solid Tumors”. In: *CPT: Pharmacometrics Syst. Pharmacol.* 8.10 (Aug. 2019), p. 720. DOI: 10.1002/psp4.12450.
- [You+13] Steve Young et al. *POMDP-Based Statistical Spoken Dialog Systems: A Review*. 2013.
- [Zha+19] Jingsong Zhang et al. “Integrating evolutionary dynamics into treatment of metastatic castrate-resistant prostate cancer (mCRPC): Updated analysis of the adaptive abiraterone (abi) study (NCT02415621).” In: *J. Clin. Oncol.* (May 2019).
- [ZT20] Kaiyu Zheng and Stefanie Tellex. “pomdp_py: A Framework to Build and Solve POMDP Problems”. In: *ICAPS 2020 Workshop on Planning and Robotics (PlanRob)*. Arxiv link: "<https://arxiv.org/pdf/2004.10099.pdf>". 2020.

Résumé

La prise de décision séquentielle dans des systèmes partiellement observables et à dynamique inconnue est particulièrement difficile, notamment lorsque les données sont limitées. C'est le cas en médecine, où l'état des patients est observé de façon intermittente et où l'évolution des maladies varie fortement d'un individu à l'autre. Les stratégies de traitement personnalisées visent alors à adapter le suivi à la dynamique propre de chaque patient afin de réduire les traitements inutiles et de favoriser une détection précoce des rechutes. Les Processus de Markov Déterministes par Morceaux (PDMPs) sont adaptés pour modéliser des maladies comme le cancer, dont l'évolution est en grande partie déterministe mais ponctuée d'événements aléatoires. Toutefois, leur contrôle optimal devient complexe en présence d'observations partielles ou d'incertitude sur le modèle. Cette thèse propose d'abord une méthode numérique pour calculer une stratégie ϵ -optimale dans un PDMP complètement observé à dynamique connue. Le problème est ensuite étudié en cadre partiellement observable via une reformulation en POMDP, résolu grâce à l'apprentissage par renforcement profond. Enfin, pour intégrer l'incertitude sur la dynamique, nous introduisons un POMDP Bayésien Adaptatif (BAPOMDP), permettant d'apprendre des stratégies de contrôle robustes et personnalisées à partir de données limitées et incertaines.

Mots-clés Processus semi-Markovien Déterministe par Morceaux, Etats Cachés, Processus de Décision Markovien, Contrôle Stochastique, Apprentissage par Renforcement Profond, Optimisation de Traitement

Abstract

Sequential decision-making in partially observable systems with unknown dynamics is particularly difficult, especially when data is limited. This is the case in medicine, where patients' conditions are observed intermittently and the progression of diseases varies greatly from one individual to another. Personalised treatment strategies therefore aim to adapt follow-up to each patient's specific dynamics in order to reduce unnecessary treatment and promote early detection of relapses. Piecewise Deterministic Markov Processes (PDMPs) are suitable for modelling diseases such as cancer, whose progression is largely deterministic but punctuated by random events. However, their optimal control becomes complex in the presence of partial observations or uncertainty about the model. This thesis first proposes a numerical method for calculating an ϵ -optimal strategy in a fully observed PDMP with known dynamics. The problem is then studied in a partially observable setting via a reformulation into a POMDP, solved using deep reinforcement learning. Finally, to incorporate uncertainty about the dynamics, we introduce an Adaptive Bayesian POMDP (BAPOMDP), which allows robust and customised control strategies to be learned from limited and uncertain data.

Keywords Piecewise Deterministic semi-Markov Processes, Hidden State, Markov Decision Processes, Stochastic Control, Deep Reinforcement Learning, Treatment Optimisation